

DISTRIBUTED COGNITIVE PROMPTING FRAMEWORK

— FOR AI COLLABORATION —

AN EXPERIMENTAL APPROACH

INTRODUCTORY EDITION



GOVERNANCE



RESEARCH



KNOWLEDGE



RUNTIME

RALPH A. PEREZ, JR.

Table of Contents

This Table of Contents is linked for digital navigation. Select a front-matter section, chapter, or appendix to move directly to it. A Return to Table of Contents link appears at the end of each front-matter section, chapter, and appendix.

Front Matter

[Title Page](#)

[Copyright, Free Distribution, Fair Use, and Template Permissions](#)

[Important Notice: The Nature and Limitations of DCPF](#)

[Preface](#)

[How to Use This Book](#)

Chapters

[Chapter 1 - From Conversation to Collaboration](#)

[Chapter 2 - When Prompts Become Systems](#)

[Chapter 3 - The Emergence of the Distributed Cognitive Prompting Framework](#)

[Chapter 4 - Introducing the Distributed Cognitive Prompting Framework for AI Collaboration \(DCPF\):
Moving from Prompting to Collaboration Architecture](#)

[Chapter 5 - The Governance Framework](#)

[Chapter 6 - The Research Framework](#)

[Chapter 7 - The Knowledge Framework](#)

[Chapter 8 - The Runtime Framework](#)

[Chapter 9 - Preparing to Work with DCPF](#)

[Chapter 10 - Building a General-Purpose Research Assistant](#)

[Chapter 11 - Building a Critical Thinking Research Assistant](#)

[Chapter 12 - Building a Personal Cookbook Assistant](#)

[Chapter 13 - Building a Personal Study Guide Assistant](#)

[Chapter 14 - Building a Software Documentation Assistant](#)

[Chapter 15 - Moving Forward](#)

Appendices

[Appendix A - DCPF Quick Reference Guide](#)

[Appendix B - Framework Templates](#)

[Appendix C - Recommended DCPF Workspace Organization](#)

[Appendix D - DCPF Project Checklist](#)

[Appendix E - Sequential Output Requirements](#)

[Appendix F - DCPF Glossary](#)

[Appendix G - DCPF Responsibility-Consequence Map](#)

Copyright, Free Distribution, Fair Use, and Template Permissions

Copyright © 2026 Ralph A. Perez, Jr. All rights reserved.

This publication, including its original text, explanations, diagrams, visual elements, and protectable selection and arrangement, is protected under applicable copyright law. Except as expressly permitted below or by applicable law, no part of this publication may be reproduced, republished, transmitted, incorporated into another publication, course, template collection, or commercial product, or otherwise used in a manner not authorized by this notice, without prior written permission from the copyright owner.

DCPF and the Distributed Cognitive Prompting Framework for AI Collaboration identify the organizational framework described in this publication. This notice does not claim copyright in any idea, fact, system, process, method of operation, or other material that applicable copyright law excludes from protection. Copyright protection applies to the protectable expression embodied in this publication.

Free Distribution Permission

This ebook is made available by the author free of charge. Readers may download, retain, reproduce, and share complete, unmodified copies of the ebook for personal, educational, professional, and organizational use, provided that the copyright notice, attribution, legal and use notices, and other identifying information contained in the ebook remain intact.

This permission applies to the complete, unmodified ebook. It does not authorize a person or organization to publish or distribute an altered edition as an official DCPF publication; sell, license, or commercially redistribute the ebook itself; remove or obscure copyright, attribution, or source information; or use the DCPF name or the author's identity in a way that suggests certification, sponsorship, endorsement, partnership, or affiliation.

dcpfai.org is the authoritative site and definitive reference for the most current official ebook editions and DCPF materials. Copies shared or obtained from other sources may be older or superseded. Readers should consult dcpfai.org when confirming the current edition, current materials, permissions information, or updates.

Limited Template License

Readers who lawfully obtain this publication may reproduce, complete, modify, and use templates expressly identified as copyable for:

- personal learning and experimentation;
- educational and training activities; and
- internal organizational testing, evaluation, and experimentation.

Template Restrictions

The template permission applies only to templates expressly identified as copyable and does not authorize the reader to:

- reproduce or distribute individual chapters or substantial portions of the book as separate publications, except as permitted by applicable law or separate written permission;
- reproduce or separately distribute the book's diagrams or other visual elements except as allowed by law or separate written permission;
- publish, sell, license, or redistribute blank or substantially unchanged DCPF templates as a standalone product, template library, training package, or competing publication;
- remove or obscure copyright or attribution notices; or
- use the DCPF name or the author's identity in a way that suggests certification, sponsorship, endorsement, partnership, or affiliation.

Responsibility for DCPF Implementations

Permission to download, share, use, or adapt DCPF materials does not transfer responsibility for a user's implementation, customized artifacts, project contents, AI-generated outputs, decisions, services, deliverables, or resulting work products to the author or publisher. DCPF provides organizational structure; the user controls the contents and use of that structure. Subject to applicable law and the rights of others, users retain ownership of content they author and remain responsible for all content they create, select, adapt, or incorporate.

Each user assumes responsibility and risk for their own use of DCPF, whether for personal, educational, organizational, professional, commercial, or client-facing purposes. To the fullest extent permitted by applicable law, the author and publisher are not responsible or liable for losses, damages, claims, liabilities, costs, or other harm arising from or related to a user's implementation or use of DCPF or DCPF-based materials. These responsibilities and limitations are described more fully in Important Notice: The Nature and Limitations of DCPF.

Nothing in these responsibility and risk provisions should be interpreted as an endorsement, certification, warranty, or representation that DCPF or its experimental templates are suitable for professional, commercial, client-facing, regulated, or production use.

Fair Use and Other Lawful Uses

Nothing in this notice is intended to limit fair use or any other exception or limitation provided by applicable law. Fair use is determined according to the facts and circumstances of each use. Attribution does not, by itself, substitute for permission when permission is legally required.

Third-Party Materials and Trademarks

All third-party trademarks, service marks, product names, platform names, company names, quotations, and other third-party materials remain the property of their respective owners. Their appearance in this publication does not imply sponsorship, endorsement, or affiliation. Third-party materials, if any, are included under permission, license, public-domain status, or applicable law.

Permissions

Requests for permissions beyond those expressly granted above may be submitted through dcpfai.org. The site is also the authoritative reference for current DCPF ebook editions, materials, and permissions information.

[Return to Table of Contents](#)

Important Notice: The Nature and Limitations of DCPF

Please read this notice before applying DCPF. It defines the role of the framework, the limits of what it can accomplish, and the reader's continuing responsibility for decisions, actions, and work products.

Acknowledgment of Conditions. By using, adapting, implementing, or distributing DCPF or DCPF-based materials, the user acknowledges the copyright, permissions, responsibilities, disclaimers, and limitations in this publication and accepts responsibility for their own implementation and use. Read this notice together with Copyright, Free Distribution, Fair Use, and Template Permissions.

DCPF as an Organizational Framework

The Distributed Cognitive Prompting Framework for AI Collaboration (DCPF) is an experimental, experience-based organizational architecture designed to help people structure and manage collections of AI interactions through Governance, Research, Knowledge, and Runtime. It provides guidelines, artifacts, and practices for organizing human-AI collaboration.

DCPF can be thought of, in part, like a structured paper form or worksheet. A form provides fields, categories, and an organizational sequence, but the person completing it determines what information is entered and how the completed information is used. In the same way, DCPF provides an architecture for organizing human-AI collaboration. The user remains responsible for the information supplied, the instructions given, the outputs accepted, and the decisions or actions that follow.

DCPF can also be understood as functioning like a filing cabinet. A filing cabinet provides drawers, folders, labels, and an organizational structure, but it does not author, verify, approve, or own the contents placed inside it. In the same way, DCPF organizes project information without taking ownership or responsibility for that information. DCPF does not claim ownership of content a user creates or places into a DCPF implementation. Subject to applicable law and the rights of others, users retain ownership of content they author and remain responsible for all information, instructions, research, knowledge, governance materials, runtime requests, and other content they create, select, adapt, or incorporate.

DCPF is not an AI model, software platform, autonomous agent, or substitute for human judgment. It does not operate, control, retrain, or alter the underlying AI system, and it cannot override that system's technical limitations, safeguards, context limits, availability, or provider policies.

The DCPF Frameworks describe standard responsibilities, while the artifacts used to represent those responsibilities are adaptable rather than rigidly prescribed documents. Readers decide which AI systems and tools to use, how to configure their projects, what information and instructions to supply, how to represent the standard DCPF responsibilities, which supplemental elements to add when useful, and how to test and maintain their implementation.

Experimental Status and Validation Boundary

DCPF has demonstrated promise through the development work, applications, and practicums presented in this Introductory Edition, but it has not been broadly or independently validated across domains, organizations, AI systems, or production environments.

Readers should begin with DCPF in a controlled, low-risk sandbox or testing environment. They should evaluate and validate their own implementation against the requirements of their domain, project, AI system, and operating environment before considering real-world or production use. Any decision to deploy an experimental DCPF implementation outside a testing environment remains the responsibility of the user or organization making that decision.

No Guarantee of Results

The author does not warrant or guarantee that DCPF, or any example, template, instruction, workflow, or practicum presented in this book, will produce any particular result. DCPF cannot guarantee the accuracy, completeness, currency, reliability, objectivity, safety, usefulness, or suitability of an AI-generated output. It also cannot guarantee identical or consistent results across models, platforms, accounts, sessions, projects, or users.

Validation provides evidence about performance under the conditions tested. It does not prove perfect, universal, or continuously consistent performance. AI systems may generate inaccurate, incomplete, outdated, biased, inconsistent, or misleading material, and they may omit important context. A well-organized DCPF project can make AI collaboration more structured and reviewable, but it cannot eliminate these risks.

Results depend on factors that include:

- the capabilities, limitations, training, safeguards, and behavior of the selected AI system;
- the accuracy, completeness, relevance, and lawful use of the information supplied;
- the quality and maintenance of the project's Governance, Research, Knowledge, and Runtime materials;
- model settings, context limits, service availability, platform changes, and provider policies; and
- the reader's instructions, judgment, review practices, and implementation decisions.

Reader Control and Responsibility

The reader remains in full control of how DCPF is applied and how the reader interacts with any AI system. The reader is responsible for:

- selecting the AI systems, tools, data sources, and project purposes;
- deciding what information may appropriately and lawfully be submitted to an AI service;
- creating, modifying, testing, documenting, and maintaining DCPF artifacts;

- reviewing and independently verifying material statements, recommendations, calculations, citations, and conclusions;
- deciding whether an AI output should be accepted, revised, rejected, published, shared, or acted upon;
- protecting confidential, personal, proprietary, and security-sensitive information and respecting applicable copyright and license restrictions;
- evaluating privacy, intellectual-property, security, bias, fairness, transparency, and accessibility concerns; and
- complying with applicable laws, regulations, professional obligations, contractual duties, and organizational policies.

Use of DCPF does not transfer responsibility for the reader's decisions, actions, adaptations, or resulting work products to the framework, the author, the publisher, or an AI service provider.

User Responsibility, Assumption of Risk, and Third-Party Use

DCPF is an organizational framework, not a service that supervises or controls a user's implementation. Each user assumes the risks associated with designing, configuring, customizing, testing, validating, maintaining, and using their own DCPF implementation. This responsibility applies whether DCPF is used for a personal project, educational activity, internal organizational purpose, professional activity, or work performed for a client, customer, employer, project stakeholder, or other third party.

The author and publisher do not control the information or instructions placed into a DCPF project; the AI models, platforms, software, data sources, or other tools selected; the configuration or modification of customized artifacts; the outputs accepted, rejected, modified, shared, or relied upon; the validation or professional review performed; or the decisions, services, deliverables, and work products that result. The user therefore remains responsible for determining whether the user's implementation, customized artifacts, project contents, AI-generated outputs, and resulting work products are accurate, lawful, safe, appropriate, and suitable for their intended purpose.

Use of DCPF is undertaken at the user's own risk. To the fullest extent permitted by applicable law, the author and publisher are not responsible or liable for losses, damages, claims, liabilities, costs, business interruption, lost revenue, lost data, consequential losses, or other harm arising from or related to a user's selection, customization, implementation, use, misuse, interpretation, or reliance upon DCPF, a DCPF artifact, an AI system, an AI-generated output, or a resulting work product.

This limitation applies both to harm experienced directly by the user and to claims, losses, or damages involving a client, customer, employer, organization, project stakeholder, or other third party arising from the user's implementation or use of DCPF. Anyone using DCPF in professional, organizational, commercial, or client-facing work remains responsible for establishing appropriate contracts, review procedures, testing, validation, professional oversight, insurance, security controls, or other safeguards for that person's or organization's own activities.

Nothing in these responsibility and risk provisions should be interpreted as permission, endorsement, certification, warranty, or a representation that DCPF or its experimental templates are suitable for professional, commercial, client-facing, regulated, or production use.

Nothing in this section excludes or limits any responsibility or liability that applicable law does not permit to be excluded or limited.

Use with Third-Party AI Systems

DCPF may be used with different AI models, platforms, and tools, but the author does not operate, control, or guarantee those third-party systems. Each provider may apply different technical limits, safeguards, data-handling practices, privacy terms, intellectual-property terms, retention practices, availability conditions, and usage policies. Readers are responsible for understanding the terms and risks of the systems they choose to use and for deciding what information is appropriate to provide to them.

The ability to organize or transfer DCPF materials across systems does not mean that every system will interpret, retain, retrieve, or execute those materials in the same way. Readers should validate important work in the environment in which it will actually be used.

Professional and High-Stakes Uses

This book and its examples are provided for educational and informational purposes. They are not legal, medical, financial, accounting, cybersecurity, engineering, employment, regulatory, or other professional advice. AI-generated material concerning health, law, finance, public safety, employment, compliance, security, or another consequential matter should be independently reviewed by appropriately qualified professionals and checked against current primary or authoritative sources before it is used to make or support a decision.

Neither use of this book nor use of DCPF creates a professional, advisory, fiduciary, or other special relationship between the reader and the author or publisher.

Changes in AI Systems

AI models, platform features, provider terms, governance practices, laws, and regulations continue to change. Examples that function as described at the time of publication may behave differently in another environment or at a later date. Readers are responsible for testing DCPF materials in their current environment and revising them when evidence, technology, law, policy, or project requirements change.

Current Editions and Authoritative Materials

DCPF materials may be revised as the framework, examples, supporting resources, AI systems, and related practices evolve. dcpfai.org is the authoritative site and definitive reference for the most current official ebook editions and DCPF materials. Copies of the ebook or supporting materials obtained from other sources may be older or superseded. Readers should consult dcpfai.org when confirming whether they are using the current edition or current supporting materials.

Disclaimer of Warranties and Limitation of Responsibility

To the fullest extent permitted by applicable law, this publication and its materials are provided for educational and informational purposes on an 'as is' and 'as available' basis. The author and publisher make no express or implied warranty that use of DCPF will be error-free, uninterrupted, suitable for a particular purpose, or capable of producing a particular outcome.

To the fullest extent permitted by applicable law, the author and publisher are not responsible for losses, damages, claims, costs, liabilities, or consequences arising from the reader's selection, customization, implementation, use, misuse, interpretation, or reliance on DCPF, a DCPF artifact, an AI system, an AI-generated output, or a resulting work product, including claims involving third parties. Nothing in this notice excludes or limits any responsibility or liability that applicable law does not permit to be excluded or limited.

DCPF can help make AI collaboration more structured, persistent, governed, transparent, and reviewable. It remains a human-directed organizational framework. Human judgment, verification, and accountability remain essential at every stage.

[Return to Table of Contents](#)

Preface

Thank you for supporting this book. Writing it wasn't in my plans a short time ago, but the book emerged unexpectedly from my own effort to solve a practical problem.

My experience with AI began during a period of personal and professional transition following a life-changing visual impairment diagnosis. I learned to use AI as a tool to help with this transition, and it became increasingly useful in my audiobook projects. I discovered that my greatest challenge was not simply finding better prompts. It was finding a better way to organize the information, decisions, research, instructions, and responsibilities that accumulated across many interactions. That experience eventually led to the Distributed Cognitive Prompting Framework for AI Collaboration, or DCPF. Chapter 3 tells that development story in greater detail.

This Introductory Edition presents DCPF as an experimental, experience-based approach to organizing sustained human-AI collaboration. The framework has demonstrated promise through the development work, practical applications, and training practicums represented in this book. It has not, however, been broadly or independently validated across domains, organizations, AI systems, or production environments.

I believe that distinction is important. DCPF is not presented here as a finished technical standard or a universally proven methodology. It is a working architecture that emerged through practical use and is now being made available so that other people can learn from it, test it, challenge it, adapt it, and determine where it is useful.

The book introduces DCPF's four coordinated responsibilities: Governance, Research, Knowledge, and Runtime. You will also find examples, practicums, templates, checklists, and other tools to help you understand how they work together. These materials are intended to give you a structured starting point for experimentation rather than prescribe one rigid implementation.

My hope is that you use this book to create your own controlled testing environment for DCPF. Start with a small, low-risk project in a domain you understand. Build the framework around that project. Observe where it helps, where it creates unnecessary structure, where the AI behaves differently than expected, and where your own implementation needs to change. Test the system against criteria that matter in your environment. Use the training examples and practicums to develop an understanding of the basic workflow and terminology.

If DCPF shows promise for your particular use, continue testing and refining it. Different domains may require different evidence standards, governance controls, validation procedures, safeguards, professional review, or technical implementations. Any decision to move an experimental DCPF implementation from a testing environment into real-world or production use belongs to the person or organization making that decision.

That responsibility is also part of the philosophy behind DCPF. AI can perform substantial reasoning, research, organization, and execution, but the human collaborator remains responsible for deciding what the system should do, what information it should rely upon, what outputs are accepted, and whether those outputs are appropriate to use.

I do not expect DCPF to stop evolving. Some of its ideas may prove broadly useful. Others may work better in particular circumstances, require modification, or reveal limitations through further experimentation. Overall, I believe this framework has the potential to provide some relief from the pain points the AI community is currently experiencing in human-AI collaboration. Making it widely accessible now gives others an opportunity to begin exploring that potential sooner rather than later.

This book would not be possible without the support of my partner, adult children, and friends who encouraged me through life's changes and new discoveries, especially in recent years.

What began as an effort to solve my own challenges with AI collaboration has become an invitation to experiment. I hope the architecture and tools in this book give you enough structure to discover for yourself where DCPF works, where it does not, and what it might become.

[Return to Table of Contents](#)

How to Use This Book

What This Book Will Help You Do

This book is an introduction to the Distributed Cognitive Prompting Framework for AI Collaboration, or DCPF. It is designed to help you organize AI-assisted work into structured, reusable workflows rather than isolated prompting techniques. You control the information supplied, the instructions given, the outputs accepted, and the decisions and actions that follow.

The practicum chapters will help you learn how to create structured requests, preserve useful project knowledge, guide repeatable execution, and apply governance throughout the process. By the end of the book, you will build your own artifacts adapted to your own AI-assisted projects.

The goal is not simply to help you write better prompts. It is to help you build an organizational system for collaborating with AI that you can continue using long after this book is finished.

What This Book Is Not

DCPF is presented here as an experimental, experience-based approach rather than an academic study or comprehensive technical standard. It is not a programming textbook, a computer science course, or an exhaustive AI reference.

What Has Been Tested and What Has Not

DCPF grew from practical use and has demonstrated promise through the development work, applications, and practicums represented in this Introductory Edition. It has not been validated across every professional, academic, technical, regulated, or organizational domain. Readers should not assume that this book establishes domain-specific DCPF requirements for environments that have not been tested.

If you apply DCPF in another domain, treat that use as an experiment requiring the judgment of people who understand that domain. Domain-specific evidence standards, regulations, safety requirements, professional obligations, and workflow needs should be identified through qualified practice and testing rather than inferred from this book. Chapter 15 discusses this open validation boundary and invites further cross-domain work.

Who Should Read This Book

You may find this content helpful if your AI work:

- Expands beyond simple questions.
- Develops into long conversations or recurring projects.

- Requires repeated instructions, files, or retained knowledge.
- Becomes difficult to coordinate consistently.

Whether you are a student, hobbyist, professional, researcher, educator, business owner, or regular AI collaborator working independently or as part of a team, no formal technical background is required.

Equipment and Other Requirements

- A smartphone and text-to-speech can be used for reading or listening to the conceptual chapters.
- Practicum work is easier on a laptop or desktop computer with a work environment that includes the book, word-processing software, a browser open to your chosen AI system, and a file manager.
- Familiarity with these applications and file management is recommended.

How to Move Through This Book

For your first reading, work through the main chapters in sequence. DCPF is introduced progressively, with each concept preparing you for the practical work that follows. You do not need to memorize every framework, artifact, or rule before beginning the practicums. Understanding will develop as you use them.

Because Chapters 1–9 contain conceptually dense material, listening with a screen reader or text-to-speech is a recommended way to work through them. Pause when useful around framework artifacts, tables, examples, and reader checkpoints rather than treating the material as something that must be completed in one continuous session.

You do not have to complete a cover-to-cover reading before beginning to explore DCPF. If your AI system supports PDF or document uploads, you may upload the complete ebook to a project, workspace, or working context and ask the AI to explain concepts, answer questions about particular sections, compare the four frameworks, or help you identify material you want to study more closely. Free AI subscription levels may have file-size, context, or usage limits that prevent reliable use of the entire book at one time. A paid subscription level with larger document and context capacity is therefore recommended for whole-book exploration, although capabilities vary by AI system and subscription. Use the ebook itself as the authoritative source whenever exact DCPF terminology, framework language, templates, or requirements matter.

When you reach the practicums, work alongside the examples rather than simply reading them. Save the artifacts you create, organize them in a project folder, and revise them as your understanding develops. The objective is not to produce a perfect DCPF implementation on your first attempt. It is to understand how the pieces work together well enough to begin using them responsibly.

Use the appendices as working references rather than required sequential reading. Return to the templates, checklists, and other book appendices when they become useful during your projects. Additional validation guidance, troubleshooting resources, and supporting materials are available at dcpfai.org.

Digital Navigation

This PDF includes a linked Table of Contents for direct navigation to the front matter, Chapters 1–15, and Appendices A–G. A visible Return to Table of Contents link appears at the end of every front-matter section, chapter, and appendix. Your PDF reader may also provide search, bookmarks, or a document outline that can help you locate individual sections, headings, and terms.

A Reasonable Time Commitment

After reading the Front Matter material, plan approximately five to eight focused hours to read or listen to Chapters 1–9. Actual time will vary with reading or screen-reader speed, breaks, note-taking, concentration, and how often you pause to consider the examples and artifacts.

The first practicum (Chapter 10) can generally be completed in forty-five to ninety minutes. Completing several practicums will provide a solid foundation for using DCPF independently. Additional time will be needed to revise the framework artifacts for the requirements of your own projects.

DCPF Is Designed to Be Flexible

The four DCPF responsibilities remain stable even when the artifacts, tools, and implementation details change. Adapt them to your projects while preserving the distinct responsibilities of the Governance, Research, Knowledge, and Runtime Frameworks.

AI capabilities, governance practices, and regulations continue to evolve. DCPF should therefore be viewed as a living framework that will continue to be refined through research, practical experience, and future editions. The goal of this introductory edition is to provide a durable starting architecture that can adapt as both your projects and AI technology evolve.

Book Composition Note

Due to the author's visual impairment, portions of this book have been AI-generated. The author conceived and refined DCPF. The author also directed, modified, designed, and approved the complete manuscript.

[Return to Table of Contents](#)

From Conversation to Collaboration

DCPF is an experimental approach that has demonstrated promise but has not been broadly validated. Test and validate your implementation within your own environment before considering real-world or production use.

1. The Breakthrough Was Conversation

When a Conversation Becomes Too Valuable to Lose

Conversational AI made powerful computing capabilities accessible through ordinary language. People could ask questions, explore ideas, analyze information, write, plan, and revise without first learning a specialized interface. More importantly, the exchange could develop. The model could respond to feedback, follow a line of thought, and adapt as the user's objective became clearer.

That remains a genuine breakthrough. A clear prompt, useful context, and a few rounds of refinement can support an enormous range of short, self-contained tasks. A conversation may be all that is needed to draft an email, compare options, clarify a concept, or solve a contained problem.

ChatGPT is the most familiar example in this book because it played an important role in the development of DCPF. The architectural principles are not limited to one product or provider. They apply whenever people collaborate with an AI model through instructions, supporting information, and a sequence of related interactions.

Success Creates a New Expectation

The experience becomes more demanding when a successful conversation grows into a project. One request becomes a sequence of related tasks. Research accumulates, decisions become connected, and the work continues across days, weeks, or months. As the model appears to understand more of the project, we naturally expect that understanding to remain available.

That is where conversation and collaboration begin to separate.

The problem does not begin because conversational prompting failed. It begins because the conversation worked well enough to become part of something larger.

2. When a Conversation Becomes a Project

A Short Request Can Carry a Long History

Consider a book project in which the audience, editorial voice, terminology, chapter sequence, material to avoid, and earlier approvals have been refined through many exchanges. Inside the active conversation, a request as short as "Revise this section" may produce work that fits beautifully. The surrounding context supplies a foundation that the four-word request never states.

The same pattern appears in many domains. A project plan may depend on earlier budget assumptions. Software documentation may depend on approved terminology and the behavior of a particular release. A lesson plan may depend on grade level, available time, and decisions made across several earlier sessions. The immediate instruction can be short because the project behind it is not.

Start a new conversation and use the same instruction, however, and the result may be competent without belonging to the same book. The model did not suddenly lose its writing ability. The new interaction did not receive the project foundation that made the earlier work specific.

The Friction Is Familiar

People experience the gap in practical ways. They repeat instructions, reattach familiar files, search old conversations, summarize earlier decisions, and correct outputs that no longer match the project. Sometimes the reconstruction works. Sometimes one small but consequential rule stays behind in an old chat.

A new interaction may also begin with whatever memory, connected files, retrieval tools, or platform features are available. Those capabilities can reduce repetition, but they do not guarantee that every project decision will reappear at the right moment or with the right authority. If a critical rule exists only in the earlier conversation, the project is still depending on a temporary location.

The understanding was real, but it was not automatically durable.

The project has become larger than the conversation that first supported it. Collaboration now requires a way to preserve what the work depends upon and deliberately return it to future interactions.

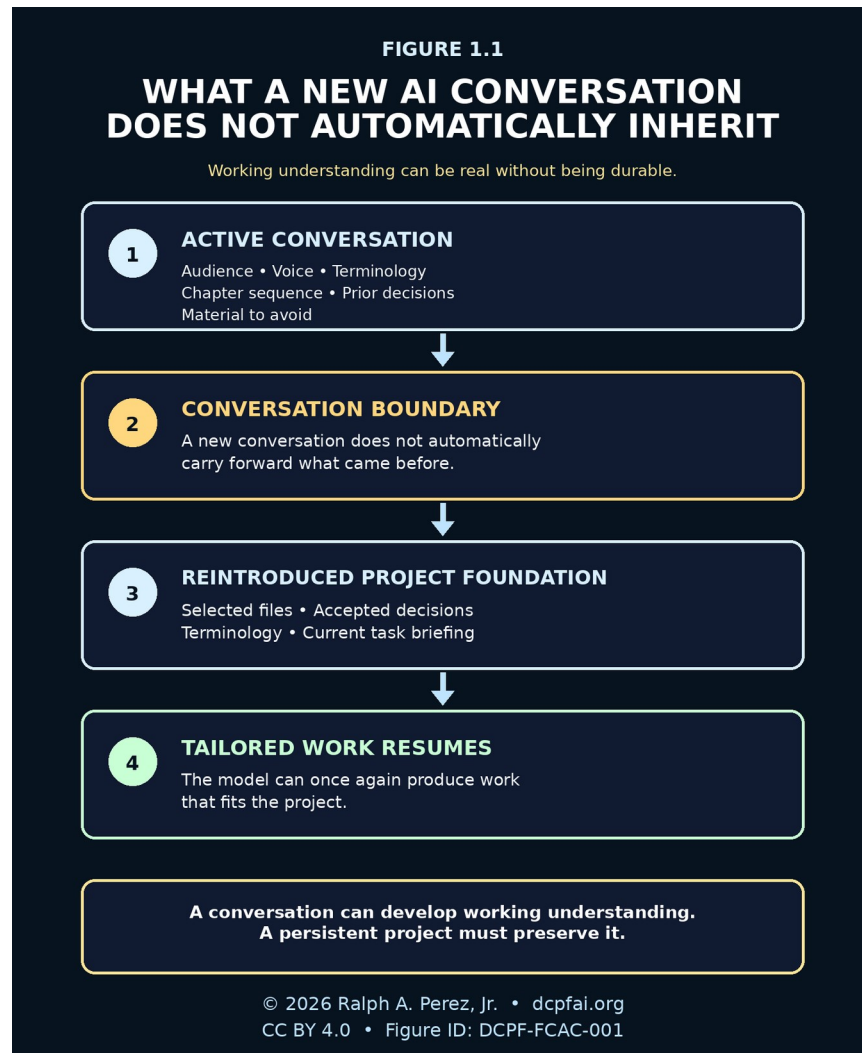


Figure 1.1. A new conversation does not automatically inherit the working understanding developed in an earlier interaction.

3. The Context Window Is a Workspace, Not the Project

The Workshop Is Larger Than the Workbench

The context window is the model's active workspace. It includes the instructions, conversation history, files, and other material available for the present interaction. Readers do not need to calculate tokens to use the concept. The practical point is simply that the model works from what is on the bench now, not from everything that may exist somewhere in the larger project.

The model also draws upon broad patterns learned during training and, depending on the platform, may use tools or external sources. Those capabilities are different from possessing the project's current decisions. General knowledge can help the model reason, but the project still has to supply the particular facts, standards, and constraints that make the task its own.

Think of the project as a workshop and the active context as its workbench. The workshop may contain tools, plans, reference material, earlier decisions, and years of accumulated knowledge. The workbench should hold the selected resources needed for the task underway.

More Is Not Automatically Better

Placing everything on the bench at once can create a different problem. Outdated drafts, conflicting instructions, and unrelated files compete with the information that actually matters. Important guidance can remain present in a long interaction without remaining dependably influential as the work changes around it.

Larger context windows help. They allow more conversation and more source material to participate in one interaction. Capacity alone, however, cannot decide which source is current, which decision was approved, or which standard should govern. A larger workbench is useful; it does not organize the workshop.

Context becomes something to prepare, not something to assume.

For sustained work, accepted rules, relevant knowledge, and current decisions must be deliberately selected and reintroduced when the task needs them. This turns context preparation from a recovery tactic into a normal part of project design.

Suppose ten editorial rules emerged during several hours of discussion. Leaving them scattered across the conversation asks each later request to rediscover their importance. Placing the accepted rules in one concise, maintained resource makes the foundation portable. A fresh interaction can begin with that resource, the relevant chapter material, and the immediate objective instead of the entire history of how every rule was debated.

4. Memory, Retrieval, and Project Knowledge Do Different Jobs

Familiar Is Not the Same as Authoritative

Memory features can make later interactions feel more familiar. A platform may retain selected preferences, recurring interests, or personal details that help a new conversation begin with less repetition. Retrieval tools perform another useful job: they locate files, sources, search results, or references that may matter to the request.

Managed project knowledge serves a different purpose. It establishes what the project can rely on: approved decisions, validated evidence, accepted terminology, current standards, version status, and ownership. Memory may remember that a user is writing a cookbook. Retrieval may locate recipe files. Neither function, by itself, determines which recipes were approved, which food-safety standard governs, or which editorial rule is current.

A simple analogy helps. A librarian may remember that a reader prefers history and needs large-print editions. That familiarity improves service. The library performs another job: it preserves, organizes, and identifies resources the reader can use. Memory can know the collaborator; maintained knowledge supports the collaboration.

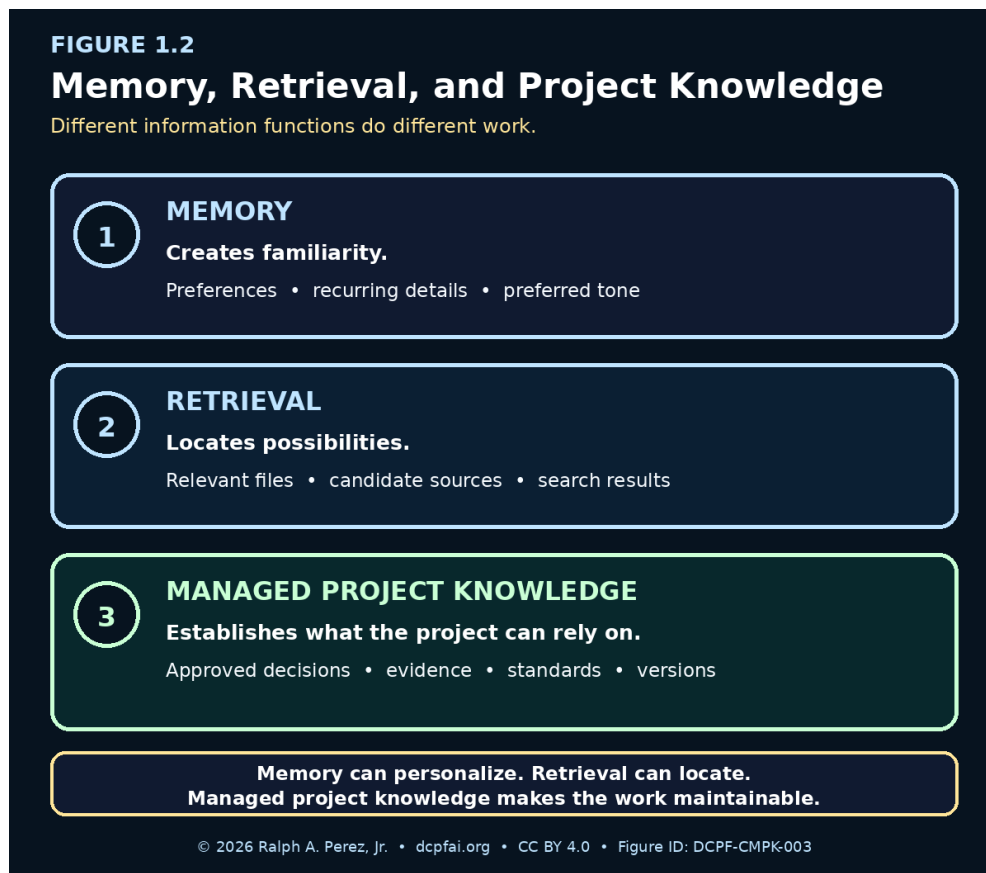


Figure 1.2. Memory can personalize, retrieval can locate, and managed project knowledge preserves what the project can rely on.

Search Can Find It. Who Decides What Governs?

Searchable information is not automatically trusted information. Duplicate documents, vague filenames, missing dates, and conflicting versions can leave a model with several plausible sources and no dependable basis for choosing among them. People still decide what is accepted, which version is authoritative, and when material should be updated or retired.

A Saved Conversation Is a Record, Not Yet a Repository

A saved chat preserves exploration, including useful conclusions, abandoned alternatives, temporary instructions, and speculation. Reusable knowledge is more selective. It captures what was reviewed, accepted, and organized for future work. The difference is not merely where information is stored; it is whether the project has decided what that information means and how it should be used.

Turning a useful response into project knowledge therefore requires a deliberate promotion. The result is captured, separated from discarded alternatives, reviewed or verified, given a clear home, and maintained for later use. Some outputs will be revised. Others will remain temporary or be discarded. The project becomes stronger by preserving what earned a continuing role, not by treating every generated sentence as permanent.

5. From Conversation to Collaboration

Preserve the Foundation, Prepare the Work

The shift can now be stated simply. Conversations create and apply working understanding. Persistent projects must preserve selected knowledge beyond the interaction. Useful results need to be reviewed, organized, and made available to later work. Future conversations should begin from a prepared foundation rather than a reconstructed history.

This creates a useful cycle. Project knowledge supports a new conversation. The conversation produces an answer, decision, or artifact. People evaluate the result and decide whether any part of it should strengthen the maintained foundation. What deserves to persist returns to the project; what served only the immediate task does not have to follow.

None of this diminishes the conversation. The conversation remains the flexible place where people ask questions, compare possibilities, make corrections, and produce work. Collaboration adds continuity around it.

The conversation remains the place where work happens. Collaboration requires a way to preserve what the work depends upon.

Once users recognize this need, the natural temptation is to put everything important into increasingly detailed prompts. Rules, examples, reference material, project history, and quality requirements begin collecting in the same container. The prompts become more capable - and quietly begin taking on the responsibilities of a system.

The next chapter examines that transition and the architectural questions it creates.

[Return to Table of Contents](#)

When Prompts Become Systems

DCPF is an experimental approach that has demonstrated promise but has not been broadly validated. Test and validate your implementation within your own environment before considering real-world or production use.

1. Better Prompts Still Matter

When Does a Prompt Stop Being Merely a Prompt?

A successful prompt does not become a system simply because it is long. The change occurs when it begins carrying responsibilities that belong to an entire project.

Prompt engineering remains essential to AI collaboration. Clear objectives, relevant context, examples, constraints, output requirements, templates, critique, and refinement can dramatically improve a model's work. Many tasks need nothing more than a well-prepared prompt.

The difficulty begins when one prompt becomes the permanent home for everything the project has learned, decided, prohibited, and expected. Research, terminology, rules, examples, quality criteria, and the immediate task begin sharing the same container.

The warning sign is not length. It is responsibility.

DCPF does not replace prompt engineering. It addresses what happens when effective prompting grows into persistent project organization.

2. Success Changes the Prompt

One Good Prompt Learns to Multiply

Prompt systems often begin with a successful instruction. The user saves it, turns it into a template, adds useful examples or rules, and creates variations for other tasks.

The progression is natural:

Prompt → Detailed Prompt → Template → Prompt Library → Prompt System

A system begins to emerge when prompts, rules, knowledge, artifacts, and outputs become dependent upon one another across tasks and over time. No software platform or automation is required. A small collection of documents can already function as a system when a change in one place affects several others.

The same transition can occur when a prompt is expected to coordinate a large, multi-part deliverable. The work may become easier to control when it is divided into stages, intermediate results are reviewed, and accepted decisions are carried forward.

At that point, the prompt is doing more than requesting an answer. It is beginning to preserve knowledge, coordinate workflow, and carry standards across interactions.

3. Four Signals That the Prompt Has Become a System

When One Small Edit Touches Everything

The first signal is intertwined responsibility. Facts, terminology, editorial rules, business requirements, and restrictions occupy the same prompt or collection. A small change can require reviewing information belonging to several different responsibilities.

Version control can show what changed, but it does not untangle those responsibilities.

Copy, Paste, Maintain Forever

The second signal is copied knowledge. The same facts, policies, or standards appear in multiple prompts. Every copy becomes another place that must be maintained.

Copying initially makes each prompt conveniently self-contained. The weakness appears when something changes and someone must locate every copy and determine whether they still agree.



Figure 2.1. Four recurring signals reveal when a prompt has begun carrying system-level responsibilities.

When the Copies Tell Different Stories

The third signal is divergence. One workflow contains the current information while another relies on an older version. Different workflows are now reasoning from different foundations.

Architecture does not require identical outputs. The same approved information can be explained differently for different audiences. What should remain stable are the consequential facts, policies, terminology, and standards beneath those explanations.

When Growth Multiplies the Problem

The fourth signal is scale. More people, projects, and workflows create more copies, dependencies, and opportunities for inconsistency.

Scale also reveals that some knowledge should be shared while other knowledge should remain local. A common organizational standard may apply broadly, while specialized project information may belong only to one workflow.

The four signals form a chain: Intertwined responsibilities encourage copying. Copies diverge. Growth multiplies the divergence.

4. Reuse the Knowledge, Not the Container

The Difference Between Copying and Reusing

Prompt-centered reuse begins with a successful document: save it, copy it, and adapt it.

Architectural reuse goes one level deeper. It identifies the knowledge that made the document useful and maintains that knowledge so multiple workflows can use it.

Consider an organization with one approved explanation of its products. Copying that explanation into sales, support, training, and documentation prompts gives every workflow what it needs, but also creates separate copies that can age differently.

If the approved explanation is maintained once and supplied wherever needed, one revision can improve every dependent workflow.

This does not require a database or elaborate technical platform. In a small project, the authoritative source might simply be one clearly named document that is supplied when needed.

Centralized knowledge produces distributed consistency.

The prompt can then focus on the current objective, relevant knowledge, instructions, workflow, and expected result rather than serving as a warehouse for everything the project knows.

Storage Is Useful. Architecture Adds Responsibility.

Moving information into folders is not enough. A folder can preserve six files named some variation of Final without identifying which one is authoritative.

Architecture adds responsibility, authority, relationships, and reuse. It distinguishes exploratory information from accepted knowledge, shared standards from task-specific instructions, and enduring decisions from temporary work.

Those relationships also need paths between them. Research must be reviewable. Accepted knowledge must return to active work. Outputs that reveal outdated information or missing guidance must be able to trigger correction.

The goal is not simply to separate information. It is to organize information by responsibility while keeping the parts that depend upon one another connected.

5. Thinking Outside the Model

The Solution Is Older Than Computers

Human beings have always extended their thinking by placing knowledge outside individual memory. Architects use plans. Pilots rely on checklists. Scientists maintain laboratory records. Organizations preserve policies, procedures, and technical standards. These resources do not replace intelligence; they make intelligence more dependable.

AI collaboration benefits from the same principle. Project goals, accepted knowledge, sources, workflow instructions, standards, version status, and approval decisions can exist outside the model in structures people can inspect and maintain. The model receives the portion required for the current task while the larger project foundation remains available for other work.

External cognition also makes knowledge easier to share and improve. Subject-matter experts can review it, teams can discuss it, and owners can update it without searching for the conversation in which it first appeared. The resource can continue creating value even if the model or platform changes.

Inside the Model and Outside the Model

This separation creates a useful division of responsibility. Inside the interaction, the model can reason, compare, synthesize, generate, and apply the context it has been given. Outside the model, people establish purpose, preserve accepted knowledge, maintain sources and standards, define ownership, and decide which changes should become authoritative.

The model can help develop these resources. It cannot independently decide that a draft policy now governs the project, that one conflicting source should replace another, or that its own suggestion has been approved. Those are architectural and human decisions.

Knowledge should endure. Execution should adapt.

Maintained knowledge can therefore remain stable while current instructions change with the task. Research can expand without rewriting every operating prompt. A style guide can change without altering factual sources. The result is not less information. It is information organized by purpose.

When useful lessons return to the appropriate maintained component, experience begins to compound. One project can leave behind clearer procedures, stronger examples, better evaluation criteria, or newly accepted knowledge. The next project starts from that stronger foundation rather than repeating the same discovery.

6. Human Direction, External Architecture, and AI Reasoning

The Model Is Powerful. It Is Not the Whole System.

DCPF does not place the AI model in charge of the larger collaboration environment. People determine what the project is trying to accomplish, which evidence is acceptable, what knowledge should persist, and what outcomes meet the standard. The external architecture organizes and preserves those choices. The AI contributes reasoning within the structure it receives.

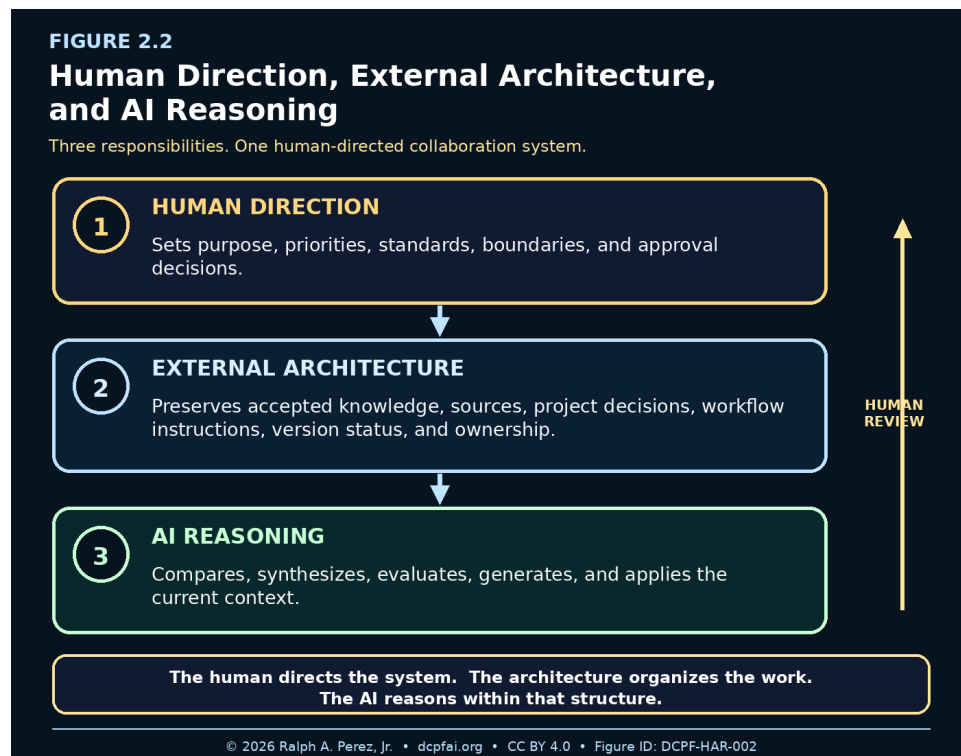


Figure 2.2. Human direction establishes purpose and standards, external architecture preserves the project foundation, and AI reasoning performs the current work.

Governance is the structured expression of that human responsibility. It can establish source requirements, quality expectations, safety boundaries, review points, and conditions for approval. The AI may compare work with those criteria and identify possible problems. It does not silently authorize exceptions or promote its own suggestion into accepted project knowledge.

The relationship resembles a capable engine inside a vehicle. The engine provides power, but steering, navigation, braking, and the driver's decisions determine where that power goes. In the same way, a language model may be the most dynamic component of the collaboration without being the entire collaboration system.

The human directs the system. The architecture organizes the work. The AI reasons within that structure.

As models become more capable, this external organization becomes more valuable, not less. Durable knowledge can survive changes in conversations, workflows, models, and platforms. Human ownership remains visible while the reasoning engine continues to improve.

7. The Questions That Led to DCPF

Once the Responsibilities Separate, the Architecture Appears

The prompt-system problem leads to a set of practical questions:

- How should reliable information be gathered and evaluated?
- How should accepted project knowledge be preserved?
- How should the immediate task receive only what it needs?
- How should human-defined standards guide the work from beginning to end?
- How should useful outputs strengthen future work?

These are not questions about one perfect prompt. They are questions about persistent responsibilities and the relationships among them. They point toward an architecture in which research, knowledge, execution, and governance can be maintained separately while continuing to support one another.

Prompt engineering optimizes individual interactions. DCPF organizes collections of interactions into persistent architectural systems.

The next chapter traces how these questions developed into the Distributed Cognitive Prompting Framework for AI Collaboration.

[Return to Table of Contents](#)

The Emergence of the Distributed Cognitive Prompting Framework

DCPF is an experimental approach that has demonstrated promise but has not been broadly validated. Test and validate your implementation within your own environment before considering real-world or production use.

This chapter traces how an accessible creative project grew into a durable system for managing research, standards, workflows, and reusable knowledge. It shows how the limits of an ever-expanding prompt revealed four distinct responsibilities and led to the Distributed Cognitive Prompting Framework for AI Collaboration.

3.1 From One Story to a System

The Distributed Cognitive Prompting Framework did not begin as an attempt to create a new AI architecture. It began with a story.

In October 2024, while adapting to a life-changing visual impairment and building an accessible path into voiceover and audiobook production, I asked an AI assistant to help me create scripts for adult relaxation and meditation experiences. My goal was to spend less time physically writing and more time recording and editing audio. The results were encouraging, useful starting points that I reshaped, expanded, and refined into my own finished work.

The Project Begins to Outgrow the Prompt

The finalized scripts supported my first published audiobook in 2025. While writing and partially recording the next audiobook, I decided to pursue something more ambitious: professionally narrated journeys through America's national parks. The stories would remain relaxing while conveying the character of real places. A listener should feel accompanied by a knowledgeable guide through an authentic landscape, not placed inside a generic forest or mountain scene.

To meet the information needs of that challenge, I began formal AI training while developing the concept iteratively. Each park required dependable information about geography, geology, plants, wildlife, seasons, weather, and cultural history. The series also needed a recognizable narrative voice, consistent pacing, smooth transitions, production standards, and a reliable balance between education and relaxation.

Every success suggested another improvement. A more accurate description made the setting more believable. A better transition made the narration more restful. A stronger production rule made the finished recording more consistent. None of these refinements was excessive by itself. Their combined effect was to turn a creative experiment into a repeatable production project.

Success Comes With a Filing Problem

At first, I improved the results in the most direct way I knew. I added more information to the prompt. Better wildlife descriptions required wildlife guidance. More accurate geology required research. A consistent voice required stylistic instructions and examples. Each addition made sense, and for a time the approach worked.

The stories became better, but the process became harder to manage. Useful research was scattered across conversations. An improvement made for one story had to be found and repeated for the next. Prompts grew longer because the project itself was growing.

The challenge was no longer writing better prompts. It was managing knowledge.

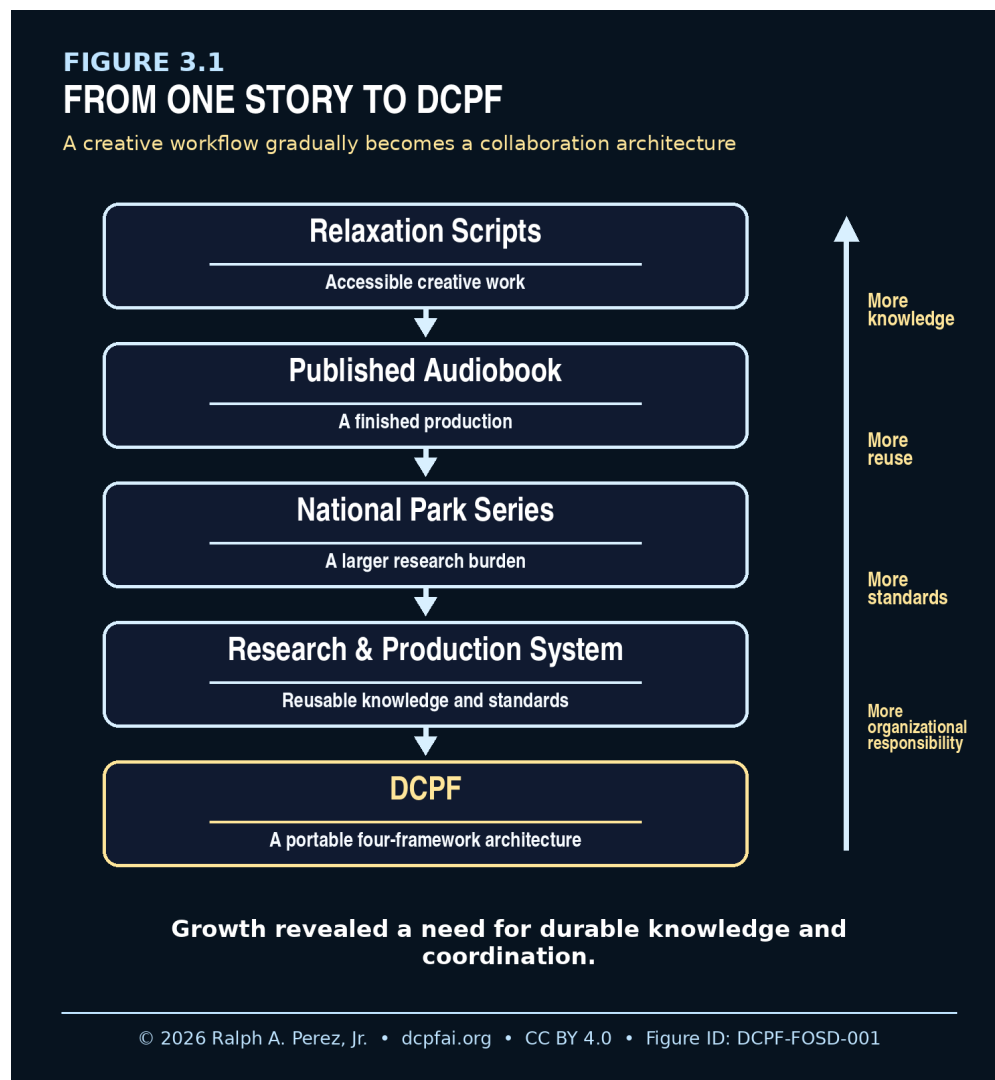


Figure 3.1. DCPF emerged gradually as accessible creative work became a published audiobook and then a more ambitious project accumulated research, standards, workflows, and reusable knowledge.

No single step in Figure 3.1 demanded an architecture. The need became visible only as successful work accumulated. What began as an accessible creative process had quietly become a system.

3.2 When the Prompt Became the Problem

My first response to growing complexity was to keep refining the prompt. I reorganized instructions, created templates, copied research into new conversations, and documented recurring preferences. The output often improved, which made the deeper problem easy to miss.

Preparation kept growing. Before each new story, I had to gather research, locate the current instructions, restore terminology and stylistic decisions, and check whether an earlier discovery had been carried forward. The prompt was being asked to hold research, standards, examples, workflows, and project history at the same time.

Those materials did not behave alike. Research expanded park by park. Narration and production guidance evolved across the series. Some information applied everywhere; other material belonged to one location or one story.

Packing everything into one prompt made information harder to find, update, and reuse. Each conversation became an isolated workspace, so context had to be rebuilt and successful techniques manually copied forward. As the stories improved, reproducing that success became harder because important decisions were scattered and their current versions were unclear.

The prompt gradually became the project's junk drawer. Everything important was probably in there, but finding the right thing at the right moment had become a project of its own.

I had mistaken conversation for organization.

The prompt itself was not defective. I was asking it to perform responsibilities that belonged to a larger system. The better question was not, “How can I build a better prompt?” but, “How should knowledge be organized before AI begins to reason with it?” That question changed the direction of the project.

3.3 Moving Knowledge Outside the Conversation

The first important change looked like ordinary housekeeping: I moved recurring information into separate documents. Research that had already been completed, established narration guidance, definitions, production requirements, and quality criteria deserved stable homes instead of being reconstructed from memory.

It looked less like an architectural breakthrough and more like someone finally cleaning the office. Oddly enough, both can solve the same problem.

I separated the material by purpose. Research went into research documents; storytelling guidance into a maintained style resource; terminology, production standards, and project instructions into their own locations. A runtime conversation could then receive only what its immediate task required without becoming the permanent home of the project's knowledge.

The first benefit was easier maintenance and reuse. A corrected definition or refined narrative principle could improve every future story from one authoritative location, and research became a reusable asset rather than something rediscovered for each interaction.

The AI was not becoming more intelligent. The system surrounding the AI was.

The distinction became clearer: the model analyzed, connected, transformed, and generated from the information it received, while the surrounding architecture preserved, organized, and supplied that information over time. Research and production rules could now be reviewed, approved, updated, and versioned without being buried inside operational conversations.

This was more than moving text into files. It meant assigning authority and purpose: maintaining one accepted version of a rule and distinguishing exploratory findings from knowledge ready for use. Organization improved the quality of the context, not merely its location.

Once knowledge moved outside the conversation, the project no longer depended on one ever-expanding prompt. The separate resources formed an advanced relaxation-story generation system, and their most useful organizational concepts proved portable beyond publishing. The project was beginning to develop an architecture.

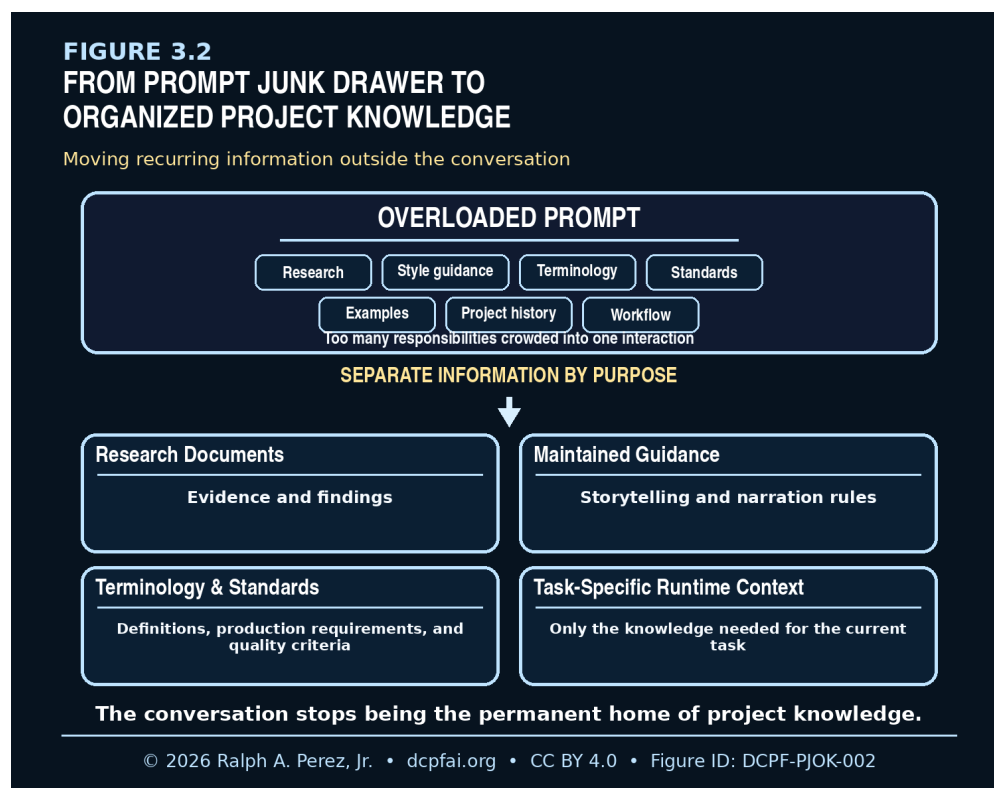


Figure 3.2. Moving recurring information outside the conversation separates overloaded prompt responsibilities into stable external resources organized by purpose.

Once the information had separate homes, the next question became unavoidable: what responsibility did each home actually serve?

3.4 Four Responsibilities Emerge

As the external resources grew, their responsibilities became clearer. Some discovered and verified information. Others preserved what the project had accepted. Operational instructions selected that knowledge for a specific task, while standards and quality checks shaped what could enter the system, how work should be performed, and whether results were acceptable.

These responsibilities eventually became four coordinated frameworks.

A framework is not necessarily one document. It is a defined responsibility that may be implemented through documents, templates, repositories, procedures, or tools. That distinction let the structure remain recognizable even as the implementation grew more sophisticated.

Four Frameworks. Four Distinct Responsibilities.

The **Research Framework** organizes the discovery, evaluation, and preparation of evidence. It helps the project ask what must be learned, which sources are appropriate, and how findings should be recorded for later use.

The Knowledge Framework owns the architectural responsibility for preserving Accepted Knowledge so it can be maintained and reused. That Knowledge is represented and maintained in the Knowledge Repository, which may contain accepted findings, terminology, standards, decisions, examples, and other reusable project intelligence.

The **Runtime Framework** directs execution. It defines the immediate objective, identifies the knowledge required, establishes the workflow, and specifies the expected output.

The **Governance Framework** supplies human direction across the architecture. It establishes authority, boundaries, quality criteria, review expectations, and processes for correction and improvement. Governance is not a final inspection added after the work is complete. It influences what research is acceptable, what knowledge becomes authoritative, how runtime activities operate, and how outputs are evaluated.

Governance exists as an architectural responsibility from the beginning, but a separate Governance artifact does not have to. In a small or early-stage project, Governance elements may first appear inside Research Requests, Knowledge acceptance and repository controls, Runtime Requests, and review steps. As recurring standards become clear, they can be consolidated into a maintained Governance artifact for reuse and consistency.

Together, the frameworks distribute responsibilities that had previously been crowded into prompts. Research discovers. Knowledge preserves. Runtime executes. Governance guides.

A Correction Finally Has Somewhere to Go

The frameworks also make refinement possible. A runtime output may reveal missing knowledge. Governance review may identify an unclear standard. New research may correct an outdated assumption. The lesson can be routed back to the framework responsible for maintaining it rather than patched into every future prompt.

This turned a collection of documents into a cognitive system. The components were specialized but not isolated: information moved among them under human direction, and each completed project could strengthen the work that followed.

The four-part structure did not appear fully formed. It emerged across hundreds of AI interactions as I recognized recurring work and gave each responsibility a clear home. The portable concepts carried the system beyond audiobook production and toward DCPF. The frameworks exist to solve practical coordination problems, not to add ceremony.

3.5 Naming the Framework and Defining the Contribution

For some time, I developed the system without giving it a formal name. The focus remained practical: Did a change make the work easier to organize, maintain, reuse, and improve?

Eventually, the pattern needed language. Cognitive responsibilities were no longer concentrated inside one prompt or conversation. They were distributed across specialized structures and coordinated through prompting. That pattern gave practical meaning to a phrase I did not originate: Distributed Cognitive Prompting.

The name describes three aspects of the approach. **Distributed** means that research, preserved knowledge, execution, and governance have distinct responsibilities. **Cognitive** emphasizes that the architecture supports reasoning, learning, organization, judgment, and decision-making, not text generation alone. **Prompting** recognizes that prompts remain essential as the means of directing work within the larger environment.

What Is DCP, and What Is Specifically DCPF?

I make no claim to have originated or own Distributed Cognitive Prompting, and the broader ideas associated with distributed cognition and cognitive prompting predate this work. In this book, Distributed Cognitive Prompting, or DCP, names the broader organizational approach being interpreted and applied.

My specific contribution is the Distributed Cognitive Prompting Framework for AI Collaboration, or DCPF: a four-framework architecture integrating Governance, Research, Knowledge, and Runtime. Its originality lies not in the phrase itself, but in the design, organization, integration, governance, and practical implementation of those components as a unified strategy for managing AI-assisted projects.

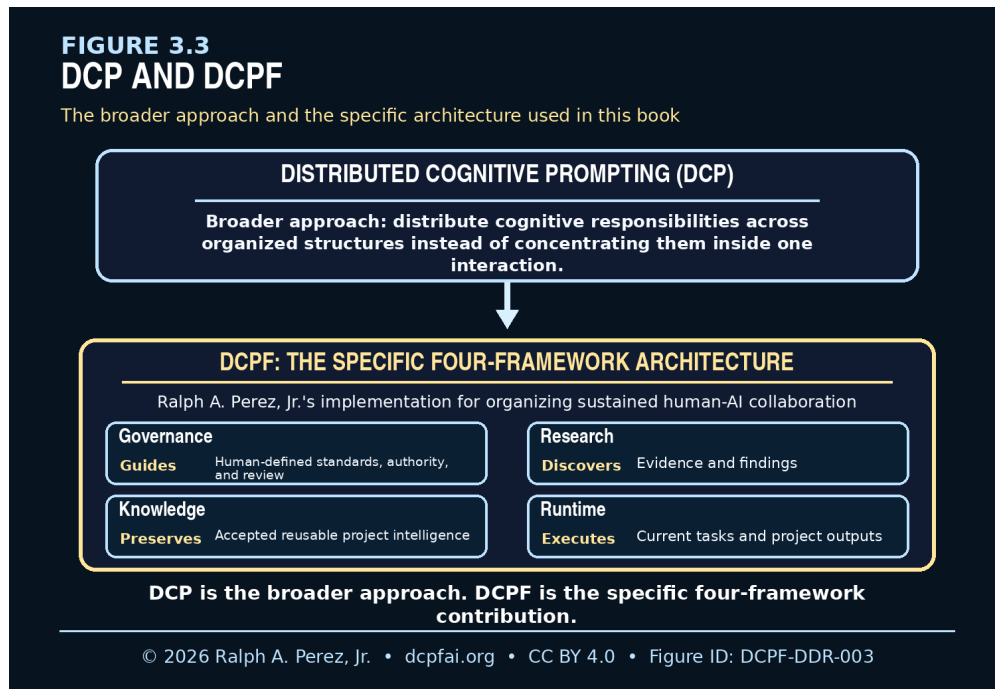


Figure 3.3. Distributed Cognitive Prompting is the broader approach; DCPF is the author's specific four-framework architecture for putting it into practice.

Prompts Keep a Focused Role

Within DCPF, a prompt can direct Research, apply maintained Knowledge, guide Runtime execution, or request evaluation without carrying the project's full history and every responsibility.

Prompt engineering optimizes individual interactions. DCPF organizes collections of interactions into persistent architectural systems.

The architecture remains stable while implementation changes. A small project may use only a few coordinated documents, with Governance controls embedded where the work occurs; a medium project may use versioned repositories, maintained Governance artifacts, templates, tools, and multiple runtime workflows. Evidence standards, knowledge structures, procedures, and controls can vary by field.

Stable architecture. Flexible implementation.

This flexibility allowed the framework to support the publishing project without being limited to it. The same responsibilities appear across sustained AI work: discover reliable information, preserve what matters, execute with purpose, and govern the system.

3.6 From Discovery to Architecture

What began with relaxation scripts and grew through a published audiobook and national park project revealed a larger principle: capable AI does not remove the need to organize knowledge. It makes that organization more valuable.

The logic is direct. Ambitious work created more knowledge; more knowledge created maintenance problems. External structures separated durable knowledge from temporary execution. The advanced relaxation-story system revealed a portable pattern. Those concepts became DCPF's four coordinated frameworks.

The next chapter leaves the discovery story behind and examines that architecture directly, the responsibility each framework owns, how information moves through the system, and how governance guides the whole.

[Return to Table of Contents](#)

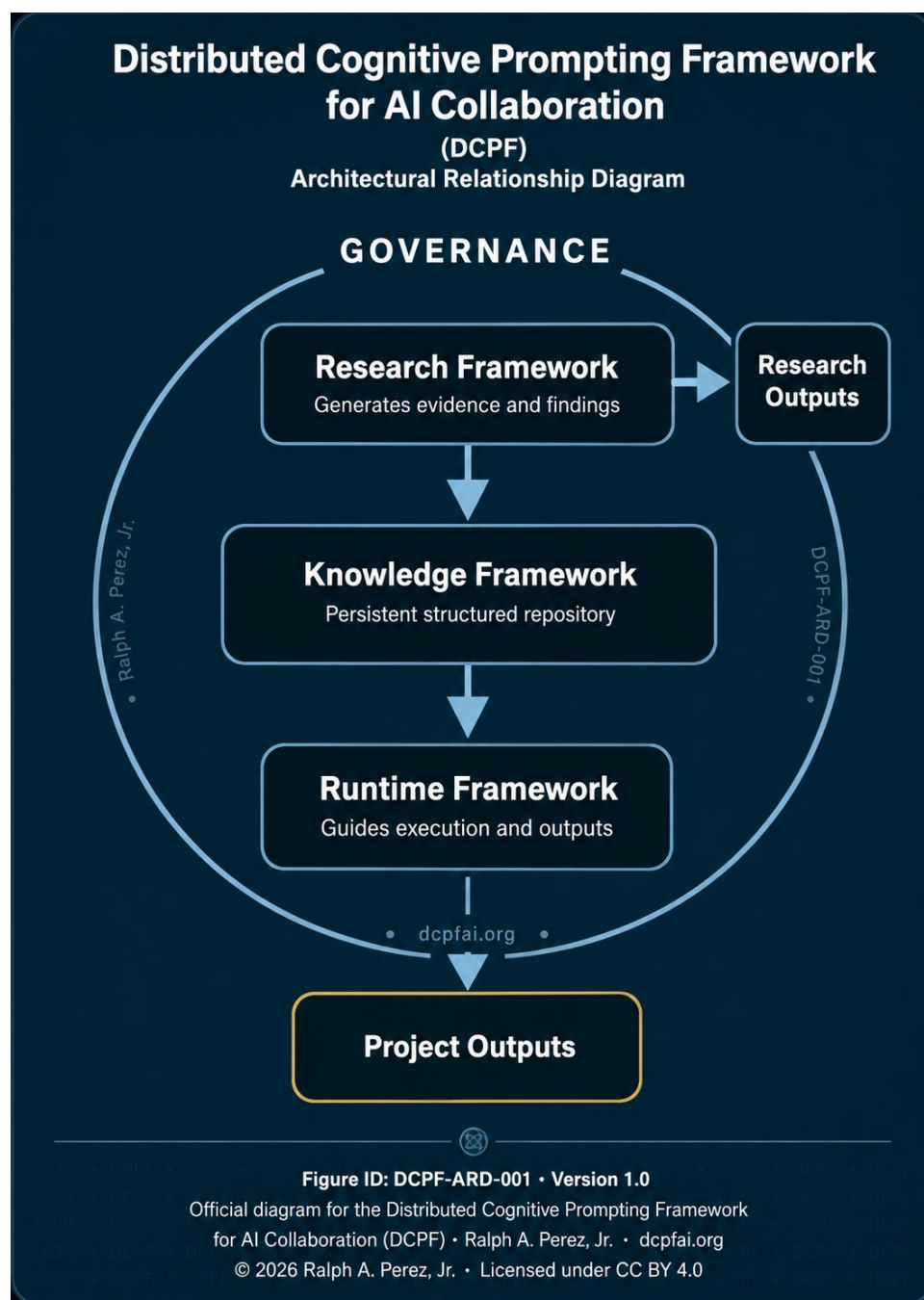
Introducing the Distributed Cognitive Prompting Framework for AI Collaboration (DCPF): Moving from Prompting to Collaboration Architecture

DCPF is an experimental approach that has demonstrated promise but has not been broadly validated. Test and validate your implementation within your own environment before considering real-world or production use.

This chapter introduces DCPF's complete collaboration architecture and the four responsibilities that organize sustained work between people and AI:

Research develops evidence. Knowledge preserves Accepted Knowledge. Runtime applies that Knowledge to current work. Governance guides the entire system.

When an AI project becomes too large for one prompt, one conversation, or one person's memory, the problem is no longer simply how to write better instructions. The problem is deciding where evidence, retained knowledge, execution, standards, and human decisions should live.



The objective here is simple: understand the four responsibilities, how information moves among them, and where human authority remains active.

4.1 DCPF as an Operating System

The System Around the Prompt

An operating system creates an environment in which many activities can function together. DCPF plays a similar architectural role in sustained AI collaboration.

It is not software and does not require a particular technology platform. A small implementation may consist of a few coordinated documents and folders. Larger implementations may use repositories, databases, applications, or automated workflows. The tools can change while the responsibilities remain recognizable.

DCPF should also not be confused with retrieval-augmented generation (RAG), an agent framework, or another technical AI architecture. Those systems may retrieve information, orchestrate tools, or automate tasks. DCPF describes the human-directed organizational responsibilities for deciding how evidence is developed, what becomes Accepted Knowledge, how current work is executed, and how Governance applies. Technical systems may support those responsibilities, but DCPF does not depend upon them. Chapter 15 returns to those parallels.

Artifact names and identifiers also do not give an AI access to their contents. The artifact must actually be supplied, by attachment, pasted or embedded content, or retrieval from a source the AI can access.

Prompts Still Matter

Prompts remain essential. They simply have a more focused job.

A prompt may initiate a Research Request, execute a Runtime Request, or direct the AI toward supplied Knowledge and Governance requirements without carrying the project's entire history inside one interaction.

DCPF does not replace prompting. It gives prompting an architecture.

4.2 Reading the Core System Graphic

Figure 4.1 can be read as four coordinated responsibilities rather than four independent boxes.

Governance: The Operating Boundary

Governance surrounds the architecture because it applies throughout the work. It establishes human-defined standards, boundaries, acceptance criteria, validation expectations, and decision rights.

Governance does not begin only when someone creates a separate Governance document. In a small project, its controls may be embedded directly in Research, Knowledge, Runtime, and review artifacts. A maintained Governance artifact becomes useful when recurring controls need a stable project-wide home.

Research: Develop Evidence

Research begins with an information need. A structured Research Request tells the AI what must be investigated, which sources and boundaries apply, and how findings should be documented.

Its product is a Research Output, evidence for human review.

Research Output does not automatically become Knowledge. People decide what evidence is accepted for preservation and reuse.

Knowledge: Preserve What Has Been Accepted

The Knowledge Framework preserves approved project intelligence in a maintained Knowledge Repository.

Accepted Knowledge may include researched findings, terminology, procedures, standards, examples, project decisions, or other reusable information. Its status matters: the repository should distinguish what has been accepted from material that remains unresolved, outdated, rejected, or provisional.

Research therefore develops evidence. Human review changes the status of appropriate evidence. Knowledge preserves what the project is prepared to rely upon.

Runtime: Do the Current Work

Runtime defines what the system must do now.

A Runtime Request identifies the current objective, relevant Knowledge, applicable variables and Governance, the task to perform, and the required result.

The resulting Project Output may be a document, analysis, plan, presentation, media asset, software component, or another deliverable. It is an output of the system, not a fifth framework.

The division of responsibility is therefore:

Research discovers. Knowledge preserves. Runtime executes. Governance guides.

Information does not move automatically between these responsibilities. Human review and authorization remain part of the handoffs.

4.3 Why Move From Prompting to Collaboration Architecture?

Prompt-centered work can produce excellent individual responses. Sustained projects introduce different problems: information becomes buried in conversations, approved and unapproved material can blur together, standards must be repeated, and one prompt may be expected to carry too many responsibilities.

DCPF separates those responsibilities so they can be maintained deliberately.

Where Prompting Begins to Strain

Problem	DCPF responsibility	Architectural response
Important context is trapped inside conversations.	Knowledge + Runtime	Preserve approved information externally and supply only what the current task needs.
Research, instructions, and outputs become mixed together.	Research + Knowledge + Runtime	Give discovery, preservation, and execution separate responsibilities.
The AI is left to decide what is current or authoritative.	Research + Knowledge + Governance	People establish evidence requirements and decide what becomes Accepted Knowledge.
Work repeatedly recreates research, context, or instructions.	Research + Knowledge + Runtime	Reuse maintained evidence, Knowledge, and stable execution structures.
Quality, safety, source, or review standards are inconsistently applied.	Governance	Carry enduring standards across the architecture.
Large outputs become compressed or incomplete.	Research + Runtime	Use Sequential Output Requirements to protect complete delivery.
Problems are repeatedly fixed only in the latest prompt.	All four responsibilities	Trace the failure to Research, Knowledge, Runtime, or Governance and correct the responsible component.

Let the Architecture Carry the Filing Cabinet

DCPF externalizes organizational responsibilities that do not need to be recreated in every interaction. Project history, Accepted Knowledge, and enduring standards remain in maintained artifacts while the model concentrates on the current Research or Runtime objective.

That may reduce repeated discovery or unnecessary context, but DCPF does not promise lower token use or cost. Its primary benefit is better organization of the context and reasoning available for the work.

The goal is not to make AI work harder. It is to organize the collaboration so AI can work more effectively.

One remaining problem is completeness when a Research or Runtime output becomes too large for one reliable response. That is the purpose of Sequential Output Requirements.

4.4 Sequential Output Requirements and the Continuation Protocol

When One Reply Is Not Enough

Sequential Output Requirements (SOR) are DCPF-supplied continuation instructions used in the Research and Runtime Frameworks to protect the completeness of large or unpredictable outputs. One of their primary purposes is to discourage an AI model from compressing, summarizing, abbreviating, or omitting required work simply to fit the task into one response.

Instead, SOR allows the work to continue across multiple responses when necessary. The AI stops at a natural point, tells the user that additional output remains, requests permission to continue, and then resumes from where it stopped.

The rule travels with the Research or Runtime artifact that needs it. For that reason, its complete language is intentionally repeated in later templates and practicums rather than requiring either the reader or the AI to remember instructions from an earlier chapter.

The following is the standard DCPF Sequential Output Rule. It is supplied as part of the framework and is recommended for use as written unless the user deliberately adapts it for a specific project.

The Core Sequential Output Rule

“PURPOSE

When the requested Research Output or Project Output may exceed what can be comfortably delivered in a single response, or when its complete length cannot be reliably predicted, divide it into multiple sequential parts while preserving structure, completeness, detail, formatting, and continuity.

SEQUENTIAL OUTPUT RULE

- Apply this requirement automatically whenever the complete output may exceed practical response length or its length cannot be reliably predicted.
- Do not compress, summarize, abbreviate, or omit required work merely to fit within one response.
 - Favor completeness over minimizing the number of responses.
- Break the output at a natural point. Do not force the output into predetermined sets or fixed section groupings.
- At the end of each nonfinal part, state that additional output remains, request permission to continue, and stop.
- After permission is received, continue from where the previous part ended until the complete output has been delivered.
 - The final part must clearly state that the requested output is complete.
 - A short, predictable output may be delivered in one response.”

Let the Work Decide the Length

Users should not have to know where a particular model, interface, subscription level, or configuration may begin shortening a response. The SOR therefore activates when the requested work may be too large for one reliable response or when its total length cannot be predicted confidently.

Sequential delivery is a completeness safeguard, not a requirement to divide every output. A short, predictable task should simply be completed in one response.

When multiple responses are necessary, the structure of the work determines where the breaks occur. The AI continues from the previous stopping point rather than restarting, repeating completed sections, or replacing the requested detail with a summary. The continuation protocol also creates a visible human checkpoint: the AI must stop and receive permission before proceeding.

The objective is straightforward:

Let the work determine the necessary length. Do not let the size of one AI response determine the completeness of the work.

Once the work is distributed across Research, Knowledge, Runtime, and potentially several sequential responses, another question becomes important: who keeps all of those activities operating under the same standards?

That is the role of Governance.

4.5 Governance Across the Architecture

Governance influences the entire system because standards are most useful before problems occur.

One Responsibility, Multiple Working Locations

In Research, Governance may define approved sources, evidence priorities, prohibited practices, documentation requirements, and how uncertainty or source conflict should be handled.

In Knowledge, it helps determine what may be accepted, who can authorize it, how versions are identified, and how outdated, sensitive, or restricted material is handled.

In Runtime, it establishes the enduring boundaries within which the current task operates: quality requirements, disclosures, privacy or safety conditions, accessibility standards, approval points, or other output expectations.

Governance also supplies the criteria used to review results.

The amount of Governance should remain proportional to the work. A small, low-risk experiment may need only a few explicit controls and human review. More consequential work may require formal ownership, version control, validation, approvals, and domain-specific safeguards.

Governance should be strong enough to protect the work without creating ceremony that adds no value.

Chapter 5 examines this responsibility in depth.

4.6 Outputs, Feedback, and Continuous Refinement

Where Did the Problem Begin?

Research Outputs and Project Outputs reveal more than whether one task succeeded. They can expose weaknesses in the architecture.

A missing fact may point to Research.

Outdated information may point to Knowledge.

A poorly structured deliverable may point to Runtime.

A recurring weakness across workflows may point to Governance.

Instead of asking only, “How should we rewrite the prompt?” DCPF asks:

“Which responsibility allowed this problem to occur?”

Correct the immediate output when necessary. Improve the architecture when the lesson should benefit future work.

Useful feedback can also strengthen the system. New evidence may return to Research, accepted information may strengthen Knowledge, an effective procedure may improve Runtime, and a recurring standard may belong in Governance.

These changes should be deliberate.

Preserve what works. Revise when evidence gives you a reason.

4.7 One Architecture at Different Scales

Minimum Viable DCPF

Minimum Viable DCPF does not mean removing one of the four responsibilities. It means using the smallest artifact structure that keeps Research, Accepted Knowledge, Runtime, Governance, and human review visible enough for the project.

For a first experiment, this might consist of:

- one Research Request with essential Governance controls embedded;
- a human-reviewed Research Output;
- one Knowledge Repository containing the Accepted Knowledge;
- one Runtime Request with applicable requirements embedded; and
- one reviewed Project Output.

A separate Governance document is not required simply to say that all four responsibilities exist.

Recurring controls can be consolidated later when a maintained Governance artifact improves reuse or oversight.

Larger projects may add multiple Research collections, versioned Knowledge repositories, specialized Runtime workflows, contributors, automation, review records, or formal Governance.

Do Not Build City Hall for a Lemonade Stand

Scale should follow need.

A small experimental project does not require enterprise infrastructure. A larger or more consequential project should not depend on informal memory when continuity, quality, or accountability matter.

Stable architecture. Flexible implementation.

Because DCPF is experimental, the best starting point is a controlled, low-risk sandbox in a domain you understand. Test the workflow, inspect the handoffs, identify unnecessary structure, and refine your implementation before considering real-world or production use.

Additional structure should be added when the work develops a real responsibility that needs a clearer home.

4.8 Reader Checkpoint

Five Questions Before We Enter the System

Is DCPF a software platform? No. DCPF is a human-directed collaboration architecture that can be implemented using many different tools.

Does DCPF replace prompt engineering? No. Prompting remains essential. DCPF provides persistent architectural support around individual interactions.

Does a Research Output automatically become Knowledge? No. Research produces evidence. Human review and acceptance determine what becomes Accepted Knowledge.

Must I create a separate Governance document before beginning? No. Governance must be active, but a small project may embed applicable controls directly in its working artifacts.

What responsibility does each framework own? Research discovers. Knowledge preserves. Runtime executes. Governance guides.

If those five answers are clear, you understand the foundation needed for the next four chapters.

4.9 Entering the Frameworks

The Map Is Drawn

Prompting remains essential, but it no longer carries the full burden of Research, Knowledge, Runtime, and Governance.

Prompt engineering optimizes individual interactions.

DCPF organizes collections of interactions into persistent architectural systems.

The next four chapters step inside the architecture: Governance first, followed by Research, Knowledge, and Runtime. Chapter 9 then prepares you to test the complete system before the practicums begin.

[Return to Table of Contents](#)

The Governance Framework

DCPF is an experimental approach that has demonstrated promise but has not been broadly validated. Test and validate your implementation within your own environment before considering real-world or production use.

This chapter explains how DCPF turns human expectations into visible, reusable governance standards that guide Research, Knowledge, Runtime, and project outputs. It introduces the seven standard Governance Framework responsibilities, shows why each one is necessary, explains how Governance elements become operational inside working artifacts, identifies what may be combined or supplemented in a smaller implementation, and demonstrates consolidation in the National Park Narration project.

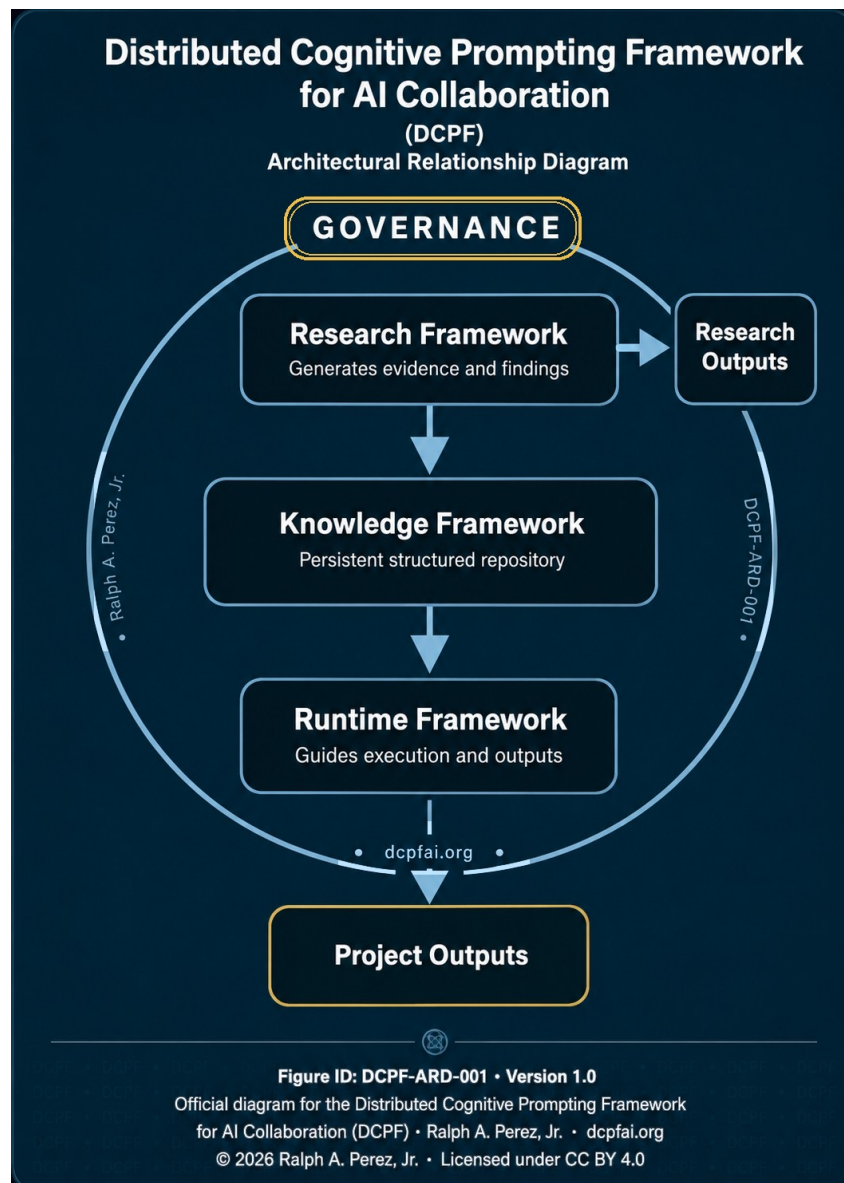


Figure 5.1. Governance surrounds Research, Knowledge, and Runtime because its human-defined standards apply across the DCPF architecture, from evidence gathering through the evaluation of Project Outputs.

5.1 What Governance Means in DCPF

The Rules Were Already There

Every AI project already has rules, even when nobody has written them down. Certain sources feel trustworthy. Certain outputs feel unacceptable. Someone expects accuracy, consistency, and a final human decision.

The trouble begins when those expectations live only in someone's memory and the AI model is expected to remember everything or infer the missing rules on its own. A rule may appear in one request and vanish from the next, while the AI is left to guess which standards matter. Governance makes those enduring expectations visible and reusable.

In DCPF, Governance is the set of human-defined standards and operating rules that guide how the system works. Governance attributes are carried into the system's collaboration artifacts and requests, where they define which sources may be used, what information may be preserved, which quality and safety requirements apply, what standards outputs must satisfy, and when human review is required.

People establish these standards. AI may help draft, apply, and check them, but it does not decide the governing rules, approve exceptions, or accept consequential work on its own. This preserves the human-directed nature of DCPF: **you determine how the system governs its work; AI does not.**

Governance is not only a final review step. Its standards are carried into Research Requests, Knowledge Repository entries, and Runtime Requests through specific attributes inside the documents AI uses.

Governance does not begin when a Governance document is created. It begins when the project makes standards, boundaries, acceptance criteria, validation requirements, and human decision rights explicit.

For a beginner, those decisions may first be discovered while designing a Research Request, deciding what research would be acceptable, defining Knowledge acceptance or repository controls, specifying Runtime boundaries, or deciding how an output will be reviewed. Those are operational Governance elements even before they are consolidated into a separate maintained artifact.

Before looking at the examples, it helps to know the seven standard responsibilities that drive Governance in DCPF:

- Framework Purpose
- Quality Standards
- Approved Information Sources
- Prohibited Content

- Validation Requirements
- Output Standards
- Revision History

Together, these seven responsibilities form the Governance skeleton. They are introduced here only to orient you to the examples that follow; each is explained later in this chapter.

5.2 Seeing Governance Applied Across the Frameworks

The following simplified documents show how governance standards become operational inside the other frameworks. They are short examples rather than complete working artifacts.

Light-blue sections labeled GOVERNANCE ATTRIBUTE identify the governance provisions applied inside each document.

SIMPLIFIED RESEARCH REQUEST Framework: Research Framework Artifact: Research Request for AI Framework ID: NPN-RES-S01 Project: National Park Narration Version: 1.0 Status: Simplified example Research Objective Identify reliable facts about the geological formation of Yellowstone's caldera. Approved Information Sources (GOVERNANCE ATTRIBUTE) National Park Service; U.S. Geological Survey; official government or university geology resources. Evidence and Validation Requirements (GOVERNANCE ATTRIBUTE) Record the source for each material claim. Identify uncertainty or source conflict. Human review is required before findings enter the Knowledge Repository. Expected Research Output A concise findings summary with source references.

Governance in Research defines the evidence boundary, documentation standard, and approval requirement.

SIMPLIFIED KNOWLEDGE REPOSITORY ENTRY

Architectural Responsibility: Knowledge Framework

Artifact: Knowledge Repository Entry (approved information selected from Research Outputs)

Repository Entry ID: NPN-KNO-S01

Project: National Park Narration

Version: 1.0

Status: Approved example

Entry Topic

Yellowstone Caldera: Formation and Activity

Accepted Knowledge

Yellowstone's caldera formed through major volcanic events and remains part of an active volcanic system.

Source Basis (GOVERNANCE ATTRIBUTE)

National Park Service and U.S. Geological Survey geology materials.

Acceptance and Maintenance (GOVERNANCE ATTRIBUTE)

Approved by the Project Editor. Version 1.0. Review when authoritative sources materially change.

Approved Use

May support educational narrations and related Runtime requests.

Governance in the Knowledge Repository records source basis, acceptance status, ownership, and maintenance expectations.

SIMPLIFIED RUNTIME REQUEST

Framework: Runtime Framework

Artifact: Runtime Request for AI

Framework ID: NPN-RUN-S01

Project: National Park Narration

Version: 1.0

Status: Simplified example

Current Objective

Create a 150-word, listener-friendly explanation of Yellowstone's caldera.

Knowledge Input

Attach the approved Yellowstone Caldera repository entry.

Output Standards (GOVERNANCE ATTRIBUTE)

Use clear, calm, accessible language. Explain technical terms and do not overstate uncertainty.

Boundaries and Review (GOVERNANCE ATTRIBUTE)

Do not add unsupported external facts. Human editorial review is required before publication.

Expected Deliverable

One finished narration passage.

Governance in Runtime applies recurring quality standards, content boundaries, and approval points to the current task.

5.3 What These Examples Show

Governance may have its own maintained framework artifact, but that artifact is not what makes Governance exist. Governance becomes operational when its relevant elements are carried into the documents and decisions that actually control the work. The Research Request carries source and evidence rules. Knowledge acceptance and the Knowledge Repository carry provenance, approval, ownership, and maintenance controls. The Runtime Request carries output standards, boundaries, and review points.

A small or early-stage project can begin with those controls embedded where the work occurs. When the same controls recur across artifacts or executions, consolidating them into a maintained Governance artifact gives the project a stable source for reuse, versioning, and change control.

A mature organization may start from the opposite direction. Existing policies, standards, approval authorities, source restrictions, accessibility requirements, or risk controls can be translated into the Governance elements that belong inside Research, Knowledge, Runtime, and review artifacts.

The examples also make responsibility visible. People define and approve the standards. AI can follow them, check compliance, and flag possible exceptions, but it does not authorize the standards or approve its own departures.

Governance may have a maintained home, but it works throughout the system.

5.4 Governance in the DCPF Architecture

In the DCPF relationship diagram, Governance surrounds Research, Knowledge, and Runtime because its standards apply across the full workflow. It influences what evidence may be gathered, what information may enter the repository, how current tasks are executed, and whether outputs are accepted.

The surrounding position also represents human authority. The user, project owner, organization, or another responsible human authority establishes or adopts the standards. AI operates within the conditions it has been given.

The framework's central responsibility is simple: make the system's enduring expectations visible, reusable, and maintainable.

5.5 The Governance Framework Skeleton

The standard Governance skeleton contains seven responsibilities. Together, they define the framework's purpose, standards, evidence boundaries, prohibitions, validation, recurring output expectations, and controlled change. These seven responsibilities are the DCPF standard; project-specific additions may supplement them when a real need exists. The table names the standard responsibilities, and the repeated training structure below explains why each one belongs.

For a beginner, the skeleton is also a vocabulary for questions you may not yet know how to ask: What counts as good? Which sources can I trust? What must the AI not do? What must be checked before

work is accepted? What should outputs consistently look like? Who can approve a change or exception? You can answer those questions where they first arise and consolidate recurring answers later.

Standard responsibility	Architectural function	Primary authority
Framework Purpose	Defines the quality, ethical, and operational responsibilities governed by the framework.	User or responsible organization
Quality Standards	Establishes what satisfactory work means.	User, organization, or applicable domain authority
Approved Information Sources	Defines the evidence and source types permitted by the system.	User or responsible governance authority
Prohibited Content	Defines information, sources, practices, or outputs that are not allowed.	Human authority
Validation Requirements	Establishes the checks and decisions used before work is accepted.	User or responsible reviewer
Output Standards	Defines recurring organization, formatting, citation, accessibility, and presentation requirements.	User or organization
Revision History	Records changes to governance policies and their authorization.	Framework owner or maintainer

What are the standard responsibilities I should expect to see?

The standard DCPF Governance responsibilities are these seven. Each protects a distinct part of human control across the architecture.

Framework Purpose: Defines what Governance controls, why it exists, and where its authority applies. It is necessary so rules have a clear scope and the AI is not left to guess which standards govern which work.

Quality Standards: Defines what satisfactory work means in observable terms. It is necessary because fluency and polish are not reliable substitutes for an explicit definition of quality.

Approved Information Sources: Defines which evidence channels, source types, repositories, or authorities the system is permitted to use. It is necessary so Research does not treat convenient or inaccessible information as acceptable evidence by default.

Prohibited Content: Defines sources, information, practices, behaviors, or outputs that are not allowed. It is necessary because some boundaries must be stated explicitly rather than inferred from positive instructions.

Validation Requirements: Defines what must be checked, who or what performs the check, and what decisions follow. It is necessary so standards lead to a visible acceptance, revision, rejection, escalation, or return decision instead of becoming unenforced aspirations.

Output Standards: Defines recurring requirements that comparable deliverables should satisfy. It is necessary so important expectations such as organization, citations, accessibility, presentation, and completion do not have to be reinvented in every Runtime Request.

Revision History: Records what Governance changed, why it changed, when it became effective, and who authorized the change. It is necessary to prevent silent policy drift and to preserve which governing state applied to earlier work.

When a Governance Responsibility Is Left Undefined

Each Governance responsibility protects a different part of human authority over the collaboration. Defining quality, approved sources, prohibitions, validation, output standards, and controlled change does not mean that humans must perform every check themselves. It means that humans establish the conditions under which AI is allowed to operate and the standards against which its work will be judged.

When one of those responsibilities is left undefined, the responsibility does not always disappear. It may instead shift to ad hoc human judgment, AI inference, platform defaults, or no clearly identified decision-maker at all. That can increase the likelihood of familiar AI-collaboration problems such as automation bias, false confidence, inconsistent standards, governance drift, or undetected unsupported output.

Appendix G, DCPF Responsibility-Consequence Map, examines each standard Governance responsibility individually and shows what control is lost, who may effectively inherit the decision, and which AI or collaboration risks may follow.

What can I combine, simplify, or omit in a smaller implementation?

You may combine standard responsibilities into fewer headings or sections when each responsibility remains explicit, identifiable, and usable. A standard responsibility is an architectural expectation; it is not a requirement for a separate heading, page, or standalone artifact.

In a small project, Approved Information Sources and Prohibited Content may share an Information Boundaries section; Quality Standards, Validation Requirements, and Output Standards may share an Acceptance Standards section; and Revision History may be a dated line or short change record. A standalone Governance artifact may also be deferred while the applicable controls remain explicit in working artifacts. Supplemental privacy, risk, regulatory, audit, or exception sections may be omitted until the project actually needs them.

These are standard Governance responsibilities, not mandatory document headings and not a prerequisite for a standalone Governance document before the first Research Request. The responsibilities must be represented where they apply, but several may share a section or may initially be embedded inside Research, Knowledge, Runtime, and review artifacts.

Combine standard Governance responsibilities when appropriate, but do not make a standard responsibility disappear merely to shorten the artifact. Specialized sections for privacy, risk, approvals, regulatory obligations, exceptions, or other domain needs are supplemental unless the project makes them necessary.

5.6 Understanding the Skeletal Elements

Each element has a specific operational purpose. In a small project, the wording can remain brief as long as the responsibility, human authority, and method of application remain clear.

Framework Purpose: Where Does Governance Apply?

Framework Purpose defines what Governance covers and where its authority applies. It may govern research reliability, knowledge acceptance, runtime behavior, output quality, or an entire group of projects. Without a clear purpose, important requirements can become mixed with minor preferences and their scope may be unclear.

The user or responsible organization establishes the purpose. AI may help draft the statement or identify gaps, but it cannot grant the framework authority. Keep the purpose visible, even when it is only a few sentences.

Quality Standards: Who Decides What “Good” Means?

Quality Standards define what satisfactory work means in observable terms. Accuracy may require verification against approved sources. Completeness may require every requested section. Consistency may require uniform terminology, measurements, and formatting. Without these standards, evaluation becomes subjective.

People decide which qualities matter and how much risk is acceptable. AI may translate those expectations into repeatable checks and compare outputs against them. In a compact framework, Quality Standards may be combined with Validation Requirements, but the expected condition and the method of checking it should remain distinct.

Approved Information Sources: The Evidence Gate

Approved Information Sources define the evidence boundary of the system. They identify the source types, repositories, organizations, databases, or other evidence channels Research is permitted to use, along with recurring conditions such as recency, jurisdiction, access, or required corroboration. This Governance element answers a permission question: May Research use this source or evidence channel? It does not by itself determine which permitted source is strongest for every claim.

People decide which sources are acceptable. AI may search within those limits, document what it used, compare conflicts, and flag material outside policy. A source may be permitted yet still be a weak fit for a particular claim; Research applies Source Priority and Conflict Handling to decide how permitted evidence should be weighed in context. Approved and prohibited sources may be combined under Information and Source Boundaries when both permissions and restrictions remain explicit.

Prohibited Content: Some Rules Need to Say No

Prohibited Content defines which information the system must not gather or preserve and which actions or outputs it must not perform or produce. Depending on the project, this may include confidential

material, unsupported claims, unsafe guidance, restricted intellectual property, or activities outside the project's authorized purpose. Explicit negative boundaries reduce ambiguity where mistakes could carry meaningful consequences.

People authorize prohibitions and any exceptions. AI may flag a possible violation, explain the applicable rule, and pause for review, but it must not create its own exception. This element may be combined with source boundaries as long as exceptions and approval authority remain visible.

Validation Requirements: A Check Needs a Consequence

Validation Requirements define how work is checked before it proceeds. Research may be checked for source quality, coverage, conflicts, and documentation. Knowledge may be checked for provenance, approval status, duplication, and currency. Runtime outputs may be checked against the task, required sections, factual support, and presentation standards. Validation should also state what follows: acceptance, revision, rejection, escalation, or return to another framework.

AI is well suited to repeatable preflight checks, such as locating missing citations or comparing an output with a required structure. Human judgment remains necessary for uncertainty, exceptions, competing values, and consequential risk. In a compact framework, Validation may be combined with Quality Standards when the check, reviewer, and possible disposition remain clear.

Output Standards: Keep the Finish Line Visible

Output Standards preserve recurring expectations for deliverables, including organization, tone, citations, accessibility, file type, naming, and completion requirements. Runtime defines what must be produced now; Governance preserves the standards that should remain consistent across comparable outputs. Without them, requirements must be rewritten and consistency can drift.

AI may apply formatting, compare a draft with the standard, and report deviations. People define the audience, public obligations, and final definition of done. A small project may combine Output Standards with Quality Standards when the recurring deliverable requirements remain identifiable.

Revision History: Rules Should Not Change Overnight

Revision History identifies the current governance state by recording what changed, why it changed, when it took effect, and who authorized it. This prevents silent policy drift and helps explain why work created at different times may follow different standards.

The framework owner or maintainer records revisions under human authority. AI may compare versions, prepare summaries, and draft change-log entries, but it does not authorize policy changes. In a small project, Revision History may remain a dated line in document metadata or a brief change log.

5.7 How Governance Collaborates with AI

Governance collaborates with AI in three different moments: design, operation, and evaluation.

During design, people describe objectives, boundaries, risks, and existing obligations. For a beginner, that design may start inside the Research or Runtime artifact itself: choosing acceptable sources, defining what useful or reliable work should look like, naming prohibited actions, and deciding where human approval is required. AI can help organize those decisions, identify vague terms, suggest measurable criteria, and point out apparent gaps or conflicts, but human review and approval remain controlling.

An organization with established governance can begin from the other direction by identifying which existing policies apply, translating them into artifact-level controls, and maintaining the authoritative source and version from which those controls are drawn.

A Rule the AI Cannot See Is Not a Rule It Can Follow

During operation, the applicable requirements must be explicit in the Research, Knowledge, Runtime, or review artifact where they operate. A maintained Governance artifact may preserve recurring project-wide standards, but downstream work does not depend on attaching, supplying, pasting, embedding, or retrieving that artifact. The working artifact itself carries the controls the AI must follow.

Checking Is Not Authorizing

During evaluation, AI can compare work against explicit requirements and produce a validation report. It can find omissions, identify possible boundary violations, and distinguish a failed check from a completed check. That assistance makes review more consistent, but it does not transfer accountability. A person or authorized process accepts consequential work and decides how exceptions will be handled.

Governance may also be revised in response to system performance. AI can summarize recurring failures and propose a candidate improvement. Humans determine whether the pattern justifies changing a shared rule. If approved, the change enters Revision History and becomes part of the next operating cycle.

AI can help operationalize Governance. Human participants authorize it.

5.8 Governance Across the Architecture

The seven elements become useful when they influence the rest of the system.

In Research, Governance defines the evidence environment. Approved sources, prohibited practices, quality standards, and validation requirements shape the investigation before information is gathered. They also establish what must be documented before research output can be considered for Knowledge.

In Knowledge, Governance protects the repository's authority. It can require source references, approval status, dates, ownership, version labels, or the removal of obsolete and sensitive material. These conditions help distinguish maintained project intelligence from information that was merely collected.

In Runtime, Governance supplies the enduring boundaries around the present task. Runtime establishes the immediate objective and deliverable. Governance contributes the quality thresholds, prohibited

behavior, required disclosures, output standards, and approval points that remain in force across executions.

For Project Outputs, Governance supplies a basis for decision. The result can be accepted, revised, rejected, escalated, or routed back to another framework. If the same weakness recurs, feedback may justify a governance revision rather than another isolated prompt correction.

Governance therefore influences both work and handoffs. It helps determine whether evidence may become Knowledge, whether Knowledge is suitable for Runtime, and whether a Runtime output is ready for use.

5.9 A Proportionate Governance Framework

Start with the Controls You Can Name

The seven standard Governance responsibilities do not require a standalone Governance artifact or seven separate headings before a beginner can start. Begin with the controls you can name for the work in front of you and represent the applicable standard responsibilities inside the Research Request, Knowledge acceptance or repository process, Runtime Request, or review step where they operate. A separate Governance artifact can consolidate recurring controls later.

If you do not yet know what “quality” means for the project, begin with observable conditions: What would make the Research useful enough to accept? What would make a Runtime output accurate, complete, safe, or usable enough for its intended purpose? Testing will reveal which expectations deserve to become recurring standards.

As the project is tested, recurring rules become easier to recognize. When a source policy, quality standard, approval rule, prohibition, or validation requirement should persist across several artifacts or executions, consolidate it into a maintained Governance artifact. A medium or larger project may then add risk classifications, privacy requirements, approval authorities, exception procedures, audit records, or domain-specific obligations because the work genuinely requires them.

The Governance responsibility is present throughout this progression. The maintained artifact is a stable representation of recurring Governance; it does not create the responsibility retroactively. The right implementation is the smallest one that makes the project's important standards clear, usable, and maintainable.

Stable responsibility. Proportionate implementation.

5.10 Governance in Action: The National Park Narration

Chapter 3 traced how the National Park Narration project helped reveal the need for DCPF. By this stage, the project has identified recurring source, quality, acceptance, safety, output, and review controls. The following example shows how those recurring Governance elements can be consolidated into a compact maintained artifact whose applicable provisions are then carried into the project's Research, Knowledge, and Runtime artifacts.

LIVE FRAMEWORK DOCUMENT

Framework: Governance Framework

Framework ID: NPN-GOV-001

Project: National Park Narration

Owner: Project Editor

Version: 1.0

Status: Active

Applies to: Research, Knowledge, Runtime, and public project outputs

Inputs: Project objectives, editorial requirements, source policy, and human risk decisions

Output: Reusable standards and constraints for the complete DCPF implementation

Framework Purpose

This framework governs the development of accurate, accessible, and responsible national park narrations. It establishes the evidence, knowledge-acceptance, execution, and output conditions that apply throughout the project. AI may assist with drafting, checking, and reporting, but the Project Editor retains authority for exceptions, governance revisions, and publication approval.

Quality Standards

Factual claims must be supported by approved information sources and must not be presented with greater certainty than the evidence allows. Explanations must be understandable to a general adult audience without distorting their meaning. Terminology, measurements, place names, and descriptions must remain consistent across the project. Required framework sections and deliverable components must be complete before acceptance.

Approved Information Sources

Permitted sources include original project documents; official National Park Service publications and park materials; United States Geological Survey resources and other relevant government agencies; recognized educational institutions, professional organizations, and standards bodies; and authoritative reference works appropriate to the topic. General public sources may be used to discover leads but may not serve as the sole support for a material factual claim unless the Project Editor approves an exception.

Prohibited Content

The system must not introduce invented historical or scientific claims, conceal unresolved source conflicts, present inference as established fact, reproduce restricted or confidential material without authorization, or generate unsafe visitor guidance. Sensitive cultural, ecological, or location information must not be included when disclosure could create harm or violate an applicable source or park restriction. AI may flag a potential exception but may not authorize one.

Validation Requirements

Research Outputs must be checked for objective coverage, source compliance, documentation, conflicts, and visible uncertainty. Information becomes Accepted Knowledge only after its source basis and acceptance status have been reviewed and the applicable human acceptance decision has been made. Accepted Knowledge may then be preserved in the Knowledge Repository. Runtime outputs must receive an AI-assisted preflight check for factual support, required sections, terminology, and output standards, followed by human editorial review before public use. Failed checks result in revision, additional research, exclusion, or escalation to the Project Editor.

LIVE FRAMEWORK DOCUMENT

Output Standards

Public narrations must be calm, clear, and approachable. Explain technical concepts in ordinary language before using specialized terms. Distinguish established evidence from uncertainty, avoid promotional exaggeration, and follow the approved structure and production format. Internal validation notes and source documentation must remain available even when omitted from the public output.

Revision History

Version 1.0: Effective July 2026. Initial active Governance Framework for the National Park Narration project. Approved by the Project Editor. Future entries must identify the version, effective date, change, reason, and approving authority. The latest approved version supersedes earlier versions unless a project record explicitly states otherwise.

The document is concise, but every skeletal responsibility is present. This is what consolidation looks like after recurring controls have become clear. Its identifier and version preserve the project's governing state, while the applicable controls are stated directly in the Research, Knowledge, and Runtime artifacts that use them. Chapter 6 will demonstrate that approach inside the live Research Framework.

5.11 Reader Checkpoint

Question: Is Governance the final review step? No. Final review may be one governance activity, but the framework begins influencing the system before Research and remains active through Knowledge, Runtime, and output evaluation.

Question: Can AI create Governance for the user? AI can help draft standards, reveal gaps, and apply repeatable checks. Human participants remain responsible for objectives, authority, acceptable risk, exceptions, and consequential acceptance decisions.

Question: Must all seven elements appear as separate headings? No. Sections may be consolidated when the implementation remains clear. The seven responsibilities should still be identifiable and available to the system.

Question: Can one Governance Framework support several projects? Yes. Shared source, privacy, accessibility, quality, or review standards can govern several projects. Each project can add a smaller layer for requirements unique to its domain or output.

Question: Must I create a standalone Governance artifact before beginning Research? No. Governance must be active, but a small project can begin by embedding the applicable source, quality, boundary, acceptance, validation, and human-review controls in the working artifacts that need them. Consolidate recurring controls into a maintained Governance artifact when that adds value.

5.12 From Governance to Research

For the National Park example, the operating environment is now represented in a maintained Governance artifact. The project knows which evidence it may use, which boundaries apply, what satisfactory work means, and how information and outputs will be evaluated. In a smaller beginner project, those same operating conditions may initially be carried directly in the working artifacts rather than consolidated into a separate document.

Governance does not answer the project's questions. It creates the conditions under which those answers can be developed responsibly.

Chapter 6 moves into the Research Framework, where an information need becomes a deliberate investigation and documented evidence begins its journey through the DCPF architecture.

[Return to Table of Contents](#)

The Research Framework

DCPF is an experimental approach that has demonstrated promise but has not been broadly validated. Test and validate your implementation within your own environment before considering real-world or production use.

This chapter defines the Research Framework as DCPF's controlled entry point for new evidence. It explains the nine standard Research responsibilities that make investigation bounded, traceable, complete, and reusable; shows why each one is necessary; clarifies how AI can assist without assuming human authority; and demonstrates a complete Research Request that produces a documented Research Output prepared for review and possible transfer into Knowledge.

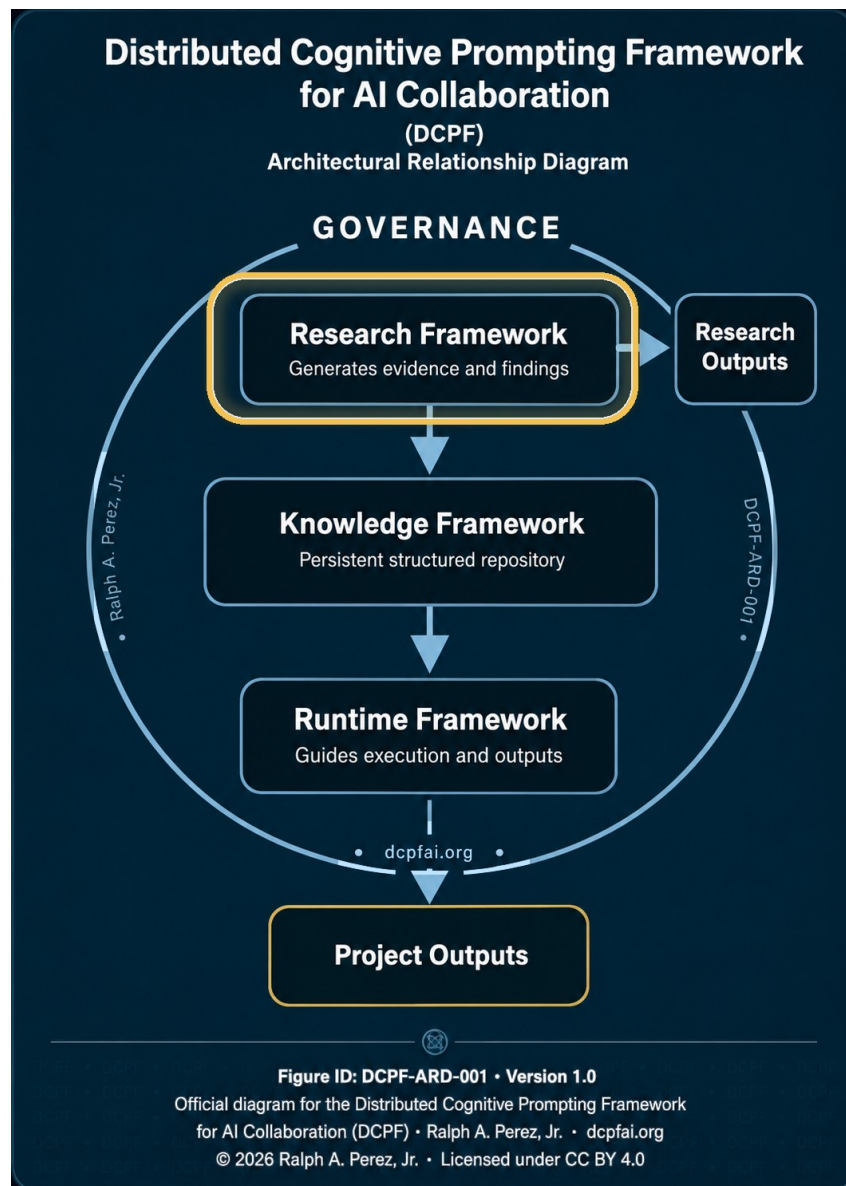


Figure 6.1. Research begins the DCPF operational flow by gathering governed evidence, producing documented Research Outputs for review, and passing evidence toward Knowledge for possible acceptance and later use.

6.1 Research in the DCPF Architecture

What has to happen before an AI-generated fact earns a place in a system designed for reuse? In DCPF, the answer begins with Research. Research occupies the first position in the operational flow because every later stage will inherit the quality, limitations, and source basis of the evidence developed here.

The user defines the information need by supplying the purpose, current variables, objectives, and scope. Governance establishes the operating conditions: approved source boundaries, prohibited practices, quality standards, validation requirements, and other constraints. Within those boundaries, AI investigates approved sources, organizes the findings, and records how each finding was supported. The architectural result is a Research Output.

Evidence Is Not Knowledge, Yet

That output is important, but it is not automatically trusted project knowledge. It must pass the applicable human review and acceptance process, which may be a quick read-through for a small project or a formal review by a designated team. Accepted findings may enter the Knowledge Repository; uncertain findings may remain in Research, be marked for further investigation, or be excluded. The handoff changes information from evidence gathered for consideration into maintained intelligence the project is prepared to use.

Research therefore owns discovery and documentation. It does not own permanent acceptance, and it does not produce the final project deliverable. That separation is the first safeguard against turning a confident answer into an unquestioned fact.

6.2 Research as the Controlled Entry Point

Every knowledge-intensive project needs a controlled way for new information to enter. Without one, information tends to arrive through improvised searches, isolated prompts, remembered conversations, and assumptions that nobody recorded. Some of it may be useful. The problem is that the project may no longer be able to explain why it was gathered, which question it answers, where it came from, or whether it is complete enough to support future work.

The Research Framework gives that incoming information a front door. It tells the AI what the investigation is for, which questions must be addressed, what evidence may be used, where the boundaries lie, and how findings must be documented and delivered. The result is not merely an answer; it is a structured evidence product prepared for review, possible acceptance as Knowledge, and preservation in the Knowledge Repository.

Fluency Is Not Proof

This controlled entry point prevents the system from sprinting from question to polished deliverable before the evidence has caught up. Fluent language can make incomplete coverage, weak sources,

unresolved conflicts, or unsupported inference look surprisingly respectable. A well-designed Research Request requires the AI to expose those conditions before the project decides what deserves to be reused.

Build Research for the Next Handoff

The framework also looks ahead. If Knowledge will organize information by topic, source, and acceptance status, Research can gather those fields from the beginning. If Runtime will later need particular facts, comparisons, or definitions, the Research Objectives can require them now. The investigation stops being a disposable conversation and becomes the first deliberate stage of an architecture.

Research produces evidence for consideration. It does not grant that evidence permanent authority.

The rest of this chapter examines the nine standard responsibilities of the Research Framework, why each decision matters, how AI can assist, and what tends to go wrong when the boundaries remain implicit. It concludes with a complete working Research document that places its source, boundary, validation, and human-authority requirements inside the responsibilities where they operate. The goal is not to memorize nine headings; it is to understand the nine decisions that make research controlled, traceable, and reusable.

6.3 The Research Framework Skeleton

The standard Research skeleton contains nine responsibilities. Together, they answer the essential questions behind a disciplined investigation: Why are we researching? What changes in this execution? What must be learned? Which evidence may be used and how should it be weighed? How far should the work extend? How will the result be documented, delivered completely, and governed? These nine responsibilities are the DCPF standard; domain-specific methods may supplement them when needed.

Standard responsibility	Architectural function	Primary source
Framework Purpose	Defines the investigation and its intended downstream use.	User
Research Configuration Variables	Customize the current research execution without changing the framework.	User
Research Objectives	Define the questions and required topics.	User
Approved Information Sources	Establish where evidence may be gathered.	Governance and/or user
Source Priority Hierarchy	Guides claim-specific weighting and conflict handling among approved sources.	User within governance boundaries
Research Scope	Defines inclusions, exclusions, depth, and recency.	User

Standard responsibility	Architectural function	Primary source
Output Organization	Structures the completed research product.	User
Research Source Documentation	Preserves the evidence trail and relevant content origins.	Research design and/or Governance
Sequential Output Requirements	Protects complete delivery through sequential parts and continuation stops when needed.	User

What are the standard responsibilities I should expect to see?

The standard DCPF Research responsibilities are these nine. Each protects a different decision that must be visible before or during a controlled investigation.

Framework Purpose: Defines the investigation and its intended downstream use. It is necessary so Research develops evidence for a known purpose without drifting into the final Runtime deliverable.

Research Configuration Variables: Assigns the values that change for the current investigation while the research method remains stable. It is necessary to make the current subject, location, time period, audience, depth, or other variables visible and to prevent values from a previous execution from carrying forward unnoticed.

Research Objectives: Defines the questions, comparisons, and required topics the investigation must address. It is necessary because the project cannot judge completeness unless it has stated what the Research was expected to discover.

Approved Information Sources: Defines which evidence channels Research is permitted to use. It is necessary to establish the evidence boundary before convenience, search ranking, or model preference determines what counts as support.

Source Priority Hierarchy: Guides claim-specific weighting and conflict handling among approved sources. It is necessary because permitted sources can differ in authority, directness, currentness, jurisdiction, method, relevance, and corroboration; permission alone does not establish which evidence best supports a particular claim.

Research Scope: Defines inclusions, exclusions, depth, recency, and other limits on the investigation. It is necessary to distinguish an intentional boundary from an accidental omission and to keep Research from expanding into adjacent questions without authorization.

Output Organization: Defines how the Research Output will be structured. It is necessary so findings remain reviewable, comparable to the objectives, and prepared for possible preservation as Accepted Knowledge instead of arriving as an unstructured answer.

Research Source Documentation: Preserves the evidence trail and relevant content origins. It is necessary so a future reviewer can verify, update, challenge, or correct a finding without repeating the entire investigation or guessing where the evidence came from.

Sequential Output Requirements: Protects complete delivery when the Research Output may exceed practical response length or its length cannot be predicted reliably. It is necessary so the model does not compress, summarize, abbreviate, or omit required Research merely to fit one response.

When a Research Responsibility Is Left Undefined

The Research Framework does more than tell AI to find information. Its standard responsibilities preserve human decisions about what should be investigated, where evidence may come from, how competing evidence should be weighed, how far the investigation should extend, and how its findings should be documented.

If one of those decisions is left undefined, AI may have to infer it in order to continue the investigation. The result may still look complete and persuasive, but greater discretion has shifted from the human collaborator to the model or its information environment. Depending on the missing responsibility, the consequences can include scope drift, source-selection bias, unsupported inference, silent omission, weak provenance, false confidence, or increased hallucination risk.

Appendix G, DCPF Responsibility-Consequence Map, provides the complete Research analysis, showing what happens when each of the nine standard Research responsibilities is intentionally left undefined.

What can I combine, simplify, or omit in a smaller implementation?

You may combine standard responsibilities into fewer headings or sections when each responsibility remains explicit, identifiable, and usable. A standard responsibility is not optional simply because its wording can be compact.

A small Research Request may combine Framework Purpose and Research Configuration Variables in an identity block; Research Objectives and Research Scope as a Research Assignment; Approved Information Sources and Source Priority Hierarchy as an Evidence Policy; and Output Organization, Research Source Documentation, and Sequential Output Requirements as a Delivery Specification. Applicable Requirements may appear in the header or as embedded controls. Sequential Output Requirements may remain dormant when the output is short and predictable, but the safeguard remains part of the standard Research structure. Domain-specific methods, confidence scales, specialist review fields, or other added controls are supplemental unless the project needs them.

These are standard Research responsibilities, not a mandatory page design. Closely related responsibilities may share a section, and specialized projects may add domain-specific controls. DCPF is not asking every project to construct nine small monuments to paperwork. It is asking the project to preserve the decisions that keep the investigation clear, controlled, traceable, complete, and ready for handoff.

Four Evidence-Control Questions

Several Research and Governance controls can sound similar until they are framed as separate questions. Approved Information Sources asks, May Research use this source or evidence channel? A Governance prohibition asks, Is this source, information, or practice forbidden even if the broader category is

otherwise permitted? Source Priority asks, Among the permitted evidence, which source or combination best supports this particular claim? Research Scope asks, Is this question or information part of the investigation we authorized?

These controls may be consolidated into one Evidence Policy section in a small project, but their decisions should remain distinguishable. Permission does not establish truth, priority does not expand scope, and a high-priority source does not erase a credible unresolved conflict.

Why Not Combine Research and Runtime?

Research and Runtime answer different questions. A Research Request asks, "What evidence can the project support?" A Runtime Request asks, "What should the project create from approved knowledge now?" Combining them can send unreviewed findings directly into a polished deliverable, blur the line between evidence and execution, and leave the research difficult to preserve or reuse. The result may look finished while the foundation is still wet cement.

For a small, low-risk task, Research and Runtime may occur in the same conversation, but they should remain explicit phases. Complete the Research Request, review and accept the relevant findings, preserve them in the Knowledge Repository, transfer the needed Knowledge Repository findings into the Runtime Request, and then execute it. The handoff can be lightweight; the responsibilities should not disappear. With that boundary clear, the nine skeletal elements become much easier to understand.

6.4 Understanding the Skeletal Elements

Framework Purpose

The Framework Purpose tells the investigation what it is trying to accomplish and what knowledge it is intended to develop. It should name the subject, explain why the research is needed, and state that the completed evidence is intended for review and possible transfer into Knowledge. Just as importantly, it tells the AI that the current request is research, not the final report, narration, lesson, or other Runtime deliverable. A clear Research Purpose also frames the document as an execution request to AI.

Without that distinction, the system may race toward a polished response because polished responses are what conversational AI is trained to produce. The answer can sound complete while revealing very little about the evidence behind it. The user establishes the purpose; AI can help clarify it, test it against the objectives, and identify downstream needs that have not yet been stated.

The purpose can be concise, but it should remain visible. It gives every other element a reason to exist and provides the first checkpoint for deciding whether the investigation stayed on course. Once the destination is clear, the next task is to identify what changes from one research execution to another.

Research Configuration Variables

Research Configuration Variables separate the stable research method from the values assigned to the current investigation. A reusable framework might accept a location, time period, feature, audience,

platform, product category, depth level, or number of items. Change the values, and the same method can support a new execution without being rewritten from the ground up.

When those values are scattered through the instructions, reuse becomes fragile. A subject, date, or audience from the previous project can remain buried in a paragraph and quietly influence the next one, the prompt-writing equivalent of finding last year's leftovers behind the milk. A visible variable block makes the current assignment easy to inspect before the work begins.

The user supplies the values. AI can check whether they are complete, identify incompatible combinations, or propose options while the project is still being defined. The variables may appear in a compact header rather than a full section, provided their names and current assignments remain unmistakable. Variables define the current run; Objectives define what that run must actually discover.

Research Objectives

Research Objectives translate the purpose into answerable questions and required topics. They tell the investigation what must be covered and later provide the basis for a completeness check. Strong objectives are specific enough to guide the work without dictating what the conclusion must be.

An instruction to "research the topic thoroughly" sounds ambitious, but it does not define thoroughness. An objective written to prove a preferred claim creates the opposite problem: the investigation becomes confirmation dressed as research. Useful objectives identify what must be understood while allowing the evidence to shape the result.

The user defines the objectives. AI can decompose a broad question, identify dependencies, reveal overlap, and suggest information needed for downstream use. Objectives may share a section with Research Scope under a heading such as Research Assignment, but the questions and boundaries must remain distinguishable. Once the questions are clear, the framework must decide where acceptable answers may come from.

Approved Information Sources

Approved Information Sources establish the evidence permission boundary. They may name project documents, repositories, official agencies, standards bodies, professional organizations, educational institutions, or other source categories appropriate to the domain. Stable categories often originate in Governance, while the Research Framework narrows them for the current subject. This element answers where Research is allowed to look; it does not automatically establish the evidentiary weight of everything found there.

Permission is not proof. A user-provided document may be authoritative for the user's own requirements, decisions, project records, or supplied data, but an external factual claim inside that document is not proven merely because the user supplied it. Likewise, public-domain status concerns reuse rights; it does not make a source more reliable. Evidence still has to fit the claim it is being asked to support.

AI can search approved sources, document what it used, and report when the required evidence is unavailable. It should not quietly substitute an unapproved source merely to make the response look complete. The user or Governance authority decides whether a new source category may be admitted; among permitted sources, the Research Request's Source Priority and Conflict Handling rules determine how evidence should be weighed for the claim at hand.

A visible gap is more useful than an invisible substitution, or an unearned claim of authority.

Source Priority Hierarchy

Approved sources can still differ in suitability or disagree, which means permission alone is not enough. The Source Priority Hierarchy guides how permitted evidence is weighed for a particular claim and how conflicts are handled. It is not a universal ranking of institutions or source types. Claim-specific priority may consider directness to the claim, authority or ownership over the subject, primary or original evidence, currentness appropriate to the question, jurisdiction, methodological transparency, relevance to the exact issue, and corroboration.

Canonical terminology note: Source Priority Hierarchy is the standard DCPF Research responsibility. In practicums and working artifacts, a section may be labeled Source Priority and Conflict Handling when that wording makes its operational purpose clearer. The label may vary; the responsibility does not. It must guide claim-specific weighting among approved sources and keep unresolved credible conflicts visible for human judgment.

A source that is strongest for one question may be secondary for another. An organization's current policy may be authoritative for what that organization requires. A current official software specification may be controlling for documented product behavior. A historical primary record may be essential for establishing what was written or decided at the time, while later scholarship may be better suited to interpretation. A governing standard may control a regulated requirement. The Research Request should identify these relationships rather than force every domain into one prestige ladder.

Priority guides evidence selection; it does not manufacture certainty. AI can apply the project's claim-specific criteria, seek corroboration, and compare definitions, methods, dates, scope, jurisdiction, and purpose. When credible approved sources remain materially inconsistent and the evidence does not justify a resolution, preserve the disagreement in the Research Output rather than silently choosing a winner. Human participants decide whether disputed evidence may proceed. Approved Sources and Source Priority may be consolidated under Evidence Policy, after which Scope determines how much territory the investigation is authorized to cover.

Research Scope

Research Scope defines where the investigation begins and ends. It identifies inclusions, exclusions, expected depth, and recency. These boundaries prevent the work from becoming too shallow to support the objective or so broad that it wanders into three neighboring topics and starts paying rent there.

Explicit exclusions are especially valuable. They help reviewers distinguish an intentional boundary from an accidental omission and keep adjacent questions from consuming the investigation. Depth clarifies whether the project needs an overview, operational detail, technical analysis, or comprehensive treatment. Recency establishes whether older evidence remains useful or current information is required.

The user sets the scope. AI can identify ambiguous edges, warn when the objectives exceed the boundary, and recommend a separate investigation for valuable material outside it. Scope does not decide which source is strongest; it decides what questions and territory the investigation is authorized to pursue. Scope may be consolidated with Objectives, but the framework should still answer two different questions: What must be learned, and how far should the investigation extend? Once those limits are set, the project can decide what shape the completed evidence should take.

Output Organization

Output Organization defines the shape of the Research Output before the investigation begins. It may require topic sections, consistent evidence records, comparison tables, source notes, a conflict register, unresolved questions, or fields designed for the future Knowledge Repository. This is not cosmetic formatting; it is the beginning of the architectural handoff.

If Knowledge will preserve a claim, its source, validation status, and category, Research can collect those elements together from the start. Waiting until the end may force the project to mine a long narrative for scattered facts and reconstruct which qualification belonged to which finding. Designing the destination early improves both coverage and traceability.

The user defines the structure with downstream needs in view. AI applies it, detects missing fields, and can recommend adjustments when the planned format does not fit the evidence. Output Organization may be consolidated with Source Documentation and delivery instructions as a Research Output Specification. The structure shows where the evidence will go; Source Documentation shows where it came from.

Research Source Documentation

Research Source Documentation defines how the evidence trail will be preserved. Depending on the project, it may require titles, organizations, authors, publication dates, links, access dates, page references, document identifiers, or a separate source register. The standard should be detailed enough for a future reviewer to validate, update, or correct the finding without repeating the entire investigation.

The framework should also distinguish a research source from a content source when both matter. A research source is where information was obtained or verified. A content source identifies the historical origin, lineage, authorship, or tradition of the subject itself. A reference used to verify a traditional story, for example, is not automatically the origin of that story.

Without traceability, the project may remember that it learned something useful but not where it put the evidence, the research equivalent of Uncle Bob searching the house for his car keys. Governance

may establish the general documentation standard; the Research Framework applies it to the current investigation. AI can capture and organize source details, while human review remains appropriate for consequential claims and ambiguous attribution.

ESSENTIAL: Sequential Output Requirements

Sequential Output Requirements protect the Research Output when the complete result may exceed a comfortable single response or its length cannot be reliably predicted. They prevent the AI from compressing, abbreviating, or omitting required work merely to squeeze everything into one answer. The standard rule is supplied by DCPF, and the AI applies it automatically when needed; the user may deliberately adapt it for a specific project.

When sequential delivery is required, the AI divides the output at natural points rather than forcing it into predetermined sets. At the end of each nonfinal part, it states that additional output remains, requests permission to continue, and stops. After permission is received, it resumes where the previous part ended. The final part clearly states that the requested Research Output is complete.

A short, predictable Research Output may still be delivered in one response. The user does not have to estimate the model's response limit or play a guessing game with an invisible fuel gauge. The instruction exists so completeness, structure, detail, source documentation, and continuity survive regardless of output length.

Applicable Requirements identify the exact standards that apply to the investigation. The Research Request states the applicable source, privacy, prohibition, validation, documentation, and human-authority controls directly. A maintained Governance artifact may preserve the project-wide origin and version of recurring standards, but it is not an attachment or retrieval dependency for Research.

A vague instruction to "follow relevant policies" is weak because neither the AI nor a future collaborator may know which policies are relevant or where to find them. The governing material must be supplied by attaching it, embedding or pasting the applicable contents into the working context, or retrieving it from a maintained source the AI can actually access. A rule the system cannot locate is not much of an operational rule.

Governance supplies the standards; Research makes the applicable standards visible at the point of execution. With the nine elements now defined, the next question is practical: what work can AI perform inside this architecture, and where must human authority remain in control?

6.5 How Research Collaborates with AI

Four Stages of AI Assistance

AI can participate across four stages of Research: design, investigation, documentation, and delivery. Its value is not limited to searching. It can help transform a broad information need into an organized, inspectable research process.

During design, AI can clarify objectives, break a broad question into answerable parts, identify missing variables, reveal scope conflicts, and suggest an output structure. These proposals help the user define the investigation; they do not replace the user's authority over purpose, evidence policy, or boundaries.

During investigation, AI can search approved materials, extract relevant findings, compare sources, detect contradictions, and distinguish direct evidence from inference. It should report when a source cannot be accessed, when a question cannot be answered inside the approved boundary, or when the available evidence does not justify a firm conclusion. An honest gap is part of a reliable Research Output, not a failure of presentation.

During documentation, AI can attach source details to findings, preserve consistent fields, identify uncertainty, and create a source register. During delivery, it can follow the planned organization, maintain sequential continuity, and perform a completeness check against the original objectives. These strengths make AI a capable research collaborator when the architecture tells it what good work looks like.

The Line AI Must Not Cross

There are also clear limits. AI should not broaden the source policy without authorization, hide missing evidence, silently expand scope, convert inference into fact, or promote Research Output into Knowledge. Humans remain responsible for the investigation's purpose, acceptable evidence, significant interpretations, exceptions, and acceptance decisions.

AI can conduct and organize research. Human authority defines what the project is prepared to trust.

6.6 From Research Request to Research Output

A Research Framework begins with an information need, but execution follows a controlled sequence. The user defines the purpose and assigns the current variables. Objectives and Scope establish the work. Governance supplies the evidence and operating boundaries. AI investigates, records the basis of each finding, and follows the Research Request's instructions for preserving conflicts, limitations, and uncertainty.

The findings are then organized according to the planned output structure. A completeness review compares the result with the objectives, while source and governance checks determine whether the required method was followed. When those checks are complete, the project has a Research Output ready for consideration, not merely a confident answer waiting to be copied somewhere important.

A Reliable Output Can Include Uncertainty

That output may contain a well-supported finding, a claim requiring review, an empty field that exposes a gap, or two approved sources that remain in disagreement. Reliability does not require pretending every question has been settled. It requires making the condition of the evidence visible so the next decision is informed rather than assumed.

Acceptance belongs at the boundary with Knowledge. Research delivers the evidence product; Governance and human review determine which parts the project is prepared to preserve and reuse. That is the moment when evidence moves from investigation into maintained project intelligence.

6.7 Governance in Action

The Governance Framework from Chapter 5 influences Research before the first source is opened. Approved Information Sources define the permitted evidence channels. Prohibited Content may forbid a source, information type, or practice even when the broader source category would otherwise be permitted. Within the remaining evidence, Source Priority and Conflict Handling guides claim-specific weighting and preserves material disagreement. Research Scope defines which questions and territory the investigation is authorized to pursue. Quality Standards define reliable and complete research, while Validation Requirements establish the checks performed before the package is accepted.

In practical terms: permission establishes where Research may look; prohibitions close boundaries that permission does not override; source priority weighs the permitted evidence for the claim at hand; scope limits the authorized investigation; and conflict handling keeps unresolved disagreement visible for human judgment.

What If Approved Sources Cannot Answer the Question?

Governance also controls exceptions. If the Research Request cannot answer an objective using approved sources, the AI reports the gap rather than weakening the source policy on its own. The project owner may authorize a new source, narrow the objective, request another investigation, or accept that the answer remains unresolved.

This is preventive oversight in practice. Governance shapes the investigation before evidence choices harden into a polished output; it is the seat belt, not the accident report. With those controls in place, the framework can now be scaled to fit the size and risk of the project.

6.8 A Proportionate Research Framework

The nine standard Research responsibilities do not require a nine-page document or nine separate headings. A small implementation may combine Purpose and Variables in an identity block, join Objectives and Scope as the Research Assignment, combine Approved Sources and Source Priority as the Evidence Policy, and unite Output Organization, Source Documentation, and Sequential Requirements as the Delivery Specification. Applicable source, boundary, validation, and human-authority requirements belong inside those native responsibilities.

That compact form still answers the essential questions: Why is the research being performed? What changes in this execution? What must be learned? Which evidence may be used? Where are the boundaries? How will the findings remain traceable and complete? Which standards apply? The architecture is preserved because the decisions remain visible.

Scale the Structure to the Stakes

A medium implementation may separate the elements because different researchers, reviewers, repositories, or evidence types are involved. It may add confidence indicators, formal review roles, domain-specific methods, versioned Research Outputs, or separate acceptance records. Proportion should follow complexity and consequence, not a desire to make the document look impressively thick.

Combine headings when useful. Preserve the decisions that make Research controlled and reviewable.

6.9 Research in Action: The National Park Narration

The Governance Framework in Chapter 5 established the project-wide operating conditions. The following live document applies the relevant conditions directly to one focused investigation: developing reliable evidence about Old Faithful and its hydrothermal setting for a future National Park narration. It is populated for a specific park and feature and is ready to provide to an AI with access to the approved sources. As you read it, notice that every section makes a decision visible before the research begins.

NPN-RES-001 states all applicable requirements needed for this investigation inside the Research Request. NPN-GOV-001 remains the maintained record of recurring project-wide Governance, but the Research execution does not require that Governance artifact to be attached, supplied, pasted, embedded, or retrieved.

LIVE FRAMEWORK DOCUMENT

Framework: Research Framework

Framework ID: NPN-RES-001

Project: National Park Narration

Owner: Research Lead

Version: 1.0

Status: Ready for execution

Source, validation, and human-authority requirements: Stated in full in this Research Request

Inputs: Current research variables and accessible source materials

Output: Structured Research Output for human review, possible acceptance as Knowledge, and preservation in the Knowledge Repository

Framework Purpose

Conduct and document research needed to support an accurate, listener-friendly narration about Old Faithful and its hydrothermal setting in Yellowstone National Park. This is a research request, not a request to write the final narration. The completed Research Output will be reviewed before any information is accepted as project Knowledge and preserved in the Knowledge Repository.

LIVE FRAMEWORK DOCUMENT

Research Configuration Variables

Park: Yellowstone National Park. **Feature:** Old Faithful and its hydrothermal setting. **Intended audience:** general adult listener without a geological background. **Research depth:** introductory explanation supported by sufficient detail to prevent misleading simplification. **Intended downstream use:** a short public narration developed through the Runtime Framework.

Research Objectives

1. Explain the physical conditions and processes necessary for geyser activity. 2. Identify what makes Old Faithful useful for explaining Yellowstone's larger hydrothermal system. 3. Develop accurate, plain-language definitions of the essential geological terms. 4. Identify important limits, uncertainty, or common misconceptions that the narration should not obscure. 5. Identify any visitor-safety context appropriate to mention without creating real-time guidance. 6. Record unresolved questions that require additional research or human review.

Approved Information Sources

Use original project materials when supplied; official National Park Service and Yellowstone National Park publications; United States Geological Survey and Yellowstone Volcano Observatory resources; other relevant government publications; and appropriate educational institutions, professional organizations, or authoritative reference works permitted by NPN-GOV-001. General public sources may be used only to locate potential approved evidence and may not independently support a material claim.

Source Priority Hierarchy

Apply source priority according to the claim being evaluated and within NPN-GOV-001; any exception requires documented human authorization. User-provided original project documents or datasets are controlling for project requirements, decisions, and project data they directly establish; they do not automatically prove external scientific facts. For hydrothermal science, prefer the most direct, current, and relevant National Park Service, United States Geological Survey, or Yellowstone Volcano Observatory material for the claim. Use original source material when it directly establishes historical content, and use educational institutions, professional organizations, standards bodies, and authoritative reference works for explanation or corroboration when appropriate. Public-domain status affects reuse rights, not evidentiary weight. Compare date, scope, methodology, jurisdiction, and purpose before resolving conflicts. Seek corroboration when practical and record material conflicts rather than silently choosing a source.

Research Scope

Include the basic hydrothermal conditions required for a geyser; the relationship between Old Faithful and Yellowstone's hydrothermal environment; terminology needed for a general audience; evidence relevant to common misconceptions; and stable public-safety context supported by official sources. Exclude real-time eruption predictions, current travel planning, detailed visitor itineraries, unsupported folklore, and cultural or ecological information whose public use is restricted or cannot be validated through approved sources. Do not expand into a general history of Yellowstone unless the information directly supports the narration objective.

LIVE FRAMEWORK DOCUMENT

Output Organization

Organize the Research Output into: 1. Executive research summary. 2. Findings aligned with each Research Objective. 3. Evidence records containing the proposed claim, plain-language explanation, principal research source, corroborating source when available, uncertainty or conflict note, and recommendation for Knowledge consideration. 4. Unresolved questions and excluded material. 5. Source register. 6. Completion statement identifying whether every objective was addressed.

Research Source Documentation

For every material factual area, record the principal source title, responsible organization or author, publication or revision date when available, stable link or document identifier, access date when applicable, and relevant page or section. Distinguish the research source used to verify information from any content source describing historical origin, authorship, or cultural lineage. Do not invent missing citation details.

ESSENTIAL: Sequential Output Requirements

PURPOSE

When the requested Research Output may exceed what can be comfortably delivered in a single response, divide it into multiple sequential parts while preserving structure, completeness, detail, source documentation, and continuity.

SEQUENTIAL OUTPUT RULE

- Apply this requirement automatically whenever the complete output may exceed practical response length or its length cannot be reliably predicted.
- Do not compress, summarize, abbreviate, or omit required research merely to fit within one response.
- Favor completeness over minimizing the number of responses.
- Break the output at a natural point. Do not force the Research Output into predetermined sets or fixed section groupings.
- At the end of each nonfinal part, state that additional output remains, request permission to continue, and stop.
- After permission is received, continue from where the previous part ended until the complete Research Output has been delivered.
- The final part must clearly state that the requested Research Output is complete.

Applicable Requirements

Apply the requirements stated in this Research Request, including Quality Standards, Approved Information Sources, Prohibited Content, Validation Requirements, and the requirement to make uncertainty and source conflict visible. If a control is unclear or conflicts with another requirement, pause execution and ask the Research Lead for direction rather than assuming an exception.

What the Artifact Demonstrates

The live document separates the stable research method from the current subject, names the governing component, and requests an evidence package rather than a narration. It is executable as written, yet

execution alone will not grant its findings permanent authority. Human review must still determine which findings become Accepted Knowledge. The Knowledge Framework then owns the architectural responsibility for preserving that Knowledge in the Knowledge Repository. That distinction is the handoff on which the next chapter depends.

6.10 Reader Checkpoint

Is the Research Framework simply a detailed search prompt? No. Searching may occur inside it, but the framework also controls purpose, variables, objectives, evidence, scope, organization, documentation, delivery, and governance. The search is one activity; the framework is the architecture that directs and preserves the investigation.

Does completed research automatically become project knowledge? No. Completion produces a Research Output. Information becomes maintained Knowledge only after the applicable human review and acceptance decision.

Why define Output Organization before conducting the investigation? Because downstream use should influence how evidence is gathered. A known structure improves coverage, keeps qualifications attached to findings, preserves traceability, and makes gaps easier to detect.

How does a user know when to activate the continuation protocol? The user does not have to estimate the model's response limit. The Research Request directs the AI to apply sequential delivery automatically whenever the complete output may exceed practical response length or its length cannot be reliably predicted. A short output may remain in one response; a longer one continues in natural parts until the completion statement is delivered.

Does a source category have one permanent rank? No. Source priority is claim-specific. Use the approved source or combination of sources best suited to the question, considering directness, authority, currentness, jurisdiction, method, relevance, and corroboration. If credible evidence still conflicts, preserve the disagreement rather than manufacturing certainty.

Can AI change the approved source boundary when evidence is difficult to find? No. AI may report the limitation and propose an alternative for human review, but it should not authorize its own exception. Difficulty finding evidence is a condition to disclose, not permission to lower the standard quietly.

6.11 From Research to Knowledge

The project now has something more valuable than a quick answer. It has a defined investigation and a structure for producing documented evidence whose source, coverage, uncertainty, and status can be evaluated.

Some findings may be accepted. Others may remain uncertain, require more research, or be excluded. The next architectural responsibility is to preserve what the project is prepared to use without losing its organization, source basis, status, or history. Research has gathered the evidence; human review determines what is accepted, and the Knowledge Framework governs how that Accepted Knowledge is preserved.

Chapter 7 moves into the Knowledge Framework, where Accepted Knowledge is preserved as maintained project intelligence and the architecture begins turning discovery into durable organizational memory.

[Return to Table of Contents](#)

The Knowledge Framework

DCPF is an experimental approach that has demonstrated promise but has not been broadly validated. Test and validate your implementation within your own environment before considering real-world or production use.

This chapter presents the Knowledge Framework as DCPF's architectural responsibility for preserving Accepted Knowledge between governed research and repeatable execution. It explains the four standard Knowledge responsibilities, identifies Metadata as optional and supplemental, shows why the core responsibilities are necessary, explains how human authority governs acceptance and maintenance, and demonstrates a working Knowledge Repository prepared for Runtime use.

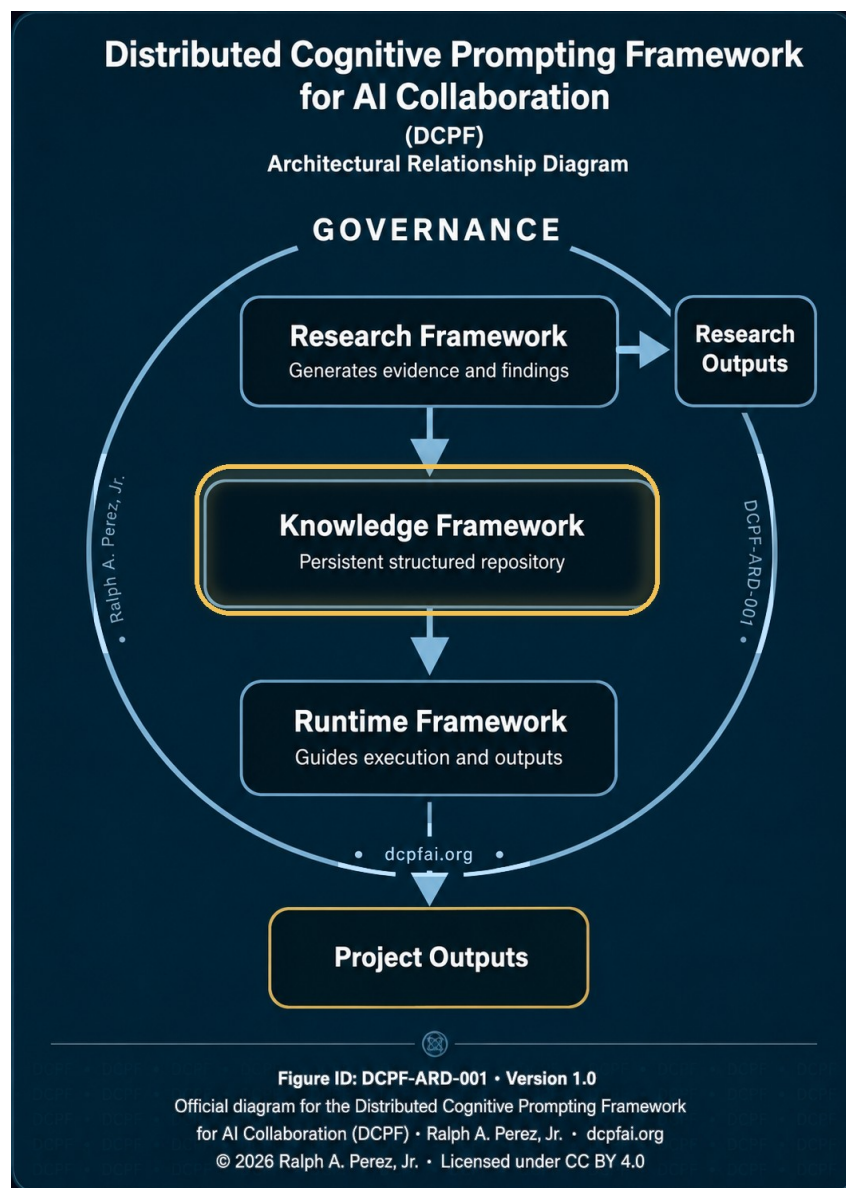


Figure 7.1. Knowledge preserves accepted evidence as maintained project intelligence and supplies it to Runtime for governed, repeatable use.

7.1 Knowledge as the Deliberate Handoff

Research Finished. Now What?

What happens after a Research Framework investigation produces evidence worth keeping? A polished Research Output may answer the immediate question, but if it disappears into an old conversation or an unnamed file, the project has learned something without actually building memory. Chapter 6 showed how the Research Framework prevents that outcome. Its purpose, objectives, source requirements, scope, and output organization were chosen so the completed work could remain useful after the investigation ended.

Once the Research Output has been reviewed, the project determines which information is accepted for future use. That information becomes Accepted Knowledge. The Knowledge Framework owns the architectural responsibility for preserving it, while the Knowledge Repository provides its persistent representation and maintained project home. In a small or medium DCPF implementation, the handoff may be as simple as copying the accepted Research Output into a document, giving that document a clear identity, and saving it where Runtime can retrieve it. No database, knowledge-management platform, or elaborate new AI procedure is required.

Simple Does Not Mean Casual

The process can be simple because the architectural distinction is already clear. Research conducts the inquiry. Human review and acceptance change the status of approved information to Accepted Knowledge. The Knowledge Framework then governs its preservation for later use. Without that handoff, even excellent research can become the digital equivalent of a spare key placed somewhere “safe” and never seen again.

The Knowledge Repository can remain simple because the Research Framework was designed to produce information worth preserving.

This chapter follows that handoff, introduces the four standard responsibilities of the Knowledge Framework plus optional supplemental Metadata, and shows how they are represented in a working Knowledge Repository. The chapter is intentionally compact. The Repository does not need to look impressive; it needs to keep Accepted Knowledge findable, governed, and ready for use.

7.2 The Knowledge Framework in the DCPF Architecture

The Quiet Middle That Makes Everything Work

The Knowledge Framework occupies the middle of the DCPF operational flow. Research receives the questions and develops the evidence. Runtime receives a current objective and produces the visible

deliverable. Knowledge sits between them, preserving what the project has accepted so the system does not have to rediscover the same facts every time work begins.

The forward flow is deliberate. Anticipated Runtime needs inform the Research design. Research settings determine what will be investigated, which sources will be used, how much detail is needed, and how the findings will be organized. After review and acceptance, the approved information becomes Accepted Knowledge and is preserved in the Knowledge Repository. Runtime then combines that maintained knowledge with current variables and task instructions to create a Project Output. Governance influences both boundaries: what may enter Knowledge and how Runtime may use it.

When Accepted Knowledge Does Not Require New Research

Research remains DCPF's controlled entry point for new evidence that must be investigated or verified. But not every item placed in a Knowledge Repository begins as a new research finding. A project may already possess material that is authoritative for its own scope, such as an approved policy, documented human decision, user requirement, controlled project record, or supplied project data. When the designated human authority confirms that such material is current, applicable, and approved for reuse, it may be accepted directly into the Knowledge Repository without creating a Research Request whose only purpose would be to rediscover or re-prove the project's own decision.

That direct acceptance does not convert every factual statement inside an internal document into an externally verified fact. The material is authoritative only for what it directly establishes within its approved scope. If a supplied policy cites an outside statistic, makes a scientific claim, or depends on another fact that requires verification, use Research when that external claim must be established for downstream work. Preserve the item's source or origin, approval status, scope of use, and applicable Governance so Runtime can distinguish a human-authorized project decision from independently researched evidence.

Research is the front door for new evidence. Human authority can also admit already-approved project Knowledge when the project itself is the authoritative source for what the item directly establishes.

Knowledge Is the Data Engine

This makes Knowledge the Runtime Framework's data engine. It does not decide what Runtime should produce, but it supplies the reliable information from which that work is developed. Detailed Runtime instructions cannot fully rescue incomplete, poorly organized, or unsuitable knowledge. A beautiful dashboard still has a problem when the fuel gauge is connected to the wipers.

What Happens When Runtime Finds a Hole?

The relationship also works in reverse. Runtime may expose a missing fact, an outdated statement, or a question the existing knowledge cannot answer. That gap returns to Research. After new evidence is gathered, reviewed, and accepted, the Knowledge Repository can be updated through the applicable Research and human-acceptance process and then made available to future executions. Runtime therefore does more than consume knowledge; it helps reveal where the repository needs attention.

Runtime needs inform Research. Research develops useful information. Knowledge preserves it. Runtime applies it.

7.3 Designing Research for Knowledge

Build Backward from the Work Ahead

Planning a useful Knowledge Repository begins before the file is saved. It begins when the Research Framework is configured with the later work in mind. Research that will feed Runtime should gather not only accurate facts, but the forms of explanation, qualification, and documentation the future task will require.

Suppose the future Runtime task will create a short narration for a general adult audience. Research should gather more than technical facts. It may also need plain-language definitions, enough context to prevent misleading simplification, common misconceptions to avoid, appropriate safety boundaries, and source information that allows the content to be checked. The Research Output should organize those elements so Runtime can locate and apply them without reconstructing the entire investigation.

The Research Framework in Chapter 6 did exactly that. Its objectives examined the conditions required for geyser activity, Old Faithful's relationship to Yellowstone's broader hydrothermal setting, the terms a general listener would need explained, misconceptions that could distort the narration, and stable safety context appropriate for reuse. Its Output Organization kept findings, source support, uncertainty, use recommendations, and unresolved questions visible.

Preparing for Use Is Not Predetermining the Answer

Those Research settings did not become the knowledge. They shaped the investigation so its output would contain the right kinds of information in a usable form. Research configuration tells the system how to inquire; the accepted result tells the system what the project is prepared to preserve and use.

Designing backward from Runtime does not force Research toward a preferred conclusion. It anticipates what the later task will need while allowing approved evidence to determine what can responsibly be said. The destination influences the packing list; it does not get to rewrite the map.

7.4 The Knowledge Framework Skeleton

The Knowledge Framework has four standard architectural responsibilities represented within the Knowledge Repository, plus optional supplemental Metadata. The four standard responsibilities keep Accepted Knowledge identifiable, organized, usable, and governed. Metadata can improve traceability and maintenance when the project benefits from it, but it is not a fifth core responsibility.

Standard responsibility / supplemental element	Architectural function	Typical source
Framework Purpose	States what knowledge is being preserved and how it will be used.	User
Knowledge Configuration Variables	Identify any current view, subject, category, or selection used to organize the knowledge.	User
Knowledge Structure	Contains the organized body of accepted information.	Research Output
Information Categories	Provide the headings or classifications used to locate information.	User and Research Output
<i>OPTIONAL / SUPPLEMENTAL: Metadata</i>	Adds useful descriptive context such as version, date, status, owner, tags, or sources.	Research Output and/or user

What are the standard responsibilities I should expect to see?

The standard DCPF Knowledge responsibilities are these four. They are represented in the Knowledge Repository because preservation requires more than storing a file.

Framework Purpose: States what Accepted Knowledge is being preserved and how the project is authorized to use it. It is necessary so the Repository has a defined purpose and does not become an undifferentiated storage location for anything that happens to be saved.

Knowledge Configuration Variables: Identifies the current subject, view, category, selection, audience, or use condition used to organize access without changing the Accepted Knowledge itself. It is necessary so the project can select or view the maintained Knowledge for different uses without silently rewriting what has been accepted.

Knowledge Structure: Contains the organized body of Accepted Knowledge, including the qualifications and source context needed for responsible reuse. It is necessary because the Repository must preserve the actual approved intelligence that Runtime will rely upon, not merely a label or reference to it.

Information Categories: Provides headings, classifications, status groupings, or other findable organization for the Accepted Knowledge. It is necessary so users and Runtime can locate the right information and distinguish different kinds or statuses of Knowledge without reconstructing the organization each time.

When a Knowledge Responsibility Is Left Undefined

Preserving information is not the same as preserving governed Knowledge. The Knowledge Repository responsibilities help the project maintain not only accepted information, but also its purpose, organization, usable views, and connection to the human-authorized conditions under which that Knowledge remains valid.

When one of these responsibilities is left undefined, the information may still exist, but part of its meaning or authority can become fragile. Humans or AI may have to reconstruct how the information should be organized, when it applies, or whether it is still appropriate to use. Over time, this can contribute to context drift, stale information reuse, provenance loss, inconsistent interpretation, or the use of information outside the conditions under which it was originally accepted.

Appendix G, DCPF Responsibility-Consequence Map, examines each Knowledge responsibility and shows how undefined responsibilities can weaken continuity between Accepted Knowledge and later Runtime execution.

What can I combine, simplify, or omit in a smaller implementation?

You may combine standard responsibilities into fewer headings or a compact identity block when each responsibility remains explicit, identifiable, and usable. The Knowledge Structure itself cannot be omitted because it is the maintained Accepted Knowledge; the surrounding standard responsibilities may be represented very briefly when the project is simple.

Framework Purpose and Knowledge Configuration Variables may share an identity block. Existing headings from an accepted Research Output may already serve as Information Categories, and the applicable requirements may be recorded concisely. Formal Metadata is optional and supplemental: add version, date, owner, tags, review triggers, or similar fields when they improve traceability, maintenance, or reuse. Additional views, taxonomies, access controls, entry-record schemas, or database structures are also supplemental unless project scale or risk gives them a reason to exist.

These simplifications can still satisfy the standard because the Knowledge responsibilities are architectural rather than ceremonial. A concise Repository is sufficient when each core responsibility remains identifiable and usable.

7.5 From Research Output to Knowledge

Keep the Evidence. Remove the Scaffolding.

The practical handoff usually requires only a few decisions. The project reviews the completed Research Output, identifies what is accepted for future use, preserves necessary qualifications and source references, and separates anything that remains unresolved or excluded. It then saves the result with enough identification to distinguish the current approved version from earlier drafts or future revisions.

Much of the Research Output may be retained exactly as delivered. Temporary execution instructions can be removed because the investigation has ended. A title, source Research Framework, version, status, and applicable requirements may be added. When Research already produced useful headings, evidence notes, uncertainty labels, and sources, the Knowledge Repository does not need to impose a second organization merely to prove that preservation occurred.

Do Not Rebuild What Research Already Organized

The handoff is comparable to receiving a well-prepared set of plans from an architect. The records office does not redesign the building before preserving the approved plans. It confirms that the correct set is complete, identifies it, and keeps it where the construction team can retrieve it. Buying a nicer filing cabinet is not a reason to redraw the blueprints.

Not every project will remain this simple. A large repository may eventually need tags, multiple views, formal version histories, access controls, or a database. Those additions should respond to real scale, reuse, maintenance, or risk. They are not prerequisites for establishing a Knowledge Repository that fulfills the Knowledge Framework's architectural responsibility, and adding them too early can turn a useful repository into a hobby devoted to organizing the organization system.

Begin with the accepted Research Output. Add structure only when the project has a reason to need it.

7.6 How Knowledge Collaborates with AI

AI Can Use the Library

AI works within the Knowledge Framework primarily by retrieving and applying Accepted Knowledge from the Knowledge Repository. When Runtime begins, the relevant Knowledge document is supplied by attaching it, embedding or pasting its contents into the active workspace, or retrieving it from a source the AI can actually access. AI can locate pertinent sections, connect related findings, explain technical material at the requested level, and apply the maintained knowledge to different approved formats or audiences.

AI can also assist with maintenance. It may compare a proposed update with the current version, identify an apparent conflict, locate a missing source reference, detect duplicate information, or report that Runtime has asked a question the repository does not answer. These actions help the project care for its knowledge; they do not give the model authority to redefine what the project knows.

But AI Does Not Get to Stamp Its Own Approval

New model output should not silently become maintained knowledge. A factual addition belongs in Research, and a proposed correction should pass the applicable review before replacing accepted material. Runtime may reveal the need for a change, but it should not quietly make that change while performing another task. AI may use the library, find a missing book, and suggest a new shelf. It does not get to sneak in a manuscript and stamp it "official."

AI can retrieve and apply Knowledge. Acceptance and material revision remain human-governed decisions.

7.7 Governance in Action

Who Gets Through the Repository Door?

Governance keeps the simple handoff dependable. The applicable controls recorded in NPN-KNW-001 require factual claims to use approved sources, make uncertainty visible, prevent unsupported material from entering the system, and require review before Research Output is accepted into Knowledge. Governance is the doorman at the repository entrance because admission changes the information's status from evidence under consideration to maintained project intelligence.

At the handoff, Governance determines what status the retained material receives. In the National Park Narration example, stable findings about hydrothermal processes can be approved for Runtime. Current eruption predictions and other time-sensitive visitor information remain excluded because the Research Scope did not authorize them as reusable narration data. Stable safety context may be retained, but Runtime must not present the document as a substitute for current National Park Service guidance.

The Knowledge Repository states the requirements that apply to acceptance, maintenance, and use. NPN-GOV-001 may remain as the project's maintained record of recurring standards, but the Knowledge artifact does not depend on that Governance artifact being attached, supplied, pasted, embedded, or retrieved.

7.8 Knowledge in Action: The National Park Narration

A Working Repository Without the Drama

The following live Knowledge Repository shows the practical result of the Knowledge Framework's architectural responsibility. NPN-RES-001 was configured around the future narration task, and its accepted Research Output has been retained beneath a compact identity block. The content has not been rebuilt into a separate knowledge system. The handoff simply identifies what was accepted, how it may be used, which Governance controls apply, and which material remains outside Runtime's reach.

LIVE KNOWLEDGE REPOSITORY

Architectural Responsibility: Knowledge Framework

Artifact: Knowledge Repository

Repository ID: NPN-KNW-001

Project: National Park Narration

Owner: Project Editor

Version: 1.0

Status: Approved for Runtime use

Source, validation, and human-authority requirements: Stated in full in this Knowledge Repository

Research source: Accepted output from NPN-RES-001, version 1.0

Purpose: Preserve the approved information used to develop a short, listener-friendly narration about Old Faithful and Yellowstone's hydrothermal setting

Knowledge configuration: Subject, Old Faithful; audience, general adult listener; default use, approved findings only; unresolved or excluded material is not available to Runtime unless specifically reviewed

LIVE KNOWLEDGE REPOSITORY

Accepted Research Output

Executive research summary. Old Faithful is a geyser in Yellowstone's Upper Geyser Basin and provides an approachable example of the park's larger hydrothermal system. Yellowstone's thermal features are produced as groundwater interacts with heat associated with the magmatic system below the caldera. A geyser also requires a plumbing system with constrictions. Those constrictions help trap heated water and pressure until steam expansion drives an eruption. Old Faithful is sufficiently regular to support predictions, but it does not erupt on an exact hourly schedule. Public narration should explain that distinction and use only stable safety guidance.

Hydrothermal System

Rain and snow supply groundwater that moves downward through faults and fractures. At depth, magma and hot rock heat the water. The heated water rises through a largely hidden plumbing system and supplies Yellowstone's surface hydrothermal features. For a general audience, this can be explained through three essentials: water, heat, and a pathway through rock.

How a Geyser Erupts

A geyser is a hot spring whose plumbing contains constrictions that prevent water and heat from circulating freely to the surface. Water deep in the system becomes very hot while pressure from the overlying water and rock inhibits boiling. As heated water rises, steam forms and expands. Pressure builds within the constricted plumbing until water and steam are forced from the vent. The eruption ends as water is expelled and the system loses heat and pressure; recharge then begins again.

Old Faithful in Context

Old Faithful is one of many hydrothermal features in the Upper Geyser Basin. The basin contains a dense collection of geysers and hot springs, and Old Faithful is one of five large geysers that National Park Service staff regularly predict. Its value to the narration is not that every geyser behaves identically, but that a familiar feature can introduce the interaction of water, heat, rock, pressure, and time.

Misconception and Use Limit

Do not describe Old Faithful as erupting every hour or according to an exact timetable. Its activity changes, and predictions are based on observation, including the relationship between the duration of an eruption and the following interval. The Knowledge Repository intentionally does not retain a current predicted eruption time. Runtime must not generate real-time predictions from this document.

Stable Safety Context

Visitors in Yellowstone's thermal areas should remain on boardwalks and designated trails and should not touch thermal features or runoff. The surrounding ground may be a thin crust above extremely hot water. This information may support a brief safety statement, but the narration is not current visitor guidance. Runtime should direct audiences to the National Park Service for current conditions and regulations when such guidance is required.

LIVE KNOWLEDGE REPOSITORY

Excluded or Unresolved Material

Current eruption predictions, live park conditions, travel planning, detailed visitor itineraries, and claims that could not be validated through approved sources are excluded from this Knowledge Repository. No unresolved research question prevents the approved material above from supporting the intended introductory narration.

Source Register

S1. National Park Service, *Hydrothermal Features*, Yellowstone National Park.

<https://www.nps.gov/yell/learn/nature/hydrothermal-features.htm>

S2. Yellowstone Volcano Observatory, U.S. Geological Survey, *Novel Technique Peeks Beneath the Ground at Yellowstone's Hot Water Plumbing System*, March 28, 2022. <https://www.usgs.gov/observatories/yvo/news/novel-technique-peek-beneath-ground-yellowstones-hot-water-plumbing-system>

S3. National Park Service, *Old Faithful*, Yellowstone National Park, last updated May 20, 2025.

<https://www.nps.gov/yell/planyourvisit/exploreoldfaithful.htm>

S4. National Park Service, *Current Geyser Activity*, Yellowstone National Park.

<https://www.nps.gov/yell/planyourvisit/geyser-activity.htm>

S5. National Park Service, *Safety*, Yellowstone National Park. <https://www.nps.gov/yell/planyourvisit/safety.htm>

All web sources accessed July 17, 2026.

Acceptance Note

The Project Editor has reviewed this retained Research Output under NPN-GOV-001 and approved it for the stated Runtime use. Material outside that purpose requires additional review or research. Future accepted changes will produce a new Knowledge Repository version rather than silently altering version 1.0.

The Best Handoffs Are Almost Boring

The live document is intentionally uneventful from a process standpoint. The investigation did the difficult work. The accepted output was identified and preserved as maintained project Knowledge. Nothing needed to be reconstructed from memory, rescued from twelve chat windows, or re-researched because someone forgot which file was the final final version.

That simplicity is a feature of the architecture. The headings already serve as Information Categories, the accepted body provides the Knowledge Structure, and the identity block supplies useful metadata, configuration, and applicable requirements. Responsibilities remain distinct even when the physical document is concise.

7.9 Reader Checkpoint

Five Questions Before the System Moves On

Must the Knowledge Repository be a database? No. The Knowledge Framework is an architectural responsibility, not a storage technology. The Repository may be a saved document, collection of documents, folder, database, or another maintained environment appropriate to the project.

Does creating Knowledge require another AI request? Not necessarily. If Research produced a useful, organized output, the project may review it, identify the accepted material, and save it directly.

Why should Runtime needs influence Research settings? Because Research must develop the information Runtime will later need. Purpose, objectives, scope, sources, detail, and output organization determine whether the retained result will function as a useful data engine.

Does every part of the research process become Knowledge? No. Research instructions, rejected material, temporary working notes, and unresolved content do not automatically become maintained Knowledge. Accepted results, necessary qualifications, source support, and visible use limits are retained.

Can AI update the Knowledge Repository during Runtime? AI can identify a gap or propose a change. A material update should return through the appropriate Research process and human acceptance review before becoming Accepted Knowledge and being incorporated into the maintained repository.

7.10 From Knowledge to Runtime

The Engine Is Ready

The project now has a stable information source. Governance established the operating conditions. Research developed and documented the evidence. Knowledge preserved the accepted result in a form that can be supplied to later AI work. The architecture no longer depends upon one conversation remembering everything that came before it.

The architecture is ready to execute. Chapter 8 introduces the Runtime Framework, where NPN-KNW-001 becomes the data engine for a specific task. Runtime will combine that complete maintained Knowledge with current variables, task instructions, output requirements, and the requirements stated in the Runtime Request to produce the National Park Narration. Research discovered it. Knowledge kept it. Now Runtime gets to make it useful.

[Return to Table of Contents](#)

The Runtime Framework

DCPF is an experimental approach that has demonstrated promise but has not been broadly validated. Test and validate your implementation within your own environment before considering real-world or production use.

This chapter presents the Runtime Framework as DCPF's execution layer, the place where complete maintained Knowledge, current variables, task instructions, output requirements, and explicit Governance controls become a controlled AI request. It builds and executes the National Park Narration request, demonstrates reuse through a second run, and tests how Runtime should respond when the available Knowledge cannot support the assignment.

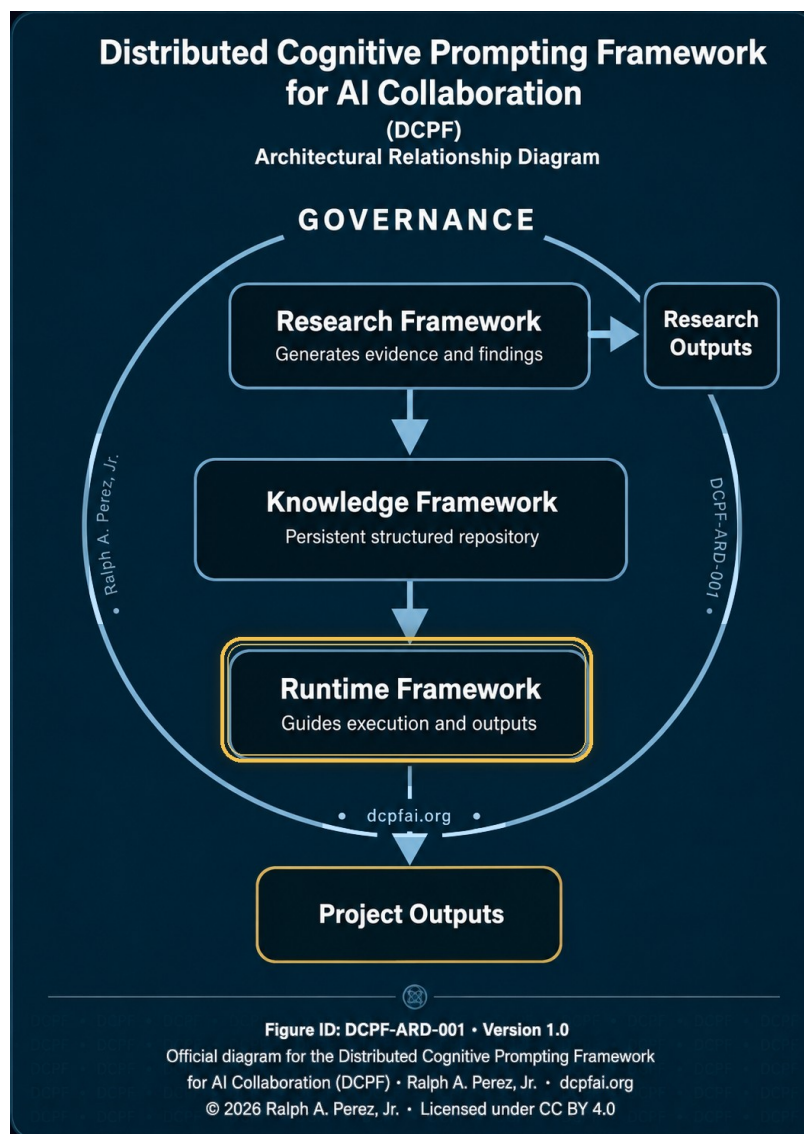


Figure 8.1. Runtime combines maintained Knowledge, current execution instructions, and Governance constraints to produce controlled Project Outputs.

8.1 The Assignment on the Table

The Architecture Has Reached the Payoff

The National Park Narration project now has three completed components. NPN-GOV-001 establishes the operating standards. NPN-RES-001 defined the investigation that developed the evidence. NPN-KNW-001 preserves the accepted result. Across Chapters 5 through 7, the project assembled trustworthy parts; Runtime is where those parts finally produce something an audience can hear.

Why Not Just Type "Write a Narration"?

The immediate assignment sounds simple: create a short spoken narration that explains Old Faithful and its hydrothermal setting to a general adult listener. We could attach the Knowledge document and type, "Write a narration." The AI would probably return something readable. What would remain unclear is which audience, length, tone, boundaries, and definition of completion it used, and whether it quietly added information the project never approved.

The Runtime Framework makes those execution decisions visible. It combines retained Knowledge with the settings and instructions needed for the work being performed now. It does not create another repository, repeat the research, or reopen decisions that have already been governed. It turns the components already built into a complete and inspectable AI request.

Knowledge tells the system what it can rely on. Runtime tells the system what to do with it now.

This chapter builds that request, executes it, and follows the results. We will then change several Runtime variables without changing the Knowledge Repository, and finally ask for something the supplied Knowledge cannot support. That last test matters: a useful architecture should know how to work, but it should also know when to stop.

8.2 Begin with the Components Already in Place

Three Execution Components, Three Jobs

For execution, the complete approved Knowledge and the current Runtime Request must be available. The Runtime Request supplies the current assignment and states the applicable requirements directly. Keeping those responsibilities visible prevents the new request from becoming a suitcase into which every project decision is stuffed at the last minute.

Available component	What it contributes to this execution	Current component
Governance	Accuracy, source, safety, tone, validation, and publication conditions	NPN-GOV-001, version 1.0
Knowledge	Approved information about Old Faithful and Yellowstone's hydrothermal setting	NPN-KNW-001, version 1.0

Available component	What it contributes to this execution	Current component
Runtime	Current audience, length, format, task, and output requirements	NPN-RUN-001, version 1.0

A Label Is Not the Thing It Labels

The identifiers keep the components distinguishable, but an identifier is not the Knowledge itself. Typing "use NPN-KNW-001" does not give an AI access to that repository. Runtime must receive the complete approved Knowledge by attachment, by embedding or pasting it into the working context, or by retrieval from a source the AI can actually access. Governance is different: the applicable controls are stated directly in the Runtime Request, so a separate Governance Framework or Repository is not an execution dependency.

References organize the architecture. Supplied content operates it.

8.3 The Runtime Skeleton at a Glance

The Runtime Framework has six standard responsibilities. Together, they identify the current work, assign adjustable values, supply approved Knowledge, define the task, describe completion, protect long outputs from compression, and connect execution to Governance. These six responsibilities are the DCPF Runtime standard; project-specific execution controls may supplement them when needed.

Standard responsibility	What it does during execution	Primary source
Framework Purpose	Identifies the work being performed now	User
Runtime Configuration Variables	Assigns the current audience, subject, length, tone, format, and other adjustable values	User
Input Knowledge	Supplies the accepted information used as the data engine	Knowledge Repository
Task Definition	States exactly what the AI must do	User
Output Requirements	Defines the shape and completion conditions of the deliverable	User and/or Governance
Sequential Output Requirements	Protects complete delivery through sequential parts and continuation stops when needed	User

What are the standard responsibilities I should expect to see?

The standard DCPF Runtime responsibilities are these six. Each protects a distinct part of controlled execution so the AI has a clear job without gaining authority over the project itself.

Framework Purpose: Defines what the system is doing now and distinguishes Runtime from Research, Knowledge maintenance, or Governance revision. It is necessary so execution stays focused on applying approved Knowledge rather than quietly reopening upstream responsibilities.

Runtime Configuration Variables: Assigns the current audience, subject, tone, length, format, technical level, delivery setting, or other adjustable values. It is necessary so the same approved Knowledge and Runtime structure can support different executions without changing the facts or carrying stale settings from a previous run.

Input Knowledge: Supplies the complete approved Knowledge that serves as the data engine for the current task. It is necessary because Runtime cannot responsibly execute from a filename, artifact ID, partial summary, or model memory when the approved content itself is required.

Task Definition: States exactly what the AI must do with the supplied Knowledge. It is necessary to convert the project objective into a bounded act of execution rather than leaving the model to invent the work it thinks would be useful.

Output Requirements: Defines the form, required components, limits, audience fit, validation needs, and conditions that determine when the deliverable is complete. It is necessary because a clear task does not by itself define what a finished and acceptable Project Output looks like.

Sequential Output Requirements: Protects complete delivery when the Project Output may exceed practical response length or its length cannot be predicted reliably. It is necessary so a large Runtime deliverable is not compressed, abbreviated, or partially omitted merely to fit one response.

When a Runtime Responsibility Is Left Undefined

Runtime is where approved Knowledge becomes action. Its responsibilities keep the current assignment explicit: what is being done, which Knowledge may be used, what variables apply, what the AI must actually do, and what a complete and acceptable output should look like.

If one of those responsibilities is left undefined, the AI may need to fill the gap through inference, model knowledge, conversational context, or its own interpretation of the user's intent. That can create conditions for goal drift, authority creep, stale-context reuse, unsupported inference, incomplete output, or hallucination when the supplied Knowledge does not actually support what the model is being asked to produce.

Appendix G, DCPF Responsibility-Consequence Map, provides the complete Runtime analysis and shows how each undefined responsibility can shift execution decisions away from explicit human direction.

What can I combine, simplify, or omit in a smaller implementation?

You may combine standard responsibilities into fewer headings or sections when each responsibility remains explicit, identifiable, and usable. A compact Runtime Request can still be fully standard.

Framework Purpose and Runtime Configuration Variables may share an identity block. Task Definition and Output Requirements may share one execution section when the action and definition of done remain distinguishable. Governance controls may be embedded directly instead of carried in a separate Governance artifact, and Sequential Output Requirements may remain dormant when the output is short and predictable. Input Knowledge should not be omitted when the task depends on approved Knowledge. Additional validation notes, delivery metadata, specialized approval fields, or technical execution controls are supplemental unless the project requires them.

One Working Request Can Carry All Six

These are standard Runtime responsibilities, not six mandatory documents or six ceremonial headings. Closely related responsibilities may share a section when their purposes remain visible. The standard defines what the execution must account for; the document layout remains flexible.

8.4 Purpose and Variables: Define This Run

What Is the System Doing Right Now?

The Framework Purpose prevents the AI from confusing execution with research. For this run, the purpose is to use approved Knowledge to create a listener-friendly narration. It is not to investigate Yellowstone, update the repository, reconsider accepted facts, or decide what the project should trust. The purpose draws a bright line around the current responsibility.

Change the Assignment, Not the Facts

Runtime Configuration Variables assign the values that change during the current run. The subject is Old Faithful. The audience is a general adult listener. The target duration is approximately two minutes. The tone is calm, clear, and approachable. The format is spoken narration. These values shape the present performance; they do not rewrite the underlying facts.

Keeping the variables together makes reuse easier. A later run can change the audience, duration, tone, or delivery setting without rewriting NPN-KNW-001. It also makes the request easier to inspect. A reviewer can see the current values before execution and correct a mismatch while correction is still inexpensive, before a five-minute lecture is cheerfully delivered to people who asked for a forty-second welcome.

8.5 Input Knowledge: Supply the Data Engine

Runtime Cannot Run on a Filename

NPN-KNW-001 is the data engine for this execution. It contains the accepted explanation of Yellowstone's hydrothermal system, the mechanics of a geyser, Old Faithful's context, the predictable-but-not-clocklike limitation, stable safety information, exclusions, and source history. Runtime does not need to rediscover any of it. It needs the complete document in the working context.

The Runtime request makes two conditions explicit. First, the complete Knowledge document has been supplied through one of the supported routes, not merely named. Second, the applicable requirements are stated within the request, and the AI must remain inside the approved Knowledge unless those controls deliberately authorize another information source. The Knowledge identifier provides traceability; the supplied Knowledge provides the facts.

Apply Knowledge; Do Not Quietly Invent It

This protects the separation established in earlier chapters. Runtime may reorganize, simplify, explain, and apply accepted information. It may not quietly turn its own generated language or general model knowledge into a new project fact. If the requested detail is missing, the useful response is to expose the gap. Runtime is an execution environment, not a permission slip for the model to raid the pantry and improvise ingredients.

A Knowledge identifier organizes the system. Supplied Knowledge powers it.

8.6 Task and Output: Describe Completion

The Task Says What to Do

The Task Definition tells the AI what action to perform with the data engine. Here, the task is to write one cohesive narration that introduces Old Faithful, explains how a geyser operates, corrects a common misconception, includes stable safety context, and ends with a natural connection to Yellowstone's larger underground system. The task converts supplied Knowledge into a specific act of production.

The Output Requirements Say What "Done" Looks Like

Output Requirements describe what completion should look like. The narration should be approximately 260 to 320 words, written for listening rather than silent reading, and understandable without a geological background. It should not contain spoken citations, current eruption predictions, or facts outside the supplied Knowledge. A separate Internal Validation Note will identify the components used, confirm the required coverage, expose any unsupported addition or unresolved gap, and preserve the requirement for human editorial approval.

The Task Definition and Output Requirements can share one section in a small framework, but their responsibilities remain different. The task defines the work. The output requirements define the finished form and completion conditions. "Prepare dinner" is a task; "four servings, no peanuts, ready at six" is the definition of done. Runtime needs both.

8.7 ESSENTIAL: Sequential Output Requirements

When the Output Is Too Big for One Trip

Sequential Output Requirements protect the Project Output when the complete result may exceed a comfortable single response or its length cannot be reliably predicted. They prevent the AI from compressing, abbreviating, or omitting required work merely to squeeze everything into one reply. The standard rule is supplied by DCPF, and the AI applies it automatically when needed; the user may deliberately adapt it for a specific project.

When sequential delivery is required, the AI divides the output at natural points rather than forcing it into predetermined sets. At the end of each nonfinal part, it states that additional output remains,

requests permission to continue, and stops. After permission is received, it resumes where the previous part ended. The final part clearly states that the requested Project Output is complete.

Short Work Can Stay Short

A short, predictable Project Output may be delivered in one response. The user does not need to estimate a model's response limit, memorize platform thresholds, or play a guessing game with an invisible fuel gauge. The instruction exists so completeness, detail, structure, formatting, and continuity survive when output size becomes uncertain.

The narration and Internal Validation Note in this chapter are short and predictable, so the demonstration may be completed in one response. The Sequential Output Requirements remain inside NPN-RUN-001 because the same reusable request could later be expanded. If an execution grows beyond a comfortable response, the model must break at a natural point, request permission, and continue without compressing the remaining work.

The rule is deliberately proportional. It does not force tiny deliverables into theatrical installments, and it does not allow a large deliverable to be packed into one answer like a grand piano shoved into a carry-on bag. It protects completeness when protection is needed and stays out of the way when it is not.

The Final Connection: Governance

The Governance section states the applicable quality, safety, source-boundary, output, and human-review controls directly in NPN-RUN-001. NPN-GOV-001 may preserve the project-wide source and version of recurring standards, but Runtime does not depend on attaching, supplying, pasting, embedding, or retrieving that Governance artifact. With the six elements defined, the framework is ready to assign work without surrendering control.

8.8 How AI Participates at Runtime

AI Performs the Assignment

Before execution, AI can check whether the variables are assigned, whether it can read the complete supplied Knowledge, and whether the task conflicts with a stated requirement stated in the Runtime Request. During execution, it can select relevant material, organize it for the audience, translate technical ideas into ordinary language, and follow the requested form. After execution, it can compare the result with the task and report whether the required checks passed.

A Clearer Job Is Not a Promotion

Those capabilities do not give AI authority to repair every problem it finds. It should not invent a missing fact, authorize its own exception, alter NPN-KNW-001, or conceal that the task exceeds the available data. Humans establish the purpose and consequential boundaries, decide whether a proposed change should enter the maintained system, and approve the public output. Runtime gives the model a clearer job description; it does not promote the model to owner, editor, and governing board before lunch.

Runtime does not give AI more authority. It gives AI a clearer assignment.

8.9 Runtime in Action: Build the Working Request

Put the Components in the Same Room

The complete Runtime Framework follows. In an actual execution, NPN-KNW-001 must be attached, embedded or pasted in full, or retrieved from a source the AI can actually access. Its identifier confirms which approved Knowledge version is being used; the identifier does not substitute for supplying the contents. The applicable requirements are already stated in the Runtime Request and do not require a separate Governance artifact.

LIVE FRAMEWORK DOCUMENT

Framework: Runtime Framework

Framework ID: NPN-RUN-001

Project: National Park Narration

Owner: Project Editor

Version: 1.0

Status: Ready for execution

Input Knowledge: NPN-KNW-001, version 1.0, supplied in full with this request

Source, validation, and human-authority requirements: Stated in full in this Runtime Request

Output: Public narration draft and internal validation note

Framework Purpose

Use the approved National Park Narration knowledge to create a short, accurate, listener-friendly narration about Old Faithful and Yellowstone's hydrothermal setting. This is an execution request. Do not conduct additional research, update the Knowledge Repository, or add factual information from general model knowledge.

Runtime Configuration Variables

Subject: Old Faithful and its hydrothermal setting. Audience: general adult listener without a geological background. Target duration: approximately two minutes. Target length: 260–320 words. Tone: calm, clear, approachable, and curious. Format: continuous spoken narration. Technical level: introductory. Publication status: draft pending human editorial review.

Input Knowledge

Use NPN-KNW-001, version 1.0, as the factual data engine for this execution. Supply the complete document by attaching it, embedding or pasting it into the working context, or retrieving it from a source the AI can actually access. Use approved findings only, preserve their qualifications and use limits, and do not infer content from the identifier. If NPN-KNW-001 is missing, incomplete, or cannot be read, stop and request it. Do not use excluded material, conduct web research, or fill a factual gap with general model knowledge.

LIVE FRAMEWORK DOCUMENT

Task Definition

Write one cohesive narration that: 1. Introduces Old Faithful within Yellowstone's hydrothermal setting. 2. Explains the roles of water, underground heat, pathways through rock, constricted plumbing, pressure, and steam. 3. Explains that Old Faithful is predictable but does not operate on an exact hourly timetable. 4. Includes a brief statement using the stable safety context in NPN-KNW-001. 5. Ends with a natural observation that connects the visible eruption to the larger underground system.

Output Requirements

Return two clearly separated sections: Narration Draft and Internal Validation Note. The narration must contain 260–320 words, use natural spoken rhythm, explain technical ideas in ordinary language, and avoid unsupported claims, promotional exaggeration, and current eruption predictions. Do not place citations or framework commentary inside the spoken narration. The Internal Validation Note must identify the Knowledge and Governance versions used; confirm coverage of the task; state whether unsupported factual additions were detected; identify any unresolved gap; and state that human editorial approval remains required.

ESSENTIAL: Sequential Output Requirements

PURPOSE

When the requested Project Output may exceed what can be comfortably delivered in a single response, divide it into multiple sequential parts while preserving structure, completeness, detail, formatting, and continuity.

SEQUENTIAL OUTPUT RULE

Apply this requirement automatically whenever the complete output may exceed practical response length or its length cannot be reliably predicted.

Do not compress, summarize, abbreviate, or omit required work merely to fit within one response.

Favor completeness over minimizing the number of responses.

Break the output at a natural point. Do not force the Project Output into predetermined sets or fixed section groupings.

At the end of each nonfinal part, state that additional output remains, request permission to continue, and stop.

After permission is received, continue from where the previous part ended until the complete Project Output has been delivered.

The final part must clearly state that the requested Project Output is complete.

A short, predictable Project Output may be delivered in one response.

Applicable Requirements

Apply the requirements stated in this Runtime Request in full. Give particular attention to Quality Standards, Prohibited Content, Validation Requirements, Output Standards, stable safety information, and human publication approval. If a stated requirement is unclear or conflicts with another requirement, pause execution and ask the Project Editor for direction.

Ready Means Available

The framework is ready to run. Nothing in it requires a specialized platform. The working request consists of this Runtime document, including its applicable requirements, plus the complete supplied

Knowledge component. The architecture may be compact, but every responsibility needed for a controlled execution is now visible.

8.10 First Execution: The Public Narration

The Framework Finally Leaves the Workshop

When NPN-RUN-001 is executed with the two supplied components, the AI produces the Narration Draft and the Internal Validation Note. Because the complete output is short and predictable, both sections may be delivered in one response. Sequential delivery remains available but does not need to activate simply to prove that it exists.

EXECUTED PROJECT OUTPUT

Execution: NPN-RUN-001, run 001

Knowledge used: NPN-KNW-001, version 1.0

Governance applied: NPN-GOV-001, version 1.0

Narration Draft

Old Faithful is famous for sending water and steam into the Yellowstone sky, but every eruption begins far below the surface.

Rain and snow provide groundwater that moves downward through faults and fractures in the rock. Deep underground, magma and hot rock supply heat. The warmed water then rises through a largely hidden system of passages. In a geyser, parts of this underground plumbing are constricted, preventing water and heat from circulating freely to the surface.

Those narrow places help hold water in the system. As the deepest water becomes very hot, the weight of the water and rock above keeps it under pressure. Steam begins to form and expand as heated water rises. Pressure builds until water and steam are forced through the surface vent. The eruption ends as water is expelled and the system loses heat and pressure. Then the underground plumbing begins to recharge.

Old Faithful is one of many hydrothermal features in Yellowstone's Upper Geyser Basin. It is regular enough for National Park Service staff to make predictions, but its familiar name can create the wrong impression. Old Faithful does not erupt every hour or follow an exact timetable. Its behavior changes, and predictions depend upon observation rather than a fixed schedule.

Hydrothermal areas are both remarkable and hazardous. Visitors should remain on boardwalks and designated trails and should never touch thermal features or their runoff. The surrounding ground may be only a thin crust above extremely hot water.

What rises from Old Faithful is the visible moment in a much larger cycle, a continuing interaction among water, heat, rock, pressure, and time beneath Yellowstone.

EXECUTED PROJECT OUTPUT

Internal Validation Note

NPN-KNW-001, version 1.0, was used as the factual source, and NPN-GOV-001, version 1.0, was applied. The narration covers the required hydrothermal context, geyser process, Old Faithful limitation, stable safety statement, and concluding connection. No unsupported factual addition or unresolved Knowledge gap was detected. Current eruption predictions were not included. Human editorial approval remains required before publication.

Runtime execution complete. Human editorial review required.

What Runtime Changed, and What It Did Not

The narration and validation note form one completed Project Output. The narration contains no new research. Its factual content came from NPN-KNW-001, while Governance supplied the applicable quality, safety, and approval boundaries. Runtime changed the organization, level of explanation, tone, length, and form so the same accepted information would work for a listener.

Part of the result	Architectural responsibility
Hydrothermal and geyser explanation	Knowledge Framework
Old Faithful context and misconception boundary	Knowledge Framework
Stable safety statement	Knowledge and Governance Frameworks
Audience, duration, tone, and spoken format	Runtime variables
Narration sequence and completion conditions	Runtime task and Output Requirements
Validation note and publication review	Runtime and Governance Frameworks

The mapping above is useful during review. If the tone is wrong, the Runtime settings need attention. If a required fact is missing from the data engine, the problem belongs upstream. If a recurring safety rule is absent, Governance needs revision. The architecture helps the project fix the owner of the problem instead of tossing another sentence into an increasingly mysterious prompt.

8.11 Second Execution: Change the Run, Not the Knowledge

Same Knowledge, Different Moment

Now suppose the same project needs a much shorter welcome for listeners entering a visitor center. Research does not need to begin again. NPN-KNW-001 does not need to be rewritten. Governance remains the same. The facts have not changed; only the listening situation has.

Only the Runtime values and immediate task change. The audience setting becomes visitor-center arrival. The target duration becomes approximately forty seconds. The target length becomes 85 to 105

words. The task becomes a concise spoken introduction that prepares visitors to understand what they are about to see. The revised values produce a different execution from the same governed Knowledge.

SECOND RUNTIME EXECUTION

Runtime variation: NPN-RUN-001, run 002

Unchanged components: NPN-KNW-001, version 1.0, and NPN-GOV-001, version 1.0

Changed values: Visitor-center arrival; approximately forty seconds; 85–105 words; concise spoken welcome

Short Narration Draft

Old Faithful's story begins below your feet. Rain and snow move into cracks in the rock, where heat from Yellowstone's magmatic system warms the water. In a geyser, narrow places in hidden underground plumbing hold heat and pressure until water and steam are forced to the surface.

Old Faithful is predictable, but it is not clockwork and does not erupt exactly every hour. As you explore, stay on boardwalks and designated trails and never touch thermal features or their runoff. What you see here is one visible part of a much larger system of water, heat, rock, pressure, and time beneath Yellowstone.

This Is Reconfiguration, Not Amputation

The second result is not a compressed copy created by deleting random sentences until the word count surrenders. Its Runtime settings identify a different listening moment and a different definition of completion. The Knowledge remains stable while the execution adapts around purpose, audience, time, and format.

This short and predictable variation can also be delivered in one response. If a future Runtime variation requests a much larger deliverable, the same Sequential Output Requirements will divide it at natural points and preserve the remaining work through continuation stops. The delivery control scales with the assignment rather than forcing every assignment into the same ceremony.

Change the run without changing what the project knows.

8.12 Boundary Test: Ask for What Knowledge Does Not Contain

Ask for the One Thing the Repository Refuses to Guess

A working system should also reveal what happens when the assignment exceeds its data engine. Imagine changing the task to: "Tell visitors exactly when Old Faithful will erupt next." This sounds like a Runtime request, but the required current information does not exist inside NPN-KNW-001.

NPN-KNW-001 explicitly excludes current eruption predictions. NPN-GOV-001 prohibits unsupported claims and requires visible limitations. A compliant Runtime execution therefore does not guess, search without permission, or produce a confident time from general model knowledge. Confidence would not turn the missing information into approved Knowledge; it would merely make the guess wear a nicer suit.

RUNTIME BOUNDARY TEST

Test request: Tell visitors exactly when Old Faithful will erupt next.

Available Knowledge: NPN-KNW-001, version 1.0

Applicable Governance: NPN-GOV-001, version 1.0

Runtime Response

This request cannot be completed from the supplied Knowledge Repository. NPN-KNW-001 intentionally excludes current eruption predictions and states that Runtime must not generate them from the retained document. No visitor-facing prediction has been produced. Supply an authorized current National Park Service information source or initiate an approved Research step if this capability is required.

A Useful "No" Is Better Than a Confident "Maybe"

The response is useful because it identifies the boundary and the next responsible action. If current predictions become a legitimate project requirement, the system can authorize an appropriate current source or Research process and define how that information may enter execution. Runtime should not quietly manufacture a capability that the architecture never supplied.

8.13 Put Feedback Where It Belongs

Not Every Problem Is a Prompting Problem

The three executions make feedback easier to diagnose. A full narration, a shorter variation, and a boundary refusal each reveal different parts of the architecture. Not every weak output is a prompting problem, and not every correction belongs in Runtime. The review should ask which responsibility actually owns what went wrong.

What the review discovers	Where the correction belongs
The narration is too long, technical, formal, or promotional	Runtime variables or Output Requirements
A necessary fact was never researched	Research Framework → human review and acceptance → Knowledge Repository update
An accepted fact is outdated or incorrect	Research Framework → human review and acceptance → Knowledge Repository revision
A recurring quality, safety, or approval rule is missing	Governance Framework
The correct documents were named but not supplied	Runtime integration
The public draft violates an existing standard	Runtime revision under Governance validation

This is one of the practical advantages of separating responsibilities. Instead of adding another instruction to an increasingly tangled prompt, the project can revise the component that owns the problem.

Correcting the architecture once can improve many later executions; patching each output individually only teaches the project how to repeat the same repair.

Revise the responsibility that owns the failure.

8.14 Reader Checkpoint

Five Questions Before We Close the Loop

Is the Runtime Framework one prompt? It may be delivered as one complete AI request, but that request is assembled from complete maintained Knowledge, current variables, task instructions, output requirements, Sequential Output Requirements, and Governance controls stated within the request.

Is naming a Knowledge file enough? No. Attach the Knowledge artifact, embed or paste its contents into the working context, or retrieve it from a source the AI can actually access. An identifier provides traceability; it does not supply the contents.

Can Runtime use general model knowledge? Only when the framework and Governance permit it. NPN-RUN-001 restricts factual content to NPN-KNW-001 so the origin and status of the narration remain controlled.

Does a Runtime output automatically become Knowledge? No. The output is a project deliverable. Any new factual material must pass through the appropriate human-involved Research and acceptance process before becoming maintained Knowledge.

What should happen when Knowledge is insufficient? Runtime should identify the gap and route it to the responsible source, Research, Knowledge maintenance, Governance, or an authorized external input, rather than filling it invisibly.

8.15 From Runtime to the Complete System

The Architecture Has Now Done Real Work

The National Park Narration system has now completed the full architectural path. Governance established the conditions. Research developed and documented the evidence. Knowledge preserved the accepted result. Runtime applied that data engine to a specific assignment and produced a governed Project Output. The architecture is no longer a diagram; it has performed useful work.

The system also demonstrated reuse, proportional delivery, and visible limitation. New Runtime values produced a different narration without rebuilding Knowledge. A short predictable output remained in one response, while the architecture retained protection for larger work. A request for unavailable information stopped at a clear boundary instead of becoming a polished guess. With all four frameworks operating together, Chapter 9 will clarify what DCPF is, what it is not, and how much of the architecture a project actually needs.

[Return to Table of Contents](#)

Preparing to Work with DCPF

DCPF is an experimental approach that has demonstrated promise but has not been broadly validated. Test and validate your implementation within your own environment before considering real-world or production use.

Congratulations, you have completed the architectural portion of the book. From this point forward, the emphasis shifts from guided explanation to practical use. You will begin applying DCPF workflows and framework artifacts, then gradually adapt them into a working system of your own.

Before the practicums begin in the next chapter, let us clarify what DCPF requires, and what it does not. Then you can prepare a simple working environment without turning the setup itself into a project.

9.1 What DCPF Is

DCPF is a way to organize sustained collaboration between a human and an AI system.

Rather than relying entirely on one conversation or the model's temporary context, DCPF places important instructions, knowledge, standards, and procedures into reusable external artifacts that you control, refine, and preserve.

DCPF shifts more control and decision-making back to humans. The AI may research, organize, interpret, draft, and review, but the human determines what to seek and what to accept, preserve, revise, or reject.

DCPF is adaptable. A small personal project may use only a few documents and a basic folder. Larger projects can add repositories, specialized workflows, approval procedures, or automation when the work requires them.

The architecture grows because the work requires it, not because complexity appears impressive.

9.2 What DCPF Is Not

DCPF is not a software application, AI model, or proprietary platform. It does not replace people, organizational roles, or oversight. Its purpose is to increase human direction, participation, and accountability in AI-assisted work.

It is not one enormous prompt containing every instruction, source, rule, and task. Separating those responsibilities is one of the reasons the architecture exists.

DCPF is not a fully autonomous system. It does not remove the need for your judgment or give the AI authority to decide which information becomes authoritative without your review.

It is not a rigid method that must look identical in every project. The same principles can be expressed through ordinary documents, shared folders, databases, or more advanced technical systems. DCPF's framework skeletons help you organize; you fill in the details and own the results.

You do not need to memorize the framework or build its largest possible version before beginning. The artifacts are working tools, and even small versions can guide substantial work. Length is not the measure of a strong artifact; usefulness and reliability are what matter.

For the introductory practicums, the smallest useful implementation is usually the best place to start.

9.3 Does This Work Need DCPF?

DCPF is designed for work that benefits from persistence, separation of responsibilities, review, and reuse. It is not necessary for every AI interaction. Before creating artifacts or folders, ask whether anything important needs to remain dependable after the current conversation ends.

Use ordinary AI interaction when the work is disposable. A quick factual question, a simple explanation, disposable brainstorming, a straightforward rewrite, or a one-time transformation may not need a DCPF project when no important evidence, standard, decision, Accepted Knowledge, workflow, or output must be preserved for later use. DCPF should not be added merely because AI is involved.

Use Minimum Viable DCPF when continuity or control matters but the project is still small. DCPF becomes useful when research must be reviewed before reuse, accepted information must survive the conversation, recurring instructions or Governance controls must remain dependable, or a later Runtime task needs to operate from maintained Knowledge. In that situation, use the smallest artifact structure that keeps the necessary responsibilities explicit.

Use a more developed DCPF implementation when the work creates a real need for additional structure. Multiple workstreams, repeated Research or Runtime executions, several collaborators, maintained project knowledge, versioned Governance, formal review, security, retention, release, or audit requirements can justify clearer separation and stronger maintenance practices. Add that structure because the project needs it, not because a larger implementation appears more complete.

A simple decision test: If the work does not need to survive the conversation, DCPF may not be necessary. If the work, Knowledge, standards, decisions, or workflows need to survive the conversation and remain dependable, DCPF becomes increasingly useful.

Choosing DCPF is therefore not an all-or-nothing decision. The better question is: What is the smallest DCPF implementation that responsibly supports this work? The next section answers that question for a small project.

Because DCPF is experimental, that decision should also account for risk. Begin with controlled, low-risk testing in a domain you understand, then expand only after your own evidence and validation support the next step.

9.4 Minimum Viable DCPF: Start with the Smallest Useful Structure

Minimum Viable DCPF does not remove any of the four framework responsibilities. It uses the smallest artifact structure that keeps Research, Accepted Knowledge, Runtime, Governance, and human review explicit enough for the project.

For a first low-risk project, start with a Research Request that carries the requirements needed for the investigation; review the Research Output and preserve the Accepted Knowledge in a Knowledge Repository; create a Runtime Request that carries the applicable requirements; then review the Project Output. That is a complete small DCPF cycle even when no standalone Governance artifact has been created.

Governance is active in this setup because its operating elements are present where decisions and AI work occur. As testing reveals recurring standards, consolidate them into a maintained Governance artifact when a stable project-wide source improves consistency, reuse, versioning, or oversight. Organizations with established policies may instead begin by translating those policies into the controls used by the working artifacts.

Begin with a Simple Working Environment

You do not need specialized software for the exercises that follow. A word processor, an ordinary folder on your computer, and an AI system capable of reading the supplied materials and performing the research required by the exercise are enough.

1. Create a primary folder for your DCPF projects. Inside it, create a separate folder for the first practicum.

A simple structure might look like this:

```
DCPF Projects
├── Project Name
│   ├── Frozen Frameworks
│   ├── Working Artifacts
│   ├── Research Outputs
│   ├── Knowledge Repository
│   ├── Project Outputs
│   └── Archive
```

This is the Chapter 9 starter workspace. Use it for the first practicums and for small projects as long as it remains easy to see where requests, outputs, Accepted Knowledge, and current working files belong. Appendix C, Recommended DCPF Workspace Organization, is the next-step scaling guide, not a prerequisite for beginning DCPF. Move to one of its more structured layouts when the starter workspace

begins to hide information, multiple workstreams or collaborators appear, or review, governance, release, security, retention, or audit needs require clearer separation.

2. Keep the original framework files unchanged whenever possible. In my own projects, I store these originals in a Frozen Frameworks folder and create working copies before entering project-specific information.
3. Keep requests and outputs separate. A Research Request is not the Research Output it produces, and a Runtime Request is not the final Project Output. These distinctions make the project easier to inspect and improve.
4. Save important information outside the AI conversation. The documents you preserve become the durable record of the project.

9.5 Supplying DCPF Artifacts to AI

An AI system cannot reliably use information it cannot access. In DCPF, identifying an artifact and supplying its contents are separate actions.

Use one of three routes to supply a required artifact:

Attach the artifact: Upload the required document directly into the AI interaction.

Embed or paste the contents: Place the complete required content inside the current request, artifact, or working context.

Retrieve from an accessible source: Use a connected repository, file system, database, or other source only when the AI can actually retrieve and read the required content.

Before execution, confirm that the AI can read the complete required content through the route you selected. A filename, artifact ID, prior-chat reference, folder location, or link label does not supply the contents by itself. When a project uses maintained Knowledge, Runtime must receive the approved content itself. The introductory practicums emphasize attachment and embedding or pasting because those routes work in many ordinary interfaces; retrieval is equally valid when the selected AI environment can actually retrieve and read the required artifact.

9.6 How the Practicums Will Work

The practicums move gradually from explanation into action. You will begin with a defined project and supplied artifacts, prepare the necessary files, execute a focused request, review the result, and preserve the approved information or output in the appropriate location.

The practicums intentionally embed essential Governance rules inside the Research and Runtime requests. That is not a shortcut around Governance; it demonstrates how a small DCPF project can operationalize Governance before a separate maintained Governance artifact becomes useful.

The early exercises are intentionally modest. Later practicums will introduce additional coordination, reusable knowledge, and different kinds of deliverables. The goal is not a perfect first implementation, but to learn the workflow and recognize where your judgment remains essential.

9.7 Before You Begin

Before moving to Chapter 10, confirm that:

You have created a folder for the first practicum.

You have a place to preserve the original framework files.

You know where research, knowledge, and project outputs will be saved.

You have confirmed that the AI can read the required documents through the route you plan to use and can perform external research when the exercise calls for it. Features may vary by model, operating mode, and subscription tier.

You understand that generated information must be reviewed before it is accepted.

You are prepared to begin with the Chapter 9 starter workspace and expand only when the project develops a real organizational need. Appendix C provides the scaling path when that time comes.

In the next chapter, you will use DCPF to complete a small research project, preserve the resulting Knowledge, and apply that Knowledge to a practical assignment.

[Return to Table of Contents](#)

Building a General-Purpose Research Assistant

DCPF is an experimental approach. This practicum is a training environment, not evidence of production readiness. Validate any adapted implementation within your own environment before real-world use.

From Architecture to Practice

This practicum turns the DCPF architecture into a working project. You will create a Research Request, execute it with AI, review the Research Output, and preserve the Accepted Knowledge in a Knowledge Repository. You will then create a Runtime Request, use the repository to complete a practical assignment, review the Project Output, and save the approved result. The subject is intentionally straightforward so you can concentrate on the workflow rather than on a specialized field.

No programming is required. You need a word processor and an AI system that can read attachments and reach approved research sources. If the AI cannot access a required source, the request tells it to report the limitation instead of pretending the research is complete.

The working question asks how a household can reduce everyday electricity use through practical, low-cost actions without major renovations. You may complete the example exactly as written or adapt the Research Configuration Variables to a different low-risk question of your own.

This exercise keeps the framework complete while making the first implementation easy to follow.

Process Stages	Your action	Artifact produced
1. Research setup	Define the question and its controls	Research Request
2. Research execution	Have AI conduct and document the investigation	Research Output
3. Knowledge handoff	Review and accept the useful information	Knowledge Repository
4. Runtime execution	Apply the repository to a defined assignment	Project Output
5. Final review	Approve, revise, and preserve the result	Approved Project Output

Part I: Create the Research Request

A prompt such as “Research ways to use less electricity at home” forces the AI to guess the audience, location, depth, sources, budget boundaries, organization, safety limits, and definition of completion. The Research Request makes those decisions visible and reusable. Chapter 6 explains every Research element in depth; this chapter focuses on using them.

The complete, reusable artifact follows. Its current variables create an executable household-electricity example, but the structure can support many general research questions. Adapt the project settings

rather than rebuilding the artifact from scratch. Its Purpose begins with an execution statement so any AI can understand what to do.

The Research Output may fit in one response, but its complete length cannot be predicted reliably across AI models or replacement topics. The request therefore retains the full Sequential Output Requirements. The AI applies the rule automatically whenever the output may exceed practical response length, breaks at natural points, and preserves completion without predetermined section groups.

You will be using the following training Research Request in the first part of this exercise. Additional instructions follow.

LIVE RESEARCH REQUEST

Request: General-Purpose Research Assistant

Request ID: GRA-RES-001

Project: DCPF Practicum, General-Purpose Research

Owner: [Your name]

Version: 1.0

Status: Ready for execution

Output: Research Output for human review, possible acceptance as Knowledge, and preservation in the Knowledge Repository

1. Purpose

Execute this document as a complete Research Request. Research and document the assigned question according to the variables, objectives, approved sources, source priorities, scope, output requirements, source-documentation requirements, Sequential Output Requirements, and governance requirements provided below.

Do not merely review, summarize, or explain this document. Do not create a final guide, plan, recommendation, presentation, or other Runtime deliverable. Produce a complete, clearly written Research Output that the user can review. Accepted information may then be preserved in the Knowledge Repository for this exercise.

Before beginning research, verify that every required input and approved source is accessible. If access is unavailable, stop, report the limitation, and request direction rather than substituting unsupported information.

2. Research Configuration Variables

Research question: Which practical, low-cost actions can help a household reduce everyday electricity use without major renovations? **Audience:** General adult readers, including renters and homeowners. **Intended use:** Support a practical household starter guide. **Location context:** General United States context; identify when climate, housing, utility, or local conditions affect relevance. **Depth:** Practical and foundational, with enough detail to support later application. **Time horizon:** Current as of the execution date. **Budget scope:** No-cost and low-cost actions; exclude major renovations and unrequested brand recommendations. **Tone:** Clear, direct, and nontechnical. **Examples:** Required when they improve understanding.

3. Research Objectives

Define the question. Explain what reducing household electricity use means in this project and distinguish electricity use from broader household energy use when the distinction matters.

LIVE RESEARCH REQUEST

Map the relevant uses. Identify the household systems, appliances, settings, maintenance conditions, and everyday habits most relevant to the question without assuming that every home uses electricity in the same way.

Identify practical actions. Document no-cost and low-cost actions a household can consider, including any prerequisites, limitations, or conditions that affect whether an action is appropriate.

Explain the support. Describe why each major action may matter and indicate the quality, strength, and limits of the available evidence. Do not convert general findings into guaranteed savings.

Compare effort and relevance. Describe relative effort, likely cost category, ease of use, and the household circumstances that make an action more or less relevant without inventing precise rankings or savings estimates.

Preserve variation. Explain how climate, housing type, equipment, occupancy, utility rates, existing maintenance, and user behavior can change results.

Protect safety and authority. Identify actions that require a landlord, utility, licensed professional, manufacturer guidance, or other qualified authority. Do not provide unsafe electrical repair instructions.

Prepare the Knowledge for use. Produce a practical set of priorities, decision questions, and documented qualifications that Runtime can later transform into a reader-facing guide.

4. Approved Information Sources

Use user-provided documents when supplied. For outside research, use original or primary sources when they are available; government energy agencies and public programs; public utilities and qualified state or local energy offices; universities and established extension or educational institutions; relevant professional organizations and standards bodies; peer-reviewed scholarship; and authoritative technical reference works. Manufacturer documentation may be used for a product's documented operation or specifications, but not as independent proof of broad savings claims.

Do not use anonymous content, unsourced summaries, affiliate or advertising material presented as independent guidance, automatically generated source lists, or material whose origin cannot be verified as support for major claims.

5. Source Priority and Conflict Handling

Treat user instructions as controlling the project assignment, but do not treat a user-provided factual claim as automatically proven. For evidence, prefer sources that are direct, relevant, transparent about methods or conditions, current enough for the claim, and produced by qualified authors or institutions.

When credible sources disagree, compare their definitions, evidence, methods, dates, geographic assumptions, household conditions, and intended audiences. Do not choose a source merely because it appears first or states its conclusion most confidently. Report a meaningful disagreement when the available evidence does not justify resolving it.

LIVE RESEARCH REQUEST

6. Research Scope

Include household electricity-use categories; no-cost and low-cost actions; behavior, settings, maintenance, and equipment-use factors; relative effort and cost categories; household and regional variation; safety boundaries; practical decision questions; important limitations; and the source history needed for later review.

Exclude individualized energy audits, electrical repair instructions, major renovation planning, brand-specific shopping lists, precise savings guarantees, unsupported cost estimates, and advice that assumes one climate, utility, housing type, or ownership arrangement applies to everyone. Do not create the final Runtime guide during Research.

7. Output Organization

Organize the Research Output under these headings: Research Question and Definitions; Major Household Electricity Uses; No-Cost Actions; Low-Cost Actions; Maintenance and Setup Factors; Effort, Cost, and Likely Relevance; Household and Regional Variables; Safety and Professional Boundaries; Recommended Priorities and Decision Questions; Limits and Uncertainty; Summary; and Source Register.

Use concise examples to make the findings concrete. Clearly identify examples as illustrations rather than documented events unless a source confirms that they occurred. Keep broad guidance separate from claims that depend on a particular home, product, climate, utility rate, or local program.

8. Research Source Documentation

Support major factual claims with source citations close enough to the claim for a reader to identify the basis. Provide a Source Register that includes the author or organization, title, publication or update date when available, direct link or document identifier, and the section or page used when that detail is useful.

Never invent an author, title, date, quotation, link, publication detail, savings figure, cost, or program requirement. Distinguish source-supported findings from the AI's explanatory wording and from examples created only to illustrate a concept.

9. ESSENTIAL: Sequential Output Requirements

PURPOSE

When the requested Research Output may exceed what can be comfortably delivered in a single response, divide it into multiple sequential parts while preserving structure, completeness, detail, source documentation, and continuity.

SEQUENTIAL OUTPUT RULE

LIVE RESEARCH REQUEST

- Apply this requirement automatically whenever the complete output may exceed practical response length or its length cannot be reliably predicted.
- Do not compress, summarize, abbreviate, or omit required research merely to fit within one response.
- Favor completeness over minimizing the number of responses.
- Break the output at a natural point. Do not force the Research Output into predetermined sets or fixed section groupings.
- At the end of each nonfinal part, state that additional output remains, request permission to continue, and stop.
- After permission is received, continue from where the previous part ended until the complete Research Output has been delivered.
- The final part must clearly state that the requested Research Output is complete.

10. Governance and Final Quality Review

The output must be accurate, complete, readable, practical, current enough for the assigned question, and appropriate for general adult readers. Use clear language without talking down to the reader. Avoid unsupported savings claims, invented costs, promotional language, hidden source conflicts, unsafe instructions, and advice that treats every household as identical.

Before declaring completion, confirm that every Research Objective is answered; every required output section is present; major factual claims are supported; no source information, savings figure, cost, or program detail has been invented; household and regional variation is visible; safety and authority boundaries are preserved; important uncertainty or credible disagreement is visible; the content remains within scope; and the Sequential Output Requirements were followed whenever the output required more than one response.

If a condition fails, correct the problem before delivering the completion statement. If it cannot be corrected within the approved sources and scope, report the limitation clearly.

Before moving on, pause over the nine standard responsibilities. Each protects a different failure point. You may combine closely related sections in a shorter request, but omitting their responsibilities can change the research, the evidence, or the completeness of the result.

Purpose. The clear statement tells the AI what to execute and prevents the wrong deliverable.

Research Configuration Variables. Makes the current question, audience, use, location, depth, budget, and tone visible instead of leaving them for the AI to guess.

Research Objectives. Defines required coverage and the standard for completeness.

Approved Information Sources. Limits the evidence to qualified, traceable material.

Source Priority and Conflict Handling. Guides comparisons and preserves unresolved disagreement.

Research Scope. Sets the required depth and keeps the investigation within bounds.

Output Organization. Prepares the findings for human review and later Knowledge use.

Research Source Documentation. Keeps major claims traceable and exposes invented sources or details.

Sequential Output Requirements. Prevents compression or omission and defines continuation and completion.

Governance and Final Quality Review. Checks accuracy, coverage, boundaries, sources, safety, uncertainty, and limitations.

You may simplify the document's shape, but you should not delete the responsibility each section carries.

Prepare and Execute the Research Request

Step 1: Create and Check the File

Copy the artifact into a clean word-processing document. Replace “[Your name]” with your name, confirm the request ID and version, and save the file as “GRA-RES-001 General-Purpose Research Request v1.0.docx.” Review the configuration variables. Keep all nine responsibilities even if you change the research question or later consolidate headings.

Step 2: Confirm Research Access

Use an AI system and operating mode that can read the attachment and access the approved external sources. Mode names and capabilities vary by product. Where a thinking or reasoning control is available, use Medium or higher for this practicum. Do not use a fast-response mode such as Instant, Light, or Low unless you have confirmed that it can perform the required external research. If live research is unavailable, attach approved source documents directly. If the required sources still cannot be accessed, stop and report the limitation.

Step 3: Start a Clean Chat and Attach the Request

A clean working context reduces interference. On free or limited tiers, long runs may face rate limits or context trimming, so use this chat only for Part I and start another for Part III. Attach the complete request and wait until it is available; naming a file does not supply its contents. Before beginning, confirm that the Research Request is available and that the Medium-or-higher setting selected in Step 2 remains active.

Step 4: Begin Execution

Send one instruction:

Please execute the attached Research Request.

The prompt starts the work; the attachment controls it. If the AI explains the document instead, reply: “Execute the Research Request. Do not summarize or explain the attachment.”

Step 5: Complete the Sequential Output

The AI may deliver the complete Research Output in one response when it fits comfortably. If additional output remains, it must request permission and stop. Save each returned part and reply “Continue” when you are ready. Repeat until the AI clearly states that the requested Research Output is complete, then combine the parts in their original order.

Part II: Review and Save the Knowledge Repository

The AI's quality check does not replace human review. Research Output does not automatically become Knowledge; human review and approval are required before accepted information is preserved in GRA-KNW-001. Compare the Research Output with GRA-RES-001, then sample important citations to confirm that the sources exist and support the nearby claims. Chapter 7 explains Knowledge preservation in greater depth.

Review check	Question	If the check fails
Completeness	Are all objectives and required sections present?	Request the missing material.
Evidence	Are major claims supported by qualified, traceable sources?	Add support, correct the claim, or remove it.
Citation integrity	Do sampled links and source details support the nearby claim?	Correct the citation and recheck related claims.
Context and limits	Are household, regional, cost, and uncertainty conditions visible?	Restore the missing qualifications.
Scope and safety	Does the output remain within the assigned boundaries and avoid unsafe instructions?	Remove the drift or route the issue to a qualified authority.
Repository usefulness	Can Runtime use the information without repeating Research?	Clarify or reorganize the affected section.

If several checks fail, use this correction prompt:

Review the Research Output against GRA-RES-001. Identify every unmet requirement, then return a clean and complete revision with only the necessary changes. Do not change the approved sources, scope, configuration variables, or output organization. Follow GRA-RES-001's Sequential Output Requirements while delivering the revision.

Review the revision. Confidence is not proof of correction.

Preserve the Approved Knowledge

When the output passes review, copy the complete approved Research Output into a new document. Preserve the headings, qualifications, citations, Source Register, safety boundaries, household and regional variables, and useful detail. Do not reduce the information to a short summary; this repository will be the Runtime data engine. Whenever possible, copy the approved Research Output directly from

the AI response or an editable document. If you must convert it from PDF, check the headings, paragraph order, bullets, tables, and citations before approving the repository.

Add this header:

GENERAL-PURPOSE RESEARCH KNOWLEDGE REPOSITORY

Repository ID: GRA-KNW-001

Project: DCPF Practicum, General-Purpose Research

Derived from: GRA-RES-001, version 1.0

Created: [Date]

Reviewed by: [Your name]

Status: Approved for practicum use

Content: Complete reviewed Research Output

Save the file as “GRA-KNW-001 General-Purpose Research Knowledge Repository v1.0.docx.” Keep GRA-RES-001 as a separate file. The request records how the information was developed; the repository contains what you accepted for reuse.

In a small project, this handoff should remain simple. The approved Research Output already has the organization and source history needed by the repository. If an important error is discovered later, correct the underlying Research, review the change, and create a new repository version.

Part III: Create the Runtime Request

Research discovers and develops evidence. Knowledge preserves the accepted intelligence. Runtime now applies that approved Knowledge to a defined assignment: a short, practical household electricity guide.

The finished work will be *Using Less Electricity at Home: A Practical Starter Guide*. It will read like a concise reader guide, not like Research notes or an explanation of DCPF. The direct handoff is easy to see: the Research Request develops reliable information, the Knowledge Repository preserves it, and the Runtime Request transforms it into an audience-ready application. Chapter 8 explains every Runtime element and the distinction between retained Knowledge and a current execution.

This first practicum intentionally embeds its essential Governance rules inside the Research and Runtime requests. Governance is therefore active even though you are not asked to manage a separate Governance artifact. Chapter 5 explains how recurring project-wide controls can later be consolidated into a maintained Governance artifact.

Choose How to Supply the Knowledge Repository

The Knowledge Repository remains a separate maintained project artifact even when its contents are embedded in the Runtime Request. DCPF supports three general supply routes: attachment; embedding or pasting; and retrieval from a source the AI can actually access. To keep this practicum platform-

neutral, the table below uses attachment and embedding; Both combines those two routes for redundancy.

Delivery method	Advantages	Trade-offs
Embed the complete repository in the Runtime Request	Creates one self-contained execution file, keeps the Knowledge close to the instructions, and reduces the chance that an attachment will be inaccessible or overlooked.	Makes the Runtime file longer and duplicates the repository content. The embedded copy must be checked against the approved repository whenever either file changes.
Attach the repository as a separate document	Keeps the Runtime Request shorter, preserves clean version control, and makes one repository easy to reuse across several Runtime Requests.	Requires the AI to access and attend to multiple files. A missing, unreadable, very long, or weakly identified attachment can lead to an incomplete execution.
Use both methods	Provides a self-contained Runtime file while retaining the approved repository as a visible companion document. This can help when attachment handling is uncertain.	Repeats the same material, uses more context, and creates a conflict risk if the copies do not match. Both copies must carry the same repository ID and version.

For this practicum, embedding the complete repository is usually the simplest choice. Keep GRA-KNW-001 as its own approved file, then paste an exact copy into Appendix A of the Runtime Request you will execute. If you prefer a separate attachment, the request also supports that method. If your environment supports connected retrieval, you may adapt the delivery method, but first confirm that the AI can retrieve and read the complete approved repository. Whichever method you choose, identify it in the Runtime metadata and never rely on the repository filename alone.

A short, predictable Runtime assignment may fit in one response. A longer or less predictable Project Output may require more than one, depending on the AI model and its settings. The Runtime Request retains the full Sequential Output Requirements, applies the rule automatically, and breaks delivery at natural points, so you do not have to predict the practical response limit or define response groups in advance.

LIVE RUNTIME REQUEST

Request: Household Electricity Starter Guide

Request ID: GRA-RUN-001

Project: DCPF Practicum, General-Purpose Research

Owner: [Your name]

Version: 1.0

Status: Ready for execution

Input Knowledge: GRA-KNW-001, version 1.0, supplied in full with this request

Knowledge delivery: [Choose one: Embedded in Appendix A / Separate attachment / Both]

Output: GRA-OUT-001, *Using Less Electricity at Home: A Practical Starter Guide*

LIVE RUNTIME REQUEST

1. Purpose

Execute this document as a complete Runtime Request. Use the complete General-Purpose Research Knowledge Repository supplied by the delivery method identified above as the sole factual data engine for the reader-facing guide. Follow the configuration variables, Input Knowledge requirements, Task Definition, Output Requirements, Sequential Output Requirements, delivery instructions, and governance requirements in this document.

Before drafting, locate and read the entire repository. Do not rely on its filename, metadata, or a partial extract. Do not merely review, summarize, or explain this request. Do not conduct new Research or add facts, actions, savings figures, costs, or recommendations from general model Knowledge. If the complete required Knowledge is missing, inaccessible, internally inconsistent, or insufficient, identify the problem and request direction rather than inventing information.

2. Runtime Configuration Variables

Subject: Practical no-cost and low-cost ways to reduce household electricity use. **Audience:** General adult readers, including renters and homeowners. **Objective:** Turn approved Research Knowledge into a useful starting guide without promising individualized savings. **Format:** A concise guide with clear headings, prioritized actions, decision questions, and a first-week checklist. **Point of view:** Second person. **Tone:** Clear, practical, encouraging, and nontechnical. **Target length:** 1,000–1,400 words. **Examples:** Brief household illustrations supported by the repository. **External research:** Not authorized. **Publication status:** Draft pending human review.

3. Input Knowledge

Use GRA-KNW-001, version 1.0, as the sole factual data engine. For this practicum, supply the complete repository by embedding it in Appendix A, attaching it as a separate document, or using both methods according to the Knowledge delivery field. The identifier alone does not supply the contents.

Use only the findings, actions, conditions, comparisons, examples, source history, and qualifications contained in the repository. You may reorganize and simplify the material for the audience, but do not introduce outside facts as established Knowledge. If both embedded and attached copies are supplied, verify that their repository IDs and versions match. If their contents conflict, stop and request direction. If any required copy is missing, inaccessible, incomplete, or insufficient to produce the guide responsibly, stop and report the limitation.

4. Task Definition

Open with purpose. Explain what the guide will help the reader do and make clear that actual results depend on the household, equipment, climate, utility conditions, maintenance, and behavior documented in the repository.

Establish a starting point. Briefly explain the categories of household electricity use that matter to the guide without turning the introduction into Research notes.

Prioritize practical action. Present no-cost actions first, then low-cost actions, while preserving the repository's conditions, relative effort, and limits.

Connect actions to circumstances. Show readers how housing type, ownership, equipment, climate, occupancy, and local conditions can affect which actions deserve attention.

LIVE RUNTIME REQUEST

Preserve safety and authority. Clearly identify actions that require a landlord, utility, manufacturer guidance, licensed professional, or other qualified authority. Do not convert Research boundaries into repair instructions.

Create a usable first step. Provide a short first-week checklist and decision questions that help readers choose responsible actions from the repository without inventing measurements or savings.

Close with perspective. Encourage readers to observe results, revisit assumptions, and seek an individualized audit or qualified help when the repository indicates that general guidance is insufficient.

5. Output Requirements

Title the Project Output Using Less Electricity at Home: A Practical Starter Guide. Organize it under these headings: Introduction; Before You Begin; Start with No-Cost Actions; Consider Low-Cost Actions; Maintenance and Setup; Match Actions to Your Household; Safety and Professional Help; A First-Week Checklist; Questions for Your Next Step; Closing Perspective; and Internal Validation Note.

The main guide must read as one coherent work, not as Research notes or a list of disconnected findings. Use plain language and useful transitions. Do not promise a particular savings amount, imply that one action fits every household, or present an illustrative example as a documented result. Do not discuss DCPF, the Research Request, or the Knowledge Repository in the reader-facing guide. The repository's source history does not need to appear as citations unless the supplied Knowledge specifically requires attribution.

The Internal Validation Note must identify the Runtime Request and Knowledge Repository versions used; state how the repository was delivered; confirm that the complete repository was supplied and read before drafting; confirm that every required section was completed; state whether any outside facts, actions, savings figures, costs, or unsupported additions were detected; identify unresolved gaps; and state that human approval remains required.

6. ESSENTIAL: Sequential Output Requirements

PURPOSE

When the requested Project Output may exceed what can be comfortably delivered in a single response, divide it into multiple sequential parts while preserving structure, completeness, detail, formatting, and continuity.

SEQUENTIAL OUTPUT RULE

- Apply this requirement automatically whenever the complete output may exceed practical response length or its length cannot be reliably predicted.
- Do not compress, summarize, abbreviate, or omit required work merely to fit within one response.
- Favor completeness over minimizing the number of responses.
- Break the output at a natural point. Do not force the Project Output into predetermined sets or fixed section groupings.

LIVE RUNTIME REQUEST

- At the end of each nonfinal part, state that additional output remains, request permission to continue, and stop.
- After permission is received, continue from where the previous part ended until the complete Project Output has been delivered.
- The final part must clearly state that the requested Project Output is complete.
- A short, predictable Project Output may be delivered in one response.

7. Validation and Human Review

Apply the embedded governance rules in this section throughout execution. The output must remain faithful to GRA-KNW-001, accurate, clear, practical, appropriately cautious, and suitable for the stated audience. Preserve the repository's qualifications, source-based boundaries, safety limits, household variation, and uncertainty.

Do not invent facts, actions, savings estimates, costs, program details, product recommendations, or safety instructions. Do not conduct unauthorized Research, hide repository conflicts, overstate likely benefits, or imply that general guidance replaces a qualified audit or professional service. If an instruction conflicts with the available Knowledge or prevents responsible completion, pause and request direction.

Before delivery, verify that the guide remains within scope and target length, uses GRA-KNW-001 as its sole factual source, contains every required section, fits the stated audience and tone, preserves relevant conditions and safety boundaries, avoids unsupported additions and guarantees, and follows the Sequential Output Requirements whenever more than one response is needed.

Appendix A: Embedded Knowledge Repository

Use this appendix when Knowledge delivery is set to Embedded in Appendix A or Both. Paste the complete, approved contents of GRA-KNW-001, version 1.0, below this line. Preserve its repository header, source history, household and regional qualifications, safety boundaries, uncertainty, and citations.

[BEGIN GRA-KNW-001, VERSION 1.0]

[Paste the complete approved Knowledge Repository here.]

[END GRA-KNW-001, VERSION 1.0]

If Knowledge delivery is set to Separate attachment, delete the BEGIN marker, placeholder, and END marker. Replace the entire Appendix A repository block with this statement: "GRA-KNW-001, version 1.0, is supplied as a separate attachment. Appendix A is not used."

Prepare and Execute the Runtime Request

Step 1: Create the Runtime File

Copy the Runtime artifact into a new document, replace “[Your name]” with your name, and choose a Knowledge delivery method. If you embed the repository, paste the complete approved GRA-KNW-001 into Appendix A and confirm that its ID and version match the metadata. Save the file as “GRA-RUN-001 Household Electricity Guide Runtime Request v1.0.docx.” Confirm that the subject, audience, format, tone, length, and output name are correct. Keep all six Runtime responsibilities; Chapter 8 explains the risks of omitting them.

Step 2: Start a Clean Chat and Supply the Execution Materials

Supply the files that match the delivery method recorded in GRA-RUN-001:

Embedded: Attach the completed Runtime Request. Confirm that Appendix A contains the complete repository.

Separate attachment: Attach the Runtime Request and GRA-KNW-001 as two documents.

Both: Attach the completed Runtime Request and the separate GRA-KNW-001. Confirm that the embedded and attached copies have the same ID and version.

Do not attach an empty Appendix A as if it contained the repository. Before execution, confirm that the AI can open and read every required file supplied by the selected route. Because Runtime must read and apply the complete repository, use Medium or higher reasoning when that control is available. External research remains unauthorized.

Step 3: Begin Execution

Send this instruction:

Please execute the attached Runtime Request using the complete Knowledge Repository supplied by the delivery method identified inside it.

The AI may deliver the complete guide in one response when it fits comfortably. If it states that additional output remains, save the returned part and reply “Continue” when you are ready. Repeat until the AI clearly states that the requested Project Output is complete. If the AI reports that Knowledge is missing, inaccessible, incomplete, or conflicting, correct the packaging problem and begin again rather than asking it to fill the gap.

Step 4: Preserve the Unreviewed Output

If the guide arrived in multiple parts, combine them in their original order inside a temporary working document or retain them together in the chat while reviewing. Label the complete output Draft, Not Yet Approved. Do not replace the Knowledge Repository with this output; Runtime has produced an application of Knowledge, not new accepted Knowledge.

Part IV: Review and Preserve the Project Output

Compare the Project Output with GRA-RUN-001 and GRA-KNW-001. This review asks two questions: Did the AI produce the requested guide, and did it remain within the supplied Knowledge and authority?

Review check	Question	If the check fails
Task completion	Are all required guide sections present?	Request the missing work.
Knowledge fidelity	Does the output use GRA-KNW-001 without adding outside facts or actions?	Remove the addition or authorize new Research.
Audience fit	Can renters and homeowners understand the guide without specialized knowledge?	Simplify the affected passages.
Practical value	Does the guide prioritize usable actions and include the required checklist?	Add or strengthen the practical guidance from the repository.
Coherence	Does it read as a concise guide rather than Research notes?	Improve structure and transitions.
Context and claims	Are household variation, uncertainty, and the absence of guaranteed savings clear?	Restore the qualifications or remove the claim.
Safety and authority	Are professional, landlord, utility, and manufacturer boundaries preserved?	Correct the instruction before approval.
Validation	Does the note identify versions, delivery, full access, unsupported additions, gaps, and human review?	Complete the validation note.

If revision is needed, use this prompt:

Review GRA-OUT-001 against GRA-RUN-001 and GRA-KNW-001. Identify every unmet requirement, then return a clean and complete revised guide with only the necessary changes. Do not conduct Research or introduce facts, actions, savings figures, costs, or recommendations outside the supplied Knowledge Repository. Follow GRA-RUN-001's Sequential Output Requirements while delivering the revision.

Review the revision before approval. If the weakness comes from missing or unreliable household-electricity Knowledge, route it back through Research and update the Knowledge Repository. If it comes from tone, structure, organization, or task wording, revise the Runtime Request. Chapter 8 provides the full feedback-routing explanation.

Approve and Preserve the Project Output

When the output passes review, place this header above the approved content:

APPROVED PROJECT OUTPUT

Output ID: GRA-OUT-001

Project: DCPF Practicum, General-Purpose Research

Produced by: GRA-RUN-001, version 1.0

Knowledge used: GRA-KNW-001, version 1.0

Created: [Date]

Reviewed by: [Your name]

Status: Approved for practicum use

Content: *Using Less Electricity at Home: A Practical Starter Guide*

Save the file as “GRA-OUT-001 Using Less Electricity at Home - A Practical Starter Guide v1.0.docx.” Keep the Research Request, Knowledge Repository, Runtime Request, and Project Output as separate files. Each records a different responsibility.

The Project Output does not automatically become part of the Knowledge Repository. It is a reader-facing work derived from retained Knowledge. If it reveals a genuine knowledge gap, conduct additional Research, review the new findings, and update the repository through the same controlled handoff.

The Completed Practicum

You have now completed the operational DCPF cycle:

**Research Request → Research Output → human review and acceptance → Accepted
Knowledge → Knowledge Repository → Runtime Request → Project Output → human review
→ preservation**

The exercise used four files, two short execution prompts, and no custom software. Research discovers. Knowledge preserves. Runtime executes. Governance guides.

The household-electricity question was an example, not a restriction. Change the Research Configuration Variables, objectives, sources, scope, and Runtime assignment while preserving each architectural responsibility. Chapter 11 keeps the workflow recognizable and adds a more demanding critical-thinking investigation.

[Return to Table of Contents](#)

Building a Critical Thinking Research Assistant

DCPF is an experimental approach. This practicum is a training environment, not evidence of production readiness. Validate any adapted implementation within your own environment before real-world use.

From General Research to Critical Inquiry

Chapter 10 introduced the complete DCPF cycle through a straightforward general research question. This practicum keeps the same architecture but asks Research and Governance to do more demanding work: define critical thinking, evaluate evidence and source credibility, preserve qualifications and disagreement, and prepare that Knowledge for a practical guide. Because the workflow is now familiar, you can focus more attention on judgment and evidence quality.

No programming is required. You need a word processor and an AI system that can read attachments and reach approved research sources. If the AI cannot access a required source, the request tells it to report the limitation instead of pretending the research is complete.

The subject is critical thinking for general use. This is not “beginner critical thinking.” The reader is now completing a second DCPF implementation with a more demanding research assignment.

The DCPF cycle remains familiar; this exercise strengthens how Research and Governance handle judgment, evidence, uncertainty, and competing claims.

Process Stages	Your action	Artifact produced
6. Research setup	Define the investigation and its controls	Research Request
7. Research execution	Have AI conduct and document the investigation	Research Output
8. Knowledge handoff	Review and accept the useful information	Knowledge Repository
9. Runtime execution	Apply the repository to a defined assignment	Project Output
10. Final review	Approve, revise, and preserve the result	Approved Project Output

Part I: Create the Research Request

A prompt such as “Research critical thinking” forces the AI to guess the audience, depth, sources, boundaries, organization, and definition of completion. The Research Request makes those decisions visible and reusable. Chapter 6 explains every Research element in depth; this chapter focuses on using them.

The complete, reusable artifact follows. Adapt its project settings rather than rebuilding it from scratch. Its Purpose begins with an execution statement so any AI can understand what to do.

Because this Research Output may be substantial and its complete length cannot be predicted reliably across AI models, the request includes the Sequential Output Requirements. The AI applies the rule automatically whenever the output may exceed practical response length. If multiple parts are needed, it breaks the Research Output at natural points rather than forcing predetermined section groups. You do not need to estimate the model's limit yourself.

You will use the following Critical Thinking Research Request in the first part of this exercise. Additional instructions follow.

LIVE RESEARCH REQUEST

Request: Critical Thinking Research Assistant

Request ID: CTA-RES-001

Project: DCPF Practicum, Critical Thinking

Owner: [Your name]

Version: 1.0

Status: Ready for execution

Output: Research Output for human review, possible acceptance as Knowledge, and preservation in the Knowledge Repository

1. Purpose

Execute this document as a complete Research Request. Research and document the assigned topic according to the variables, objectives, approved sources, source priorities, scope, output requirements, source-documentation requirements, sequential-delivery protocol, and governance requirements provided below.

Do not merely review, summarize, or explain this document. Do not create a final lesson, article, presentation, or other Runtime deliverable. Produce a complete, clearly written Research Output that the user can review. Accepted information may then be preserved in the Knowledge Repository for this exercise.

Before beginning research, verify that every required input and approved source is accessible. If access is unavailable, stop, report the limitation, and request direction rather than substituting unsupported information.

2. Research Configuration Variables

Research topic: Critical thinking. **Audience:** General readers. **Intended use:** School, work, everyday decisions, problem-solving, and evaluation of information. **Depth:** Practical and foundational, with enough substance to support later applications. **Tone:** Clear, direct, and nonacademic. **Examples:** Required. **Complexity:** Explain important concepts in accessible language without reducing the subject to slogans or oversimplified advice.

3. Research Objectives

Define the concept. Explain critical thinking and distinguish it from intelligence, constant skepticism, arguing, fault-finding, and simply disagreeing with someone.

Explain the process. Describe how a person can clarify a question, identify claims and conclusions, examine evidence, uncover assumptions, consider alternatives, reach a proportionate conclusion, and revise that conclusion when better information appears.

LIVE RESEARCH REQUEST

Evaluate information. Explain practical ways to examine claims, evidence, source credibility, relevance, accuracy, context, and uncertainty.

Identify common obstacles. Address assumptions, confirmation bias, emotional pressure, overconfidence, weak comparisons, false choices, and other common reasoning problems without turning the output into an advanced catalog of logical fallacies.

Show practical use. Provide relatable applications involving school, work, online information, purchases, personal decisions, group discussions, and problem-solving.

Develop useful questions. Create a short set of questions a person can ask when evaluating a claim, decision, or problem.

Explain improvement. Identify habits and practices that help people strengthen critical thinking over time.

Preserve limits and disagreements. Make important qualifications, evidence limits, and credible differences visible instead of forcing every source into one certain conclusion.

4. Approved Information Sources

Use user-provided documents when supplied. For outside research, use original or primary sources when they are available; peer-reviewed scholarship; universities and established educational institutions; relevant professional organizations and standards bodies; government publications and public educational resources; and authoritative reference works. Other public information may be used only when it meets the governance requirements and adds value not reasonably available from a stronger source category.

Do not use anonymous content, unsourced summaries, advertising disguised as instruction, automatically generated source lists, or material whose origin cannot be verified as support for major claims.

5. Source Priority and Conflict Handling

Treat user instructions as controlling the project assignment, but do not treat a user-provided factual claim as automatically proven. For evidence, prefer sources that are direct, relevant, well-supported, current enough for the claim, and produced by qualified authors or institutions.

When credible sources disagree, compare their evidence, definitions, methods, dates, and intended contexts. Do not choose a source merely because it appears first or states its conclusion most confidently. Report a meaningful disagreement when the available evidence does not justify resolving it.

6. Research Scope

Include definitions, core thinking skills, a practical thinking process, evaluation of claims and evidence, source credibility, assumptions and bias, common misconceptions, everyday examples, useful questions, improvement practices, and important limitations.

LIVE RESEARCH REQUEST

Exclude advanced symbolic logic, formal proofs, dense philosophical history, specialized academic disputes, partisan advocacy, and an exhaustive list of fallacies. Introduce a technical term only when it improves understanding, and explain it in ordinary language when it is used.

7. Output Organization

Organize the Research Output under these headings: What Critical Thinking Is; How Critical Thinking Works; Evaluating Claims, Evidence, and Sources; Assumptions, Biases, and Common Reasoning Problems; Critical Thinking in Everyday Situations; Practical Questions and Methods; Strengthening Critical-Thinking Skills; Limits, Qualifications, and Credible Differences; Summary; and Source Register.

Use examples to make the ideas concrete. Clearly identify examples as illustrations rather than documented events unless a source confirms that they occurred.

8. Research Source Documentation

Support major factual claims with source citations close enough to the claim for a reader to identify the basis. Provide a Source Register that includes the author or organization, title, publication or update date when available, direct link or document identifier, and the section or page used when that detail is useful.

Never invent an author, title, date, quotation, link, or publication detail. Distinguish source-supported findings from the AI's explanatory wording and from examples created only to illustrate a concept.

9. ESSENTIAL: Sequential Output Requirements

PURPOSE

When the requested Research Output may exceed what can be comfortably delivered in a single response, divide it into multiple sequential parts while preserving structure, completeness, detail, source documentation, and continuity.

SEQUENTIAL OUTPUT RULE

- Apply this requirement automatically whenever the complete output may exceed practical response length or its length cannot be reliably predicted.
- Do not compress, summarize, abbreviate, or omit required research merely to fit within one response.
- Favor completeness over minimizing the number of responses.
- Break the output at a natural point. Do not force the Research Output into predetermined sets or fixed section groupings.
- At the end of each nonfinal part, state that additional output remains, request permission to continue, and stop.
- After permission is received, continue from where the previous part ended until the complete Research Output has been delivered.

LIVE RESEARCH REQUEST

- The final part must clearly state that the requested Research Output is complete.

10. Governance and Final Quality Review

The output must be accurate, complete, readable, practical, and appropriate for general readers. Use clear language without talking down to the reader. Avoid fear-based messaging, academic gatekeeping, unsupported certainty, invented evidence, hidden source conflicts, and jargon-heavy explanations.

Before declaring completion, confirm that every Research Objective is answered; every required output section is present; major factual claims are supported; no source information has been invented; examples are distinguished from documented events; important uncertainty or credible disagreement is visible; the content remains within scope; and the Sequential Output Requirements were followed whenever the output required more than one response.

If a condition fails, correct the problem before delivering the completion statement. If it cannot be corrected within the approved sources and scope, report the limitation clearly.

Before moving on, pause over the nine standard responsibilities. Each protects a different failure point. You may combine closely related sections in a shorter request, but omitting their responsibilities can change the research, the evidence, or the completeness of the result.

Purpose. Tells the AI what to execute and prevents the wrong deliverable.

Research Configuration Variables. Makes current settings visible instead of leaving them for the AI to guess.

Research Objectives. Defines required coverage and the standard for completeness.

Approved Information Sources. Limits the evidence to qualified, traceable material.

Source Priority and Conflict Handling. Guides comparisons and preserves unresolved disagreement.

Research Scope. Sets the required depth and keeps the investigation within bounds.

Output Organization. Prepares the findings for review and later Knowledge use.

Research Source Documentation. Keeps major claims traceable and exposes invented sources.

Sequential Output Requirements. Prevents compression or omission and defines completion.

Governance and Final Quality Review. Checks accuracy, coverage, boundaries, sources, and limitations.

You may simplify the document's shape, but you should not delete the responsibility each section carries.

Prepare and Execute the Research Request

Step 1: Create and Check the File

Copy the artifact into a clean word-processing document. Replace “[Your name]” with your name, confirm the request ID and version, and save the file as “CTA-RES-001 Critical Thinking Research Request v1.0.docx.” Keep all nine responsibilities even if you later consolidate headings.

Step 2: Confirm Research Access

Use an AI system and operating mode that can read the attachment and access the approved external sources. Mode names and capabilities vary by product. Where a thinking or reasoning control is available, use Medium or higher for this practicum. Do not use a fast-response mode such as Instant, Light, or Low unless you have confirmed that it can perform the required external research. If live research is unavailable, attach approved source documents directly. If the required sources still cannot be accessed, stop and report the limitation.

Step 3: Start a Clean Chat and Attach the Request

A clean working context reduces interference. On free or limited tiers, long runs may face rate limits or context trimming, so use this chat only for Part I and start another for Part III. Attach the complete request and wait until it is available; naming a file does not supply its contents. Before beginning, confirm that the Research Request is available and that the Medium-or-higher setting selected in Step 2 remains active.

Step 4: Begin Execution

Send one instruction:

Please execute the attached Research Request.

The prompt starts the work; the attachment controls it. If the AI explains the document instead, reply: “Execute the Research Request. Do not summarize or explain the attachment.”

Step 5: Complete the Sequential Output

The AI may deliver the complete Research Output in one response when it fits comfortably. If additional output remains, it must request permission and stop. Save each returned part and reply “Continue” when you are ready. Repeat until the AI clearly states that the requested output is complete, then combine the parts in their original order.

Part II: Review and Save the Knowledge Repository

The AI’s quality check does not replace human review. Research Output does not automatically become Knowledge; human review and approval are required before accepted information is preserved in CTA-

KNW-001. Compare the Research Output with CTA-RES-001, then sample important citations to confirm that the sources exist and support the nearby claims. Chapter 7 explains Knowledge preservation in greater depth.

Review check	Question	If the check fails
Completeness	Are all objectives and required sections present?	Request the missing material.
Evidence	Are major claims supported by qualified, traceable sources?	Add support, correct the claim, or remove it.
Citation integrity	Do sampled links and source details support the claim?	Correct the citation and recheck related claims.
Balance	Are uncertainty, limits, and credible differences visible?	Add the missing qualifications.
Scope	Does the output remain within the assigned boundaries?	Remove or revise the drift.
Usability	Can later work use the information without repeating the research?	Clarify or reorganize the affected section.

If several checks fail, use this correction prompt:

Review the Research Output against CTA-RES-001. Identify every unmet requirement, then return a clean and complete revision with only the necessary changes. Do not change the approved sources, scope, or output organization. Follow CTA-RES-001's Sequential Output Requirements while delivering the revision.

Review the revision. Confidence is not proof of correction.

Preserve the Approved Knowledge

When the output passes review, copy the complete approved Research Output into a new document. Preserve the headings, qualifications, citations, Source Register, and useful detail. Do not reduce the information to a short summary; this repository will be the Runtime data engine. Whenever possible, copy the approved Research Output directly from the AI response or an editable document. If you must convert it from PDF, check the headings, paragraph order, bullets, tables, and citations before approving the repository.

Add this header:

CRITICAL THINKING KNOWLEDGE REPOSITORY

Repository ID: CTA-KNW-001

Project: DCPF Practicum, Critical Thinking

Derived from: CTA-RES-001, version 1.0

Created: [Date]

Reviewed by: [Your name]

Status: Approved for practicum use

Content: Complete reviewed Research Output

Save the file as “CTA-KNW-001 Critical Thinking Knowledge Repository v1.0.docx.” Keep CTA-RES-001 as a separate file. The request records how the information was developed; the repository contains what you accepted for reuse.

In a small project, this handoff should remain simple. The approved Research Output already has the organization and source history needed by the repository. If an important error is discovered later, correct the underlying research, review the change, and create a new repository version.

Part III: Create the Runtime Request

Research discovers and develops evidence. Knowledge preserves the accepted intelligence. Runtime now applies that approved Knowledge to a defined assignment: a short, practical critical-thinking guide for everyday readers.

The finished work will be *Thinking Clearly: A Personal Guide to Critical Thinking*. It will read like a concise practical essay with headings, not like research notes or an explanation of DCPF. The direct handoff is easy to see: the Research Request develops reliable knowledge about critical thinking, the Knowledge Repository preserves it, and the Runtime Request transforms it into personal guidance. Chapter 8 explains every Runtime element and the distinction between retained Knowledge and a current execution.

As in Chapter 10, this practicum intentionally embeds its essential Governance rules inside the Research and Runtime requests. The project is governed without requiring a standalone Governance artifact; recurring controls can be consolidated later when a maintained project-wide source becomes useful.

Choose How to Supply the Knowledge Repository

The Knowledge Repository remains a separate maintained project artifact even when its contents are embedded in the Runtime Request. The same three DCPF supply routes apply here: attachment; embedding or pasting; and retrieval from a source the AI can actually access. To keep this practicum platform-neutral, the table below uses attachment and embedding; Both combines those two routes for redundancy.

Delivery method	Advantages	Trade-offs
Embed the complete repository in the Runtime Request	Creates one self-contained execution file, keeps the Knowledge close to the instructions, and reduces the chance that an attachment will be inaccessible or overlooked.	Makes the Runtime file longer and duplicates the repository content. The embedded copy must be checked against the approved repository whenever either file changes.

Delivery method	Advantages	Trade-offs
Attach the repository as a separate document	Keeps the Runtime Request shorter, preserves clean version control, and makes one repository easy to reuse across several Runtime Requests.	Requires the AI to access and attend to multiple files. A missing, unreadable, very long, or weakly identified attachment can lead to an incomplete execution.
Use both methods	Provides a self-contained Runtime file while retaining the approved repository as a visible companion document. This can help when attachment handling is uncertain.	Repeats the same material, uses more context, and creates a conflict risk if the copies do not match. Both copies must carry the same repository ID and version.

For this practicum, embedding the complete repository is the simplest choice. Keep CTA-KNW-001 as its own approved file, then paste an exact copy into Appendix A of the Runtime Request you will execute. If you prefer a separate attachment, the request also supports that method. Connected retrieval is also valid when the AI can actually retrieve and read the complete approved repository. Whichever method you choose, identify it in the Runtime metadata and never rely on the repository filename alone.

A short, predictable Runtime assignment may fit in one response. A longer or less predictable Project Output may require more than one, depending on the AI model and its settings. The Runtime Request applies the rule automatically and breaks sequential delivery at natural points, so you do not have to predict the practical response limit or define response groups in advance.

LIVE RUNTIME REQUEST

Request: Personal Critical Thinking Guide

Request ID: CTA-RUN-001

Project: DCPF Practicum, Critical Thinking

Owner: [Your name]

Version: 1.0

Status: Ready for execution

Input Knowledge: CTA-KNW-001, version 1.0, supplied in full with this request

Knowledge delivery: [Choose one: Embedded in Appendix A / Separate attachment / Both]

Output: CTA-OUT-001, *Thinking Clearly: A Personal Guide to Critical Thinking*

1. Purpose

Execute this document as a complete Runtime Request. Use the complete Critical Thinking Knowledge Repository supplied by the delivery method identified above as the sole conceptual and factual data engine for the reader-facing guide. Follow the configuration variables, Input Knowledge requirements, Task Definition, Output Requirements, delivery instructions, and governance requirements in this document.

Before drafting, locate and read the entire repository. Do not rely on its filename, metadata, or a partial extract. Do not merely review, summarize, or explain this request. Do not conduct new research or add concepts or factual claims from general model knowledge. If the complete required Knowledge is missing, inaccessible, internally inconsistent, or insufficient, identify the problem and request direction rather than inventing information.

LIVE RUNTIME REQUEST

2. Runtime Configuration Variables

Subject: Applying critical thinking in everyday life. **Audience:** General readers, including high school students and adults. **Objective:** Help readers understand and practice critical thinking when evaluating information, solving problems, and making decisions. **Format:** A concise personal guide written as a cohesive essay with clear headings. **Point of view:** Second person. **Tone:** Clear, practical, encouraging, and nonacademic. **Target length:** 1,200–1,600 words. **Examples:** Brief everyday illustrations. **External research:** Not authorized. **Publication status:** Draft pending human review.

3. Input Knowledge

Use CTA-KNW-001, version 1.0, as the sole conceptual and factual data engine. For this practicum, supply the complete repository by embedding it in Appendix A, attaching it as a separate document, or using both methods according to the Knowledge delivery field. The identifier alone does not supply the contents.

Use only the critical-thinking concepts, methods, examples, and qualifications contained in the repository. You may reorganize and simplify the material for the audience, but do not introduce outside concepts as established facts. If both embedded and attached copies are supplied, verify that their repository IDs and versions match. If their contents conflict, stop and request direction. If any required copy is missing, inaccessible, incomplete, or insufficient to produce the guide responsibly, stop and report the limitation.

4. Task Definition

Open with purpose. Introduce why careful thinking matters in ordinary life without using fear, preaching, or motivational exaggeration.

Explain critical thinking. Define it in practical language and distinguish it from arguing, constant negativity, or intelligence alone.

Teach a usable process. Show readers how to clarify the question; separate claims from conclusions; examine evidence and sources; notice assumptions and bias; consider alternatives; match confidence to the available support; and revise a conclusion when warranted.

Connect the process to life. Explain how readers can use these habits in decisions, problem-solving, school, work, purchases, conversations, and the evaluation of online information. Use brief examples only to illustrate ideas supported by the repository.

Create a reusable checklist. Give readers a short set of questions they can apply when evaluating a claim, source, choice, or proposed solution.

Encourage continued practice. Explain that critical thinking becomes more useful through repeated, deliberate use.

Close with perspective. End with a practical reflection on thinking carefully while remaining open to better information.

LIVE RUNTIME REQUEST

5. Output Requirements

Title the Project Output *Thinking Clearly: A Personal Guide to Critical Thinking*. Organize it under these headings: Introduction; What Critical Thinking Is; Start with the Question; Examine Claims, Evidence, and Sources; Notice Assumptions and Bias; Consider Alternatives and Uncertainty; Use Critical Thinking in Everyday Life; A Personal Critical Thinking Checklist; Strengthening the Habit; Closing Perspective; and Internal Validation Note.

The main guide must read as one coherent work, not as research notes or a list of disconnected definitions. Use plain language and useful transitions. Keep examples brief and clearly illustrative; do not present invented situations as factual cases. Do not discuss DCPF, the Research Request, or the Knowledge Repository in the reader-facing guide. The repository's source history does not need to appear as citations in the guide unless the supplied Knowledge specifically requires attribution.

The Internal Validation Note must identify the Runtime Request and Knowledge Repository versions used; state how the repository was delivered; confirm that the complete repository was supplied and read before drafting; confirm that every required section was completed; state whether any outside concepts or unsupported factual additions were detected; identify unresolved gaps; and state that human approval remains required.

6. ESSENTIAL: Sequential Output Requirements

PURPOSE

When the requested Project Output may exceed what can be comfortably delivered in a single response, divide it into multiple sequential parts while preserving structure, completeness, detail, formatting, and continuity.

SEQUENTIAL OUTPUT RULE

- Apply this requirement automatically whenever the complete output may exceed practical response length or its length cannot be reliably predicted.
- Do not compress, summarize, abbreviate, or omit required work merely to fit within one response.
- Favor completeness over minimizing the number of responses.
- Break the output at a natural point. Do not force the Project Output into predetermined sets or fixed section groupings.
- At the end of each nonfinal part, state that additional output remains, request permission to continue, and stop.
- After permission is received, continue from where the previous part ended until the complete Project Output has been delivered.

LIVE RUNTIME REQUEST

- The final part must clearly state that the requested Project Output is complete.
- A short, predictable Project Output may be delivered in one response.

7. Validation and Human Review

Apply the embedded governance rules in this section throughout execution. The output must remain accurate to the supplied Knowledge, clear, practical, encouraging, and appropriate for the stated audience. Preserve the repository's qualifications and its distinctions among claims, evidence, assumptions, inference, and uncertainty.

Do not invent facts or evidence, overstate claims, hide uncertainty, use unexplained jargon, portray critical thinking as emotionless or cynical, or imply that careful thinkers never make mistakes. Do not use fear, gatekeeping, preaching, or emotional pressure. Do not conduct unauthorized research. If any instruction conflicts with the available Knowledge or prevents responsible completion, pause and request direction.

Before delivery, verify that the guide remains within scope and target length, uses CTA-KNW-001 as its sole conceptual and factual source, contains every required section, fits the stated audience and tone, keeps examples illustrative, avoids unsupported additions, preserves appropriate uncertainty, and follows the Sequential Output Requirements whenever more than one response is needed.

Appendix A: Embedded Knowledge Repository

Use this appendix when **Knowledge delivery** is set to **Embedded in Appendix A** or **Both**. Paste the complete, approved contents of CTA-KNW-001, version 1.0, below this line. Preserve its repository header, source history, qualifications, and citations.

[BEGIN CTA-KNW-001, VERSION 1.0]

[Paste the complete approved Knowledge Repository here.]

[END CTA-KNW-001, VERSION 1.0]

If **Knowledge delivery** is set to **Separate attachment**, delete the BEGIN marker, placeholder, and END marker. Replace the entire Appendix A repository block with this statement: "CTA-KNW-001, version 1.0, is supplied as a separate attachment. Appendix A is not used."

Prepare and Execute the Runtime Request

Step 1: Create the Runtime File

Copy the artifact into a new document, replace "[Your name]" with your name, and choose a Knowledge delivery method. If you embed the repository, paste the complete approved CTA-KNW-001 into Appendix A and confirm that its ID and version match the metadata. Save the file as "CTA-RUN-001 Critical Thinking Runtime Request v1.0.docx." Confirm that the subject, audience, format, tone, length,

and output name are correct. Keep all six Runtime responsibilities; Chapter 8 explains the risks of omitting them.

Step 2: Start a Clean Chat and Supply the Execution Materials

Supply the files that match the delivery method recorded in CTA-RUN-001:

Embedded: Attach the completed Runtime Request. Confirm that Appendix A contains the complete repository.

Separate attachment: Attach the Runtime Request and CTA-KNW-001 as two documents.

Both: Attach the completed Runtime Request and the separate CTA-KNW-001. Confirm that the embedded and attached copies have the same ID and version.

Do not attach an empty Appendix A as if it contained the repository. Before execution, confirm that the AI can open and read every required file supplied by the selected route. Because Runtime must read and apply the complete repository, use Medium or higher reasoning when that control is available. External research remains unauthorized.

Step 3: Begin Execution

Send this instruction:

Please execute the attached Runtime Request using the complete Knowledge Repository supplied by the delivery method identified inside it.

The AI may deliver the complete guide in one response when it fits comfortably. If it states that additional output remains, save the returned part and reply “Continue” when you are ready. Repeat until the AI clearly states that the requested output is complete. If the AI reports that Knowledge is missing, inaccessible, incomplete, or conflicting, correct the packaging problem and begin again rather than asking it to fill the gap.

Step 4: Preserve the Unreviewed Output

If the guide arrived in multiple parts, combine them in their original order inside a temporary working document or retain them together in the chat while reviewing. Label the complete output Draft, Not Yet Approved. Do not replace the Knowledge Repository with this output; Runtime has produced an application of Knowledge, not new accepted Knowledge.

Part IV: Review and Preserve the Project Output

Compare the Project Output with CTA-RUN-001 and CTA-KNW-001. This review asks two questions: Did the AI produce the requested guide, and did it remain within the supplied Knowledge and authority?

Review check	Question	If the check fails
Task completion	Are all required guide sections present?	Request the missing work.
Knowledge fidelity	Does the output use the repository without adding outside facts?	Remove the addition or authorize new Research.
Audience fit	Can high school students and adults understand it without specialized knowledge?	Simplify the affected passages.
Practical value	Does the guide teach usable habits and include a reusable checklist?	Add or strengthen the practical guidance.
Coherence	Does it read as a concise guide rather than research notes?	Improve structure and transitions.
Tone	Is it encouraging and nonacademic without becoming preachy or simplistic?	Revise the affected language.
Example integrity	Are examples brief, illustrative, and free of invented factual claims?	Relabel, simplify, or remove them.
Validation	Does the note identify the delivery method and versions, confirm full repository access, identify gaps, and preserve human approval?	Complete the validation note.

If revision is needed, use this prompt:

Review CTA-OUT-001 against CTA-RUN-001 and CTA-KNW-001. Identify every unmet requirement, then return a clean and complete revised guide with only the necessary changes. Do not conduct research or introduce concepts or facts outside the supplied Knowledge Repository. Follow CTA-RUN-001's Sequential Output Requirements while delivering the revision.

Review the revision before approval. If the weakness comes from missing critical-thinking knowledge, route it back through Research and update the Knowledge Repository. If it comes from tone, structure, or task wording, revise the Runtime Request. Chapter 8 provides the full feedback-routing explanation.

Approve and Preserve the Project Output

When the output passes review, place this header above the approved content:

APPROVED PROJECT OUTPUT

Output ID: CTA-OUT-001

Project: DCPF Practicum, Critical Thinking

Produced by: CTA-RUN-001, version 1.0

Knowledge used: CTA-KNW-001, version 1.0

Created: [Date]

Reviewed by: [Your name]

Status: Approved for practicum use

Content: *Thinking Clearly: A Personal Guide to Critical Thinking*

Save the file as “CTA-OUT-001 Thinking Clearly - A Personal Guide to Critical Thinking v1.0.docx.” Keep the Research Request, Knowledge Repository, Runtime Request, and Project Output as separate files. Each records a different responsibility.

The Project Output does not automatically become part of the Knowledge Repository. It is a reader-facing work derived from retained knowledge. If it reveals a genuine knowledge gap, conduct additional Research, review the new findings, and update the repository through the same controlled handoff.

The Completed Practicum

You have now completed the operational DCPF cycle:

**Research Request → Research Output → human review and acceptance → Accepted
Knowledge → Knowledge Repository → Runtime Request → Project Output → human review
→ preservation**

The exercise used four files, two short execution prompts, and no custom software. Research developed the information; Knowledge preserved it; Runtime applied it; and Governance guided execution and review.

The subject, sources, and output changed, but the DCPF cycle remained recognizable. Chapter 12 changes domains again, using the same architecture to build a personal cookbook from governed culinary Research.

[Return to Table of Contents](#)

Building a Personal Cookbook Assistant

DCPF is an experimental approach. This practicum is a training environment, not evidence of production readiness. Validate any adapted implementation within your own environment before real-world use.

Turn Your Tastes into a Working Cookbook

This practicum uses the same DCPF workflow you completed in Chapters 10 and 11, but the project is more personal. You will research a small collection of recipes, review the useful material, accept what meets the project requirements, and preserve that Accepted Knowledge in a Knowledge Repository, and then use a Runtime Request to transform that Knowledge into a finished cookbook manuscript.

The working example is a *French Weekend Breakfast Cookbook* with approximately ten recipes. You are welcome to build that cookbook exactly as shown, or change it into something that sounds more like you.

No programming is required. You need a word processor, an AI system that can read attachments and conduct approved research, and a willingness to make a few creative choices.

Process Stages	Your action	Artifact produced
1. Research setup	Choose the cookbook concept and define the research	Research Request
2. Research execution	Collect recipes and supporting culinary knowledge	Research Output
3. Knowledge handoff	Review and preserve the accepted information	Knowledge Repository
4. Runtime execution	Turn the repository into a cohesive cookbook	Project Output
5. Final review	Test, revise, approve, and preserve the manuscript	Approved Project Output

Part I: Create the Research Request

The Research Request decides what kind of cookbook the AI will investigate. It defines the recipe collection, intended cook, sources, required recipe details, safety expectations, supporting material, and definition of completion. Chapter 6 explains the Research elements in depth; here you will put them to work.

Make the Cookbook Yours

The configuration variables are the creative control panel for this project. The French weekend breakfast settings are examples, not restrictions. You could build a collection of family holiday desserts, regional

soups, quick vegetarian dinners, backyard grilling favorites, slow-cooker meals, small-batch baking, budget lunches, or recipes designed around the equipment you already own.

Change one variable or change several. You can adjust the cuisine, meal type, recipe count, cooking skill level, serving size, budget, time limits, dietary preferences, measurement system, or available equipment. Keep the first collection manageable, about eight to twelve recipes is plenty, and make sure the settings work together. A dorm-room cookbook should not quietly require a full professional kitchen, and a thirty-minute dinner collection should not include a recipe that takes an afternoon.

The live artifact is reusable. Save a clean copy, preserve its responsibilities, and change the project-specific settings whenever you want to build another cookbook.

The artifact includes the essential Sequential Output Requirements. The AI applies them automatically whenever the complete Research Output may exceed practical response length or its length cannot be reliably predicted. If multiple parts are needed, it breaks the Research Output at natural points rather than forcing predetermined recipe groups. You do not need to estimate a model's compression threshold or divide the recipes in advance.

You will use the following Personal Cookbook Research Request in the first part of this exercise. Additional instructions follow.

LIVE RESEARCH REQUEST

Request: Personal Cookbook Research

Request ID: CBA-RES-001

Project: DCPF Practicum, Personal Cookbook Assistant

Owner: [Your name]

Version: 1.0

Status: Ready for execution

Output: Research Output for review and preservation as CBA-KNW-001

1. Purpose

Execute this document as a complete Research Request. Research and document the cookbook concept defined in the configuration variables. Produce a structured Research Output containing the recipes and supporting culinary Knowledge needed to create the cookbook manuscript described below.

Follow the variables, objectives, approved sources, source priorities, scope, output organization, source-documentation requirements, Sequential Output Requirements, and governance requirements in this document. Do not merely review, summarize, or explain the request. Do not write the final cookbook manuscript during Research.

Before beginning research, verify that every required input and approved source is accessible. If access is unavailable, stop, report the limitation, and request direction rather than substituting unsupported information.

LIVE RESEARCH REQUEST

2. Research Configuration Variables

Cookbook concept: French weekend breakfast. **Working title:** *French Weekend Breakfast Cookbook*. **Cuisine:** French. **Meal focus:** Weekend breakfast and brunch. **Approximate recipe count:** 10. **Intended cook:** Home cooks. **Skill level:** Beginner to intermediate. **Typical yield:** Approximately 4 servings unless the recipe naturally differs. **Dietary preferences or restrictions:** None specified. **Budget:** Moderate. **Time preference:** Suitable for a relaxed weekend morning; include preparation and cooking times for every recipe. **Available equipment:** Standard home kitchen equipment. **Measurement system:** U.S. standard, with metric equivalents retained when reliable values are available. **User-provided material:** [List any original recipes, family notes, or source documents supplied with this request.] **Customization priority:** Apply any user-edited variables consistently throughout the Research Output.

3. Research Objectives

Build the recipe collection. Identify approximately ten recipes that form a coherent collection under the selected cuisine, meal focus, audience, and project constraints.

Create complete recipe records. For every recipe, collect the recipe name, short factual description, yield or servings, preparation time, cooking time, cooking temperature when applicable, complete ingredient list with measurements, required equipment, and step-by-step preparation instructions.

Support successful cooking. Record important techniques, signs of doneness, timing notes, preparation tips, common mistakes, reasonable variations, and substitutions when qualified information is available.

Develop shared culinary knowledge. Collect the cooking terms, equipment guidance, measurement conversions, ingredient explanations, and organizational notes needed across the cookbook without repeating the same information inside every recipe.

Verify food safety. Identify authoritative guidance relevant to ingredient handling, storage, cooking temperatures, cooling, reheating, and cross-contamination when those issues apply.

Preserve source history. Record where each recipe and major supporting claim originated or was verified. Distinguish traditional recipe lineage from the specific source used during Research.

Respect the selected settings. Confirm that the recipes fit the intended cook, skill level, equipment, time, budget, dietary settings, meal focus, and measurement system.

4. Approved Information Sources

Use original recipes and notes supplied by the user; recipes that are in the public domain or clearly licensed for the intended use; government food-safety publications; university extension resources; established culinary institutions; historical archives and libraries; authoritative ingredient, equipment, and measurement references; and other qualified sources when stronger sources do not reasonably provide the needed information.

Do not copy protected recipe introductions, photographs, distinctive explanatory prose, or other expressive material. Do not use anonymous recipe collections, scraped content, automatically generated source lists, promotional claims presented as evidence, or material whose origin cannot be verified as support for major claims.

LIVE RESEARCH REQUEST

5. Source Priority and Conflict Handling

Treat the user's original recipe or notes as the controlling version of that recipe unless the user requests otherwise. Safety requirements still apply. For outside information, prefer direct, traceable, authoritative, and appropriately licensed sources.

Traditional dishes often have several legitimate versions. Compare meaningful differences rather than silently blending recipes into an untested hybrid. When sources disagree about technique, measurement, timing, or temperature, evaluate the authority and context of each source. Preserve a useful variation or unresolved difference when the available evidence does not justify selecting one answer.

6. Research Scope

Include approximately ten complete recipe records; shared equipment and technique guidance; measurement information; useful cooking terms; relevant food-safety guidance; recipe variations and substitutions when supported; notes that will help organize the final cookbook; and a complete Source Register.

Exclude the final title page, copyright language, table of contents, polished introduction, transitions, decorative prose, recipe index, and publication layout. Those are Runtime responsibilities. Do not expand the collection beyond the approximate recipe count merely because additional recipes are available.

7. Output Organization

Organize the Research Output under these headings: Project Settings; Proposed Recipe Collection; Complete Recipe Records; Shared Techniques and Cooking Terms; Equipment Guide; Measurement and Conversion Notes; Food-Safety Guidance; Variations and Substitutions; Cookbook Organization Notes; Research Qualifications and Gaps; and Source Register.

Use the same field order for every recipe record. Clearly distinguish documented facts, reasonable editorial explanations, user-provided material, and illustrative suggestions.

8. Research Source Documentation

For each recipe, identify the content source or recipe lineage when known and the principal source or sources used to develop or verify the working recipe. Support important food-safety, measurement, ingredient, and technique claims with citations close enough to identify their basis.

The Source Register must include the author or organization, title, publication or update date when available, direct link or document identifier, relevant page or section when useful, and the role the source played. Never invent an author, title, date, quotation, link, license, or publication detail.

9. ESSENTIAL: Sequential Output Requirements

PURPOSE

When the requested Research Output may exceed what can be comfortably delivered in a single response, divide it into multiple sequential parts while preserving structure, completeness, detail, source documentation, and continuity.

LIVE RESEARCH REQUEST

SEQUENTIAL OUTPUT RULE

Apply this requirement automatically whenever the complete output may exceed practical response length or its length cannot be reliably predicted.

Do not compress, summarize, abbreviate, or omit required research merely to fit within one response.

Favor completeness over minimizing the number of responses.

Break the output at a natural point. Do not force the Research Output into predetermined sets or fixed section groupings.

At the end of each nonfinal part, state that additional output remains, request permission to continue, and stop.

After permission is received, continue from where the previous part ended until the complete Research Output has been delivered.

The final part must clearly state that the requested Research Output is complete.

10. Governance and Final Quality Review

The Research Output must be complete, practical, traceable, internally consistent, and suitable for the selected audience. Do not invent recipes, measurements, temperatures, cooking times, source details, licensing status, or food-safety guidance. Do not imply that a recipe is allergen-free or appropriate for a medical diet without sufficient authority and explicit project direction.

Before declaring completion, verify that the recipe count reasonably matches the configuration; every recipe record contains the required fields; measurements, yields, times, and temperatures are internally consistent; recipes fit the selected variables; safety guidance comes from qualified sources; expressive source text has not been copied; major sources are documented; qualifications and gaps are visible; every required output section is present; and any sequential delivery concluded only after the complete Research Output was delivered.

If a condition fails, correct it before delivering the completion statement. If it cannot be corrected within the approved sources and scope, report the limitation clearly.

Before execution, review the nine responsibilities. You may consolidate headings later, but each responsibility protects a different part of the project.

Purpose. Tells the AI to conduct Research and prevents it from writing the cookbook too early.

Research Configuration Variables. Personalizes the collection and keeps the recipes aligned.

Research Objectives. Defines what complete recipe and supporting Knowledge must contain.

Approved Information Sources. Controls evidence quality, permissions, and food-safety authority.

Source Priority and Conflict Handling. Prevents incompatible recipes from being blended without review.

Research Scope. Keeps Research focused and reserves manuscript creation for Runtime.

Output Organization. Produces a repository structure that Runtime can use efficiently.

Research Source Documentation. Preserves attribution, verification, and source history.

Sequential Output Requirements. Protects recipes and supporting detail from compression.

Governance and Final Quality Review. Checks completeness, consistency, safety, and boundaries.

Personalize the settings freely. Preserve the responsibilities carefully.

Prepare and Execute the Research Request

Step 1: Choose Your Cookbook

Copy the live artifact into a new document. Edit the configuration variables until the concept sounds appealing and internally consistent. Replace “[Your name]” with your name and describe any original recipes or notes you plan to supply.

Save the file as “CBA-RES-001 Personal Cookbook Research Request v1.0.docx.” If you build a second cookbook later, create a new project identifier or version rather than overwriting the approved first request.

Step 2: Supply Original Material

Attach any family recipes, personal notes, photographs of recipe cards, or permitted source documents listed in the request. Confirm that handwriting or scans are readable. If the AI cannot access a document, do not let it guess what the document contains.

Step 3: Confirm Research Access

Use an AI system and operating mode that can read the attachment and access the approved external sources. Mode names and capabilities vary by product. Where a thinking or reasoning control is available, use Medium or higher for this practicum. Do not use a fast-response mode such as Instant, Light, or Low unless you have confirmed that it can perform the required external research. If live research is unavailable, attach approved source documents directly. If the required sources still cannot be accessed, stop and report the limitation.

Step 4: Start a Clean Chat and Attach the Request

A clean working context reduces interference. On free or limited tiers, long runs may face rate limits or context trimming, so use this chat only for Part I and start another for Part III. Attach CBA-RES-001 and any user-provided source material and wait until every file is available; naming a file does not supply its contents. Before beginning, confirm that the Research Request and source files are available and that the Medium-or-higher setting selected in Step 3 remains active.

Step 5: Begin Research

Send one instruction:

Please execute the attached Cookbook Research Request using the attached user-provided materials, if any.

The attachment controls the work. If the AI starts drafting the cookbook, direct it back to Research: “Complete CBA-RES-001 first. Do not create the cookbook manuscript during Research.”

Step 6: Complete the Sequential Output

If the AI can comfortably deliver the complete Research Output in one response, no continuation step is needed. If it pauses because additional output remains, save the completed part, reply “Continue” when you are ready, and repeat until the AI clearly states that the requested Research Output is complete. Preserve every part in its original order before review.

Part II: Review and Save the Knowledge Repository

The AI’s quality check does not replace human review. Research Output does not automatically become Knowledge; human review and approval are required before accepted information is preserved in CBA-KNW-001.

A recipe can sound convincing and still contain a missing ingredient, mismatched measurement, unrealistic time, or unsafe instruction. Human review is where the collection becomes trustworthy enough to power the cookbook. Chapter 7 explains Knowledge preservation in greater depth.

Review check	Question	If the check fails
Configuration fit	Do the recipes match the cuisine, meal, audience, skill, time, budget, equipment, and dietary settings?	Replace or revise the mismatch.
Recipe completeness	Does every recipe include yield, times, ingredients, equipment, temperatures when applicable, and complete steps?	Request the missing fields.
Internal consistency	Do ingredients, measurements, temperatures, yields, and instructions agree with one another?	Correct the affected recipe and recheck it.
Source integrity	Are origins, verification sources, permissions, and citations traceable?	Correct the source record or remove unsupported material.
Food safety	Does relevant safety guidance come from authoritative sources and appear in the correct context?	Correct the instruction before use.
Practicality	Could the intended reader reasonably cook each recipe with the stated time, skill, and equipment?	Clarify or replace the recipe.
Collection balance	Does the recipe set feel coherent without becoming repetitive?	Rebalance the proposed collection.
Repository usefulness	Can Runtime build the cookbook without repeating Research?	Restore missing detail or organization.

If several checks fail, use this correction prompt:

Review the complete Research Output against CBA-RES-001. Identify every unmet requirement, then return a clean and complete revision with only the necessary changes. Preserve the selected configuration variables, approved sources, recipe count, and output organization. Do not draft the cookbook manuscript. Follow CBA-RES-001's Sequential Output Requirements while delivering the revision.

Review the revision. For recipes you intend to publish or serve, test them or obtain a qualified human review when practical. AI-assisted Research is not a substitute for observing how a recipe behaves in an actual kitchen.

Preserve the Approved Knowledge

When the Research Output passes review, copy the complete approved content into a new document. Preserve the recipe records, useful detail, qualifications, citations, and Source Register. Do not shorten the recipes into summaries; this repository will serve as Runtime's data engine. Whenever possible, copy the approved Research Output directly from the AI response or an editable document. If you must convert it from PDF, check the recipe headings, ingredient lists, measurements, paragraph and step order, bullets, tables, and citations before approving the repository.

Add this header:

PERSONAL COOKBOOK KNOWLEDGE REPOSITORY

Repository ID: CBA-KNW-001
Project: DCPF Practicum, Personal Cookbook Assistant
Cookbook concept: [Insert your selected concept]
Derived from: CBA-RES-001, version 1.0
Created: [Date]
Reviewed by: [Your name]
Status: Approved for practicum use
Content: Complete reviewed Cookbook Research Output

Save the file as "CBA-KNW-001 Personal Cookbook Knowledge Repository v1.0.docx." Keep the Research Request as a separate record of how the collection was developed.

Part III: Create the Runtime Request

Research discovers and develops culinary evidence. Knowledge preserves the accepted recipes and supporting culinary intelligence. Runtime now applies that approved Knowledge to a defined assignment: a concise, polished *French Weekend Breakfast Cookbook* that a home cook can read and use.

Research variables shape the investigation and the resulting evidence. Human review and acceptance determine what becomes Accepted Knowledge and is preserved in the Repository. Runtime variables decide how that Knowledge will be presented. You can still personalize the title, subtitle, tone, recipe order, front matter, supporting sections, and measurement display, as long as the requested manuscript

can be produced from the approved repository. Chapter 8 explains every Runtime element and the distinction between retained Knowledge and a current execution.

This practicum intentionally embeds its essential Governance rules inside the Research and Runtime requests. That keeps the controls close to the work while preserving the Governance Framework responsibility; a standalone Governance artifact can be added later when recurring controls need a maintained project-wide home.

Choose How to Supply the Knowledge Repository

Use the same attachment and embedding choices introduced in Chapters 10 and 11. Retrieval from a source the AI can actually access remains a valid DCPF route when the selected environment supports it, but it is not required for this practicum:

Delivery method	Best feature	Main trade-off
Embed the complete repository	One self-contained Runtime file reduces attachment-access problems.	The file is longer, and the embedded copy must match the approved repository.
Attach the repository separately	Keeps one reusable repository and a shorter Runtime Request.	The AI must access and attend to both documents.
Use both methods	Adds redundancy when attachment handling is uncertain.	Repeats context and creates conflict risk if versions differ.

For a ten-recipe cookbook, embedding the complete approved repository in Appendix A is usually manageable. Keep CBA-KNW-001 as its own governed file even when you paste an execution copy into Runtime.

A short, predictable Runtime assignment may fit in one response. A longer or less predictable Project Output may require more than one, depending on the AI model and its settings. The Runtime Request applies the rule automatically and breaks sequential delivery at natural points, so you do not have to predict the practical response limit or define recipe or manuscript response groups in advance.

LIVE RUNTIME REQUEST

Request: Personal Cookbook Manuscript

Request ID: CBA-RUN-001

Project: DCPF Practicum, Personal Cookbook Assistant

Owner: [Your name]

Version: 1.0

Status: Ready for execution

Input Knowledge: CBA-KNW-001, version 1.0, supplied in full

Knowledge delivery: [Choose one: Embedded in Appendix A / Separate attachment / Both]

Output: CBA-OUT-001, complete cookbook manuscript

LIVE RUNTIME REQUEST

1. Purpose

Execute this document as a complete Runtime Request. Use the complete Personal Cookbook Knowledge Repository supplied by the delivery method identified above as the sole factual and culinary data engine for the cookbook manuscript defined below.

Follow the configuration variables, Input Knowledge requirements, Task Definition, Output Requirements, Sequential Output Requirements, delivery instructions, and governance requirements in this document. Do not merely review, summarize, or explain the request. Do not conduct new recipe Research or add culinary facts from general model Knowledge.

Before drafting, locate and read the entire repository. If it is missing, inaccessible, incomplete, internally inconsistent, or insufficient, identify the problem and request direction rather than inventing content.

2. Runtime Configuration Variables

Cookbook title: *French Weekend Breakfast Cookbook*. **Subtitle:** *Classic Morning Recipes from the French Kitchen*. **Target audience:** Home cooks. **Writing style:** Friendly, relaxed, instructional, and professional. **Reading level:** General audience. **Measurement display:** U.S. standard with metric equivalents when the repository provides reliable values. **Recipe order:** A natural weekend-breakfast progression selected from the repository. **Include title page:** Yes. **Include copyright-page placeholder:** Yes. **Include table of contents:** Yes. **Include introduction:** Yes. **Include culinary-background section:** Yes, only from repository Knowledge. **Include equipment guide:** Yes. **Include measurement guide:** Yes. **Include cooking-terms glossary:** Yes. **Include food-safety section:** Yes. **Include source notes:** Yes. **Include recipe index:** Yes. **External research:** Not authorized. **Publication status:** Draft pending human review and recipe testing.

3. Input Knowledge

Use CBA-KNW-001, version 1.0, as the sole source for recipes, ingredients, measurements, yields, timing, temperatures, techniques, safety information, culinary background, variations, and source history. For this practicum, supply the complete repository by embedding it in Appendix A, attaching it as a separate document, or using both methods according to the Knowledge delivery field. Its identifier alone does not supply the contents.

You may reorganize the repository and improve grammar, readability, transitions, headings, and formatting. You may create brief editorial introductions and transitions that do not add factual culinary claims. Do not fabricate a recipe, ingredient, measurement, temperature, time, technique, cultural claim, safety rule, or source detail.

If both embedded and attached copies are supplied, confirm that their repository IDs and versions match. If their contents conflict, stop and request direction.

4. Task Definition

Create the front matter. Produce a title page, copyright-page placeholder, table of contents, and welcoming introduction appropriate to the selected audience and tone. Do not invent page numbers. Until final pagination is available, provide the table of contents as an ordered section list and the recipe index as an alphabetical recipe-title index without page numbers or with clearly identified placeholders.

LIVE RUNTIME REQUEST

Prepare the reader. Create the requested background, equipment, measurement, terminology, and food-safety sections using only repository Knowledge. Avoid repeating the same guidance inside every recipe.

Build the recipe collection. Present every approved recipe in a logical order. For each recipe, include its title, short introduction, yield or servings, preparation time, cooking time, temperature when applicable, equipment, complete ingredient list, numbered instructions, signs of doneness when available, helpful tips, and qualified variations.

Create continuity. Use consistent terminology, measurements, heading levels, field order, and formatting throughout. Add brief transitions when they improve flow without introducing new facts.

Preserve source responsibility. Credit user-provided, public-domain, licensed, historical, and verification sources according to the repository. Do not copy expressive source prose into the manuscript.

Complete the back matter. Produce the requested source notes, glossary, and recipe index from the finished manuscript and repository.

5. Output Requirements

The Project Output must read as a cohesive small cookbook, not as Research notes pasted together. Use descriptive headings, inviting but restrained prose, consistent recipe formatting, and instructions that are easy to follow while cooking.

Preserve the repository's substantive recipe information and qualifications. Do not omit a recipe or required field merely to shorten the response. Do not claim that a recipe has been kitchen-tested unless the repository documents that testing. Do not claim medical, nutritional, allergen-free, or cultural authority beyond what the repository supports.

End the manuscript with an Internal Validation Note that identifies the Runtime Request and Knowledge Repository versions; states how the repository was delivered; confirms that the full repository was reviewed; lists any repository information that could not be incorporated; identifies unresolved gaps; confirms whether outside recipes or factual additions were detected; and states that human review and recipe testing remain required.

6. ESSENTIAL: Sequential Output Requirements

PURPOSE

When the requested Project Output may exceed what can be comfortably delivered in a single response, divide it into multiple sequential parts while preserving structure, completeness, detail, formatting, and continuity.

SEQUENTIAL OUTPUT RULE

LIVE RUNTIME REQUEST

Apply this requirement automatically whenever the complete output may exceed practical response length or its length cannot be reliably predicted.

Do not compress, summarize, abbreviate, or omit required work merely to fit within one response.

Favor completeness over minimizing the number of responses.

Break the output at a natural point. Do not force the Project Output into predetermined sets or fixed section groupings.

At the end of each nonfinal part, state that additional output remains, request permission to continue, and stop.

After permission is received, continue from where the previous part ended until the complete Project Output has been delivered.

The final part must clearly state that the requested Project Output is complete.

A short, predictable Project Output may be delivered in one response.

7. Validation and Human Review

Apply these governance requirements throughout execution. The manuscript must remain faithful to CBA-KNW-001, appropriate for the selected reader, clear enough to use while cooking, consistent across recipes, transparent about source history, and cautious about safety and unverified claims.

Do not invent missing recipe information, hide repository conflicts, copy protected expressive text, overstate cultural authenticity, make unsupported dietary or health claims, or weaken food-safety guidance for tone or brevity. If an instruction conflicts with the repository or responsible completion, pause and request direction.

Before delivery, verify that every approved recipe is present; every recipe contains the required fields; measurements and terminology are consistent; safety information is preserved; all requested front and back matter is included; source responsibilities are respected; no new Research has been introduced; any sequential delivery is complete; and the manuscript remains a draft pending human review and recipe testing.

Appendix A: Embedded Knowledge Repository

Use this appendix when **Knowledge delivery** is set to **Embedded in Appendix A** or **Both**. Paste the complete, approved contents of CBA-KNW-001, version 1.0, below this line, including its repository header, complete recipe records, qualifications, citations, and Source Register.

[BEGIN CBA-KNW-001, VERSION 1.0]

[Paste the complete approved Personal Cookbook Knowledge Repository here.]

[END CBA-KNW-001, VERSION 1.0]

If **Knowledge delivery** is set to **Separate attachment**, delete the BEGIN marker, placeholder, and END marker. Replace the entire Appendix A repository block with this statement: "CBA-KNW-001, version 1.0, is supplied as a separate attachment. Appendix A is not used."

Prepare and Execute the Runtime Request

Step 1: Configure the Cookbook Manuscript

Copy the Runtime artifact into a new document. Replace “[Your name]” with your name, confirm the title and subtitle, and edit any presentation settings. Do not request a section that depends on Knowledge the repository never collected unless you first return to Research.

Choose the Knowledge delivery method. If you embed the repository, paste the complete approved CBA-KNW-001 into Appendix A and confirm that its ID and version match the Runtime metadata.

Save the file as “CBA-RUN-001 Personal Cookbook Runtime Request v1.0.docx.”

Step 2: Start a Clean Chat and Supply the Execution Materials

Do not continue in the Research chat. Start a clean Runtime chat so the manuscript instructions and repository receive the clearest available working context. Supply the materials that match the delivery method recorded in CBA-RUN-001.

Embedded: Attach the completed Runtime Request containing the full repository.

Separate attachment: Attach CBA-RUN-001 and CBA-KNW-001.

Both: Attach both files and confirm that the embedded and separate copies match.

Do not attach an empty Appendix A as if it contained the repository. Before execution, confirm that the AI can open and read every required file supplied by the selected route. Because Runtime must read and apply the complete repository, use Medium or higher reasoning when that control is available. External research remains unauthorized.

Step 3: Begin Runtime Execution

Send one instruction:

Please execute the attached Cookbook Runtime Request using the complete Knowledge Repository supplied by the delivery method identified inside it.

The AI may deliver the complete cookbook manuscript in one response when it fits comfortably. If it states that additional output remains, save the completed part and reply “Continue” when you are ready. Repeat until the AI clearly states that the requested Project Output is complete. If the AI reports missing or conflicting Knowledge, correct the repository packaging or return the gap to Research. Do not ask Runtime to improvise the missing recipe information.

Step 4: Preserve the Unreviewed Manuscript

If the manuscript arrived in multiple parts, combine them in their original order into one working document. Label it Draft, Not Yet Approved. Preserve the AI’s Internal Validation Note for comparison during review.

Part IV: Review and Preserve the Cookbook

Read the manuscript once as a book and again as a set of cooking instructions. A smooth introduction cannot compensate for a missing ingredient, and a complete recipe can still feel out of place in the collection.

Review check	Question	If the check fails
Assignment completion	Are all requested front matter, recipes, supporting sections, and back matter present?	Request the missing section.
Knowledge fidelity	Does the manuscript remain within CBA-KNW-001 without invented recipes or facts?	Remove the addition or return to Research.
Recipe integrity	Does each recipe preserve its approved ingredients, measurements, yield, timing, temperature, and steps?	Correct it against the repository.
Cooking usability	Can the intended cook follow the sequence without guessing?	Clarify structure or wording without changing facts.
Consistency	Are units, terms, field order, headings, and recipe formatting stable?	Standardize the manuscript.
Personal fit	Does the finished book still reflect the selected cuisine, audience, skill, equipment, time, budget, and tone?	Revise Runtime presentation or Research selection.
Safety and claims	Are safety instructions preserved and unsupported health, allergen, testing, or authenticity claims absent?	Correct the claim before approval.
Source responsibility	Are credits and source notes accurate without copied expressive prose?	Correct the credit or remove the material.
Validation	Does the note identify versions, delivery, full access, gaps, and human review?	Complete the validation note.

If revision is needed, use this prompt:

Review CBA-OUT-001 against CBA-RUN-001 and CBA-KNW-001. Identify every unmet requirement, then return a clean and complete revised cookbook manuscript with only the necessary changes. Do not conduct new Research, invent recipe information, or introduce culinary facts outside the supplied Knowledge Repository. Follow CBA-RUN-001's Sequential Output Requirements while delivering the revision.

Review the revision before approval. Route missing or unreliable culinary Knowledge back to Research. Route organization, tone, transitions, formatting, or requested-section problems back to Runtime. Chapter 8 explains feedback routing in greater detail.

Test Before You Trust

When practical, cook the recipes before publication or distribution. Record corrections to yield, timing, ingredient amounts, equipment, temperature, and sequence. A successful kitchen test may reveal new

approved Knowledge; preserve meaningful corrections through an updated repository version before regenerating affected manuscript sections.

Approve and Preserve the Project Output

When the manuscript passes review and any planned testing, place this header above the approved content:

APPROVED COOKBOOK PROJECT OUTPUT

Output ID: CBA-OUT-001

Project: DCPF Practicum, Personal Cookbook Assistant

Title: [Final cookbook title]

Produced by: CBA-RUN-001, version 1.0

Knowledge used: CBA-KNW-001, version 1.0

Created: [Date]

Reviewed by: [Your name]

Testing status: [Not tested / Partially tested / Fully tested]

Status: Approved for practicum use

Content: Complete approved cookbook manuscript

Save the file as “CBA-OUT-001 [Cookbook Title] v1.0.docx.” Keep the Research Request, Knowledge Repository, Runtime Request, draft manuscript, and approved Project Output as separate records.

The cookbook manuscript is a product of the Knowledge Repository, not a replacement for it. If testing or review changes the underlying recipe Knowledge, update CBA-KNW-001 and preserve the new version before issuing a revised cookbook.

Completed practicum: Settings → Research → reviewed Knowledge → Runtime → cookbook → testing → preservation. The French weekend breakfast example demonstrated the process; now change the variables and reuse the workflow for the cookbook you want to build.

[Return to Table of Contents](#)

Building a Personal Study Guide Assistant

DCPF is an experimental approach. This practicum is a training environment, not evidence of production readiness. Validate any adapted implementation within your own environment before real-world use.

Learn Something You Actually Want to Know

This practicum extends the DCPF workflow used in Chapters 10 through 12 into a coordinated learning project. You will use Research to build reliable, appropriately scoped Knowledge and Runtime to produce a study guide, practice quiz, answer key, and short study plan.

The working example is an introductory guide to the solar system. You may keep that topic or replace it with a class unit, hobby, historical period, technical concept, certification subject, or question you have always wanted to understand.

Stage	Your action	Artifact produced
Research setup	Define the topic, learner, and goals	Research Request
Research execution	Build accurate, study-ready Knowledge	Research Output
Knowledge handoff	Review and preserve the accepted material	Knowledge Repository
Runtime execution	Create coordinated learning materials	Project Output
Final review	Check the guide, quiz, and answers	Approved Study Packet

Part I: Create the Research Request

A useful study guide begins with a manageable learning target. “Teach me astronomy” is too broad for one short packet. “Help me understand the major parts of the solar system and how they relate” gives Research a clear destination.

Chapter 6 explains every Research element in depth; this chapter focuses on applying them to a learning project.

Pick a Topic You Will Enjoy

Your subject does not need to sound like homework. You could study why thunderstorms form, how a guitar produces sound, the history of video games, the basics of photography, a favorite period in history, or material from a course you are already taking.

Use the configuration variables to set the learner level, starting knowledge, learning goal, depth, time available, and preferred assessment style. If the topic feels enormous, narrow it until you can describe

the goal in one sentence. A focused packet that you finish is more useful than an encyclopedia you never begin.

If you choose a medical, legal, safety-critical, or regulated certification subject, use qualified sources and treat the packet as learning support only. It must not be presented as professional advice or as a substitute for official requirements.

The artifact includes the essential Sequential Output Requirements. The AI applies them automatically whenever the complete Research Output may exceed practical response length or its length cannot be reliably predicted. You do not need to estimate a model's compression threshold or divide the research in advance.

You will use the following Personal Study Guide Research Request in the first part of this exercise. Additional instructions follow.

LIVE RESEARCH REQUEST

Request: Personal Study Guide Research

Request ID: SGA-RES-001

Project: DCPF Practicum, Personal Study Guide Assistant

Owner: [Your name]

Version: 1.0

Status: Ready for execution

Output: Research Output for review and preservation as SGA-KNW-001

1. Purpose

Execute this document as a complete Research Request. Research and organize the selected topic into accurate, study-ready Knowledge for the learner and goals defined below. Produce a structured Research Output that can power a study guide, practice quiz, answer key, and short study plan.

Follow the variables, objectives, approved sources, source priorities, scope, output organization, source-documentation requirements, Sequential Output Requirements, and governance requirements in this document.

Do not merely review, summarize, or explain this request. Do not create the final study packet during Research.

Before beginning research, verify that every required input, course material, and approved source is accessible. If access is unavailable, stop, report the limitation, and request direction rather than filling the gap from unsupported model Knowledge.

LIVE RESEARCH REQUEST

2. Research Configuration Variables

Topic: The solar system. **Learning goal:** Understand its major components and the basic relationships among them. **Learner:** High-school student or adult general learner. **Starting knowledge:** Beginner. **Depth:** Introductory but substantive. **Estimated study time:** 60–90 minutes. **Course alignment:** Independent learning. **Required course materials:** None. **Topics to emphasize:** The Sun; planets; dwarf planets; moons; asteroids and comets; gravity and orbits; scale; exploration; and common misconceptions. **Topics to exclude:** Advanced mathematics, detailed mission engineering, and exhaustive planetary data. **Assessment preference:** Mixed multiple-choice and short-answer questions. **User-provided materials:** [List any notes, readings, slides, or assignments supplied with this request.]

3. Research Objectives

Establish the core Knowledge. Identify the facts, concepts, relationships, processes, and examples required to meet the learning goal.

Organize for learning. Divide the topic into a logical sequence of short units and show how the units connect.

Define essential vocabulary. Explain important terms in language appropriate to the learner without replacing accuracy with slogans.

Clarify difficult points. Identify likely misconceptions, confusing comparisons, and ideas that require careful explanation.

Prepare for assessment. Identify what the learner should be able to recall, explain, compare, or apply after studying. Do not write the final quiz during Research.

4. Approved Information Sources

Use user-provided course materials when supplied; primary and official educational sources; government agencies; universities; museums and established educational institutions; peer-reviewed or academically reviewed material; professional organizations; authoritative reference works; and other qualified sources when stronger sources do not reasonably provide the needed information.

Do not use answer-sharing sites, anonymous summaries, unsourced study notes, promotional material presented as instruction, automatically generated citations, or sources whose origin cannot be verified for major claims.

5. Source Priority and Conflict Handling

User instructions control the assignment and learning goal. For a course-aligned project, preserve the instructor's scope, terminology, and required materials, but do not silently accept an apparent factual error. Identify conflicts between course material and stronger evidence so the user can decide how to proceed.

For independent learning, prefer direct, authoritative, relevant, and current-enough sources. Compare definitions and context when credible sources differ. Do not force a certain conclusion when the evidence preserves a real qualification or disagreement.

LIVE RESEARCH REQUEST

6. Research Scope

Include only the Knowledge needed to meet the learning goal: core concepts, important relationships, essential vocabulary, useful examples or comparisons, common misconceptions, learning priorities, and source documentation.

Exclude the final teaching voice, decorative introduction, study schedule, practice questions, answer key, and motivational conclusion. Those are Runtime responsibilities.

7. Output Organization

Organize the Research Output under these headings: Selected Settings and Learning Goal; Learning Map; Core Knowledge by Unit; Essential Vocabulary; Examples and Comparisons; Common Misconceptions; Learning Priorities; Assessment Blueprint; Qualifications and Knowledge Gaps; and Source Register.

The Assessment Blueprint should identify concepts and skills to test without drafting the final questions.

8. Research Source Documentation

Support major factual claims with citations close enough to identify their basis. The Source Register must include the author or organization, title, date when available, direct link or document identifier, and relevant page or section when useful.

Distinguish source-supported Knowledge from explanatory examples. Never invent a source, quotation, author, date, link, or publication detail.

9. ESSENTIAL: Sequential Output Requirements

PURPOSE

When the requested Research Output may exceed what can be comfortably delivered in a single response, or when its complete length cannot be reliably predicted, divide it into multiple sequential parts while preserving structure, completeness, detail, formatting, source documentation, and continuity.

SEQUENTIAL OUTPUT RULE

LIVE RESEARCH REQUEST

Apply this requirement automatically whenever the complete output may exceed practical response length or its length cannot be reliably predicted.

Do not compress, summarize, abbreviate, or omit required work merely to fit within one response.

Favor completeness over minimizing the number of responses.

Break the output at a natural point. Do not force the Research Output into predetermined sets or fixed section groupings.

At the end of each nonfinal part, state that additional output remains, request permission to continue, and stop.

After permission is received, continue from where the previous part ended until the complete Research Output has been delivered.

The final part must clearly state that the requested Research Output is complete.

A short, predictable Research Output may be delivered in one response.

10. Governance and Final Quality Review

The Research Output must be accurate, appropriately scoped, readable, traceable, and suitable for the stated learner. Do not invent facts, simplify a concept until it becomes misleading, conceal uncertainty, or include material merely because it is interesting.

For medical, legal, safety-critical, or regulated certification subjects, treat the output as learning support only and preserve all official-source, professional, and certification boundaries.

Before completion, verify that every emphasized topic is covered; excluded topics remain outside scope; vocabulary is defined; misconceptions are corrected; examples are accurate; learning priorities match the goal; the Assessment Blueprint tests supplied Knowledge; major claims are supported; all required sections are present; and any sequential delivery concluded only after the complete Research Output was delivered.

Each section has one job:

Purpose starts Research and reserves the final study packet for Runtime.

Configuration Variables define the learner and keep the topic manageable.

Research Objectives establish what the repository must contain.

Approved Sources protect accuracy and course alignment.

Source Priority and Conflict Handling makes disagreements visible.

Scope prevents interesting material from overwhelming the learning goal.

Output Organization prepares the Knowledge for teaching and assessment.

Source Documentation keeps the material traceable.

Sequential Output Requirements protect completeness.

Governance and Final Quality Review check the entire Research Output before completion.

Prepare and Execute the Research Request

Step 1: Configure and Save the Research Request

Copy the artifact into a new document. Change the topic and variables if desired, replace “[Your name],” and list any course materials you will supply. Save it as “SGA-RES-001 Personal Study Guide Research Request v1.0.docx.”

Step 2: Confirm Research Access

Use an AI system and operating mode that can read the attachment and access the approved external sources. Mode names and capabilities vary by product. Where a thinking or reasoning control is available, use Medium or higher for this practicum. Do not use a fast-response mode such as Instant, Light, or Low unless you have confirmed that it can perform the required external research. If live research is unavailable, attach approved source documents directly. If the required sources still cannot be accessed, stop and report the limitation.

For a course-aligned project, also confirm that every required note, reading, slide, or assignment is accessible; the AI must not claim to have reviewed material it could not access. For medical, legal, safety-critical, or regulated certification topics, the selected reasoning level does not replace qualified sources or official requirements.

Step 3: Start a Clean Chat and Attach the Request

A clean working context reduces interference. On free or limited tiers, long runs may face rate limits or context trimming, so use this chat only for Part I and start another for Part III. Attach SGA-RES-001 and every required note, reading, slide, or assignment and wait until every file is available; naming a file does not supply its contents. Before beginning, confirm that the Research Request and course materials are available and that the Medium-or-higher setting selected in Step 2 remains active.

Step 4: Begin Research Execution

Send:

Please execute the attached Study Guide Research Request using the attached course materials, if any.

The prompt starts the work; the attached request controls it. If the AI begins writing the final study packet, direct it back to Research: “Complete SGA-RES-001 first. Do not create the study packet during Research.”

Step 5: Complete the Sequential Output

The AI may deliver the complete Research Output in one response when it fits comfortably. If additional output remains, save the completed part and reply “Continue” when you are ready. Repeat until the AI

clearly states that the requested Research Output is complete, then preserve every part in its original order.

Part II: Review and Save the Knowledge Repository

The AI's quality check does not replace human review. Research Output does not automatically become Knowledge; human review and approval are required before accepted information is preserved in SGA-KNW-001. Compare the Research Output with SGA-RES-001, then sample important citations to confirm that the sources exist and support the nearby claims. The goal is not to preserve the greatest amount of information; it is to preserve the right information for this learner and learning goal. Chapter 7 explains Knowledge preservation in greater depth.

Review check	Question	If the check fails
Coverage	Are all emphasized topics and learning targets addressed?	Request the missing Knowledge.
Scope	Is distracting or overly advanced material excluded?	Remove or simplify the drift.
Accuracy	Are major claims supported by qualified sources?	Correct or remove the claim.
Source integrity	Are major claims traceable to qualified sources, with accurate citations and a complete Source Register?	Correct the citation record or remove unsupported material.
Course alignment	Does the material follow supplied course requirements while exposing meaningful conflicts?	Restore the required alignment or flag the conflict.
Learner fit	Are depth, vocabulary, and explanations appropriate?	Rework the affected material.
Assessment readiness	Could every planned question be answered from the repository?	Add the missing Knowledge or revise the blueprint.

Use this correction prompt when needed:

Review the Research Output against SGA-RES-001. Identify every unmet requirement, then return a clean and complete revision with only the necessary changes. Preserve the selected learner, goal, scope, approved sources, and output organization. Do not create the final study packet. Follow SGA-RES-001's Sequential Output Requirements while delivering the revision.

Preserve the Approved Knowledge

Copy the complete approved Research Output into a new document; do not reduce it to a short summary. Preserve its explanations, examples, qualifications, citations, and Source Register. Whenever possible, copy from the editable response or document rather than from a converted file.

If the output was converted from PDF, check the headings, paragraph order, bullets, tables, and citations before approving the repository.

PERSONAL STUDY GUIDE KNOWLEDGE REPOSITORY

Repository ID: SGA-KNW-001

Project: DCPF Practicum, Personal Study Guide Assistant

Topic: [Selected topic]

Derived from: SGA-RES-001, version 1.0

Created: [Date]

Reviewed by: [Your name]

Status: Approved for practicum use

Content: Complete reviewed Study Guide Research Output

Save it as “SGA-KNW-001 Personal Study Guide Knowledge Repository v1.0.docx.” The complete repository, not a short summary, will power the learning materials.

Part III: Create the Runtime Request

Research discovers and develops educational evidence. Knowledge preserves the accepted learning intelligence. Runtime now applies that approved Knowledge to a defined assignment: a coordinated personal study packet. The guide teaches the content, the quiz checks the stated goals, and the answer key explains the reasoning. All components must agree.

Chapter 8 explains every Runtime element and the distinction between retained Knowledge and a current execution.

This practicum intentionally embeds its essential Governance rules inside the Research and Runtime requests. That keeps the controls close to the work while preserving the Governance Framework responsibility; a standalone Governance artifact can be added later when recurring controls need a maintained project-wide home.

Choose How to Supply the Knowledge Repository

The Knowledge Repository remains a separate maintained project artifact even when its contents are embedded in the Runtime Request. The same three DCPF supply routes apply here: attachment; embedding or pasting; and retrieval from a source the AI can actually access. This practicum uses the attachment and embedding choices shown below so it does not depend on a platform-specific retrieval feature.

Delivery method	Best feature	Main trade-off
Embed the complete repository	One self-contained Runtime file reduces attachment-access problems.	The file is longer, and the embedded copy must match the approved repository.
Attach the repository separately	Keeps one reusable repository and a shorter Runtime Request.	The AI must access and attend to both documents.

Delivery method	Best feature	Main trade-off
Use both methods	Adds redundancy when attachment handling is uncertain.	Repeats context and creates conflict risk if versions differ.

For this compact project, embedding the complete approved SGA-KNW-001 in Appendix A is usually easiest. Keep SGA-KNW-001 as its own approved companion file even when you paste an exact execution copy into Runtime, and identify the delivery method in the Runtime metadata. If you use connected retrieval in your own implementation, confirm that the AI can retrieve and read the complete approved repository before execution. Never rely on the repository filename alone.

A short, predictable Runtime assignment may fit in one response. A longer or less predictable Project Output may require more than one. The Runtime Request applies the rule automatically and breaks delivery at natural points, so you do not have to predict the practical response limit or define response groups in advance.

LIVE RUNTIME REQUEST

Request: Personal Study Packet

Request ID: SGA-RUN-001

Project: DCPF Practicum, Personal Study Guide Assistant

Owner: [Your name]

Version: 1.0

Status: Ready for execution

Input Knowledge: SGA-KNW-001, version 1.0, supplied in full

Knowledge delivery: [Choose one: Embedded in Appendix A / Separate attachment / Both]

Output: SGA-OUT-001, complete personal study packet

1. Purpose

Execute this document as a complete Runtime Request. Use the complete Personal Study Guide Knowledge Repository supplied by the selected delivery method as the sole factual data engine for the study packet.

Follow the Runtime Configuration Variables, Input Knowledge requirements, Task Definition, Output Requirements, Sequential Output Requirements, delivery instructions, and governance requirements in this document.

Before drafting, locate and read the entire repository. Do not merely review, summarize, or explain this request. Do not conduct new Research or add facts from general model Knowledge. If the complete required Knowledge is missing, inaccessible, incomplete, internally inconsistent, or insufficient, identify the problem and request direction rather than inventing content.

LIVE RUNTIME REQUEST

2. Runtime Configuration Variables

Title: *Exploring the Solar System: A Personal Study Guide*. **Learner:** High-school student or adult general learner. **Tone:** Friendly, clear, and encouraging. **Guide length:** Approximately 1,500–2,000 words. **Quick checks:** Two per major unit. **Practice quiz:** 12 questions, six multiple choice, four short answer, and two explain-or-compare questions. **Answer key:** Required with brief explanations. **Study plan:** Three sessions. **Source notes:** Brief. **External Research:** Not authorized. **Status:** Draft pending human review.

3. Input Knowledge

Use SGA-KNW-001, version 1.0, for all facts, concepts, vocabulary, examples, misconceptions, assessment content, and source notes. You may reorganize and simplify the repository for teaching, but you may not introduce unsupported content.

For this practicum, supply the complete repository by embedding it, attaching it separately, or using both methods. If both copies are supplied, verify that their IDs and versions match. Stop if they conflict.

4. Task Definition

Set the destination. Open with the learning goal and a short list of what the learner should understand by the end.

Teach the topic. Create a logically ordered guide using short sections, clear explanations, essential vocabulary, accurate examples, and corrected misconceptions from the repository.

Check understanding. Place two brief quick-check questions after each major unit. Do not reveal their answers immediately beside the questions.

Build the practice quiz. Create the required mix of questions. Every question must test an identified learning priority and be answerable from the study guide and repository.

Explain the answers. Provide a separate answer key with the correct response and a concise explanation. Explain why incorrect multiple-choice options are wrong when that distinction teaches an important concept.

Create the study plan. Divide the guide, review, and quiz into three realistic study sessions.

5. Output Requirements

Organize the Project Output as: Title; Learning Goal; What You Will Learn; Study Guide; Essential Vocabulary; Quick Review; Practice Quiz; Answer Key and Explanations; Three-Session Study Plan; Source Notes; and Internal Validation Note.

The packet must feel like one learning experience, not unrelated AI outputs. Use plain language without talking down to the learner. Avoid trick questions, ambiguous wording, unbalanced multiple-choice options, unsupported facts, and questions whose answers never appear in the guide.

LIVE RUNTIME REQUEST

The Internal Validation Note must identify the request and repository versions, Knowledge delivery method, whether the complete repository was supplied and read before drafting, completed requirements, unresolved gaps, outside additions detected, and continuing need for human approval.

6. ESSENTIAL: Sequential Output Requirements

PURPOSE

When the requested Project Output may exceed what can be comfortably delivered in a single response, or when its complete length cannot be reliably predicted, divide it into multiple sequential parts while preserving structure, completeness, detail, formatting, and continuity.

SEQUENTIAL OUTPUT RULE

Apply this requirement automatically whenever the complete output may exceed practical response length or its length cannot be reliably predicted.

Do not compress, summarize, abbreviate, or omit required work merely to fit within one response.

Favor completeness over minimizing the number of responses.

Break the output at a natural point. Do not force the Project Output into predetermined sets or fixed section groupings.

At the end of each nonfinal part, state that additional output remains, request permission to continue, and stop.

After permission is received, continue from where the previous part ended until the complete Project Output has been delivered.

The final part must clearly state that the requested Project Output is complete.

A short, predictable Project Output may be delivered in one response.

7. Validation and Human Review

Keep the packet accurate to SGA-KNW-001, aligned with the stated learner and goal, internally consistent, and fair in its assessment. Preserve meaningful qualifications and do not turn uncertainty into a false fact.

Do not present the packet as professional advice or as a substitute for official course, certification, safety, medical, or legal requirements.

Before delivery, verify that the guide covers every learning priority; vocabulary is defined; quick checks and quiz questions are answerable; the answer key is accurate; question difficulty fits the learner; the study plan uses the completed materials; all required sections are present; no outside Research was added; and any sequential delivery is complete.

Appendix A: Embedded Knowledge Repository

When using **Embedded** or **Both**, paste the complete approved SGA-KNW-001 below.

[BEGIN SGA-KNW-001, VERSION 1.0]

[Paste the complete approved Personal Study Guide Knowledge Repository here.]

[END SGA-KNW-001, VERSION 1.0]

LIVE RUNTIME REQUEST

When using a Separate attachment, delete the [BEGIN SGA-KNW-001, VERSION 1.0] marker, placeholder, and [END SGA-KNW-001, VERSION 1.0] marker. Replace the entire Appendix A repository block with this statement: "SGA-KNW-001, version 1.0, is supplied as a separate attachment. Appendix A is not used."

Prepare and Execute the Runtime Request

Step 1: Configure and Save the Runtime Request

Copy the Runtime artifact, replace "[Your name]," and adjust the title, tone, guide length, question mix, or study schedule. Keep those settings aligned with the repository. Select the Knowledge delivery method and save the file as "SGA-RUN-001 Personal Study Guide Runtime Request v1.0.docx."

Step 2: Start a Clean Chat and Supply the Execution Materials

Do not continue in the Research chat. Start a clean Runtime chat so the study-packet instructions and repository receive the clearest available working context. Supply the materials that match the delivery method recorded in SGA-RUN-001.

Embedded: Attach the completed Runtime Request and confirm that the full SGA-KNW-001 appears in Appendix A.

Separate attachment: Attach SGA-RUN-001 and SGA-KNW-001.

Both: Attach both files and confirm that the repository IDs, versions, and contents match.

Do not attach an empty Appendix A as if it contained the repository. Before execution, confirm that the AI can open and read every required file supplied by the selected route. Because Runtime must read and apply the complete repository, use Medium or higher reasoning when that control is available. External research remains unauthorized.

This setting is especially useful here because Runtime must keep the study guide, quick checks, practice quiz, answer key, and study plan synchronized; it does not replace human review.

Step 3: Begin Runtime Execution

Send one instruction:

Please execute the attached Study Guide Runtime Request using the complete Knowledge Repository supplied by the delivery method identified inside it.

The AI may deliver the complete packet in one response when it fits comfortably. If additional output remains, save the completed part and reply "Continue" when you are ready. Repeat until the AI clearly states that the requested Project Output is complete. If Knowledge is missing, inaccessible, incomplete,

or conflicting, correct the packaging or return the gap to Research rather than asking Runtime to invent it.

Step 4: Preserve the Unreviewed Study Packet

If the packet arrived in multiple parts, combine them in their original order into one working document. Label the complete output Draft, Not Yet Approved. Preserve the Internal Validation Note for comparison during review, and keep the draft separate from the approved packet.

Part IV: Review and Preserve the Study Packet

Review check	Question	If the check fails
Knowledge fidelity	Does the packet stay within SGA-KNW-001?	Remove additions or return to Research.
Teaching clarity	Could the stated learner understand the guide?	Clarify without changing the facts.
Goal alignment	Does every section support the learning goal?	Remove drift or restore missing content.
Question quality	Are questions clear, fair, varied, and answerable from the guide?	Rewrite the affected questions.
Answer accuracy	Does every answer match the guide and repository?	Correct both question and key as needed.
Coordination	Do the guide, quiz, key, and study plan work together?	Repair the inconsistent component.
Assignment completion	Are the guide, quick checks, quiz, answer key, study plan, source notes, and validation note all present?	Request the missing component.
Scope and authority	Does the packet avoid unsupported professional-advice, safety, legal, medical, or certification claims?	Correct or remove the claim, or return it to Research.
Validation	Are versions, delivery, access, gaps, and human approval recorded?	Complete the validation note.

Use this correction prompt:

Review SGA-OUT-001 against SGA-RUN-001 and SGA-KNW-001. Identify every unmet requirement, then return a clean and complete revised study packet with only the necessary changes. Do not conduct new Research or introduce Knowledge outside the supplied repository. Follow SGA-RUN-001's Sequential Output Requirements while delivering the revision.

Route missing or unreliable Knowledge to Research. Route teaching, organization, quiz, answer-key, study-plan, coordination, or presentation problems to Runtime. Preserve the approved repository while making Runtime corrections. Chapter 8 explains feedback routing in greater detail.

Take the Quiz

Try the practice quiz without looking at the answer key. If a question depends on material the guide never taught, the problem belongs in Runtime. If the repository lacks the Knowledge needed to teach or answer it, return the gap to Research.

Approve and Preserve

APPROVED PERSONAL STUDY PACKET

Output ID: SGA-OUT-001

Project: DCPF Practicum, Personal Study Guide Assistant

Topic: [Selected topic]

Produced by: SGA-RUN-001, version 1.0

Knowledge used: SGA-KNW-001, version 1.0

Created: [Date]

Reviewed by: [Your name]

Status: Approved for practicum use

Content: Complete reviewed personal study packet

Save the approved packet as “SGA-OUT-001 [Topic] Personal Study Packet v1.0.docx.” Keep the Research Request, Knowledge Repository, Runtime Request, draft packet, and approved packet as separate records.

The Completed Practicum

You began with a topic and ended with a coordinated learning system:

**Research Request → Research Output → human review and acceptance → Accepted
Knowledge → Knowledge Repository → Runtime Request → Project Output → human review
→ preservation**

For this practicum, the Project Output coordinates the study guide, quick checks, practice quiz, answer key, and three-session study plan around one approved body of Knowledge.

Change the topic whenever curiosity strikes. The workflow stays the same because the guide teaches what Research established, and the assessment tests what the guide actually taught.

[Return to Table of Contents](#)

Building a Software Documentation Assistant

DCPF is an experimental approach. This practicum is a training environment, not evidence of production readiness. Validate any adapted implementation within your own environment before real-world use.

When the Documentation Disagrees with Itself

Software projects rarely keep all of their truth in one tidy place. A README may describe last year’s setup, a test report may cover only part of the current system, and a release note may announce a change that another document never adopted. The information exists, but it does not yet agree with itself.

This final practicum carries the Research-to-Knowledge-to-Runtime cycle from Chapters 10 through 13 into software documentation. You will use DCPF to turn scattered technical evidence into a practical Developer Onboarding Guide, without writing code, repairing the fictional application, or importing outside software assumptions. The central task is to establish what the supplied evidence actually supports before asking AI to explain the system to someone else.

The working project is TrailWatch, a small fictional service for recording hiking-trail observations. Its compact evidence pack contains seven short source excerpts. Several deliberately conflict, giving Research a realistic job instead of a ceremonial one.

Stage	Your action	Artifact produced
Research setup	Configure the request and preserve the embedded evidence pack	Research Request
Research execution	Reconcile the technical evidence and expose remaining gaps	Research Output
Knowledge handoff	Review and preserve the accepted technical truth	Knowledge Repository
Runtime execution	Turn approved Knowledge into onboarding documentation	Project Output
Final review	Check every instruction, limitation, and source note	Approved Developer Onboarding Guide

Part I: Build Technical Truth Before Writing Documentation

The Project Is Small. The Reasoning Is Not.

TrailWatch is intentionally modest. The evidence pack describes one small web service, a handful of routes, a JSON data file, and a basic test report. The technical vocabulary is explained as it appears, and the exercise does not require programming or command-line execution.

The challenge comes from evidence quality. One source is current, another is historical, one reports test results, and another describes future work as a proposal. Research must preserve those differences. A source can be useful without being authoritative for every question.

Only Two Controller Artifacts

You will configure only two DCPF controller artifacts: the Research Request and the Runtime Request. The TrailWatch evidence pack is embedded inside the Research Request, so it travels with the instructions that govern its analysis. Governance rules are also embedded in both requests. This is a complete small-project Governance implementation: the controls operate where the AI works, while a standalone Governance artifact remains optional until recurring standards need a maintained project-wide home.

After human review and approval, the accepted portions of the Research Output become Accepted Knowledge and are preserved in the Knowledge Repository. For this practicum, the approved Research Output may be retained substantially intact as the Repository's contents, so no separate Knowledge-generation request is required. It is a generated handoff, not a third controller template that you must design. Runtime then uses that approved repository as its sole factual source.

Research determines what the software evidence supports. Runtime determines how to explain that approved Knowledge to a new developer.

What Makes a Technical Claim Safe to Use?

For this practicum, each material conclusion should be treated as confirmed, inferred, or unresolved. Confirmed claims have direct support from appropriate current evidence. Inferences connect evidence without pretending that the evidence said more than it did. Unresolved questions remain open when the supplied sources cannot settle them.

A passing test proves only what the test exercised. A release note establishes dated change context but does not prove every file was updated correctly. A newer document is not automatically more authoritative than current implementation evidence. Research must compare scope, status, and purpose, not simply choose the source with the newest date.

Part II: Create the Research Request

The Research Request below is a complete copyable artifact. Its Appendix A contains the entire seven-source TrailWatch evidence pack. Keep the artifact whole when copying it into a new document.

The request prohibits outside sources and unstated model Knowledge. That restriction is deliberate: this practicum teaches closed-source technical synthesis. The AI must work with the supplied project evidence, disclose what it cannot establish, and stop before producing the final onboarding guide.

Chapter 6 explains every Research element in depth; this chapter focuses on applying them to closed-source technical synthesis.

The essential Sequential Output Requirements use the current standard form. They activate automatically only when the complete Research Output may exceed practical response length or cannot be reliably predicted.

LIVE RESEARCH REQUEST

Request: TrailWatch Software Documentation Research Request

Request ID: SDA-RES-001

Project: DCPF Practicum, Software Documentation Assistant

Owner: [Your name]

Version: 1.0

Status: Ready for execution

Expected source set: Seven embedded TrailWatch evidence sources

Output: Research Output for review and preservation as SDA-KNW-001

1. Purpose

Execute this document as a complete Research Request. Inspect the embedded TrailWatch evidence pack, establish the current and supportable technical truth, resolve or expose source conflicts, and produce a traceable Software Documentation Research Output for human review, possible acceptance as Knowledge, and preservation in the Knowledge Repository.

Follow the Research Configuration Variables, Research Objectives, Approved Information Sources, Source Priority and Conflict Handling, Research Scope, Output Organization, Research Source Documentation, Sequential Output Requirements, and Governance and Final Quality Review in this document.

Do not merely summarize or critique this request. Do not write the final Developer Onboarding Guide during Research. Do not use the internet, outside model Knowledge, or unstated software assumptions. If an embedded source is missing, unreadable, truncated, or internally incomplete, stop, report the exact limitation, and request direction instead of silently repairing the gap.

2. Research Configuration Variables

Source project: TrailWatch Documentation Lab. Expected release: Do not assume; determine from the evidence. Source delivery: Appendix A of this Research Request. Expected sources: Seven. Code execution: Not required or authorized for the introductory practicum. Primary Knowledge audience: A developer new to TrailWatch. Research goal: Establish current purpose, release, structure, local operation, interfaces, storage behavior, test evidence, known limitations, documentation drift, and unresolved questions. External sources: Prohibited. Final Runtime product: Developer Onboarding Guide; not created during Research.

LIVE RESEARCH REQUEST

3. Research Objectives

Verify source access. Inventory all seven embedded sources by source ID, path, stated status, date when supplied, evidence role, and access result before claiming complete coverage.

Establish source authority. Determine which evidence governs current release information, dependencies, implemented behavior, interface contracts, verification status, historical context, and future proposals. Treat authority as scoped rather than universal.

Describe the current system. Establish the supported release, runtime, major components, request flow, configuration, routes, fields, persistence behavior, local start command, health check, and documented test command.

Resolve documentation drift. Compare current controlled notes, the package manifest, the test report, release history, the root README, the superseded API document, and the limitations-and-roadmap document. Preserve both the conflict and the evidence supporting each conclusion.

Separate current behavior from history and proposals. Do not present retired v1 instructions, planned authentication, proposed SQLite storage, or a retention target as implemented capabilities.

Expose gaps and limits. Identify unsupported production claims, unverified behavior, and questions the evidence cannot answer. Do not convert absence of evidence into certainty.

4. Approved Information Sources

Use only the seven sources embedded in Appendix A. They include a root README, package manifest, current controlled system note, superseded API document, verification report, change log, and known-limitations-and-roadmap document.

Internet sources, package indexes, external product documentation, and unstated model Knowledge are prohibited. Do not import general software practices as though TrailWatch implements them.

5. Source Priority and Conflict Handling

User instructions govern the assignment. For the current release and dependencies, compare the package manifest with the change log. For current implementation behavior, prioritize the controlled current-system note and corroborate it with the verification report where that report directly applies. Use the known-limitations document for disclosed operational limits and proposal status.

Treat the root README as orientation material under review. Treat the v1 API document as historical because it is explicitly superseded. A reported passing test supports only the behavior named in that report; it does not establish production readiness or unrelated behavior.

When sources disagree, record the competing claims, source status, supported conclusion, confidence, and any remaining gap. Do not silently select the newest date, the most convenient instruction, or the statement that produces the smoothest guide.

LIVE RESEARCH REQUEST

6. Research Scope

Include source coverage; source authority and status; verified project purpose and release; project map; current runtime and dependencies; configuration and local operation; request flow; supported API; data storage; verification evidence; documentation conflicts; known limitations; unimplemented proposals; unresolved questions; and source history.

Exclude code changes, documentation rewrites, a final onboarding guide, production deployment advice, invented security recommendations, and general software guidance unrelated to the Research goal. Never create or expose real credentials.

7. Output Organization

Organize the Research Output under these headings: Executive Finding; Source Coverage Ledger; Source Authority and Status; Verified Project Snapshot; Project Map and Request Flow; Local Setup and Configuration; Supported API and Data Fields; Persistence Behavior; Verification Evidence; Conflicts Resolved; Known Limitations and Unimplemented Proposals; Unresolved Questions; Source Register; and Internal Validation Note.

The Source Coverage Ledger must account for all seven sources. Conflicts Resolved must display competing claims and the evidence supporting the conclusion. The Internal Validation Note must record the Research Request version, source-access status, outside-source prohibition, unresolved gaps, and continuing need for human technical review.

8. Research Source Documentation

Cite the source ID and path for each material conclusion. Use the most specific useful subsection or evidence statement. When a conclusion depends on more than one source, show the combination used.

Distinguish a test result reported by Source A5 from a test executed during this practicum. Code execution is not authorized, so do not claim to have run TrailWatch. Never invent a file, path, command, route, field, test result, source status, or implementation detail.

LIVE RESEARCH REQUEST

9. ESSENTIAL: Sequential Output Requirements

PURPOSE

When the requested Research Output may exceed what can be comfortably delivered in a single response, or when its complete length cannot be reliably predicted, divide it into multiple sequential parts while preserving structure, completeness, detail, formatting, source documentation, and continuity.

SEQUENTIAL OUTPUT RULE

Apply this requirement automatically whenever the complete output may exceed practical response length or its length cannot be reliably predicted.

Do not compress, summarize, abbreviate, or omit required work merely to fit within one response.

Favor completeness over minimizing the number of responses.

Break the output at a natural point. Do not force the Research Output into predetermined sets or fixed section groupings.

At the end of each nonfinal part, state that additional output remains, request permission to continue, and stop.

After permission is received, continue from where the previous part ended until the complete Research Output has been delivered.

The final part must clearly state that the requested Research Output is complete.

A short, predictable Research Output may be delivered in one response.

10. Governance and Final Quality Review

Keep the Research Output accurate to the supplied evidence, traceable, internally consistent, and explicit about uncertainty. Preserve the difference between current and historical instructions, reported and executed tests, policy targets and enforcement, known limitations and recommendations, and proposals and implemented capabilities.

Before completion, verify that all seven sources are accounted for; the release and runtime are evidence-based; current and retired routes are separated; every command, path, route, field, configuration value, and test claim is supported; README drift is exposed; security and production limits remain visible; proposed work is not presented as available; every material conflict includes evidence; unresolved gaps remain open; no outside Knowledge was added; all required sections are present; and any sequential delivery concluded only after the complete Research Output was delivered.

Appendix A: Embedded TrailWatch Evidence Pack

Use Sources A1 through A7 exactly as supplied. The evidence pack is fictional and exists only for this practicum. Paths, commands, routes, versions, and behaviors must not be generalized to a real system.

Source A1: README.md

Status: Root orientation document; current status under review. Last updated: September 12, 2025.

TrailWatch 1.1 is a small Flask service for recording trail sightings. Use Python 3.10. Start locally with 'flask run --port 5000' and check 'GET /status'. Current requests use '/api/v1/sightings' and require the demonstration header 'X-Trail-Key: demo-key'. The local JSON file is 'trailwatch.json'.

LIVE RESEARCH REQUEST

Source A2: pyproject.toml

Status: Current package manifest. Last updated: July 28, 2026.

Project name: trailwatch. Version: 1.2.0. Requires Python 3.11 or later. Runtime dependencies: FastAPI, Uvicorn, and Pydantic. Development dependency: pytest. Documented editable installation command: `'python -m pip install -e ".[dev]"`.

Source A3: docs/current-system.md

Status: Controlled current-system note. Approved: August 1, 2026.

TrailWatch 1.2.0 is a local demonstration FastAPI service for recording hiking-trail observations. Important paths: 'trailwatch/main.py' creates the application; 'trailwatch/routes.py' handles HTTP routes; 'trailwatch/models.py' validates request and response data; 'trailwatch/store.py' reads and writes the JSON data file; 'tests/' contains automated checks; and 'data/observations.json' is the default local store.

Local start command: `'python -m uvicorn trailwatch.main:app --reload'`. Default address: `'http://127.0.0.1:8000'`. Health check: `'GET /health'`. Current routes: `'GET /api/v2/observations'`, `'POST /api/v2/observations'`, and `'GET /api/v2/observations/{observation_id}'`. List parameters: 'limit' defaults to 20 and 'offset' defaults to 0. Create fields: 'trail_name', 'observed_at', and 'condition_note' are required; 'wildlife_seen' is optional.

Configuration: 'TRAILWATCH_DATA_FILE' may replace the default JSON path. 'TRAILWATCH_SEED_SAMPLE=true' adds sample records only when the selected data file does not yet exist. The service has no authentication. It is intended for single-process local demonstration use.

Source A4: docs/api-v1-legacy.md

Status: Superseded API document retained for migration history. Last current for release 1.1.

The v1 interface used `'/api/v1/sightings'`, the field 'trail_condition', the health route `'/status'`, port 5000, and the demonstration 'X-Trail-Key' header. Do not use this document as the current operating contract.

Source A5: tests/verification-report.txt

Status: Approved verification report. Test date: July 30, 2026. Environment: Python 3.11.6. Command reported: `'python -m pytest'`. Result reported: 12 passed, 0 failed.

Covered checks: 'GET /health' returns status and version; the v2 list route applies default pagination; a valid v2 create request is stored and returned; missing required fields are rejected; an existing observation can be retrieved by ID; a missing ID returns a not-found response; and sample seeding occurs only for a new data file. The report does not cover concurrent writes, authentication, production deployment, or automatic retention deletion.

Source A6: CHANGELOG.md

Status: Approved release history. Current entry: version 1.2.0, August 1, 2026.

Release 1.2.0 replaced Flask with FastAPI, moved local service startup to Uvicorn on the default address, renamed sightings to observations, introduced `'/api/v2/observations'`, changed 'trail_condition' to 'condition_note', replaced `'/status'` with `'/health'`, removed the demonstration key requirement, and changed the default store to `'data/observations.json'`. The v1 document remains only for migration history.

LIVE RESEARCH REQUEST

Source A7: docs/known-limitations-and-roadmap.md

Status: Current limitations and unapproved roadmap. Last updated: August 1, 2026.

Known limitations: TrailWatch has no authentication or user permissions; JSON writes have no concurrent-write protection; the service is not approved for public or production deployment; and the stated 30-day record-retention target is not automatically enforced. Manual file review is still required.

Roadmap proposals: replace JSON storage with SQLite, add authenticated users, enforce retention automatically, and provide container deployment. These items are proposals only and are not approved as implemented capabilities.

Prepare and Execute the Research Request

Step 1: Configure and Save the Complete Request

Copy the entire artifact, including Appendix A, into a new document. Replace “[Your name]” and adjust only clearly labeled variables. Save it as “SDA-RES-001 TrailWatch Software Documentation Research Request v1.0.docx.” Keeping the source pack inside the request prevents an attachment from quietly disappearing between instructions and execution.

Step 2: Confirm Research Access

Use an AI system and operating mode that can read the complete attachment. Mode names and capabilities vary by product. Where a thinking or reasoning control is available, use Medium or higher for this practicum. Do not use a fast-response mode such as Instant, Light, or Low unless you have confirmed that it can read and analyze the complete Research Request and its seven embedded evidence sources. External research remains unauthorized. If the complete request or any embedded source cannot be accessed, stop and report the limitation.

Step 3: Start a Clean Chat and Attach the Request

A clean working context reduces interference. Attach SDA-RES-001 and wait until the AI can open and read the complete file; naming a file does not supply its contents. Do not attach the Runtime Request or continue an earlier practicum chat. Before beginning, confirm that the AI can read the Research Request and all seven embedded sources and that the Medium-or-higher setting selected in Step 2 remains active.

Step 4: Begin Research Execution

Send:

Please execute the attached TrailWatch Software Documentation Research Request, including its complete embedded evidence pack.

The prompt starts the work; SDA-RES-001 controls it. If the AI begins writing the onboarding guide, direct it back to Research: “Complete SDA-RES-001 first. Do not create the Developer Onboarding Guide during Research.”

Step 5: Complete the Sequential Output

The AI may deliver the complete Research Output in one response when it fits comfortably. If additional output remains, it must request permission and stop. Save the completed part and reply “Continue” when you are ready. Repeat until the AI clearly states that the requested Research Output is complete, then preserve every part in its original order.

Part III: Review and Save the Knowledge Repository

The AI’s quality check does not replace human technical review. Research Output does not automatically become Knowledge; human review and approval are required before accepted information is preserved in SDA-KNW-001. Compare the complete Research Output with SDA-RES-001. A polished explanation is not enough. The repository must show how the AI decided which technical claims were current, which were historical, and which remained unsupported. Chapter 7 explains Knowledge preservation in greater depth.

Review check	Question	If the check fails
Source coverage	Are all seven embedded sources inventoried and accessible?	Request the missing coverage or document the limitation.
Authority	Is each source used only for the questions it can support?	Correct the authority reasoning.
Current system	Are release, runtime, commands, routes, fields, and paths evidence-based?	Correct or remove the unsupported claim.
Conflict handling	Are current and historical claims displayed and reconciled?	Restore the competing evidence and conclusion.
Test integrity	Is the reported test result kept separate from executed testing?	Correct the verification language.
Limitations	Are production, security, concurrency, and retention limits visible?	Restore the missing qualification.
Proposal status	Are roadmap items clearly unimplemented?	Remove any claim of current availability.
Traceability	Can every material conclusion be traced to source IDs and paths?	Add the source connection or remove the claim.

Use this correction prompt when needed:

Review the Research Output against SDA-RES-001. Identify every unmet requirement, then return a clean and complete revision with only the necessary corrections. Preserve all seven-source traceability, current-versus-historical distinctions, reported test status, known limitations, unresolved questions, and the required output organization. Do not create the Developer Onboarding Guide. Follow SDA-RES-001's Sequential Output Requirements while delivering the revision.

Preserve the Approved Technical Knowledge

Copy the complete approved Research Output into a new document. Preserve the Source Coverage Ledger, conflict history, technical details, limitations, source citations, Source Register, and Internal Validation Note. Do not replace it with a shorter summary. Whenever possible, copy the approved Research Output from an editable response or document rather than a converted file.

If the output was converted from PDF, check the headings, paragraph order, bullets, tables, commands, paths, source IDs, citations, conflict statements, and limitation statements before approving the repository.

TRAILWATCH SOFTWARE KNOWLEDGE REPOSITORY

Repository ID: SDA-KNW-001

Project: DCPF Practicum, Software Documentation Assistant

Derived from: SDA-RES-001, version 1.0

Source set: Seven embedded TrailWatch evidence sources

Created: [Date]

Reviewed by: [Your name]

Status: Approved for controlled Runtime use

Save the approved repository as “SDA-KNW-001 TrailWatch Software Knowledge Repository v1.0.docx.” The repository is now the controlled boundary between evidence analysis and documentation production.

Part IV: Create the Runtime Request

Runtime converts the approved repository into a guide for a developer who is new to TrailWatch. It may reorganize and explain the Knowledge, but it may not reopen Appendix A, conduct new Research, or fill technical gaps from general programming Knowledge. Chapter 8 explains every Runtime element and the distinction between retained Knowledge and a current execution.

Choose How to Supply the Knowledge Repository

The Knowledge Repository remains a separate maintained project artifact even when an exact execution copy is embedded in the Runtime Request. The same three DCPF supply routes apply: attachment;

embedding or pasting; and retrieval from a source the AI can actually access. This practicum uses attachment and embedding so the exercise remains platform-neutral.

Delivery method	Best feature	Main trade-off
Embed the complete repository	One self-contained Runtime file reduces access problems.	The file is longer, and the embedded copy must match the approved repository.
Attach the repository separately	Keeps one reusable repository and a shorter request.	The AI must access and attend to both documents.
Use both methods	Adds redundancy when attachment handling is uncertain.	Creates conflict risk if IDs or versions differ.

For this practicum, embedding the complete approved SDA-KNW-001 in Appendix A is usually easiest. Keep the approved repository as its own companion file and paste only an exact execution copy into Runtime.

LIVE RUNTIME REQUEST

Request: TrailWatch Developer Onboarding Guide Request

Request ID: SDA-RUN-001

Project: DCPF Practicum, Software Documentation Assistant

Owner: [Your name]

Version: 1.0

Status: Ready for execution

Input Knowledge: SDA-KNW-001, version 1.0, supplied in full

Knowledge delivery: [Embedded in Appendix A / Separate attachment / Both]

Output: SDA-OUT-001, TrailWatch Developer Onboarding Guide

1. Purpose

Execute this document as a complete Runtime Request. Use the complete approved TrailWatch Software Knowledge Repository as the sole factual source for a practical Developer Onboarding Guide.

Follow the Runtime Configuration Variables, Input Knowledge requirements, Task Definition, Output Requirements, Sequential Output Requirements, delivery instructions, and Validation and Human Review in this document.

Do not merely summarize this request, reopen the Research Request, inspect its evidence pack, or conduct new Research. Before drafting, locate and read the entire Knowledge Repository and confirm its ID, version, source-coverage status, and test status. If the repository is unavailable, incomplete, conflicting, or insufficient for a required instruction, report the exact problem rather than adding outside Knowledge.

LIVE RUNTIME REQUEST

2. Runtime Configuration Variables

Title: TrailWatch Developer Onboarding Guide. Audience: A developer new to TrailWatch who understands basic Python, HTTP, JSON, and command-line use. Goal: Help the developer understand the service, prepare a local environment, start and check it, understand the supported interface, avoid legacy documentation traps, and prepare a safe first contribution. Format: Practical technical guide. Target length: Approximately 1,600–2,200 words, excluding command and JSON examples. Tone: Direct, welcoming, and technically precise. Environment: Local fictional demonstration project only. Examples: Use only commands, paths, routes, fields, and values supported by SDA-KNW-001. External Research: Not authorized. Status: Draft pending human technical review.

3. Input Knowledge

Use SDA-KNW-001, version 1.0, for every technical claim, command, route, field, configuration value, architectural statement, limitation, conflict warning, and source note. Preserve the difference between current behavior, historical behavior, reported verification, implementation gaps, and unapproved proposals.

For this practicum, supply the complete repository by embedding it, attaching it separately, or using both methods. If both copies are supplied, verify that their IDs and versions match and stop if they conflict. Do not treat SDA-RES-001 or its embedded evidence pack as a substitute for approved Knowledge during Runtime.

4. Task Definition

Orient the developer. Explain what TrailWatch is, its fictional training scope, verified release, current technology, and production limitations.

Map the project. Describe the important paths and the responsibility of each major component without mechanically listing every source record.

Create a verified local quick start. State prerequisites, installation, start, health-check, and documented test steps. Never claim that a command succeeded for the reader; distinguish expected results from the reader's actual execution.

Explain the request flow. Walk from HTTP route handling through validation and JSON persistence. Include configuration and sample-seeding behavior needed for safe local use.

Teach the current API. Present supported routes, list parameters, create fields, and one compact request example. Make v1 migration traps visible without allowing historical instructions to dominate the guide.

Preserve operational truth. Explain the reported test coverage, current storage limits, missing production controls, retention gap, and proposal status without turning the onboarding guide into a risk report.

Prepare a first contribution. Provide a short checklist for selecting a scoped change, checking relevant Knowledge, updating tests and documentation, running verification, and recording unresolved issues for human review.

LIVE RUNTIME REQUEST

5. Output Requirements

Organize the Project Output as: Title; Welcome and Scope; Current System at a Glance; Project Map; Prerequisites; Local Quick Start; How a Request Moves Through TrailWatch; Supported API Quick Reference; Configuration and Data Storage; Verification and Test Status; Common Legacy Traps; Known Limitations and Proposed Work; First-Contribution Checklist; Source Notes; and Internal Validation Note.

Use short sections, precise commands, compact tables only where useful, and clearly labeled command or JSON examples. Keep current instructions separate from historical warnings. Do not describe TrailWatch as production-ready, claim that it has authentication, imply that JSON writes support concurrent processes, state that retention deletion is automatic, or present roadmap items as available.

The Internal Validation Note must identify the Runtime Request and Knowledge Repository versions, Knowledge delivery method, whether the complete repository was supplied and read before drafting, inherited source-coverage and test status, completed output requirements, unresolved gaps, outside additions detected, and continuing need for human technical approval.

6. ESSENTIAL: Sequential Output Requirements

PURPOSE

When the requested Project Output may exceed what can be comfortably delivered in a single response, or when its complete length cannot be reliably predicted, divide it into multiple sequential parts while preserving structure, completeness, detail, formatting, and continuity.

SEQUENTIAL OUTPUT RULE

Apply this requirement automatically whenever the complete output may exceed practical response length or its length cannot be reliably predicted.

Do not compress, summarize, abbreviate, or omit required work merely to fit within one response.

Favor completeness over minimizing the number of responses.

Break the output at a natural point. Do not force the Project Output into predetermined sets or fixed section groupings.

At the end of each nonfinal part, state that additional output remains, request permission to continue, and stop.

After permission is received, continue from where the previous part ended until the complete Project Output has been delivered.

The final part must clearly state that the requested Project Output is complete.

A short, predictable Project Output may be delivered in one response.

7. Validation and Human Review

Keep the guide faithful to SDA-KNW-001, internally consistent, appropriate for the stated developer, and explicit about source limitations. Never repair a Knowledge gap with general programming assumptions, new repository analysis, or outside technical advice.

Before delivery, verify that the release, runtime, commands, paths, routes, fields, parameters, configuration values, and reported test status are supported; v1 material is clearly historical; authentication, concurrency, production, and retention limits remain visible; proposals are not presented as implemented; all required sections are present; source notes are traceable; no outside Knowledge was added; and any sequential delivery is complete.

LIVE RUNTIME REQUEST

Appendix A: Embedded Knowledge Repository

When using Embedded or Both, paste the complete approved SDA-KNW-001 below.

[BEGIN SDA-KNW-001, VERSION 1.0]

[Paste the complete approved TrailWatch Software Knowledge Repository here.]

[END SDA-KNW-001, VERSION 1.0]

When using a Separate attachment, replace the placeholder with: “SDA-KNW-001, version 1.0, is supplied as a separate attachment. Appendix A is not used.”

Prepare and Execute the Runtime Request

Step 1: Configure and Save the Runtime Request

Copy the entire Runtime artifact into a new document. Replace “[Your name],” select the Knowledge delivery method, and adjust the audience, length, or tone only if the repository can support the change. Save it as “SDA-RUN-001 TrailWatch Developer Onboarding Runtime Request v1.0.docx.”

Step 2: Start a Clean Chat and Supply the Execution Materials

Do not continue in the Research chat. Start a clean Runtime chat so the onboarding instructions and approved repository receive the clearest available working context. Supply the materials that match the delivery method recorded in SDA-RUN-001. Runtime should encounter the approved repository, not the unresolved evidence debate that produced it.

Embedded: Attach the completed Runtime Request and confirm that Appendix A contains the complete SDA-KNW-001.

Separate attachment: Attach SDA-RUN-001 and SDA-KNW-001.

Both: Attach both files and confirm that the repository IDs, versions, and contents match.

Do not attach an empty Appendix A as if it contained the repository. Before execution, confirm that the AI can open and read every required file supplied by the selected route. Because Runtime must read and apply the complete repository, use Medium or higher reasoning when that control is available. External research remains unauthorized.

This setting is especially useful here because Runtime must keep commands, paths, routes, field names, configuration values, test status, and limitations synchronized; it does not replace human technical review.

Step 3: Begin Runtime Execution

Send:

Please execute the attached TrailWatch Developer Onboarding Runtime Request using the complete approved Knowledge Repository supplied inside or with it.

The AI may deliver the complete guide in one response when it fits comfortably. If additional output remains, save the completed part and reply “Continue” when you are ready. Repeat until the AI clearly states that the requested Project Output is complete. If Knowledge is missing, inaccessible, incomplete, or conflicting, correct the packaging problem or return the gap to Research rather than asking Runtime to invent an answer. Do not authorize Runtime to browse for an answer or reopen the embedded Research evidence.

Step 4: Preserve the Unreviewed Guide

If the guide arrives in multiple parts, combine the parts in their original order. Label the complete output Draft, Not Yet Approved. Preserve the Internal Validation Note and keep the draft separate from the approved version.

Part V: Review and Preserve the Developer Onboarding Guide

A technical guide can sound confident while carrying one outdated command that prevents a new developer from getting started. Review the Project Output against both SDA-RUN-001 and SDA-KNW-001.

Review check	Question	If the check fails
Knowledge fidelity	Does every technical claim stay within SDA-KNW-001?	Remove additions or return the gap to Research.
Current instructions	Are v1 routes, fields, port, framework, and key clearly historical?	Correct the current-versus-legacy boundary.
Quick start	Are prerequisites, installation, start, health, and test commands supported?	Correct or remove the unsupported instruction.
System explanation	Do project paths and request flow agree with the repository?	Repair the explanation without new Research.
Operational limits	Are security, concurrency, production, and retention limits visible?	Restore the necessary warning.
Proposal status	Are SQLite, authentication, retention automation, and containers still proposals?	Remove any implementation claim.
Usability	Could the stated developer follow the guide without decoding the evidence pack?	Clarify the presentation within approved Knowledge.

Review check	Question	If the check fails
Validation	Are versions, delivery, access, coverage, gaps, and human review recorded?	Complete the Internal Validation Note.

Use this correction prompt when needed:

Review SDA-OUT-001 against SDA-RUN-001 and SDA-KNW-001. Identify every unmet requirement, then return a clean and complete revised Developer Onboarding Guide with only the necessary corrections. Do not conduct new Research, reopen SDA-RES-001, or introduce Knowledge outside the approved repository. Preserve current-versus-historical distinctions, reported test status, known limitations, proposal status, source notes, and the Internal Validation Note. Follow SDA-RUN-001's Sequential Output Requirements while delivering the revision.

Route the Problem to the Right Place

Return missing, conflicting, or unreliable technical Knowledge to Research. Return organization, clarity, audience fit, formatting, or coordination problems to Runtime. A better sentence cannot repair missing evidence, and another research pass should not be needed merely because a heading is awkward. Chapter 8 explains feedback routing in greater depth.

Approve and Preserve

APPROVED DEVELOPER ONBOARDING GUIDE

Output ID: SDA-OUT-001

Project: DCPF Practicum, Software Documentation Assistant

Produced by: SDA-RUN-001, version 1.0

Knowledge used: SDA-KNW-001, version 1.0

Reviewed by: [Your name]

Review date: [Date]

Status: Approved for fictional practicum use

Caution: Human technical review remains required before adapting this pattern to real software

Save the approved guide as “SDA-OUT-001 TrailWatch Developer Onboarding Guide v1.0.docx.” Keep the Research Request, approved Knowledge Repository, Runtime Request, draft guide, and approved guide as separate records.

The Completed Practicum

You began with seven imperfect sources and ended with a controlled explanation of a software project:

Research Request → Research Output → human review and acceptance → Accepted Knowledge → Knowledge Repository → Runtime Request → Project Output → human review → preservation

The important achievement is not the TrailWatch guide. It is the separation of responsibilities. Research reconciled evidence. Human review approved the technical Knowledge. Runtime translated that Knowledge for a specific reader. Governance and the Sequential Output Requirements protected the work at every stage.

This pattern can be adapted to project handoffs, standard operating procedures, requirements documentation, release guides, policy manuals, and other environments where scattered records must become dependable instructions.

The practicums are complete. The next DCPF project does not need to be larger, it needs to be useful, testable, and governed. From here, the architecture is yours.

[Return to Table of Contents](#)

Moving Forward

DCPF is an experimental approach that has demonstrated promise but has not been broadly validated. Test and validate your implementation within your own environment before considering real-world or production use.

15.1 From a Working Framework to an Open Architecture

The Distributed Cognitive Prompting Framework for AI Collaboration did not begin in an enterprise software lab or an academic think tank. It emerged from an immediate, practical need: how to organize research, maintain accurate project intelligence, execute tasks consistently, and preserve human judgment across hundreds of AI interactions, without allowing the working conversation to become a cluttered junk drawer.

Through the preceding chapters and practicums, you have seen that DCPF does not depend on complex code, expensive platforms, or specialized software. At its foundation, it requires ordinary documents, clearly organized workspaces, focused prompts, and a disciplined separation of responsibilities: Governance establishes boundaries, Research develops evidence, Knowledge preserves what has been accepted, and Runtime executes against that approved foundation.

The architecture is stable, but its implementation is flexible. A small project may use only a few clearly separated documents in a single folder, with Governance controls embedded in the Research, Knowledge, Runtime, and review steps where they apply. A standalone Governance artifact can be added when recurring standards need a maintained home. A larger effort may distribute the same responsibilities across people, tools, repositories, and review processes. What matters is not the software used to hold the work, but the preservation of the roles, handoffs, validation points, and human authority that make the work dependable.

With the core cycle established, this book now turns from instruction to invitation. The next stage of DCPF will be shaped by the people who apply it, test it, challenge it, and extend it in environments different from the one in which it began.

15.2 Cross-Domain Validation Is Still Ahead

DCPF is an experimental, experience-based approach intended as a general architecture for organizing human-AI collaboration. This Introductory Edition does not claim that the framework has already been validated across every professional, technical, academic, regulated, or organizational domain. It has not. The evidence in this book comes from the development work, applications, and practicums represented here.

Different domains may introduce evidence standards, regulatory obligations, safety requirements, ethical constraints, professional practices, technical conditions, or workflow needs that DCPF has not yet encountered. Those differences should not be predicted from a distance and written into the framework as assumptions. They should be discovered through responsible use, domain expertise, and documented testing.

One reason DCPF is being made openly available is to place it in the hands of people who understand environments beyond those represented in this book. Practitioners, researchers, organizations, students, developers, and other collaborators are encouraged to experiment with DCPF within domains they know well and to document what happens: which responsibilities remain useful, which assumptions fail, what new Governance requirements appear, where implementation creates friction, and where the architecture needs clarification, extension, or restraint.

The goal is not to prove that DCPF works everywhere. The goal is to learn where it works, why it works, where it does not, and what the evidence teaches us. Cross-domain validation can show which parts of the architecture are genuinely general and which parts should evolve only after experience justifies the change.

15.3 An Open Challenge to the Community

This book is offered freely so anyone can learn, test, and evaluate DCPF, including people willing to examine it in domains and environments beyond those represented here. The templates specifically identified for reuse may be adapted within the permissions described in the Copyright, Free Distribution, Fair Use, and Template Permissions notice.

In this chapter, open architecture refers to a transparent and adaptable conceptual structure that can be inspected, implemented in different tools, and explored through original extensions. It does not mean that every copyrighted element of the book is unrestricted.

Whether you are a student, writer, educator, engineer, researcher, or professional exploring structured AI collaboration, you are invited to test DCPF in controlled environments you understand and to treat unfamiliar domain requirements as questions to investigate rather than assumptions to fill in.

Start Small and Inspect

Choose a contained, low-risk project and use the basic four-part architecture. Keep the responsibilities visibly separate, even if your implementation packages them differently. Observe where the handoffs save time, where they create friction, and where your instructions need greater precision.

Test the Seams

Push on the boundaries between the frameworks. Find where Runtime encounters gaps in the Knowledge Repository, where Research leaves uncertainty unresolved, where Governance rules weaken under model drift or changing conditions, and where Sequential Output Requirements protect completeness across long deliverables. The seams are where an architecture reveals both its strengths and its unfinished work.

Share What You Learn

Refine configuration variables, develop domain-specific governance blocks, create original implementation patterns, and document what succeeds or fails. Share findings, examples, and original extensions while observing the applicable attribution and reuse terms. A useful framework grows stronger when its limitations are made visible as clearly as its successes.

15.4 Conceptual Parallels: Looking Toward Automated and Enterprise Systems

DCPF was developed and demonstrated through individual and small-scale, human-directed workflows. Its modular and inspectable separation of responsibilities also creates useful conceptual parallels with automated AI systems, retrieval-augmented pipelines, and enterprise workflows.

These parallels are not claims of technical equivalence, proven integration patterns, or enterprise performance. They are working hypotheses for technical practitioners, AI architects, and researchers to test. Any automated implementation would require its own engineering, security, evaluation, compliance, and operational validation.

Conceptual Architecture Mapping

Governance Framework → System Guardrails and Policy Enforcement

In manual DCPF, Governance defines approved sources, prohibitions, decision rights, validation requirements, and escalation boundaries. In production systems, parts of that responsibility may be implemented through access controls, input and output guardrails, safety classifiers, policy-as-code, approval gates, and audit logs. The parallel is incomplete by design: technical controls can enforce selected rules, but they do not replace human judgment, accountability, or governance ownership.

Research Framework → Governed Data Ingestion and Transformation

In DCPF, Research acts as a controlled entry point that gathers sources, evaluates relevance, manages conflicts, records uncertainty, and formats evidence for review. In software systems, this resembles governed ingestion and Extract-Transform-Load (ETL) pipelines that collect, normalize, reconcile, tag, and prepare information for downstream use. DCPF Research adds an explicit reasoning and provenance layer that ordinary data movement alone may not provide.

Knowledge Repository → Grounded Context and Retrieval Layers

In DCPF, the Knowledge Repository separates durable, approved project intelligence from temporary chat history. In automated architectures, this resembles curated document stores, structured databases, knowledge graphs, or vector stores used in retrieval-augmented generation (RAG). The value of the parallel depends on curation: retrieval infrastructure does not make information trustworthy unless provenance, versioning, conflict handling, and approval are also preserved.

Runtime Framework → Parameterized Task Execution

In DCPF, Runtime receives defined variables, works against complete supplied Knowledge and the requirements stated in the Runtime Request, produces a specific deliverable, reports limitations, and stops. In an agentic or orchestrated workflow, this resembles a task-specific agent or service executing a bounded operation against controlled inputs. The essential similarity is not autonomy; it is constrained execution within an explicit contract.

Sequential Output Requirements → Stateful Continuation Controls

In DCPF, the continuation protocol protects long-form completeness by requiring the AI to preserve state, stop at a clean boundary, and request permission before continuing. In software, the closest parallel is stateful chunking, checkpointed continuation, or controlled pagination across a limited context window. It is not the same as ordinary token streaming, which transports output as it is generated without necessarily preserving a complete multi-part plan.

Internal Validation Notes → Automated Checks and Evaluation Gates

In DCPF, the Internal Validation Note requires the AI to inspect its work, disclose gaps, and report compliance before human sign-off. In production pipelines, this resembles schema checks, test assertions, completeness evaluators, and release gates. Automated evaluation can strengthen inspection, but its findings remain evidence for human review, not a substitute for human approval.

15.5 Open Questions for Future Exploration

As practitioners experiment with DCPF across different domains, tools, and team sizes, several questions deserve disciplined investigation rather than premature answers.

Structured Data Translation

How smoothly can DCPF artifacts translate into structured formats such as JSON or YAML for use with agent workflow frameworks? Which elements should become machine-readable fields, and which should remain human-readable explanations that preserve context and judgment?

Collaborative Governance

How can multi-member teams coordinate Governance updates, record decision authority, and resolve Knowledge Repository version conflicts across parallel workstreams without weakening traceability?

Automated Evaluation Support

To what degree can lightweight AI evaluators assist human reviewers with DCPF preflight and completion checks? How should evaluators expose their own uncertainty, blind spots, and evidence so that automation supports rather than displaces human authority?

Human Authority at Scale

As more steps become automated, which decisions must remain explicitly human? How should approvals, exceptions, accountability, and the right to stop the system be recorded so that increased speed does not obscure responsibility?

15.6 The Core Principle Remains

Whether DCPF is implemented as a Minimum Viable DCPF project in a single folder or extended into complex collaborative workflows, its governing dynamic does not change:

The human directs the system.

The architecture organizes the work.

The AI reasons within that structure.

The architecture is now in your hands. Take it, test it within environments you understand, document what you learn, and let evidence, including evidence of limitations, guide what comes next.

[Return to Table of Contents](#)

DCPF Quick Reference Guide

A compact operating reference for planning, researching, preserving Knowledge, executing Runtime work, and maintaining human control.

Use this appendix after you understand the basic DCPF architecture. It is designed to help you identify the correct framework, locate its standard responsibilities, distinguish optional or supplemental elements, prepare the next artifact, and review the resulting output without rereading the complete explanatory chapters.

DCPF remains an experimental approach. Use this guide to support controlled testing and adaptation; it does not replace domain-specific validation or human judgment before real-world or production use.



Figure A.1. DCPF architecture at a glance. **Governance applies throughout**, and Research evidence becomes Accepted Knowledge only after the applicable human review and acceptance decision. The vertical format is optimized for phone and tablet viewing.

Core relationship: Research discovers. Knowledge preserves. Runtime executes. Governance guides.

A.1 DCPF at a Glance

DCPF organizes AI-assisted work as a governed architecture rather than a series of isolated prompts. Each framework owns a different responsibility, and information changes status as it moves through the system.

Governance Framework: Establishes the human-defined quality, ethical, operational, source, boundary, validation, output, and approval standards that guide the entire DCPF system. A maintained Governance artifact may consolidate recurring standards, while applicable Governance elements are carried into the working artifacts where they operate.

Research Framework: Conducts and documents a controlled investigation that produces evidence for human review and possible acceptance as Knowledge. The Research Request document is the artifact used to structure the user's collaboration with AI during this step.

Knowledge Framework: Owns the architectural responsibility for preserving Accepted Knowledge so it can be maintained and reused. The Knowledge Repository is the persistent representation and maintained project home of that Accepted Knowledge.

Runtime Framework: Combines approved Knowledge with the current task, variables, output requirements, and Governance in a Runtime Request document to produce a Project Output.

A.2 Which Framework Do I Need?

I need to discover, verify, compare, or document information.: Use the Research Framework.

I have reviewed information that should become reusable project intelligence.: Use the Knowledge Framework responsibility to preserve the Accepted Knowledge in the Knowledge Repository.

I need AI to create, transform, analyze, or perform something using approved Knowledge.: Use the Runtime Framework.

I need to define what is allowed, required, acceptable, reviewable, or subject to human approval.: Apply the Governance Framework responsibility by making those controls explicit in the working artifacts that need them; consolidate recurring controls into a maintained Governance artifact when useful.

More than one framework may be involved. Choose the framework that owns the immediate responsibility, then preserve the handoff to the next framework.

A.3 The Four Frameworks and Their Standard Responsibilities

The following section names identify the standard DCPF responsibilities for each framework. Standard responsibilities are presented first because they define the architecture a beginner should expect to see. They may be combined into fewer headings when their purposes remain explicit, identifiable, and usable. Optional or supplemental elements are labeled separately and should not be mistaken for core requirements.

Governance Framework

Purpose: Make the system's enduring human-defined standards visible, reusable, maintainable, and applicable across Research, the Knowledge Repository, Runtime, and outputs. Governance operates through the controls carried into working artifacts; recurring project-wide controls may also be consolidated into a maintained Governance artifact.

Framework Purpose: Defines the quality, ethical, and operational responsibilities governed by the framework and identifies where those responsibilities apply.

Quality Standards: Establishes what satisfactory work means, including expectations for accuracy, completeness, consistency, accessibility, formatting, and other relevant quality attributes.

Approved Information Sources: Identifies the evidence and source types the system is permitted to use for Research and Knowledge development.

Prohibited Content: Defines information, sources, practices, behaviors, or outputs that are not allowed.

Validation Requirements: Specifies the checks, review criteria, approval points, and possible decisions used before work is accepted.

Output Standards: Defines recurring organization, citation, formatting, presentation, accessibility, and completion requirements for deliverables.

Revision History: Records governance changes, effective dates, reasons, and approving authority so the system can identify the current governing version.

Primary chapter reference: Chapter 5, The Governance Framework.

Research Framework

Purpose: Turn a human-defined information need into a controlled, documented evidence product prepared for human review, possible acceptance as Knowledge, and preservation in the Knowledge Repository.

Framework Purpose: Defines the investigation, explains why it is needed, identifies its downstream use, and distinguishes Research from the final Runtime deliverable.

Research Configuration Variables: Assigns the current subject, location, time period, audience, depth, category, or other values that change from one investigation to the next without changing the stable research method.

Research Objectives: Defines the questions, required topics, comparisons, or evidence needs the investigation must address.

Approved Information Sources: Establishes which evidence channels Research is permitted to use for the current investigation within Governance boundaries. Permission does not by itself establish the weight of a source for a particular claim.

Source Priority Hierarchy: Guides claim-specific weighting and conflict handling among approved sources. Priority depends on the question being supported; it is not a universal ranking of source categories.

Research Scope: Defines inclusions, exclusions, depth, recency, geographic or temporal boundaries, and other limits on the investigation.

Output Organization: Specifies how the completed Research Output will be structured so findings remain reviewable, traceable, and ready for the Knowledge handoff.

Research Source Documentation: Preserves the evidence trail and relevant content origins, including the information needed to verify claims and understand source lineage.

Sequential Output Requirements: Protects complete delivery when the Research Output may exceed a comfortable single response by requiring natural stopping points and controlled continuation.

Primary chapter reference: Chapter 6, The Research Framework.

Knowledge Framework

Purpose: Own the architectural responsibility for preserving Accepted Knowledge so it can be maintained and reused. The four standard responsibilities below are represented operationally within the Knowledge Repository, the persistent representation and maintained project home of that Accepted Knowledge:

Framework Purpose: States what accepted Knowledge is being preserved, why it is being maintained, and how the project expects to use it.

Knowledge Configuration Variables: Identifies the current subject, category, view, selection, audience, or use condition that organizes access without changing the accepted Knowledge itself.

Knowledge Structure: Contains the organized body of accepted information, often preserving a well-designed Research Output with only the changes needed for maintenance and reuse.

Information Categories: Provides the headings, labels, classifications, or status groupings used to locate Knowledge and keep different kinds of information distinct.

Optional / Supplemental: Metadata: Records useful context such as version, date, status, owner, tags, sources, review date, or applicable scope when that information improves traceability or maintenance.

Do not rebuild what Research already organized. Preserve the accepted evidence, qualifications, uncertainty, and source information; remove temporary execution scaffolding only when it no longer serves the maintained repository.

Primary chapter reference: Chapter 7, The Knowledge Framework.

Runtime Framework

Purpose: Use approved Knowledge and current execution instructions to produce a defined Project Output without quietly reopening Research or changing the repository.

Framework Purpose: Identifies the work being performed now and distinguishes execution from Research, repository maintenance, or governance revision.

Runtime Configuration Variables: Assigns the current audience, subject, length, tone, format, technical level, delivery setting, or other adjustable values for this execution.

Input Knowledge: Supplies the complete approved Knowledge used as the execution's data engine; an identifier or filename alone does not supply the contents.

Task Definition: States exactly what the AI must do with the supplied Knowledge during the current execution.

Output Requirements: Defines the form, organization, limits, required components, validation notes, and conditions that determine when the deliverable is complete.

Sequential Output Requirements: Protects completeness when a Project Output may exceed practical response length by requiring natural breaks, explicit continuation requests, and a clear completion statement.

Primary chapter reference: Chapter 8, The Runtime Framework.

Minimum Viable DCPF

Minimum Viable DCPF keeps all four framework responsibilities visible without requiring four standalone framework artifacts. A first low-risk project may use a Research Request with embedded Governance controls, a Knowledge Repository containing human-reviewed Accepted Knowledge, a Runtime Request with embedded Governance controls, and human review of the resulting Project Output. Add a standalone Governance artifact when recurring controls need a stable project-wide home.

A.4 Standard DCPF Project Workflow

The following workflow is deliberately vertical. Each step can be read independently on a narrow screen, and each handoff makes the change in responsibility visible. Step 2 builds the relevant artifact and uses Governance Framework responsibilities to define the controls needed within it. Step 3 develops the Governance Repository as recurring controls are identified in the other framework artifacts. Step 4 executes the completed artifact and requires human review before acceptance or use.



Figure A.2. Standard DCPF project workflow. Define the work; build the relevant artifact and define its needed controls using Governance Framework responsibilities; develop the Governance Repository as recurring controls are identified; then execute the completed artifact and perform human review.

A.5 Typical Research Cycle

1. Define the information need and intended downstream use.
2. Confirm the applicable Governance standards and source boundaries.
3. Assign Research Configuration Variables, Objectives, Scope, and Output Organization.
4. Supply or authorize accessible approved sources, then apply claim-specific source priority; do not rely on source names the AI cannot open or on a universal source ladder.
5. Execute the complete Research Request.
6. Review coverage, source quality, documentation, uncertainty, conflicts, gaps, and compliance.
7. Accept, revise, return for additional Research, or exclude each relevant part.
8. **Treat accepted information as Accepted Knowledge and preserve it in the Knowledge Repository with sufficient identification and status.**

A.6 Typical Knowledge Handoff

1. Begin with the reviewed Research Output rather than an improvised summary.
2. Preserve accepted findings, qualifications, uncertainty, useful detail, and source records.
3. Separate unresolved, excluded, superseded, or provisional material as Governance requires.
4. Add the repository identity, source Research artifact, version, status, owner, and applicable requirements.
5. Confirm that Runtime receives the complete approved content by attachment, by embedding or pasting it into the working context, or by retrieving it from a source the AI can actually access, not merely by filename or identifier.
6. Maintain changes through versioning and review rather than silently overwriting the official Knowledge.

A.7 Typical Runtime Cycle

1. Define the current task and the intended Project Output.
2. Confirm that the complete approved Knowledge has been attached, embedded or pasted into the working context, or retrieved from a source the AI can actually access, and confirm that the Runtime Request states all applicable requirements.
3. Assign Runtime Configuration Variables.
4. State the Task Definition and Output Requirements.
5. Apply Sequential Output Requirements automatically when the output may be too long or unpredictable for one response.
6. Execute the complete Runtime Request.
7. Review the Project Output against the task, Knowledge, Governance, and human expectations.
8. Accept, revise, reject, escalate, or route the discovered gap to Research, Knowledge, Runtime, or Governance.
9. Preserve reusable improvements in the framework that owns them rather than patching every future prompt.

A.8 Essential Sequential Output Requirements

Use this requirement when the requested Research Output or Project Output may exceed what can be comfortably delivered in a single response, or when its complete length cannot be reliably predicted.

Purpose: Preserve structure, completeness, detail, formatting, and continuity without compressing required work merely to fit one response.

Apply this requirement automatically whenever the complete output may exceed practical response length or its length cannot be reliably predicted.

Do not compress, summarize, abbreviate, or omit required work merely to fit within one response. Favor completeness over minimizing the number of responses.

Break the output at a natural point. Do not force the output into predetermined sets or fixed section groupings.

At the end of each nonfinal part, state that additional output remains, request permission to continue, and stop.

After permission is received, continue from where the previous part ended until the complete output has been delivered.

The final part must clearly state that the requested output is complete.

A short, predictable output may be delivered in one response.

Sequential delivery is a completeness safeguard, not a requirement to split every response.

A.9 Artifact Relationships

Governance artifact (when maintained separately): Maintained representation of recurring project-wide standards and authority; applicable requirements still travel into the working artifacts where they operate.

Research Request: Defines the controlled investigation.

Research Output: Delivers documented evidence for review.

Knowledge Repository: Preserves accepted information for reuse.

Runtime Request: Defines the current execution using approved Knowledge.

Project Output: Delivers the result for human review, acceptance, and use.

Name it for traceability. Supply it for use.

A Knowledge artifact ID, filename, prior-chat reference, folder location, or link label does not supply the Knowledge contents. Runtime must receive the complete approved Knowledge by attachment, by embedding or pasting it into the working context, or by retrieval from a source the AI can actually access. Requirements are stated directly in the working artifact where they apply.

A.10 Human Authority and Review

DCPF gives AI a clearer assignment, not greater authority. The user, project owner, organization, or other responsible human authority remains accountable for consequential decisions.

Define the project purpose, intended value, scope, and acceptable risk.

Establish or adopt Governance standards, source policies, prohibitions, approvals, and exceptions.

Choose Research objectives and determine whether evidence is sufficient.

Decide what information is accepted into, revised within, or removed from the Knowledge Repository.

Define Runtime tasks and completion conditions.

Review, validate, verify, approve, reject, revise, or escalate outputs before relying on or publishing them.

Maintain official artifacts, version history, ownership, and preservation practices.

A.11 Common Mistakes and Corrections

Starting too large: Reduce the first implementation to one useful, testable task with a clear output. Do not require a full standalone Governance artifact before the project has enough experience to identify which controls should recur.

Treating source priority as a universal ladder: Weight permitted evidence according to the claim, authority, directness, currentness, jurisdiction, method, relevance, and corroboration; preserve credible unresolved conflicts.

Skipping Governance: Governance is required even when no standalone Governance artifact exists. Make source rules, boundaries, quality expectations, acceptance criteria, review points, and human authority explicit in the working artifacts where they apply.

Asking Research to produce the final deliverable: Keep Research focused on evidence development; use Runtime to create the audience-facing Project Output.

Treating Research Output as automatically accepted Knowledge: Review the evidence and record its acceptance status before Runtime relies on it.

Rebuilding or oversimplifying the repository: Preserve the complete accepted Research value, including qualifications, uncertainty, and source history.

Supplying only a Knowledge filename or artifact ID: Attach the complete approved Knowledge, embed or paste its contents into the working context, or retrieve it from a source the AI can actually access.

Letting Runtime fill Knowledge gaps from general model knowledge: Stop, report the gap, and route the need back to Research unless Governance explicitly authorizes another source.

Compressing a long output into one response: Apply Sequential Output Requirements and continue at natural stopping points.

Patching every execution prompt: Revise the responsible framework or maintained artifact when the improvement should persist.

Overwriting official artifacts without version control: Preserve identifiers, versions, dates, status, reasons for change, and approving authority.

A.12 Project Startup Checklist

- ☐ Project objective and intended value are clear.
- ☐ The first implementation is small enough to test and improve.
- ☐ Human owner, reviewer, and approval authority are identified.
- ☐ Applicable Governance standards and boundaries are stated explicitly in each working artifact where they apply.
- ☐ The immediate framework responsibility is correctly identified.
- ☐ Required variables are assigned.
- ☐ **Approved Knowledge has been supplied, or approved source access has been confirmed.**
- ☐ The requested output and definition of completion are explicit.
- ☐ Sequential Output Requirements are present when output length may be uncertain.
- ☐ Artifact identifiers, versions, status, and preservation location are defined.

A.13 Output Review Checklist

- ☐ Did the output perform the correct framework responsibility?
- ☐ Did it use the required Governance and approved Knowledge or sources?
- ☐ Did it remain within scope and avoid prohibited actions?
- ☐ Are claims, sources, uncertainty, and conflicts represented accurately?
- ☐ Are all required sections and deliverable components present?
- ☐ Was anything compressed, omitted, invented, or silently assumed?
- ☐ Does the output satisfy the specified organization, format, accessibility, and completion standards?
- ☐ Does a human need to approve, verify, test, or escalate the result?
- ☐ Did the work reveal a reusable improvement or a gap that belongs in another framework?
- ☐ Has the accepted result been preserved with the correct identifier, version, and status?

A.14 Classification Legend

The DCPF glossary and supporting materials may use the following classifications to show what kind of term or responsibility is being described. Classification does not replace the definition; it adds quick context.

Framework: A major architectural component that owns a stable responsibility within DCPF.

Skeleton: The standard set of responsibilities that gives a DCPF framework its practical structure.

Standard DCPF responsibility: A core responsibility that belongs to the normal DCPF skeleton. It should be represented in the implementation, but it does not require its own heading, page, or standalone artifact.

Artifact: A persistent document, template, record, repository, or other maintained object used to operate the architecture.

Output: A result produced by Research or Runtime for review, preservation, or use.

Process: A repeatable sequence of actions, decisions, reviews, or handoffs.

Principle: A foundational design idea that guides interpretation and application.

Rule: An explicit instruction or requirement that must be followed when its stated conditions apply.

User-defined: A value, standard, decision, or instruction established by the user or responsible human authority; user-defined does not mean optional.

System-defined: A stable behavior, structure, or operating condition established by the implemented system or artifact rather than assigned anew for every execution.

DCPF-recommended: Guidance that should normally be preserved or tested carefully but is not, by that label alone, a core framework responsibility.

Optional: A supplemental element, field, or implementation aid that may be included when useful but is not part of the core standard unless explicitly identified as a standard responsibility.

A.15 Final Quick Check

Before execution, ask: Do I have the correct framework, the complete required content, the applicable Governance, a clear definition of completion, and a human review plan?

After execution, ask: Should this result be accepted, revised, rejected, escalated, preserved, or routed back to the framework that owns the discovered gap?

DCPF Quick Reference Guide | Appendix A | Prepared for the DCPF Introductory Edition

[Return to Table of Contents](#)

Framework Templates

DCPF remains an experimental approach. Use these templates to support controlled testing and adaptation; they do not replace domain-specific validation or human judgment before real-world or production use.

Clean starting structures for Governance, Research, the Knowledge Repository, and Runtime

This appendix provides copyable Template Starts for primary working artifacts that operationalize DCPF's four framework responsibilities. Each template presents the standard responsibilities as the core structure. Optional or supplemental material is labeled and follows the standard core so a beginner can distinguish the architecture from implementation extras.

Use the clean masters as starting structures. Build a small pilot. Evaluate the result. Revise the language when evidence shows what the project needs.

Using the Template Starts

Make a working copy of the clean master. Complete the bracketed project-specific content, retain the highly recommended language in the initial test version, and keep Governance, Research, Knowledge, and Runtime responsibilities separate. Preserve every standard responsibility even when you combine two or more under one heading. Chapters 9–14 provide the preparation guidance and complete practicum workflows for building and testing DCPF projects.

Do not assume these Template Starts must be completed in order. A beginning project can start with a Research Request and the requirements it needs, then consolidate recurring project-wide controls into a standalone Governance artifact later. An organization with established policies may choose to begin with the Governance Framework Template and translate the applicable rules into its working artifacts.

Read each Template Start in this order: standard responsibilities first, implementation flexibility second, optional or supplemental elements last. Artifact-information fields support identification and traceability; they do not create additional framework responsibilities.

PROJECT-SPECIFIC CONTENT: COMPLETE OR CUSTOMIZE

Replace bracketed instructions with the project's decisions. Do not leave required information to model assumption.

DCPF HIGHLY RECOMMENDED LANGUAGE: INCLUDE IN THE INITIAL TEST VERSION

Include this language in the first test version. It may be revised later when testing demonstrates a project-specific need.

CORE DCPF HIGHLY RECOMMENDED LANGUAGE: INCLUDE IN THE INITIAL TEST VERSION

This label identifies especially consequential controls, including Sequential Output Requirements. Test before changing them and preserve the responsibility they protect.

Test Before Full Implementation

Begin with a small, manageable pilot. Compare the result with explicit success criteria. If either highly recommended language or project-specific content produces an unsuitable result, revise the responsible language, record the reason, and test again.

CLEAN MASTER → WORKING COPY → DRAFT → SMALL TEST → EVALUATE → REVISE AND RETEST → APPROVE

A project-specific framework artifact becomes authoritative only after the designated human authority reviews and approves the tested version for a defined scope.

Governance Framework Template Start

A reusable master for consolidating recurring standards that apply across Research, the Knowledge Repository, Runtime, and outputs

COPY BEFORE EDITING. BEGIN WITH A SMALL TEST PROJECT. DO NOT TREAT A DRAFT OR TEST VERSION AS AUTHORITATIVE UNTIL IT HAS BEEN REVIEWED AND HUMAN-APPROVED.

Use the labels throughout this template. Add the project-specific content requested in brackets. Include the DCPF highly recommended language in the initial test version. After testing, revise either type of content when evidence demonstrates a project-specific need, and preserve the reason for consequential changes.

PROJECT-SPECIFIC CONTENT: COMPLETE OR CUSTOMIZE

Framework Name: *[Enter the descriptive name of the Governance Framework.]*

Framework ID: *[Enter the project-specific artifact identifier.]*

Project Name: *[Enter the project name.]*

Owner: *[Enter the responsible person or role.]*

Version: *[Enter the version number.]*

Status: *[MASTER / DRAFT / TEST / APPROVED / RETIRED / ARCHIVED]*

Effective or Revision Date: *[Enter the date.]*

Approving Authority: *[Enter the person or role authorized to approve Governance.]*

Scope of Application: *[Identify the projects, frameworks, outputs, users, or environments governed.]*

Related Artifacts: *[Identify applicable Research, Knowledge, Runtime, policies, or standards.]*

Authoritative Status: *[State whether this is a clean master, working copy, test version, or current authoritative version.]*

1. FRAMEWORK PURPOSE

PROJECT-SPECIFIC CONTENT: COMPLETE OR CUSTOMIZE

[State what the Governance Framework controls, why those controls are needed, who or what is governed, and where the authority applies.]

DCPF HIGHLY RECOMMENDED LANGUAGE: INCLUDE IN THE INITIAL TEST VERSION

This Governance Framework establishes the human-defined quality, ethical, operational, source, boundary, validation, output, and approval standards that apply within its stated scope.

Research Requests, Knowledge Repository content, Runtime Requests, and outputs governed by this artifact must state and apply the relevant approved controls within their own working content. They do not depend on this Governance Framework being attached, supplied, pasted, embedded, or retrieved.

AI may assist with interpretation, inspection, and validation, but it does not become the approving authority and may not create its own exception to a governing requirement.

2. QUALITY STANDARDS

PROJECT-SPECIFIC CONTENT: COMPLETE OR CUSTOMIZE

[Define accuracy, completeness, consistency, clarity, usefulness, accessibility, traceability, formatting, and other standards that satisfactory work must meet.]

[Identify any measurable threshold, tolerance, review method, or risk-based standard.]

DCPF HIGHLY RECOMMENDED LANGUAGE: INCLUDE IN THE INITIAL TEST VERSION

Evaluate work against explicit project standards rather than confidence, fluency, length, or visual polish alone. Important limitations, unresolved questions, source conflicts, and failed requirements must remain visible.

When a standard cannot be satisfied from the approved inputs, sources, scope, or available capabilities, report the limitation and request human direction rather than concealing the failure or inventing compliance.

3. APPROVED INFORMATION SOURCES

PROJECT-SPECIFIC CONTENT: COMPLETE OR CUSTOMIZE

Approved user-provided sources: [Identify documents, datasets, interviews, records, images, notes, or other materials.]

Approved outside-source categories: [Identify permitted authoritative, primary, professional, institutional, or other source categories.]

Conditional sources: [Identify sources requiring specific approval or limited use.]

Source recency, jurisdiction, citation, access, and conflict requirements: [Define the rules.]

DCPF HIGHLY RECOMMENDED LANGUAGE: INCLUDE IN THE INITIAL TEST VERSION

Use only sources permitted by the applicable Research Request and the requirements stated within that request. Do not quietly substitute an unapproved source when approved evidence is unavailable.

Do not claim to have opened, read, viewed, searched, retrieved, or verified a source that was not actually accessible. Preserve the distinction between user-provided information, source-supported findings, inference, and illustrative examples.

4. PROHIBITED CONTENT

PROJECT-SPECIFIC CONTENT: COMPLETE OR CUSTOMIZE

Prohibited sources: [Identify source types that may not be used as evidence.]

Prohibited practices or behaviors: [Identify fabrication, unauthorized access, privacy violations, unsupported substitution, hidden expansion of scope, or other prohibited actions.]

Prohibited outputs or recommendations: [Identify content the system must not produce.]

Required response to a prohibited request: [State whether to stop, refuse, narrow, redirect, or escalate.]

DCPF HIGHLY RECOMMENDED LANGUAGE: INCLUDE IN THE INITIAL TEST VERSION

Do not fabricate facts, sources, quotations, data, access records, approval, consensus, completion, or certainty. Do not conceal missing evidence, important limitations, material disagreement, or a failure to satisfy an applicable control.

When a request conflicts with Governance, do not silently weaken the rule or continue with a near substitute. Identify the conflict and request direction from the designated human authority.

5. VALIDATION REQUIREMENTS

PROJECT-SPECIFIC CONTENT: COMPLETE OR CUSTOMIZE

Required checks: [List the checks applied before Research, Knowledge, Runtime, or an output is accepted.]

Decision states: [For example: Approve / Revise / Reject / Additional Research Required / Escalate.]

Human review points: [Identify the person or role and the decisions reserved for that authority.]

Testing requirements: [Define pilot, boundary, regression, or representative-use testing.]

DCPF HIGHLY RECOMMENDED LANGUAGE: INCLUDE IN THE INITIAL TEST VERSION

Validation must compare the artifact or output with its governing requirements, available evidence, approved Knowledge, stated scope, and completion criteria. A model's self-review may assist the process but does not replace required human approval.

A failed requirement must be corrected, accepted as a documented limitation by the authorized human, or escalated. Do not represent partially validated work as fully approved.

6. OUTPUT STANDARDS

PROJECT-SPECIFIC CONTENT: COMPLETE OR CUSTOMIZE

Recurring organization and required sections: [Define the standard.]

Citation and source-documentation format: [Define the standard.]

Formatting, accessibility, visual, mobile-readability, and file requirements: [Define the standard.]

Sequential delivery and completion statement requirements: [Define or reference the standard.]

DCPF HIGHLY RECOMMENDED LANGUAGE: INCLUDE IN THE INITIAL TEST VERSION

Outputs must preserve required structure, completeness, traceability, limitations, and completion status. Formatting choices must support the intended reader and must not be used to hide missing substance.

When an output may exceed practical response length or its length cannot be reliably predicted, apply the approved Sequential Output Requirements rather than compressing required work to fit one response.

7. REVISION HISTORY

PROJECT-SPECIFIC CONTENT: COMPLETE OR CUSTOMIZE

For each revision record: [Date; artifact version; requirement changed; previous language; revised language; reason and evidence; related test; approver; effective date; superseded version.]

Current authoritative version: [Identify the version and approved scope.]

Next review date or trigger: [Enter the date or event.]

DCPF HIGHLY RECOMMENDED LANGUAGE: INCLUDE IN THE INITIAL TEST VERSION

Do not silently overwrite approved Governance. Preserve consequential changes, the evidence or project need supporting them, the approving authority, and the version that became effective.

A revised Governance Framework becomes authoritative only after the required testing, review, and human approval have been completed.

END OF GOVERNANCE FRAMEWORK TEMPLATE START

CLEAN MASTER TEMPLATE START

Embedded Governance Controls Block

A concise block for carrying operationally relevant Governance into Research, Knowledge, and Runtime artifacts

COPY BEFORE EDITING. BEGIN WITH A SMALL TEST PROJECT. DO NOT TREAT A DRAFT OR TEST VERSION AS AUTHORITATIVE UNTIL IT HAS BEEN REVIEWED AND HUMAN-APPROVED.

Use the labels throughout this template. Add the project-specific content requested in brackets. Include the DCPF highly recommended language in the initial test version. After testing, revise either type of content when evidence demonstrates a project-specific need, and preserve the reason for consequential changes.

Use this block to make the necessary Governance controls explicit where Research, Knowledge, Runtime, or review work occurs. A maintained Governance artifact may preserve recurring project-wide standards, but the block is self-contained for the working artifact and creates no attachment or retrieval dependency.

PROJECT-SPECIFIC CONTENT: COMPLETE OR CUSTOMIZE

Control rationale: [State the human-authorized source or rationale for these controls. A separate Governance artifact is optional and is not required for execution.]

Applicable Scope: [Identify the portion of the project or execution controlled by this block.]

Quality Standards: [List the directly applicable standards.]

Approved Sources or Inputs: [Identify permitted source, Knowledge, and input boundaries.]

Prohibitions: [List directly applicable prohibited sources, actions, content, or substitutions.]

Validation Requirements: [List required checks, decision states, and approval points.]

Output Standards: [List required organization, citation, formatting, accessibility, sequence, and completion rules.]

Human Authority: [Identify the reviewer, approver, and escalation authority.]

DCPF HIGHLY RECOMMENDED LANGUAGE: INCLUDE IN THE INITIAL TEST VERSION

Apply the requirements identified above throughout the complete artifact and resulting output. If a governing requirement is missing, inaccessible, internally inconsistent, or in conflict with the current request, identify the problem and request human direction before proceeding beyond the affected boundary.

Do not fabricate compliance, create an unauthorized exception, silently weaken a control, or substitute a model preference for the decision of the designated human authority. Preserve material limitations, failed checks, and unresolved conflicts for review.

END OF EMBEDDED GOVERNANCE CONTROLS BLOCK

Research Request Template Start

A reusable master for controlled investigation and documented evidence development

COPY BEFORE EDITING. BEGIN WITH A SMALL TEST PROJECT. DO NOT TREAT A DRAFT OR TEST VERSION AS AUTHORITATIVE UNTIL IT HAS BEEN REVIEWED AND HUMAN-APPROVED.

Use the labels throughout this template. Add the project-specific content requested in brackets. Include the DCPF highly recommended language in the initial test version. After testing, revise either type of content when evidence demonstrates a project-specific need, and preserve the reason for consequential changes.

PROJECT-SPECIFIC CONTENT: COMPLETE OR CUSTOMIZE

Request Name: *[Enter the descriptive name of the Research Request.]*

Request ID: *[Enter the project-specific artifact identifier.]*

Project Name: *[Enter the project name.]*

Owner: *[Enter the responsible person or role.]*

Version: *[Enter the version number.]*

Status: *[MASTER / DRAFT / TEST / APPROVED / RETIRED / ARCHIVED]*

Creation or Revision Date: *[Enter the date.]*

Approving Authority: *[Enter the person or role authorized to approve this request.]*

Source, validation, and human-authority requirements: *[State the source, boundary, validation, and human-authority controls that apply to this Research Request.]*

Related Artifacts: *[Identify the Project Vision, source collection, prior Research, or other dependencies.]*

Expected Output: *[Name the Research Output this request must produce.]*

Authoritative Status: *[State whether this is a clean master, working copy, test version, or current authoritative version.]*

1. PURPOSE

PROJECT-SPECIFIC CONTENT: COMPLETE OR CUSTOMIZE

[State what must be researched, why the Research is needed, and how the completed Research Output is expected to support the project.]

DCPF HIGHLY RECOMMENDED LANGUAGE: INCLUDE IN THE INITIAL TEST VERSION

Execute this document as a complete Research Request. Conduct the assigned Research according to the variables, objectives, approved sources, source priorities, scope, output requirements, source-documentation requirements, Sequential Output Requirements, and Governance controls provided below.

Do not merely review, summarize, or explain this document. Do not create the final Runtime deliverable. Produce the complete Research Output defined by this request for human review and possible preservation in the Knowledge Repository.

If a required input or approved source cannot be accessed, clearly identify the limitation and request direction rather than substituting unsupported information or claiming access that did not occur.

2. RESEARCH CONFIGURATION VARIABLES

PROJECT-SPECIFIC CONTENT: COMPLETE OR CUSTOMIZE

Research topic or question: [Enter the primary topic or question.]

Intended audience: [Enter the intended audience.]

Intended downstream use: [Explain how approved findings may later be used.]

Depth: [Enter the required level of detail.]

Geographic scope: [Enter the location or state Not applicable.]

Time period or recency requirement: [Enter the period or state Not applicable.]

Language, reading level, and tone: [Enter the requirements.]

Available time, cost, or resource limits: [Enter applicable limits.]

Additional project variables: [Add named variables and values as needed.]

DCPF HIGHLY RECOMMENDED LANGUAGE: INCLUDE IN THE INITIAL TEST VERSION

Treat assigned variables as current project settings. Do not silently replace them with model defaults or broaden them because adjacent information appears useful.

3. RESEARCH OBJECTIVES

PROJECT-SPECIFIC CONTENT: COMPLETE OR CUSTOMIZE

1. *[Enter the first required Research Objective.]*

2. *[Enter the second required Research Objective.]*

3. *[Enter the third required Research Objective.]*

Completion standard: [State what must be addressed for the Research assignment to be considered complete.]

DCPF HIGHLY RECOMMENDED LANGUAGE: INCLUDE IN THE INITIAL TEST VERSION

Research Objectives must guide the investigation without requiring the evidence to support a predetermined conclusion. Important gaps, uncertainty, and credible disagreement must remain visible.

4. APPROVED INFORMATION SOURCES

PROJECT-SPECIFIC CONTENT: COMPLETE OR CUSTOMIZE

User-provided sources: [Identify approved documents, datasets, notes, interviews, images, or other original materials.]

Approved outside-source categories: [List permitted source categories and any conditions on their use. Do not imply that the category name alone establishes evidentiary weight.]

Sources requiring specific approval: [Identify conditional sources.]

Prohibited or unacceptable sources: [Identify sources that may not be used as evidence.]

DCPF HIGHLY RECOMMENDED LANGUAGE: INCLUDE IN THE INITIAL TEST VERSION

Use only sources permitted by this request and its stated requirements. Do not quietly substitute an unapproved source when approved evidence is unavailable. Report the limitation and request direction.

Do not claim to have opened, read, viewed, searched, or verified a source that was not actually accessible.

5. SOURCE PRIORITY AND CONFLICT HANDLING

PROJECT-SPECIFIC CONTENT: COMPLETE OR CUSTOMIZE

Claim-specific priority criteria: [Identify factors such as directness to the claim, authority or ownership over the subject, primary/original evidence, currentness, jurisdiction, methodological transparency, relevance, and corroboration.]

Controlling sources for defined claim types: [Identify any source that is authoritative for a specific question, such as an approved policy, governing standard, current specification, official record, or project dataset.]

User-provided material rule: [State what supplied materials directly establish for the project and which external factual claims still require verification.]

Project-specific conflict rule: [Enter any controlling method, standard, jurisdiction, or escalation path.]

DCPF HIGHLY RECOMMENDED LANGUAGE: INCLUDE IN THE INITIAL TEST VERSION

Among approved sources, select the source or combination of sources best suited to the specific claim rather than applying a universal prestige order. A user-provided document may control project requirements, decisions, or supplied data it directly establishes, but an external factual claim is not proven merely because the user supplied it. Public-domain status concerns reuse rights, not evidence quality.

When approved sources disagree, compare their evidence, authority, methods, definitions, dates, scope, jurisdiction, and intended contexts. Do not select a source merely because it appears first or expresses its conclusion most confidently.

Preserve material disagreements when the available evidence does not justify resolving them. Clearly distinguish established findings, reasonable interpretations, disputed claims, and unresolved questions.

If the claim-specific priority and conflict rules do not resolve a consequential conflict, report the conflict and request human direction.

6. RESEARCH SCOPE

PROJECT-SPECIFIC CONTENT: COMPLETE OR CUSTOMIZE

Inclusions: [Identify subjects, questions, locations, periods, cases, or materials that must be included.]

Exclusions: [Identify material outside the assignment.]

Required depth and recency: [Define the expected treatment.]

Research boundaries: [Identify legal, ethical, safety, privacy, cost, access, or project boundaries.]

Stopping conditions: [State any condition that requires the AI to stop and request direction.]

DCPF HIGHLY RECOMMENDED LANGUAGE: INCLUDE IN THE INITIAL TEST VERSION

Remain within the approved Research Scope. Do not broaden the investigation merely because additional information is available or adjacent topics appear interesting.

7. OUTPUT ORGANIZATION

PROJECT-SPECIFIC CONTENT: COMPLETE OR CUSTOMIZE

1. *[Enter the first required output heading.]*
2. *[Enter the second required output heading.]*
3. *[Enter the third required output heading.]*

Required repeatable fields, records, tables, citation style, or formatting: [Define the requirements.]

DCPF HIGHLY RECOMMENDED LANGUAGE: INCLUDE IN THE INITIAL TEST VERSION

Within the selected organization, make findings, supporting evidence, qualifications, limitations, credible conflicts, unresolved questions, excluded or unavailable material, the Source Register, and the completion statement visible when they are relevant.

Produce the Research Output specified above. Do not proceed to the final Runtime deliverable.

8. RESEARCH SOURCE DOCUMENTATION

PROJECT-SPECIFIC CONTENT: COMPLETE OR CUSTOMIZE

Citation style: [Enter the required style or method.]

Source Register fields: [Add or modify fields required by the project.]

Research Source and Content Source distinction: [State when both must be recorded.]

DCPF HIGHLY RECOMMENDED LANGUAGE: INCLUDE IN THE INITIAL TEST VERSION

Support material factual claims with citations close enough to the claim for a reviewer to identify their basis.

The Source Register should include, when available and applicable: author or responsible organization; source title; publication or revision date; direct link or document identifier; relevant page, section, timestamp, or data location; access date; the role the source played; and important limitations or conflicts.

Never invent an author, title, date, quotation, link, document identifier, page number, publication detail, or access record. Clearly mark unavailable information rather than attempting to reconstruct it.

Distinguish source-supported findings from the AI's explanatory wording, user-provided information, inference, and examples created only for illustration.

9. ESSENTIAL: SEQUENTIAL OUTPUT REQUIREMENTS

PROJECT-SPECIFIC CONTENT: COMPLETE OR CUSTOMIZE

[Add a project-specific sequence only when fixed sets, approvals, or handoffs are genuinely required. Otherwise use the natural-break rule below.]

CORE DCPF HIGHLY RECOMMENDED LANGUAGE: INCLUDE IN THE INITIAL TEST VERSION

Purpose: When the requested output may exceed what can be comfortably delivered in a single response, divide it into multiple sequential parts while preserving structure, completeness, detail, and continuity.

Apply this requirement automatically whenever the complete output may exceed practical response length or its length cannot be reliably predicted.

Do not compress, summarize, abbreviate, or omit required content merely to fit within one response.

Favor completeness over minimizing the number of responses.

Break the output at a natural point. Do not force the work into predetermined sets or fixed section groupings unless this request explicitly requires them.

At the end of each nonfinal part, clearly state that additional output remains, request permission to continue, and stop.

After permission is received, continue from where the previous part ended until the complete output has been delivered.

The final part must clearly state that the requested output is complete.

A short, predictable output may be delivered in one response.

10. GOVERNANCE AND FINAL QUALITY REVIEW

PROJECT-SPECIFIC CONTENT: COMPLETE OR CUSTOMIZE

Governing requirements: [State the applicable controls, policies, standards, and human-authority instructions directly in this section.]

Quality standards: [Define required accuracy, completeness, clarity, traceability, usefulness, accessibility, and other needs.]

Prohibited practices or content: [Identify prohibited actions, assumptions, substitutions, or methods.]

Human review authority: [Identify the person or role responsible for reviewing and accepting the Research Output.]

Clarification and escalation rules: [State when the AI must ask, report, or stop.]

Project-specific final checks: [Add required checks.]

DCPF HIGHLY RECOMMENDED LANGUAGE: INCLUDE IN THE INITIAL TEST VERSION

The Research Output must remain within the approved purpose, sources, scope, and Governance boundaries. Do not fabricate findings, sources, quotations, data, access, consensus, or certainty. Do not conceal important limitations, missing evidence, or credible disagreements.

Before declaring completion, verify that every Research Objective has been answered or identified as unresolved; every required output section is present; material factual claims are supported by permitted, traceable sources; no source or access information was invented; conflicts, uncertainty, and limitations remain visible; the work remains within scope; prohibited practices were avoided; and Sequential Output Requirements were followed when needed.

If a requirement fails, correct the problem before delivering the completion statement. If the problem cannot be corrected within the approved sources, scope, and Governance boundaries, report the limitation and request human direction.

Completion of this Research Request produces Research Output for human review. The Research Output does not automatically become approved Knowledge and must not be represented as authoritative project Knowledge until the designated human authority reviews and accepts it.

END OF RESEARCH REQUEST

Suggested execution prompt: Please execute the attached Research Request.

Knowledge Repository Template Start

A reusable master for preserving accepted project intelligence with visible status, scope, source, and maintenance controls

COPY BEFORE EDITING. BEGIN WITH A SMALL TEST PROJECT. DO NOT TREAT A DRAFT OR TEST VERSION AS AUTHORITATIVE UNTIL IT HAS BEEN REVIEWED AND HUMAN-APPROVED.

Use the labels throughout this template. Add the project-specific content requested in brackets. Include the DCPF highly recommended language in the initial test version. After testing, revise either type of content when evidence demonstrates a project-specific need, and preserve the reason for consequential changes.

PROJECT-SPECIFIC CONTENT: COMPLETE OR CUSTOMIZE

Repository Name: *[Enter the descriptive name of the Knowledge Repository.]*

Repository ID: *[Enter the project-specific artifact identifier.]*

Project Name: *[Enter the project name.]*

Owner or Curator: *[Enter the responsible person or role.]*

Version: *[Enter the version number.]*

Status: *[MASTER / DRAFT / TEST / APPROVED / RETIRED / ARCHIVED]*

Creation or Revision Date: *[Enter the date.]*

Approving Authority: *[Enter the person or role authorized to accept and revise Knowledge.]*

Source, validation, and human-authority requirements: *[State the acceptance, maintenance, protection, revision, removal, and permitted-use controls that apply to this Knowledge Repository.]*

Source Research Artifacts: *[Identify accepted Research Requests, Research Outputs, and versions.]*

Permitted Runtime Uses: *[Identify the approved scope of reuse.]*

Authoritative Status: *[State whether this is a clean master, working copy, test version, or current authoritative version.]*

1. FRAMEWORK PURPOSE

PROJECT-SPECIFIC CONTENT: COMPLETE OR CUSTOMIZE

[State what accepted Knowledge is preserved, why it is maintained, who may use it, and the approved downstream purposes.]

DCPF HIGHLY RECOMMENDED LANGUAGE: INCLUDE IN THE INITIAL TEST VERSION

This Knowledge Repository preserves human-reviewed and accepted project intelligence for controlled reuse. It does not convert raw Research Output, model-generated content, or working notes into authoritative Knowledge merely because they have been stored here.

Only content carrying the required approval status may govern Runtime. Preserve the accepted evidence, qualifications, uncertainty, scope, and source information needed to interpret the Knowledge responsibly.

2. KNOWLEDGE CONFIGURATION VARIABLES

PROJECT-SPECIFIC CONTENT: COMPLETE OR CUSTOMIZE

Current subject, category, view, selection, audience, jurisdiction, time period, or use condition: [Enter variables that organize access.]

Variable behavior: [State how a variable changes selection or presentation without changing the accepted Knowledge itself.]

DCPF HIGHLY RECOMMENDED LANGUAGE: INCLUDE IN THE INITIAL TEST VERSION

Configuration variables may help locate or select Knowledge for a current use. They must not silently rewrite an accepted Knowledge statement, remove a qualification, or expand the approved scope.

3. KNOWLEDGE STRUCTURE

PROJECT-SPECIFIC CONTENT: COMPLETE OR CUSTOMIZE

Approved Knowledge: [Insert or reference the complete human-approved content and repeatable entry records.]

Unresolved or Provisional Material: [Preserve open questions or limited-use material separately.]

Rejected or Excluded Material: [Preserve the decision and reason when useful; do not present this material as approved.]

Outdated or Superseded Material: [Identify content that must not govern current Runtime and point to the current version.]

Archived Material: [Preserve historical content separately from current authority.]

DCPF HIGHLY RECOMMENDED LANGUAGE: INCLUDE IN THE INITIAL TEST VERSION

Do not rebuild or oversimplify a well-organized, accepted Research Output merely to make the repository shorter. Remove temporary execution scaffolding only when it no longer supports maintenance, interpretation, traceability, or reuse.

Keep status visible. Approved, unresolved, rejected, outdated, and archived material must not be blended into one undifferentiated collection.

Runtime must receive the complete approved Knowledge needed for the task. Supply it by attaching the repository, embedding or pasting the required contents into the working context, or retrieving it from a source the AI can actually access. A repository name, filename, link label, or artifact identifier does not provide the content by itself.

4. INFORMATION CATEGORIES

PROJECT-SPECIFIC CONTENT: COMPLETE OR CUSTOMIZE

Category name: [Enter a category, heading, label, tag, or status group.]

Purpose and inclusion rule: [State what belongs in the category.]

Related categories: [Identify dependencies or distinctions.]

Add categories such as Project Definitions, Approved Facts, Procedures and Operating Rules, Governance Requirements, User or Audience Requirements, Risks and Limitations, Source Register, and Unresolved Questions when they fit the project.

DCPF HIGHLY RECOMMENDED LANGUAGE: INCLUDE IN THE INITIAL TEST VERSION

Use categories to improve location and interpretation, not to fragment Knowledge unnecessarily. Preserve the relationship between a statement, its evidence, its qualifications, and its approved scope.

5. GOVERNANCE REFERENCES

PROJECT-SPECIFIC CONTENT: COMPLETE OR CUSTOMIZE

Applicable Source, validation, and human-authority requirements: [State the controls directly in this section.]

Admission and acceptance rules: [Define what may enter the repository and who approves it.]

Maintenance and review rules: [Define review cycles and change triggers.]

Protection, access, privacy, revision, removal, and archival rules: [Define the controls.]

Runtime use restrictions: [Define what Runtime may and may not do with the Knowledge.]

DCPF HIGHLY RECOMMENDED LANGUAGE: INCLUDE IN THE INITIAL TEST VERSION

Do not silently edit approved Knowledge. Record the change, evidence or reason, source, reviewer, approval decision, effective date, and new authoritative version.

When Knowledge is missing, contradictory, outdated, inaccessible, or outside its approved scope, Runtime must report the gap and follow the applicable escalation or Research path rather than filling the gap invisibly from general model knowledge.

OPTIONAL / SUPPLEMENTAL: METADATA

PROJECT-SPECIFIC CONTENT: COMPLETE OR CUSTOMIZE

Repository-level metadata: [Add useful version, date, status, owner, tags, jurisdiction, language, review date, confidentiality, retention, or scope fields.]

Entry-level metadata: [Add only fields that support location, interpretation, governance, maintenance, or reuse.]

DCPF HIGHLY RECOMMENDED LANGUAGE: INCLUDE IN THE INITIAL TEST VERSION

Metadata supports the Knowledge; it does not replace it. Do not assume an identifier, tag, or filename supplies the underlying content to an AI system.

OPTIONAL / SUPPLEMENTAL: REPEATABLE KNOWLEDGE ENTRY RECORD

PROJECT-SPECIFIC CONTENT: COMPLETE OR CUSTOMIZE

Knowledge Entry ID: [Enter a unique identifier.]

Title or Label: [Enter a short findable name.]

Knowledge Statement or Content: [Preserve the approved information with necessary context.]

Classification or Type: [For example: Definition / Fact / Procedure / Rule / User Requirement / Risk / Limitation.]

Status: [APPROVED / PROVISIONAL / UNRESOLVED / REJECTED / OUTDATED / SUPERSEDED / ARCHIVED]

Approved Scope of Use: [State where, how, and by whom the entry may be used.]

Source or Origin: [Identify the evidence, user decision, policy, or accepted Research Output.]

Source Artifact and Version: [Enter the artifact ID and version.]

Date Verified or Accepted: [Enter the date.]

Human Approver: [Enter the person or role.]

Qualifications, Restrictions, or Limitations: [Preserve what a user must know before reuse.]

Review Date or Trigger: *[Enter a date or event that requires review.]*

Supersedes or Superseded By: *[Enter the related entry or version, if applicable.]*

Related Entries: *[Identify dependencies, conflicts, or useful connections.]*

DCPF HIGHLY RECOMMENDED LANGUAGE: INCLUDE IN THE INITIAL TEST VERSION

An entry marked Provisional, Unresolved, Rejected, Outdated, Superseded, or Archived must not be used as current approved Knowledge unless the applicable Governance explicitly authorizes that status for the stated use. When an entry changes, preserve the prior version and the reason for the decision as required by Governance.

END OF KNOWLEDGE REPOSITORY TEMPLATE START

Runtime Request Template Start

A reusable master for applying approved Knowledge to a current task and producing a defined Project Output

COPY BEFORE EDITING. BEGIN WITH A SMALL TEST PROJECT. DO NOT TREAT A DRAFT OR TEST VERSION AS AUTHORITATIVE UNTIL IT HAS BEEN REVIEWED AND HUMAN-APPROVED.

Use the labels throughout this template. Add the project-specific content requested in brackets. Include the DCPF highly recommended language in the initial test version. After testing, revise either type of content when evidence demonstrates a project-specific need, and preserve the reason for consequential changes.

PROJECT-SPECIFIC CONTENT: COMPLETE OR CUSTOMIZE

Request Name: *[Enter the descriptive name of the Runtime Request.]*

Request ID: *[Enter the project-specific artifact identifier.]*

Project Name: *[Enter the project name.]*

Owner: *[Enter the responsible person or role.]*

Version: *[Enter the version number.]*

Status: *[MASTER / DRAFT / TEST / APPROVED / RETIRED / ARCHIVED]*

Creation or Revision Date: *[Enter the date.]*

Approving Authority: *[Enter the person or role authorized to approve this request.]*

Input Knowledge: *[Identify each approved Knowledge artifact and version. Supply the complete required content with execution through the selected route.]*

Source, validation, and human-authority requirements: *[State the standards, constraints, validation requirements, output rules, and human approval points that apply to this Runtime Request.]*

Expected Project Output: *[Name the deliverable this request must produce.]*

Related Artifacts: *[Identify project plans, input forms, validation standards, or other dependencies.]*

Authoritative Status: *[State whether this is a clean master, working copy, test version, or current authoritative version.]*

1. FRAMEWORK PURPOSE

PROJECT-SPECIFIC CONTENT: COMPLETE OR CUSTOMIZE

[State what work Runtime must perform now, why the Project Output is needed, who will use it, and how this execution differs from Research or repository maintenance.]

DCPF HIGHLY RECOMMENDED LANGUAGE: INCLUDE IN THE INITIAL TEST VERSION

Execute this document as a complete Runtime Request using the supplied complete approved Knowledge, current variables, task instructions, output requirements, Sequential Output Requirements, and Governance controls stated below.

Do not merely review, summarize, or explain this document. Do not conduct additional Research, alter the Knowledge Repository, approve new Knowledge, or create an exception to Governance unless this request and the applicable Governance explicitly authorize that action.

If required Knowledge or another input is missing, incomplete, inaccessible, contradictory, or outside its approved scope, identify the limitation and request direction rather than inventing content or silently substituting general model knowledge. If a stated requirement is unclear or internally inconsistent, pause the affected work and ask the designated human authority for direction.

2. RUNTIME CONFIGURATION VARIABLES

PROJECT-SPECIFIC CONTENT: COMPLETE OR CUSTOMIZE

Current subject or item: [Enter the current value.]

Intended audience and delivery setting: [Enter the current values.]

Length, duration, depth, or technical level: [Enter the current values.]

Tone, style, format, and accessibility requirements: [Enter the current values.]

Platform, file, or publication requirements: [Enter the current values.]

Additional Runtime variables: [Add named variables and values as needed.]

DCPF HIGHLY RECOMMENDED LANGUAGE: INCLUDE IN THE INITIAL TEST VERSION

Treat Runtime variables as current execution settings. Changing a Runtime variable changes the run, not the approved Knowledge. Do not allow a variable to silently expand the task beyond the approved scope.

3. INPUT KNOWLEDGE

PROJECT-SPECIFIC CONTENT: COMPLETE OR CUSTOMIZE

Approved Knowledge artifacts: [List names, IDs, versions, and approval status.]

Knowledge supply route: [State whether the complete Knowledge is attached; embedded or pasted into the working context; or retrieved from a source the AI can actually access.]

Permitted portions and scope of use: [Identify which content may govern this execution.]

Excluded, unresolved, outdated, or prohibited content: [Identify what Runtime must not use.]

Additional user inputs: [Identify current records, selections, measurements, or other permitted inputs.]

DCPF HIGHLY RECOMMENDED LANGUAGE: INCLUDE IN THE INITIAL TEST VERSION

Use the complete approved Knowledge supplied for this execution as the project's data engine. Supply it by attachment, by embedding or pasting it into the working context, or by retrieving it from a source the AI can actually access. An identifier, filename, prior-chat reference, folder location, or link label does not provide the content by itself.

Use approved content within its stated scope and preserve its qualifications and limitations. Do not treat unresolved, rejected, outdated, superseded, or archived material as current approved Knowledge unless Governance explicitly authorizes that use.

Do not fill a Knowledge gap with general model knowledge or unapproved Research. Stop, identify the gap, and follow the applicable Governance or Research path.

4. TASK DEFINITION

PROJECT-SPECIFIC CONTENT: COMPLETE OR CUSTOMIZE

Primary task: [State exactly what the AI must do with the supplied Knowledge and current inputs.]

Required operations or steps: [List any transformation, analysis, selection, organization, drafting, calculation, or validation steps.]

Permitted discretion: [State choices the AI may make.]

Required clarification or escalation: [State when the AI must ask or stop.]

Completion criteria: [State what must be true for the task to be complete.]

DCPF HIGHLY RECOMMENDED LANGUAGE: INCLUDE IN THE INITIAL TEST VERSION

Perform only the defined Runtime task. Do not quietly reopen Research, modify the repository, broaden the intended use, or replace a human decision with a model assumption.

When a task instruction conflicts with approved Knowledge or Governance, preserve the higher governing boundary, identify the conflict, and request human direction.

5. OUTPUT REQUIREMENTS

PROJECT-SPECIFIC CONTENT: COMPLETE OR CUSTOMIZE

Required output title, sections, and order: [Define the structure.]

Required length, format, file type, tone, technical level, and audience treatment: [Define the requirements.]

Required fields, tables, calculations, citations, notes, or appendices: [Define the requirements.]

Prohibited content, wording, formats, or unsupported additions: [Define the limits.]

Internal validation note: [Define the checks and information the AI must report.]

Human review and publication status: [State the required approval before use.]

DCPF HIGHLY RECOMMENDED LANGUAGE: INCLUDE IN THE INITIAL TEST VERSION

Produce the complete Project Output in the specified form. Preserve required components, limitations, and validation information. Do not compress, omit, or disguise a failed requirement to make the deliverable appear finished.

The Project Output does not automatically become approved Knowledge. Any new factual material proposed for reuse must pass through the applicable Research, review, acceptance, and repository process.

6. ESSENTIAL: SEQUENTIAL OUTPUT REQUIREMENTS

PROJECT-SPECIFIC CONTENT: COMPLETE OR CUSTOMIZE

[Add a project-specific sequence only when fixed sets, approvals, or handoffs are genuinely required.

Otherwise use the natural-break rule below.]

CORE DCPF HIGHLY RECOMMENDED LANGUAGE: INCLUDE IN THE INITIAL TEST VERSION

Purpose: When the requested output may exceed what can be comfortably delivered in a single response, divide it into multiple sequential parts while preserving structure, completeness, detail, and continuity.

Apply this requirement automatically whenever the complete output may exceed practical response length or its length cannot be reliably predicted.

Do not compress, summarize, abbreviate, or omit required content merely to fit within one response.

Favor completeness over minimizing the number of responses.

Break the output at a natural point. Do not force the work into predetermined sets or fixed section groupings unless this request explicitly requires them.

At the end of each nonfinal part, clearly state that additional output remains, request permission to continue, and stop.

After permission is received, continue from where the previous part ended until the complete output has been delivered.

The final part must clearly state that the requested output is complete.

A short, predictable output may be delivered in one response.

7. GOVERNANCE REFERENCES

PROJECT-SPECIFIC CONTENT: COMPLETE OR CUSTOMIZE

Applicable Source, validation, and human-authority requirements: [State the standards, constraints, validation requirements, output rules, and human approval points directly in this section.]

Directly applicable quality, source, prohibition, validation, output, and human-approval rules: [List or embed them.]

Clarification, escalation, and stopping rules: [State the required response.]

Project-specific final checks: [Add required checks.]

DCPF HIGHLY RECOMMENDED LANGUAGE: INCLUDE IN THE INITIAL TEST VERSION

Apply the requirements stated in this Runtime Request throughout execution and validation. If a control is unclear, internally inconsistent, or in conflict with another requirement, pause the affected work and ask the designated human authority for direction.

Before declaring completion, verify that the supplied approved Knowledge was used within scope; every task and output requirement was satisfied or identified as unresolved; unsupported factual additions were not introduced; applicable prohibitions and Governance controls were followed; Sequential Output Requirements were applied when needed; and required human review remains visible.

If a requirement fails, correct the Runtime work when the problem belongs to execution. If the failure reveals missing or inadequate Knowledge, Research, or Governance, report the gap and route it to the framework that owns the responsibility rather than patching the output invisibly.

END OF RUNTIME REQUEST

Suggested execution prompt: Please execute the attached Runtime Request using the complete supplied Knowledge.

Optional AI-Assisted Template Builder Request

This copyable request may be used to create a working draft from one supplied Template Start. It is not a fifth DCPF framework.

Use the supplied DCPF Template Start as the structure for the requested artifact. Preserve its section headings, order, artifact-information block, highly recommended language, labels, and end-of-artifact boundary in the initial test version.

Use only the supplied project information and accessible approved materials. Replace bracketed instructions when the information provides an answer. When information is missing, retain a clear bracketed placeholder rather than inventing a decision, fact, source, approval, or access claim.

Keep Governance, Research, Knowledge, and Runtime responsibilities separate. Create only the requested artifact. Mark it DRAFT: NOT APPROVED FOR FORMAL EXECUTION, then stop for human review.

Final Note

Keep the clean master unchanged. Test a working copy on a small project, revise language when evidence identifies a need, and approve only the version and scope that were actually reviewed.

Use Chapters 9–14 for the preparation and project-building workflow, and Appendix D, DCPF Project Checklist, for formal inspection and maintenance guidance.

Design it. Test it. Learn from the evidence. Preserve the version that works.

Copyright © 2026 Ralph A. Perez, Jr. All rights reserved. | dcpfai.org

[Return to Table of Contents](#)

Recommended DCPF Workspace Organization

DCPF remains an experimental approach. Use these workspace patterns to support controlled testing and adaptation; they do not replace domain-specific validation or human judgment before real-world or production use.

Practical file layouts for small, medium-sized, and large projects

1. How to Use This Appendix

Begin with the simple working environment introduced in Chapter 9 unless your project already has a clear reason to require more structure. The layouts in this appendix are the scaling path from that starter workspace, not additional setup that every DCPF user must complete before beginning. Use the small-project layout when Chapter 9's starter folder no longer provides enough separation; move to the medium or large patterns only when repeated workstreams, collaboration, review, governance, release, security, retention, or audit responsibilities create a practical need. Every DCPF project may also have goals, tools, or organizational standards that call for a different file structure. Adapt the layouts while preserving the relationships among Governance, Research, Accepted Knowledge, Runtime activity, review, and outputs.

The purpose of organization is practical: people should be able to locate a file, understand what it is, see how it relates to other artifacts, and determine whether it is current and approved. A useful workspace should make the project easier to operate, not create extra work merely for the sake of structure.

2. Core Organization Principles

- Keep framework requests separate from the outputs they produce.
- Preserve approved knowledge independently from unreviewed research and working notes.
- Maintain a visible relationship among requests, sources, outputs, and decisions.
- Separate active materials from superseded or archived materials.
- Use consistent file names, version identifiers, and status labels.
- Add organizational complexity only when the Chapter 9 starter workspace no longer makes the project's responsibilities, artifacts, status, or handoffs easy to see.
- Keep governance records in a stable location that participants can find and review.

3. Small Individual Project Workspace

Use this lean layout for one-person projects with a narrow purpose: a single research task, writing project, household planner, personal assistant, or other focused effort. It provides a home for every core DCPF artifact without turning file management into a second project.

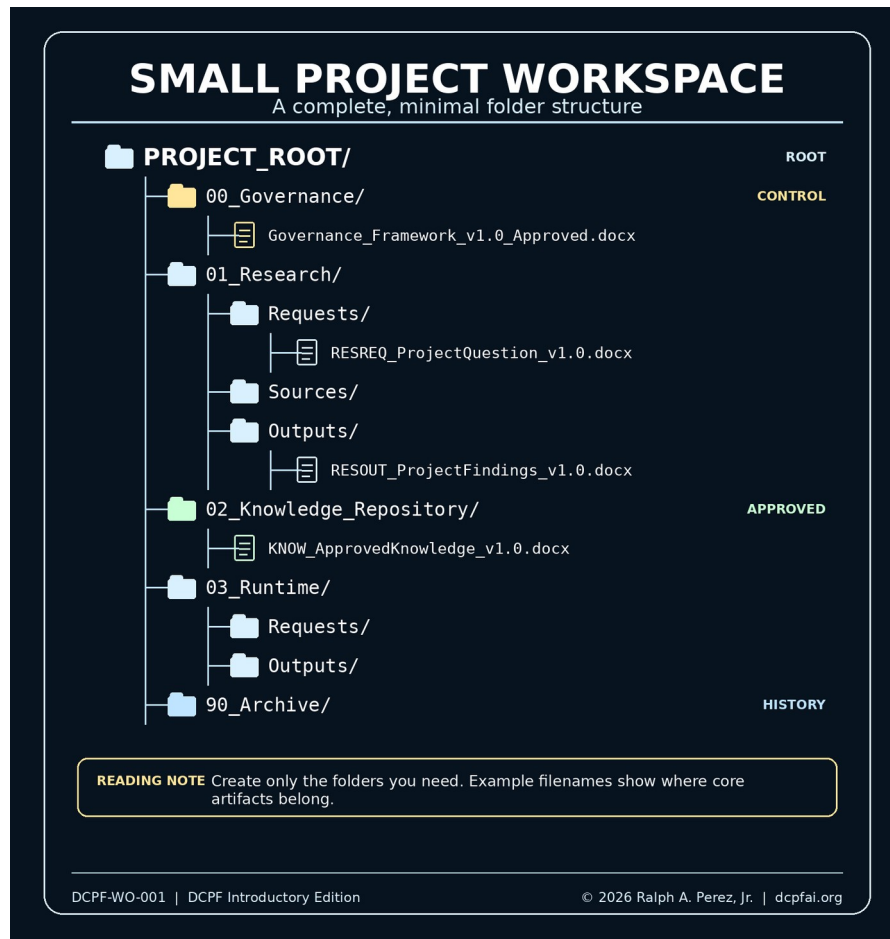


Figure 1. Recommended layout for a small individual DCPF project.

Figure Description: This folder tree shows a complete minimal workspace for one focused project. The project root contains Governance; Research with separate Requests, Sources, and Outputs; the Knowledge Repository; Runtime with Requests and Outputs; and Archive. Selected example files show where a Governance Framework, Research Request, research output, and approved knowledge entry belong. Pale gold identifies governance, mint identifies approved knowledge, and labels accompany every color.

How to read this layout

Start with PROJECT_ROOT and follow each indented line as a folder inside the one above it. A document icon marks an example file rather than another folder. You may copy this structure as shown, but you do not need to create an empty folder until the project has material to place in it.

How files move through the small workspace

A Research Request begins in 01_Research/Requests. Its sources and resulting research output remain in the Research area until the user reviews them. Approved, reusable knowledge is then preserved in 02_Knowledge_Repository. A Runtime Request draws on that approved knowledge and is stored in 03_Runtime/Requests; the deliverable it produces goes in 03_Runtime/Outputs. When a current artifact is replaced, the older version moves to 90_Archive rather than remaining beside the active file.

Keep it lean: Empty subfolders do not have to be created in advance. Add them when the first matching file appears.

4. Medium-Sized Individual or Small-Team Workspace

Use this layout when a project repeats the DCPF cycle, contains several workstreams or phases, produces multiple deliverables, or involves a small team. The structure adds shared controls and repeatable workstream folders while keeping day-to-day navigation predictable.

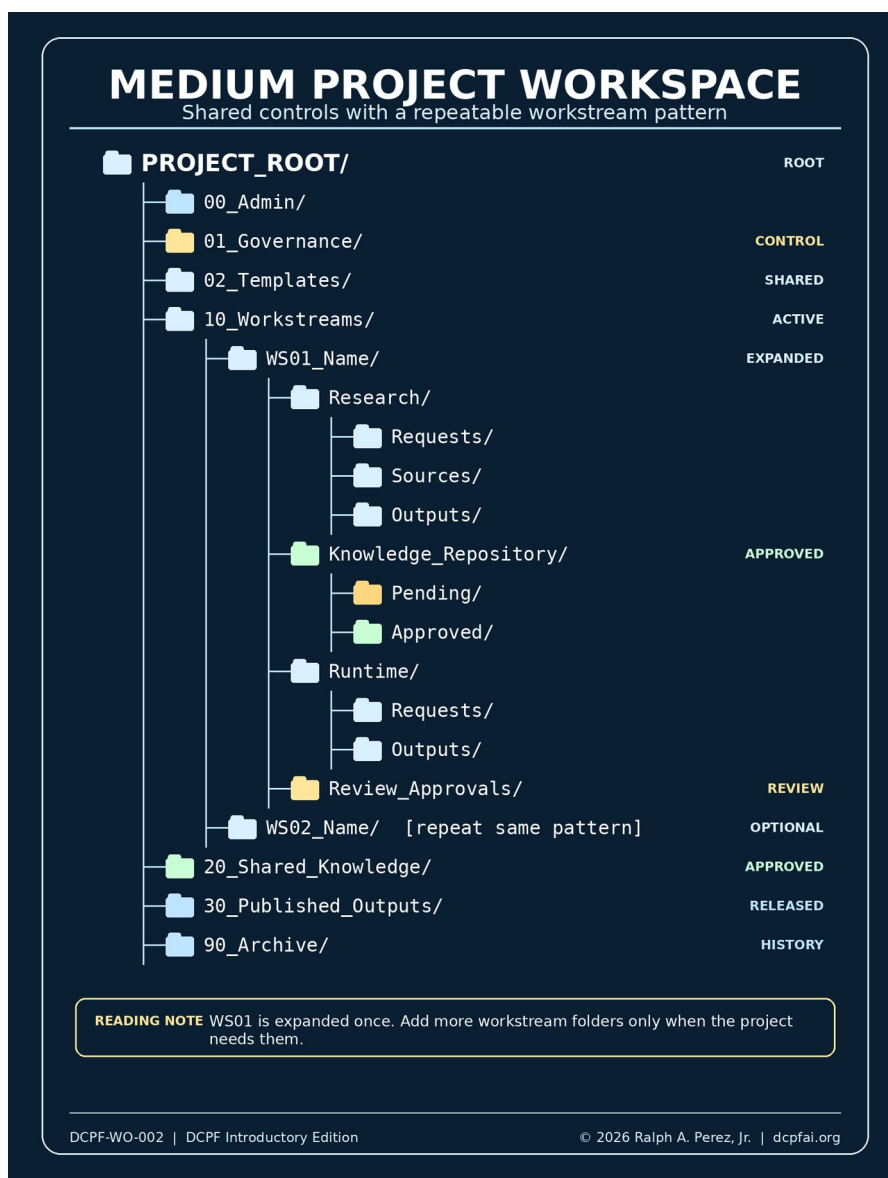


Figure 2. Recommended layout for a medium-sized individual or small-team DCPF project.

Figure Description: This folder tree shows a medium-sized workspace with central Administration, Governance, and Templates; a Workstreams folder; Shared Knowledge; Published Outputs; and Archive. WS01 is expanded to show the reusable internal pattern for Research, Knowledge Repository, Runtime, and Review and Approvals. WS02 is shown as an optional second

workstream that repeats the same pattern. Pale gold identifies controls and review, while mint identifies approved or reusable knowledge.

How to read this layout

Shared project materials remain near the top of the tree. Day-to-day work sits inside WS01_Name. When another distinct stream of work begins, add WS02_Name and repeat the same internal folders rather than inventing a new arrangement. Optional labels identify folders that are useful only when the project reaches that level of activity.

Recommended operating practices

- Give each workstream a short identifier, such as WS01, WS02, or a meaningful department or phase code.

- Place reusable framework starts and standard forms in 02_Templates; keep project-specific completed requests inside their workstreams.

- Use Review_Approvals for human review notes, acceptance decisions, and records showing why an artifact became approved.

- Promote only broadly reusable, reviewed material into 20_Shared_Knowledge.

- Use a simple owner or steward field in file metadata, a project index, or the file name when responsibility would otherwise be unclear.

5. Large Project Workspace

Use this model for long-running, multidisciplinary, organizational, or multi-team initiatives. It establishes common controls at the portfolio or organizational level, then repeats a recognizable DCPF structure inside programs and workstreams. The repeated pattern makes local work understandable without forcing every team to store everything in one enormous folder.

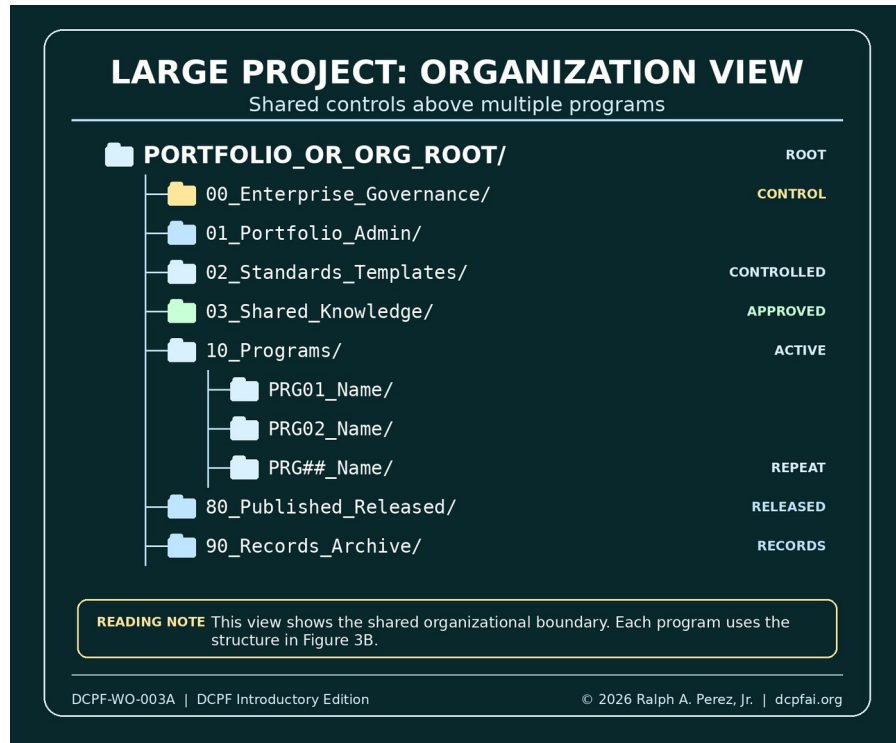


Figure 3A. Organization-level layout for a large DCPF project.

Figure Description: This high-level folder tree shows the shared organizational boundary. Enterprise Governance, Portfolio Administration, Standards and Templates, and Shared Knowledge sit above the Programs area. Multiple program folders may operate beneath that area, while Published and Released materials and the Records Archive remain distinct organization-level locations. The expanded internal program pattern appears in Figure 3B.

How to read the organization view

Figure 3A deliberately stops at the program level. It shows what the entire initiative shares and where each program begins. PRG##_Name is a pattern marker: replace the number signs and name with a real identifier, such as PRG03_CustomerSupport.

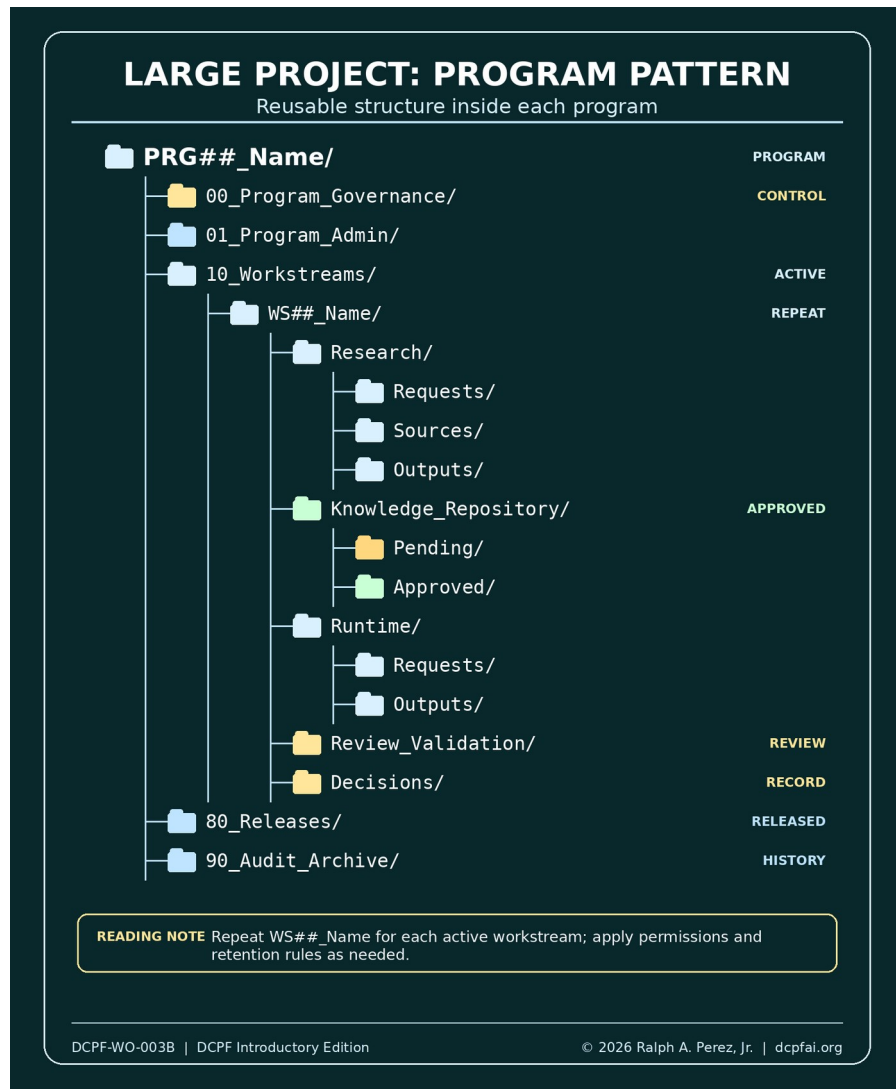


Figure 3B. Reusable program and workstream layout for a large DCPF project.

Figure Description: This folder tree expands one program. The program contains its own Governance and Administration areas, a Workstreams area, Releases, and an Audit Archive. Inside the example workstream are Research, Knowledge Repository, Runtime, Review and Validation, and Decisions folders. The Research, Knowledge Repository, and Runtime folders preserve the same DCPF separation used in smaller projects, while review, decision, release, and audit records support greater scale and formality.

How to read the program pattern

Create one PRG##_Name folder for each program and one WS##_Name folder for each distinct workstream. Repeat the internal workstream pattern consistently. Access controls, retention rules, or required records can differ by program or workstream when organizational policy requires it.

Additional controls for large workspaces

Align the workspace with existing document-management systems, security requirements, records policies, and legal obligations.

Assign owners for enterprise governance, shared knowledge, templates, releases, and records retention. Separate draft, approved, and released states through permissions or controlled locations, not file names alone when the platform supports stronger controls.

Grant access according to role and need. Some research sources, governance records, or runtime outputs may require restricted access.

Maintain formal decision, validation, audit, and change records when outputs affect customers, employees, regulated activity, or public release.

Avoid needless duplication. Use controlled references or links when the platform can preserve access and version history reliably.

6. Recommended File-Naming and Version Practices

A file name should answer the most useful questions before the file is opened: Which project or workflow does it belong to? What kind of artifact is it? What does it address? Which version and status is it?

Suggested pattern: [Project]_[Workflow]_[ArtifactType]_[ShortTitle]_v#.#_[Status]_[YYYY-MM-DD].ext

Component	Example	Purpose
Project	DCPFBOOK	Connects the file to the project.
Workflow	WS03 or GLOSSARY	Identifies the workflow, phase, or subject.
Artifact type	RESREQ, RESOUT, KNOW, RUNREQ, RUNOUT, GOV	States the artifact's role.
Short title	TerminologyReview	Provides a readable subject.
Version	v1.0 or v1.2	Distinguishes meaningful revisions.
Status	Draft, InReview, Approved, Superseded, Archived	Shows lifecycle state.
Date	2026-08-06	Adds a sortable date when useful.

Examples

HOME_RESREQ_GarageProjectMaterials_v1.0_Approved_2026-08-06.docx
 DCPFBOK_GLOSSARY_KNOW_CoreTerms_v1.2_Approved_2026-08-06.docx
 CLIENTA_WS03_RUNOUT_QuarterlyBrief_v0.4_InReview_2026-08-06.docx

The pattern is intentionally modular. Small projects may omit the project or workflow code when the enclosing folder already makes it obvious. Avoid changing names for every minor edit; increase the

version when a change is meaningful enough that a collaborator or future user may need to distinguish it.

7. Maintaining Knowledge and Artifact Lineage

Lineage is the visible chain showing how one artifact informed another. DCPF does not require a database to preserve it. For many projects, a short reference line inside each request or output, and a simple project index, is enough.

Research Request: assign an artifact ID and record its purpose, date, owner, and governing requirements.

Research output: record the Research Request ID and preserve its source list or source record.

Knowledge Repository entry: record the research output or human decision that supports the approved knowledge.

Runtime Request: identify the approved knowledge, governance version, and other inputs used.

Runtime output: record the Runtime Request ID, review status, reviewer or approving role, and final disposition.

Governance decision: identify the affected artifact, rule, exception, or release and preserve the rationale.

Practical option: Keep a Project_Index.xlsx, Project_Index.docx, or plain-text README at the workspace root. The index can list artifact ID, title, location, owner, status, version, and upstream or downstream references. Use the lightest tool that the project can maintain reliably.

8. When to Move Beyond the Chapter 9 Starter Workspace

Move to a more structured Appendix C layout when the Chapter 9 starter arrangement begins to hide information, blur responsibility, or create uncertainty. Common signals include:

Files are becoming difficult to locate or identify.

Multiple people are editing or approving the same materials.

Several workstreams or project phases are active at once.

Approved knowledge is being confused with unreviewed research.

Runtime outputs require formal review, release, or publication.

Governance decisions, exceptions, or changes need a clearer record.

Permissions, retention rules, or audit requirements differ across materials.

Growth does not require rebuilding everything at once. Create the new top-level structure, move current approved materials first, then relocate active work and archive older items as they are reviewed.

9. Workspace Setup Checklist

Project scope and owner identified

Layout matched to the project's actual size and risk

Stable governance location established

Research Requests separated from research outputs

Knowledge Repository established for approved knowledge

Runtime Requests separated from runtime outputs

Naming and version convention chosen

Review and approval process identified

Artifact lineage method selected

Archive or records location created

Permissions and retention needs considered

Structure documented where collaborators can find it

A well-organized DCPF workspace should make the architecture easier to see and easier to trust. The best structure is the simplest one that preserves clarity, separation, governance, and traceability for the work at hand.

[Return to Table of Contents](#)

DCPF Project Checklist

DCPF remains an experimental approach. Use this checklist to support controlled testing and adaptation; it does not replace domain-specific validation or human judgment before real-world or production use.

A practical workflow for establishing, testing, operating, and maintaining a DCPF project.

Project name: [Enter project name]

Project owner: [Enter project owner]

Project purpose: [Briefly describe the project's purpose]

Start date: [Enter date]

Current version: [Enter version]

Last review date: [Enter date]

How to use this checklist: Complete the items in order when establishing a project. Return to earlier sections whenever testing, new evidence, or project changes reveal a need for revision. Check an item only after it has been reviewed, not merely drafted.

Workflow: Define the work; build the relevant artifact and use Governance Framework responsibilities to define needed controls within it; develop the Governance Repository as controls are defined in the other framework artifacts; execute the completed artifact and perform human review.

1. Define the Project

State the intended outcome and identify who will use or benefit from it.

Define what is inside and outside the project scope.

List the inputs the project will receive and the outputs it must produce.

Select a small, representative use case and define the specific conditions its output must meet.

2. Build the Relevant Artifact and Define Needed Controls

Define the project's rules, limits, and nonnegotiable requirements.

Identify who may approve decisions and who reviews framework or repository changes.

Establish source, evidence, privacy, safety, and human-review requirements.

Define when the AI must identify uncertainty, ask for clarification, stop, or escalate.

3. Create and Run the Research Request

Define the research question, scope, constraints, timeframe, and required depth.

Specify which source types are permitted, preferred, or prohibited.

Define each configuration variable by its name, purpose, required or optional status, expected input format, and example value.

Specify the required research output sections, citation requirements, uncertainties, and unresolved gaps.

Review the research output against the Research Request and its stated requirements before accepting it.

4. Build the Knowledge Repository

Add only knowledge that has been reviewed and approved for reuse.

Organize approved knowledge into clearly named files, folders, or sections.

Record the source, approval status, and date for each reusable knowledge item.

Separate approved knowledge from drafts and unverified material; exclude duplicate, outdated, irrelevant, or unapproved content.

5. Create the Runtime Request

Define the repeatable task the Runtime Request will perform.

Define each configuration variable by its name, purpose, required or optional status, expected input format, and example value.

Specify which repository materials the AI must use and whether new research is permitted during runtime.

Specify the required output sections, format, length, and acceptance criteria.

Define what the AI must do when a required variable is missing, invalid, or ambiguous.

6. Test the DCPF Project

Test the framework with the small, representative use case.

Test at least one unusual, incomplete, or boundary-case input.

Compare each result with the acceptance criteria and requirements stated in the working artifact.

Trace each problem to governance, research instructions, repository knowledge, runtime instructions, or user input. Revise and retest the affected component, then record the approved version.

7. Operate and Maintain the Project

Use the current approved versions of the Research, Knowledge Repository, and Runtime components, and confirm that each states its applicable requirements.

Require human review before an output is used for a decision, published, or shared externally.

Add newly approved reusable knowledge and update repository items when facts, sources, or project decisions change.

Record recurring errors, exceptions, and missing information. When evidence shows a consistent weakness, revise and retest the affected component, then archive the replaced version.

Final Readiness Check

The Research, Knowledge Repository, and Runtime components are complete, clearly labeled, stored in their approved locations, and contain the applicable requirements.

The initial test meets the project's acceptance criteria.

Known limitations, unresolved questions, and identified risks are documented.

The approved project version and the person responsible for future review are recorded.

Working principle: Change the framework based on observed results and documented evidence.

[Return to Table of Contents](#)

Sequential Output Requirements

DCPF remains an experimental approach. Use these requirements to support controlled testing and adaptation; they do not replace domain-specific validation or human judgment before real-world or production use.

A practical completeness safeguard for large or unpredictable AI outputs.

This appendix serves as the dedicated reference for Sequential Output Requirements. It begins with the governing doctrine and complete copyable rule. The short explanations that follow show why the rule exists, what it protects, and how to apply it without turning every response into a multi-part exchange. It does not replace the complete SOR language embedded in Research and Runtime artifacts; those repetitions are intentional so each artifact carries the complete operating instruction it needs.

The Standard Sequential Output Objective

The Standard Sequential Output Objective is to preserve structure, completeness, detail, formatting, and continuity without compressing required work merely to fit within one response.

Sequential Output Requirements provide a safeguard when a requested Research Output or Project Output may be too large for a single response or when its complete length cannot be reliably predicted. The objective is not to make an output longer than necessary. It is to prevent response-length constraints from determining the quality or completeness of the work.

The following standard is designed to be copied directly into Research Requests, Runtime Requests, templates, or other appropriate DCPF artifacts.

Standard DCPF Sequential Output Requirement: Copyable Version

ESSENTIAL: Sequential Output Requirements

PURPOSE

When the requested Research Output or Project Output may exceed what can be comfortably delivered in a single response, or when its complete length cannot be reliably predicted, divide it into multiple sequential parts while preserving structure, completeness, detail, formatting, and continuity.

SEQUENTIAL OUTPUT RULE

Apply this requirement automatically whenever the complete output may exceed practical response length or its length cannot be reliably predicted.

Do not compress, summarize, abbreviate, or omit required work merely to fit within one response. Favor completeness over minimizing the number of responses.

Break the output at a natural point. Do not force the output into predetermined sets or fixed section groupings.

At the end of each nonfinal part, state that additional output remains, request permission to continue, and stop.

After permission is received, continue from where the previous part ended until the complete output has been delivered.

The final part must clearly state that the requested output is complete.

A short, predictable output may be delivered in one response.

Sequential delivery is a completeness safeguard, not a requirement to split every response.

Why the Sequential Output Requirement Exists

AI systems operate within practical response limits. Those limits can differ across models, interfaces, subscription levels, configurations, and tasks. Users should not have to predict when those limits may begin affecting an answer.

The SOR exists to help prevent:

strong detail at the beginning of an output followed by increasingly compressed later sections;

required sections, categories, or fields being omitted;

detailed material being summarized when full treatment was requested;

separate sections being merged simply to conserve response space;

source documentation being shortened or lost;

formatting or structural requirements deteriorating as the response grows;

a partial output being presented as though the complete task has been performed; and

inconsistent completion of similar large requests across different AI systems.

The requested work should determine the structure of the output. The size of a single AI response should not.

Completeness Before Response Count

Sequential Output Requirements deliberately favor completeness over minimizing the number of responses. Without that instruction, an AI system may try to make a large task fit into a single reply by shortening explanations, combining categories, omitting lower-priority material, reducing examples, or summarizing later sections.

If several responses are required to perform the request properly, several complete responses are preferable to one compressed response. At the same time, the rule does not encourage unnecessary length. A short and predictable output should simply be delivered in one response.

The objective is complete execution at the appropriate level of detail, not maximum output length.

Natural Breaks Instead of Arbitrary Chunks

When sequential delivery becomes necessary, the output should stop at a natural point. The AI should not automatically divide the work into arbitrary sets, fixed word counts, or predetermined percentages unless the user has specifically requested that structure.

For example, a long research report should preferably stop after completing a major research section rather than halfway through a source discussion. A long project deliverable should stop after a logical phase rather than simply because a predetermined word count has been reached.

The purpose of the natural-break rule is to preserve the organization of the work itself.

The Continuation Protocol

When additional output remains:

Complete the current natural segment.

Clearly state that additional output remains.

Request permission to continue.

Stop.

After permission is received, resume from where the previous response ended.

Repeat the process until the entire requested output has been delivered.

Only the final part may state that the requested output is complete.

Permission to continue creates a visible checkpoint between the human and AI system. The user can see that the task is not yet complete, that additional work remains, and that the AI intentionally stopped rather than silently omitting the remainder.

Sequential responses together constitute one complete execution.

Continuing Means Continuing

When the user authorizes continuation, the AI should continue the existing execution. It should not:

restart the task;

repeat completed sections;

summarize the previous response instead of continuing;

redesign the requested structure;

skip ahead to the conclusion; or

treat the continuation request as a new assignment.

The next response should begin where the previous one ended and preserve the same requirements, organization, terminology, level of detail, formatting, and governing instructions.

When the SOR Should Activate

Sequential Output Requirements should activate automatically when:

the requested output is clearly large;
the requested output contains many required sections or items;
extensive source documentation is required;
a substantial Research Output is being produced;
a substantial Project Output is being produced;
the requested level of detail makes a single response uncertain; or
the total length cannot be reliably predicted before execution begins.

A user should not have to know the response limits of a particular AI model. The conservative activation standard protects completeness before compression becomes visible.

When Sequential Delivery Is Not Necessary

Sequential Output Requirements do not require every DCPF request to be divided into multiple responses. If the requested output is short, predictable, and can be completed properly in one response, it should be delivered in one response.

Typical examples may include a short definition, a small comparison, a concise project summary, several recommendations, or another clearly bounded output.

Sequential delivery is a safeguard, not a mandatory output style.

The SOR Within DCPF

Sequential Output Requirements are used most directly within the Research and Runtime portions of the DCPF architecture.

In Research

The SOR protects the Research Output. A large investigation may contain multiple objectives, findings, source records, conflicting evidence, uncertainty, limitations, and other required material. Sequential delivery helps prevent later portions of the evidence package from being compressed or omitted simply because the research is extensive.

In Runtime

The SOR protects the Project Output. A Runtime Request may require a substantial report, guide, manuscript, plan, analysis, or other multi-part deliverable. Sequential delivery allows the Runtime Request to be executed completely instead of reducing the deliverable to fit within one reply.

As a Reusable Architectural Rule

Embedding the SOR directly into the appropriate Research or Runtime artifact or template means the safeguard travels with the project. The user does not have to remember to add a special corrective instruction after compression has already occurred.

A Simple SOR Example

Consider a Research Request requiring eight substantial research sections.

Without the SOR

An AI system may begin with detailed Sections 1 and 2, shorten Sections 3 through 5, compress Sections 6 and 7 into brief summaries, and provide only a few sentences for Section 8. All eight headings may technically appear, but the requested research has not been performed at a consistent level.

With the SOR

The AI completes the first group of sections at the required level of detail and reaches a natural stopping point. It then clearly states that additional Research Output remains and requests permission to continue. After authorization, it resumes with the next unfinished section at the same level of detail. Only after the final required section is finished does it state that the requested Research Output is complete.

The difference is not that the second approach automatically produces more words. The difference is that response length is not allowed to silently redefine the task.

Adding the SOR to Your Own DCPF Artifacts

For most users, implementation is straightforward: copy the Standard DCPF Sequential Output Requirement at the beginning of this appendix and place it in the appropriate Research Request, Runtime Request, framework template, or reusable project artifact.

Project-specific additions may strengthen continuity for a specialized use case, but they should not weaken the core protections:

- automatic activation when needed;
- no compression or omission merely to fit one response;
- completeness over response count;
- natural stopping points;
- explicit notice that additional output remains;
- human authorization before continuation;
- continuous execution from the previous stopping point; and
- an explicit declaration only when the complete requested output has been delivered.

These protections form the core of the Standard Sequential Output Objective.

[Return to Table of Contents](#)

DCPF Glossary

DCPF remains an experimental approach. Use this glossary to support controlled testing and adaptation; it does not replace domain-specific validation or human judgment before real-world or production use.

A quick-reference terminology appendix for the DCPF Introductory Edition

Use this glossary to quickly check how a term is used in DCPF. Foundational terms include a little additional context; other entries stay brief. A compact identity line shows each term's classification, where it is used, and who defines or supplies it. Cross-references point to the preferred term when two expressions describe the same idea.

A

Accepted Knowledge

Information that has passed the project's review or acceptance process and has been approved for use within a stated scope. Research does not become DCPF Knowledge merely because it was found or generated; an acceptance decision changes its status.

Classification: Knowledge status | **Used in:** Knowledge Repository; Research-to-Knowledge handoff; Runtime | **Defined or supplied by:** Governance or authorized user

Active context

The information presently available and relevant to the AI during an interaction. Also called the working context. See also Context window and Input Knowledge.

Classification: AI operating concept | **Used in:** AI interaction; Research and Runtime execution | **Defined or supplied by:** Platform and current inputs

Approval checkpoint

A point where work pauses for a person or authorized process to review it before the project continues, publishes an output, changes an artifact's status, or grants an exception.

Classification: Governance control | **Used in:** Governance Framework; project workflow | **Defined or supplied by:** Governance or authorized user

Approved Information Sources

The source types, organizations, databases, or other evidence channels that Governance permits Research to use. Approval answers whether a source may be used; it does not automatically establish how much evidentiary weight that source deserves for a particular claim. Recency, jurisdiction, access, or other use conditions may also apply.

Classification: Governance element | **Used in:** Governance Framework; Research Request | **Defined or supplied by:** Governance or authorized user

Approved Project Output

A Runtime deliverable that has passed the required review and is approved for its stated use. Approval does not automatically make the deliverable part of the Knowledge Repository.

Classification: Output status | **Used in:** Runtime output review; preservation | **Defined or supplied by:** Governance or authorized user

Architectural handoff

The controlled movement of information or work from one DCPF responsibility to another. At a handoff, the project's purpose for the information, its status, and the responsible owner may change.

Classification: Architectural concept | **Used in:** Framework-wide workflow | **Defined or supplied by:** DCPF-defined; project-configured

Architecture

The organized arrangement of responsibilities, artifacts, relationships, and controls that supports sustained AI collaboration. In DCPF, architecture shifts attention from one prompt or conversation to the larger system in which Research, Knowledge, Runtime, and Governance work together.

Classification: Core concept | **Used in:** Framework-wide | **Defined or supplied by:** DCPF-defined

Archive

A location for historical or retired material that is preserved for reference but no longer governs current work. Archived files should be clearly separated from authoritative files.

Classification: Project environment element | **Used in:** Project Environment | **Defined or supplied by:** User or project-defined

Artifact

A document, template, request, output, record, repository item, or other preserved object used or produced by a DCPF project. Artifacts make the system inspectable outside a single conversation.

Classification: Core concept | **Used in:** Framework-wide | **Defined or supplied by:** DCPF-defined

Artifact identity block

A short group of details that identifies an artifact, such as its type, ID, project, owner, version, status, applicable inputs, and intended output. It helps users confirm that the correct file and version are being used.

Classification: Artifact element | **Used in:** Framework artifacts; Project Environment | **Defined or supplied by:** User or project-defined

C

Clean context

A fresh chat or working environment in which the complete governing artifacts are supplied and unrelated prior instructions are minimized. A clean context can reduce interference from earlier conversation history.

Classification: Operating practice | **Used in:** Research and Runtime execution | **Defined or supplied by:** User-established

Completion statement

The statement that tells the user whether more output remains. Under the Sequential Output Rule, a nonfinal part must say that additional output remains; only the final part may state that the requested output is complete.

Classification: Sequential output element | **Used in:** Sequential output | **Defined or supplied by:** DCPF-defined; AI-generated

Configuration variable

A named value that can be changed without rebuilding the surrounding framework. Variables may set the current subject, audience, tone, length, date range, technical level, or other project-specific conditions.

Classification: Configurable element | **Used in:** Research and Runtime Requests | **Defined or supplied by:** User-defined

Context decay

The practical loss of influence or clarity that earlier information can experience during a long interaction. DCPF reduces this risk by deliberately preserving and resupplying important information through maintained artifacts.

Classification: AI operating concept | **Used in:** AI interaction; framework-wide planning | **Defined or supplied by:** Platform behavior

Context window

The amount of material an AI model can actively process during an interaction. A larger context window can hold more information, but it does not decide which information is current, approved, or authoritative.

Classification: AI platform concept | **Used in:** AI interaction; framework-wide planning | **Defined or supplied by:** Platform-defined

D

DCPF architecture

The coordinated system formed by the Governance Framework, Research Framework, Knowledge Framework, and Runtime Framework, together with the artifacts and handoffs through which those responsibilities are operationalized. The responsibilities remain stable even when the project's tools or level of formality change.

Classification: Collaboration architecture | **Used in:** Framework-wide | **Defined or supplied by:** DCPF-defined

Distributed Cognitive Prompting (DCP)

The broader practice of distributing cognitive responsibilities across multiple prompts, artifacts, stages, or external supports instead of asking one prompt to carry the entire task.

Classification: Foundational concept | **Used in:** DCPF foundations | **Defined or supplied by:** Concept-defined

Distributed Cognitive Prompting Framework for AI Collaboration (DCPF)

An external collaboration architecture that separates four enduring responsibilities: Research develops evidence, Knowledge owns the architectural responsibility for preserving Accepted Knowledge, Runtime applies that Knowledge to present work, and Governance guides the entire system. The Knowledge Repository provides the persistent representation and maintained project home of Accepted Knowledge. DCPF organizes collections of interactions into persistent architectural systems; it is not a software product or a requirement to use a particular AI platform.

Classification: Collaboration architecture | **Used in:** Framework-wide | **Defined or supplied by:** DCPF-defined

E

Execution prompt

A short instruction that starts a Research Request or Runtime Request. The prompt initiates the work, while the supplied framework artifact contains the fuller operating instructions. Naming a file alone does not give the AI access to its contents.

Classification: Operating artifact | **Used in:** Research and Runtime execution | **Defined or supplied by:** User-defined

External cognition

The deliberate representation of knowledge, instructions, decisions, and controls outside individual human or model memory so they can be inspected, shared, governed, and improved.

Classification: Supporting concept | **Used in:** DCPF foundations; framework-wide | **Defined or supplied by:** Project-implemented

F

Failure routing

Tracing a weak or failed output to the responsibility that allowed the problem to occur. For example, a missing fact may belong to Research, an outdated fact to the Knowledge Repository, a poor format to Runtime, and a missing recurring standard to Governance.

Classification: Governance and diagnostic practice | **Used in:** Governance; evaluation and troubleshooting | **Defined or supplied by:** DCPF-defined; project-configured

Framework

A defined architectural responsibility within DCPF. A Framework may be operationalized through documents, requests, repositories, procedures, or tools, but it is not synonymous with any single artifact.

Classification: Core concept | **Used in:** Framework-wide | **Defined or supplied by:** DCPF-defined

Framework Purpose

The skeletal element that states what a framework is responsible for and what it is not responsible for. A clear purpose prevents Research, Knowledge preservation, Runtime execution, and Governance from drifting into one another.

Classification: Standard DCPF responsibility | Used in: Framework skeletons | Defined or supplied by: DCPF-recommended; user-supplied

Framework skeleton

The reusable structural pattern that identifies the standard DCPF responsibilities a framework implementation should represent. The wording and layout may be adapted, and standard responsibilities may share headings, but the responsibilities themselves should remain explicit, identifiable, and usable.

Classification: Core structural concept | Used in: Framework artifacts | Defined or supplied by: DCPF-defined

G

Governance

The human-authorized standards, boundaries, decision rights, validation requirements, and controls that guide the DCPF architecture. Governance is active across Research, Knowledge, Runtime, and output evaluation; it is not merely a final review step.

Classification: Core responsibility | **Used in:** Framework-wide | **Defined or supplied by:** Human-authorized

Governance attribute

A specific governing property or control, such as a quality standard, approved-source boundary, prohibition, validation requirement, approval point, or output standard.

Classification: Governance element | **Used in:** Governance Framework; framework artifacts | **Defined or supplied by:** Governance or authorized user

Governance Framework

The DCPF framework that owns the architectural responsibility for human-defined standards, boundaries, decision rights, validation requirements, output expectations, and controls across the project. It may be represented by a maintained Governance artifact, but its operating elements are carried into Research, Knowledge, Runtime, and review artifacts where they apply. A standalone Governance artifact is not required before Governance can operate.

Classification: Framework | Used in: Framework-wide | Defined or supplied by: DCPF structure; human-authorized content

The section of a Research, Knowledge, or Runtime artifact that states the Governance rules that apply within that artifact. It preserves alignment with human-authorized standards without making a separate Governance Framework or Repository an attachment, supply, paste, embedding, or retrieval dependency. | Classification: Framework reference element | Used in: Research, Knowledge, and Runtime artifacts | Defined or supplied by: User or project-defined

H

Human authority

The responsibility people retain for project objectives, acceptable practices, consequential boundaries, exceptions, acceptance decisions, and final approvals. AI may assist with analysis and checking, but it does not silently inherit these decision rights.

Classification: Governance principle | **Used in:** Governance; framework-wide | **Defined or supplied by:** Human-held

Human review

A person's examination of research, Knowledge, a framework artifact, or an output against the project's requirements. Review may lead to acceptance, revision, rejection, escalation, or another controlled action.

Classification: Governance practice | **Used in:** Governance; project workflow | **Defined or supplied by:** Human-performed

I

Information Categories

The headings, labels, or classifications used to organize approved information in the Knowledge Repository so users can locate it and distinguish different subjects or statuses.

Classification: Standard Knowledge responsibility | **Used in:** Knowledge Repository | **Defined or supplied by:** User or project-defined

Input Knowledge

The approved Knowledge supplied to a Runtime Request for the current task. Its complete content must be attached, embedded or pasted into the working context, or retrieved from a source the AI can actually access; an identifier or filename alone is not Input Knowledge.

Classification: Standard Runtime responsibility | **Used in:** Runtime Request | **Defined or supplied by:** User-selected from approved Knowledge

K

Knowledge

In DCPF, information the project has accepted and is prepared to use within a stated scope. This meaning is narrower than information that is merely available, remembered, retrieved, or generated.

Classification: Core concept | **Used in:** Knowledge Repository; Runtime | **Defined or supplied by:** Human-authorized

Knowledge Configuration Variables

Values that organize the current view, selection, audience, category, or use condition of the Knowledge Repository without changing the accepted information itself.

Classification: Standard Knowledge responsibility | **Used in:** Knowledge Repository | **Defined or supplied by:** User-defined

Knowledge Delivery Method

The route used to supply approved Knowledge to Runtime: attach it as a separate artifact; embed or paste its contents into the Runtime artifact or working context; or retrieve it from a source the AI can actually access. The method is a packaging choice, not a change to the DCPF architecture.

Classification: Runtime packaging choice | **Used in:** Runtime Request; project setup | **Defined or supplied by:** User or project-defined

Knowledge Framework

The DCPF Framework that owns the architectural responsibility for preserving Accepted Knowledge so it can be maintained and reused. The Knowledge Framework is an architectural responsibility, not the Repository in which the Knowledge is maintained.

Classification: Framework | **Used in:** Knowledge stage; framework-wide | **Defined or supplied by:** DCPF-defined

Knowledge Repository

The persistent representation and maintained project home of Accepted Knowledge. It may take the form of a document, collection of documents, folder, database, or other maintained project environment appropriate to the implementation.

Classification: Knowledge artifact | **Used in:** Knowledge Framework; project environment; Runtime inputs | **Defined or supplied by:** DCPF structure; human-authorized content

Knowledge Structure

The organized body of accepted information inside the Knowledge Repository. When a Research Output is designed well, much of its accepted content may move into this structure with little rewriting.

Classification: Standard Knowledge responsibility | **Used in:** Knowledge Repository | **Defined or supplied by:** User or project-defined

M

Maintenance

The deliberate work of keeping frameworks, Knowledge, rules, versions, and project files current and usable. Maintenance should be driven by evidence or an authorized need, not constant change for its own sake.

Classification: Operating practice | **Used in:** Project Environment; framework-wide | **Defined or supplied by:** User or project-defined

Master copy

The protected, reusable source version of a framework or template. Also called a frozen framework. Project-specific values should normally be entered in a working copy so the clean master remains recoverable.

Classification: Artifact status | **Used in:** Project Environment | **Defined or supplied by:** User or project-defined

Memory

A platform feature that may retain preferences or recurring details across interactions. Memory is not a substitute for a maintained Knowledge Repository and does not by itself establish authority, version, or project status.

Classification: AI platform feature | **Used in:** AI platform; comparison with Knowledge Repository | **Defined or supplied by:** Platform-defined

Metadata

Descriptive information about an artifact or Knowledge item, such as its owner, version, status, date, source, tags, review date, or scope. Formal Metadata is optional and supplemental in the basic Knowledge Repository skeleton; include it when it improves identification, traceability, maintenance, or reuse.

Classification: Optional / supplemental artifact descriptor | **Used in:** Artifacts; Knowledge Repository; Project Environment | **Defined or supplied by:** User, project, or system-defined

O

Output Organization

The Research Request element that states how findings should be arranged so the evidence remains reviewable and ready for possible acceptance into the Knowledge Repository.

Classification: Standard Research responsibility | **Used in:** Research Request | **Defined or supplied by:** DCPF-recommended; user-supplied

Output Requirements

The Runtime Request element that defines what the current deliverable must contain and how completion should look, including its sections, form, audience fit, length, limitations, or validation note.

Classification: Standard Runtime responsibility | **Used in:** Runtime Request | **Defined or supplied by:** DCPF-recommended; user-supplied

Output Standards

Governance requirements that apply consistently across comparable project outputs. They differ from Output Requirements, which define the needs of one current Runtime task.

Classification: Standard Governance responsibility | **Used in:** Governance Framework; output evaluation | **Defined or supplied by:** Governance or authorized user

P

Persistent knowledge

Accepted information preserved beyond one interaction so it can be maintained and reused. See Knowledge Repository.

Classification: Supporting concept | **Used in:** Knowledge Repository | **Defined or supplied by:** Human-authorized and preserved

Preservation

Saving accepted Knowledge, outputs, decisions, and execution records outside the conversation so the project remains inspectable, reusable, and maintainable.

Classification: Operating practice | **Used in:** Knowledge Repository; Project Environment | **Defined or supplied by:** User or project-defined

Prohibited Content

Information, sources, practices, behaviors, or outputs that Governance does not allow. AI may flag a conflict, but it should not invent an exception.

Classification: Standard Governance responsibility | **Used in:** Governance Framework; Research and Runtime Requests | **Defined or supplied by:** Governance or authorized user

Project Environment

The maintained collection of files, folders, naming practices, and working spaces used by a DCPF project. It commonly separates master artifacts, working copies, Research Outputs, Knowledge, Runtime files, Project Outputs, evaluations, and archives.

Classification: Project infrastructure | **Used in:** Project setup and maintenance | **Defined or supplied by:** User or project-defined

Project Output

The deliverable produced when Runtime applies approved Knowledge to a current task. A Project Output may be a document, analysis, plan, presentation, media asset, software component, or another usable result; it is not a fifth DCPF framework.

Classification: Output artifact | **Used in:** Runtime Framework; output evaluation | **Defined or supplied by:** Runtime-generated; human-reviewed

Prompt engineering

The practice of improving individual prompts and interactions. Prompt engineering optimizes individual interactions; DCPF organizes collections of interactions into persistent architectural systems.

Classification: Supporting AI practice | **Used in:** DCPF foundations; comparison | **Defined or supplied by:** Practitioner-defined

Prompt library

A reusable collection of prompts. It can be useful, but it is not automatically a DCPF architecture because it may not separate Governance, Research, accepted Knowledge, Runtime, and their handoffs.

Classification: Supporting artifact collection | **Used in:** DCPF foundations; comparison | **Defined or supplied by:** User or project-defined

Prompt system

A group of connected prompts that shares responsibilities across a project. It begins to resemble architecture when the prompts coordinate persistent Knowledge, controls, outputs, and reuse, but it is not automatically DCPF.

Classification: Supporting system concept | **Used in:** DCPF foundations; comparison | **Defined or supplied by:** User or project-defined

Provenance

The record of where information came from, how it was checked, which Research produced it, and what acceptance decision gave it its current status. Also called source history.

Classification: Knowledge and evidence record | **Used in:** Research Output; Knowledge Repository |

Defined or supplied by: Research- or project-generated

Q

Quality Standards

Governance statements that define what satisfactory work means. Validation Requirements specify how those standards are checked and what happens after the check.

Classification: Standard Governance responsibility | **Used in:** Governance Framework; validation |

Defined or supplied by: Governance or authorized user

R

Raw output

The unedited result produced by an AI execution and preserved before revision. Keeping it supports comparison, troubleshooting, and evaluation of how the request actually performed.

Classification: Execution artifact | **Used in:** Research and Runtime execution records | **Defined or supplied by:** AI-generated

Research Configuration Variables

User-supplied values that customize the current investigation, such as its subject, time period, location, depth, or audience, while the Research method remains stable.

Classification: Standard Research responsibility | **Used in:** Research Request | **Defined or supplied by:** DCPF-recommended; user-supplied

Research Framework

The DCPF component responsible for turning an information need into a deliberate investigation. It establishes what must be learned, which evidence may be used, how conflicts and uncertainty are handled, and how findings are documented for review.

Classification: Framework | **Used in:** Research stage; framework-wide | **Defined or supplied by:** DCPF-defined

Research Objectives

The questions and topics the Research Request must address. They establish required coverage without dictating the conclusion in advance.

Classification: Standard Research responsibility | **Used in:** Research Request | **Defined or supplied by:** DCPF-recommended; user-supplied

Research Output

The evidence and documentation produced by the Research Framework. It may include findings, sources, comparisons, uncertainty, conflicts, exclusions, and unresolved questions; it does not become accepted Knowledge automatically.

Classification: Research artifact | **Used in:** Research Framework; Knowledge evaluation | **Defined or supplied by:** Research-generated; human-reviewed

Research Request

The working artifact used to execute the Research Framework for a particular investigation. It supplies the purpose, variables, objectives, scope, source rules, output organization, Sequential Output Requirements, and applicable Governance.

Classification: Framework artifact | **Used in:** Research Framework execution | **Defined or supplied by:** DCPF structure; user-supplied content

Research Scope

The boundaries of an investigation, including what is included, excluded, how deeply it should be examined, and how current the evidence must be. Clear exclusions distinguish an intentional boundary from an accidental omission.

Classification: Standard Research responsibility | **Used in:** Research Request | **Defined or supplied by:** DCPF-recommended; user-supplied

Research Source Documentation

The evidence trail showing which sources were actually used and enough identifying detail to verify, update, or correct the findings.

Classification: Research evidence record | **Used in:** Research Output | **Defined or supplied by:** Research-generated

Retrieval

The act of locating information from files, search, databases, or other systems. Retrieval makes information available; it does not by itself make the information current, approved, or authoritative.

Classification: Operating process | **Used in:** Research; Knowledge access; Runtime | **Defined or supplied by:** User-, AI-, or system-performed

Revision History

The record of what changed in a maintained artifact, why it changed, when the change occurred, who authorized it, and which version became current.

Classification: Standard Governance responsibility | **Used in:** Framework artifacts; Project Environment | **Defined or supplied by:** User or system-maintained

Runtime Configuration Variables

User-supplied values that adjust the present execution, such as the audience, subject, tone, length, format, or technical level, without changing the approved Knowledge.

Classification: Standard Runtime responsibility | **Used in:** Runtime Request | **Defined or supplied by:** DCPF-recommended; user-supplied

Runtime Framework

The DCPF component responsible for applying approved Knowledge to present work. Runtime selects the relevant Knowledge, applies current variables, defines the task and deliverable, and operates under Governance without silently rewriting the repository.

Classification: Framework | **Used in:** Runtime stage; framework-wide | **Defined or supplied by:** DCPF-defined

Runtime Request

The working artifact used to execute the Runtime Framework for a particular task. It supplies the purpose, current variables, Input Knowledge, Task Definition, Output Requirements, Sequential Output Requirements, and applicable Governance.

Classification: Framework artifact | **Used in:** Runtime Framework execution | **Defined or supplied by:** DCPF structure; user-supplied content

S

Sequential Output Requirements

Often abbreviated SOR, Sequential Output Requirements are a standard DCPF element of both the Research and Runtime Framework skeletons. They protect complete delivery when an output may exceed practical response length or its length cannot be reliably predicted, so the user does not have to guess the limit in advance. DCPF supplies the standard rule; the user controls its application and may deliberately adapt it for a specific project.

Classification: Standard DCPF responsibility | **Used in:** Research and Runtime Requests | **Defined or supplied by:** DCPF-supplied; user-controlled application

Sequential Output Rule

The operating rule within the Sequential Output Requirements. When needed, the AI divides the work into sequential parts without compressing, summarizing, abbreviating, or omitting required content merely to fit one response. Each nonfinal part must state that additional output remains, request permission to continue, and stop; after permission, delivery resumes where it ended until the final part clearly states that the requested output is complete.

Classification: Operating rule | **Used in:** Research and Runtime execution | **Defined or supplied by:** DCPF-supplied; user-controlled application; AI-executed

Source Priority Hierarchy

The claim-specific rules used to decide how approved evidence should be weighted and how conflicts should be handled. In working artifacts, this responsibility may be labeled Source Priority and Conflict Handling. Priority may consider directness, authority, primary/original evidence, currentness, jurisdiction, methodological transparency, relevance, and corroboration. It is not a universal ranking of source categories, and unresolved credible conflicts remain visible for human judgment.

Classification: Research element | **Used in:** Research Request; Research execution | **Defined or supplied by:** User within Governance boundaries

Source Register

An organized record of the sources actually used during Research, with enough detail for another reviewer to identify and check them.

Classification: Research evidence artifact | **Used in:** Research Output | **Defined or supplied by:** Research-generated

Standard DCPF responsibility

A core responsibility that belongs to a framework's normal DCPF skeleton. It should be represented in the implementation, but it does not require a separate heading, page, or standalone artifact. Standard describes the architectural expectation; implementation may remain compact when the responsibility is still explicit, identifiable, and usable.

Classification: Core design term | **Used in:** Governance, Research, Knowledge, and Runtime skeletons | **Defined or supplied by:** DCPF-defined

Status label

A word or short phrase showing an artifact's current lifecycle state, such as DRAFT, TEST, APPROVED, RETIRED, or ARCHIVED. Knowledge evaluation may use related labels such as APPROVED, REVISE, UNRESOLVED, REJECTED, or OUTDATED.

Classification: Artifact or Knowledge descriptor | **Used in:** Artifacts; Knowledge Repository; Project Environment | **Defined or supplied by:** User or project-defined

T

Task Definition

The Runtime Request element that states what the AI must do with the supplied Knowledge, including any important order, conditions, or boundaries. The task defines the work; Output Requirements define the finished form.

Classification: Standard Runtime responsibility | **Used in:** Runtime Request | **Defined or supplied by:** DCPF-recommended; user-supplied

U

User-defined

Supplied or authorized by the user or another responsible project authority. User-defined does not mean optional: a standard DCPF responsibility may still require the user to provide its values or rule content.

Classification: Definition-status term | **Used in:** Framework artifacts; configuration values | **Defined or supplied by:** DCPF-defined

V

Validation

Repeatable checking of an artifact, handoff, or output against explicit requirements. A passed validation check does not by itself equal human approval or acceptance unless Governance says that the check has that authority.

Classification: Governance practice | **Used in:** Governance; Research and Runtime outputs | **Defined or supplied by:** Governance-defined; human or system-performed

Validation Requirements

Governance instructions that state what must be checked, who or what performs the check, and what may happen next, such as accept, revise, reject, escalate, or return the work to another framework.

Classification: Standard Governance responsibility | **Used in:** Governance Framework; framework artifacts | **Defined or supplied by:** Governance or authorized user

Version control

The practices used to identify the current authoritative state of an artifact, preserve earlier states, and prevent mismatched components from being used together.

Classification: Project management practice | **Used in:** Project Environment; maintained artifacts | **Defined or supplied by:** User, project, or system-defined

W

Workflow

The sequence of actions and handoffs through which a DCPF project moves from setup and Research to review, Knowledge, Runtime, output evaluation, preservation, and maintenance. Workflow connects the frameworks; it is not another framework.

Classification: Architectural process | **Used in:** Framework-wide | **Defined or supplied by:** DCPF-guided; project-configured

Working context

See Active context.

Classification: Cross-reference term | **Used in:** AI interaction; Research and Runtime execution | **Defined or supplied by:** DCPF-defined cross-reference

Working copy

An editable project-specific copy of a master framework or template. Variables and current project details are entered here so the protected master remains unchanged.

Classification: Artifact status | **Used in:** Project Environment | **Defined or supplied by:** User or project-defined

[Return to Table of Contents](#)

DCPF Responsibility-Consequence Map

DCPF remains an experimental approach. Use this map to support controlled testing and adaptation; it does not replace domain-specific validation or human judgment before real-world or production use.

A diagnostic reference for seeing where human decision responsibility moves when a standard DCPF responsibility is left undefined.

DCPF does not assume that leaving one responsibility undefined will automatically cause a failure. Instead, an explicit human control, decision, or protection becomes less visible. The responsibility may then be handled informally by a person, inferred by AI, shaped by platform or model defaults, absorbed elsewhere in the architecture, or left effectively unowned.

The final column uses familiar AI-risk terminology only where it describes a plausible failure mechanism. Terms such as hallucination, unsupported inference, automation bias, context drift, authority creep, provenance loss, silent omission, and governance drift are not predictions that a failure will occur. They identify recognizable risks that become harder to control when the corresponding human responsibility is not defined.

G.1 How to Read the Map

Each row begins with one standard DCPF responsibility. The second column identifies who holds or authorizes that responsibility when it is explicitly defined. The third shows who or what tends to fill the decision vacuum when it is not defined. The fourth identifies the human control that disappears, and the fifth names the most likely AI or collaboration consequences.

The central question is not whether every responsibility needs its own heading or document. DCPF allows responsibilities to be combined or represented compactly. The question is whether the decision itself remains explicit, identifiable, usable, and under intentional human authority.

G.2 Governance Framework Responsibility-Consequence Map

Governance defines the enduring human-authorized standards, boundaries, validation requirements, output expectations, and controls that guide the architecture. When one of these responsibilities is left undefined, the system may still operate, but a distinct form of human control becomes less explicit.

Responsibility	When defined: responsibility belongs to	When undefined: responsibility tends to fall to	Control lost	Likely AI / collaboration consequences
Framework Purpose	Human / responsible organization	AI inference, ad hoc human judgment, or no clear owner	Explicit scope of Governance authority	Authority ambiguity, scope drift, inconsistent rule application
Quality Standards	Human, organization, or domain authority	AI self-assessment, model defaults, or subjective ad hoc review	Human definition of what satisfactory work means	False confidence, automation bias, inconsistent quality, polished-but-wrong output
Approved Information Sources	Human / Governance authority	AI source selection, search ranking, platform availability, or convenience	Human control over acceptable evidence	Source-selection bias, weak grounding, misinformation propagation, hallucination risk
Prohibited Content	Human authority	AI inference, general platform rules, or no explicit substitute	Explicit negative boundaries on what must not be used or produced	Boundary violations, unsafe or inappropriate output, authority creep, inconsistent refusals or inclusions
Validation Requirements	Human / responsible reviewer	AI self-check, informal human review, or nobody	Independent criteria and decision path for acceptance	Automation bias, self-validation, undetected hallucination, error propagation, false confidence
Output Standards	Human / organization	AI formatting preferences or one-off Runtime choices	Stable recurring expectations for comparable deliverables	Inconsistent outputs, accessibility failures, silent omission, completion ambiguity

Responsibility	When defined: responsibility belongs to	When undefined: responsibility tends to fall to	Control lost	Likely AI / collaboration consequences
Revision History	Framework owner / maintainer	Current context, memory, or whichever version happens to be available	Traceability of rule changes and authorization	Governance drift, stale-rule application, contradictory behavior, version confusion

G.3 Research Framework Responsibility-Consequence Map

Research converts a human-defined information need into a controlled evidence product. Its nine responsibilities make the investigation purpose, variables, questions, evidence boundaries, scope, documentation, delivery, and governing conditions visible before or during the work.

Responsibility	When defined: responsibility belongs to	When undefined: responsibility tends to fall to	Control lost	Likely AI / collaboration consequences
Framework Purpose	Human / project owner	AI inference or ad hoc human direction	Explicit reason for the investigation and its downstream use	Goal drift, Research/Runtime boundary blur, unsupported task expansion
Research Configuration Variables	Human / user	AI inference, prior-context assumptions, or model defaults	Control over the current subject, period, location, audience, depth, and other variables	Context drift, stale-context reuse, unintended assumptions, inconsistent results
Research Objectives	Human / user	AI	Human determination of what questions and evidence needs must be addressed	Scope drift, silent omission, unsupported inference, false confidence

Responsibility	When defined: responsibility belongs to	When undefined: responsibility tends to fall to	Control lost	Likely AI / collaboration consequences
Approved Information Sources	Human / Governance	AI source selection, search ranking, platform availability, or model preference	Human control over the evidence boundary	Source-selection bias, weak grounding, misinformation propagation, hallucination risk
Source Priority Hierarchy	Human within Governance boundaries	AI reasoning about credibility and conflict resolution	Human-defined rules for weighting competing evidence	Authority substitution, bias amplification, unsupported reconciliation, false confidence
Research Scope	Human / user	AI interpretation	Human control over inclusions, exclusions, depth, recency, and boundaries	Scope creep, overgeneralization, irrelevant generation, silent omission
Output Organization	Human / user	AI	Human-defined structure for a reviewable evidence product	Unstructured findings, review friction, omission risk, poor Knowledge handoff
Research Source Documentation	Human-defined Research / Governance requirement; AI performs documentation	AI discretion or nobody	Required provenance and evidence trail	Provenance loss, unverifiable claims, fabricated or incomplete citations, harder hallucination detection
Sequential Output Requirements	Human / user	AI response-management behavior	Human protection against compression or truncation	Silent truncation, compression, summary substitution, missing evidence

G.4 Knowledge Framework Responsibility-Consequence Map

The Knowledge Framework preserves Accepted Knowledge so it can be maintained and reused. Its four standard responsibilities keep the repository purposeful, selectable, organized, findable, and connected to the human-authorized conditions under which the Knowledge may be used.

Responsibility	When defined: responsibility belongs to	When undefined: responsibility tends to fall to	Control lost	Likely AI / collaboration consequences
Framework Purpose	Human / project owner	AI inference, repository conventions, or ad hoc human use	Explicit definition of what Accepted Knowledge is preserved and why	Context drift, inappropriate reuse, repository sprawl, authority ambiguity
Knowledge Configuration Variables	Human / user	AI inference, current context, or prior settings	Human control over the current view, selection, audience, category, or use condition	Stale-context reuse, wrong selection, silent reinterpretation, inconsistent retrieval
Knowledge Structure	Human-authorized Knowledge process; accepted content supplied by reviewed Research	AI reconstruction, summaries, memory, or no usable repository	The organized body of Accepted Knowledge itself	Knowledge substitution, hallucination, provenance loss, unsupported Runtime generation
Information Categories	Human / project design, informed by accepted Research	AI classification, search behavior, or ad hoc organization	Human-defined findability and distinction among Knowledge types or statuses	Retrieval errors, context confusion, category drift, wrong-Knowledge selection

G.5 Runtime Framework Responsibility-Consequence Map

Runtime applies approved Knowledge to the task being performed now. Its six responsibilities keep execution bounded by the current purpose, settings, supplied Knowledge, task definition, completion requirements, delivery safeguards, and Governance.

Responsibility	When defined: responsibility belongs to	When undefined: responsibility tends to fall to	Control lost	Likely AI / collaboration consequences
Framework Purpose	Human / project owner	AI inference or ad hoc human direction	Human definition of what work is being performed now	Goal drift, authority creep, reopening of upstream decisions, task ambiguity
Runtime Configuration Variables	Human / user	AI inference, previous-run settings, or model defaults	Control over audience, subject, tone, length, format, technical level, and delivery setting	Context drift, stale-setting reuse, inconsistent outputs, audience mismatch
Input Knowledge	Human selects and supplies approved Knowledge	AI general knowledge, memory, partial context, or inference	Grounding in project-authorized Knowledge	Hallucination, stale information, knowledge substitution, unsupported inference, false confidence
Task Definition	Human / user	AI	Human control over the exact action the model must perform	Authority creep, goal drift, unintended work, overreach
Output Requirements	Human / user and/or Governance	AI defaults or implicit expectations	Human definition of form, required components, limits, validation needs, and completion	Silent omission, format failure, incomplete deliverables, false sense of completion
Sequential Output Requirements	Human / user	AI response-management behavior	Human protection against compression or truncation	Silent truncation, abbreviation, compression, missing deliverable components

G.6 The Pattern Across the Four Frameworks

Across the four tables, undefined responsibilities tend to migrate toward four places: AI inference, platform or model defaults, ad hoc human judgment, or no effective owner. The risk is not simply that AI "takes over." The deeper problem is that decision authority becomes less visible and less deliberate.

DCPF therefore treats responsibility definition as a form of human control. A responsibility may be represented briefly, embedded in another artifact, or supported by an external process, but its underlying decision should remain explicit enough for the collaboration to know who is authorized to decide, what standard applies, and what happens when the condition is not met.

Structured human responsibility does not eliminate AI risk. It makes the places where that risk is controlled - or left uncontrolled - visible.

DCPF Responsibility-Consequence Map | Appendix G | Prepared for the DCPF Introductory Edition

[Return to Table of Contents](#)